ACM/ICPC

个人算法模板

与思维训练手册

poisonous-sheep / sheep-banzi

C++17 • 模板调用 • 状态设计 • 建模证明

84 专题 / 51 份速查板子 / 177 份例题

A4 打印装订版 • 2026-10-12 • v2

本地编译与样例验证记录见阅读说明。未提交在线评测，不宣称 AC。

第一部分 阅读说明与总目录

本版基于本地库及 GitHub main 的 fbd87c80b2ead1717b992c9c273153e9c20803f5。完整覆盖上一版85项知识内容：两项树上行走专用DP归入一个含三道递进例题的思维专题，活跃专题因此为84项，旧文件保留。课程原件、视频、无关软件与压缩包没有自动导入。

模板型 44 项，思维型 26 项，混合型 14 项。共 51 份不重复的速查板子；其余以题目推导组织。每个专题都有2至3道真实例题，共177份实现。F1/F2仍明确视为同题的难度版本，LCM两道题明确为相同公式的批量应用。

本版177个例题程序通过C++17编译，完成89次登记样例/边界运行。独立参考共32组、15350例，其中原17组8550例、函数版11组4850例与新增直接题解4组1950例；具体范围见附录。以上不是在线AC或全部最大数据性能保证。

第二部分提供通用板子、接口和模板型例题。第三部分围绕观察、朴素方案、突破、状态/建模、证明、初始化、优化和启发讲解；普通DP使用函数和必要全局变量。第四部分先推导模型，再把实际操作映射到已经收录的模块，不再次展开相同板子。

标为完整题解的代码含头文件、solve和signed main；若注明组合依赖，则需从第二部分取对应struct定义后再编译。源文件与catalog分别保存阅读片段和验证组合程序，不能把缺少依赖的片段称为独立可编译。旧模板的示范main读取一次solve，多测在solve内处理。

代码源272/273原题无法访问，按用户要求改用HDU5293、HDU4912、CF1675D、CF671D。它们分别讲点不相交选路、祖先点覆盖及边覆盖转换；不会声称课堂原题已经获得。

A4纵向，左26mm装订边、右18mm；正文10.2pt、等宽代码9pt，黑白打印。建议按实际大小打印。代码行号连续，标题、题号及网址可检索，电子版目录与书签可点击。

总目录

第一部分 阅读说明与总目录 2

第二部分 算法模板速查 12

数据结构 / 01Trie 12

例题 1 CF706D Vasiliy’s Multiset 14

例题 2 P4551 最长异或路径 14

数据结构 / ST表 15

例题 1 P3865 ST表 16

例题 2 P2880 Balanced Lineup 17

数据结构 / 动态开点权值线段树 18

例题 1 ABC253C Max-Min Query 21

例题 2 P3369 普通平衡树 21

例题 3 ABC241D Sequence Query 22

数据结构 / 势能线段树 23

例题 1 P4145 花神游历各国 26

例题 2 CF438D The Child and Sequence 27

数据结构 / 区间加线段树 28

例题 1 P3372 线段树1 30

例题 2 P3374 树状数组1 31

数据结构 / 对顶堆与中位数 32

例题 1 ABC127F Absolute Minima 34

例题 2 P1168 中位数 34

例题 3 LeetCode480 滑动窗口中位数 35

数据结构 / 并查集 36

例题 1 practice2_a Disjoint Set Union 37

例题 2 ABC120D Decayed Bridges 38

数据结构 / 普通线段树 39

例题 1 P1816 忠诚 41

例题 2 P1198 最大数 41

数据结构 / 权值线段树合并与分裂 42

例题 1 P5494 线段树分裂 47

例题 2 P3224 永无乡 48

数据结构 / 树状数组 49

例题 1 P3374 树状数组1 51
例题 2 P1908 逆序对 51
例题 3 P1966 火柴排队 52
数据结构 / 珂朵莉树 53
例题 1 CF896C Willem Chtholly and Seniorious 55
例题 2 P3740 贴海报 56
图论 / 2-SAT 57
例题 1 P4782 2-SAT 60
例题 2 P4171 满汉全席 60
图论 / DFS增广路匹配 61
例题 1 P3386 二分图最大匹配 63
例题 2 practice2_d Maxflow 63
图论 / Dinic最大流 65
例题 1 P3376 网络最大流 67
例题 2 P2763 试题库问题 68
图论 / Hopcroft-Karp匹配 69
例题 1 P3386 二分图最大匹配 71
例题 2 practice2_d Maxflow 72
图论 / Kosaraju强连通分量 73
例题 1 P2341 受欢迎的牛 75
例题 2 P3387 缩点 76
图论 / Prim最小生成树 77
例题 1 P3366 最小生成树 78
例题 2 ABC065D Built 79
图论 / Tarjan强连通分量 80
例题 1 P2341 受欢迎的牛 82
例题 2 P3387 缩点 83
图论 / 二分图染色 85
例题 1 ABC327D Good Tuple Problem 86
例题 2 P1330 封锁阳光大学 87
图论 / 倍增LCA 88
例题 1 P3379 最近公共祖先 89
例题 2 CF1328E Tree Queries 90
图论 / 割点 91
例题 1 P3388 割点 91

例题 2 P3469 BLO-Blockade 92
图论 / 割边 93
例题 1 P1656 炸铁路 95
例题 2 CF118E Bertown Roads 95
图论 / 差分约束 97
例题 1 P5960 差分约束 100
例题 2 P3275 糖果 101
图论 / 拓扑排序 102
例题 1 P1137 旅行计划 103
例题 2 ABC223D Restricted Permutation 104
图论 / 无向欧拉路径 105
例题 1 P2731 Riding the Fences 106
例题 2 P1341 无序字母对 107
图论 / 最小费用最大流 108
例题 1 P3381 最小费用最大流 110
例题 2 P4015 运输问题 111
图论 / 有上下界可行流与最大流 112
例题 1 CF510E Fox And Dinner 116
例题 2 P5192 Shoot the Bullet 117
图论 / 有向欧拉路径 119
例题 1 P7771 欧拉路径 120
例题 2 CF508D Tanya and Password 121
图论 / 点双连通分量 122
例题 1 P8435 点双连通分量 124
例题 2 P3225 矿场搭建 125
图论 / 边双连通分量 126
例题 1 P8436 边双连通分量 126
例题 2 P2860 Redundant Paths 126
数论 / BSGS离散对数 128
例题 1 P3846 可爱的质数 130
例题 2 P2485 计算器 130
数论 / Lucas定理 131
例题 1 P3807 Lucas定理 132
例题 2 P1313 计算系数 133
数论 / 中国剩余定理 134

例题 1 P1495 曹冲养猪 135
例题 2 P4777 扩展中国剩余定理 136
数论 / 原根 137
例题 1 P6091 原根 139
例题 2 CF284A Cows and Primitive Roots 139
数论 / 扩展Lucas 140
例题 1 P4720 扩展Lucas 142
例题 2 P2183 礼物 143
数论 / 扩展欧几里得 144
例题 1 P1082 同余方程 145
例题 2 ABC186E Throne 145
数论 / 扩展欧拉与指数塔 146
例题 1 P4139 上帝与集合的正确用法 148
例题 2 CF906D Power Tower 148
数论 / 整除分块 149
例题 1 P1403 约数研究 150
例题 2 P2424 约数和 151
数论 / 素数模组合数 151
例题 1 ABC034C 经路 152
例题 2 ABC145D Knight 153
数论 / 莫比乌斯反演与线性筛 154
例题 1 P2522 ZAP-Queries 155
例题 2 P3455 Problem b 156
动态规划 / 数位DP与有限状态 157
例题 1 P2657 windy数 159
例题 2 DP_S Digit Sum 159
字符串、线性代数与几何 / KMP字符串匹配 161
例题 1 P3375 KMP 162
例题 2 CF471D MUH and Cube Walls 162
字符串、线性代数与几何 / 点直线与线段几何 163
例题 1 CGL_1_A Projection 166
例题 2 CGL_1_B Reflection 167
字符串、线性代数与几何 / 线性基 168
例题 1 P3812 线性基 169
例题 2 P4151 最大XOR和路径 170

例题 3 P4570 元素 171

综合应用的可复用模块 172

DSU on Tree / 速查模块 172

启发式合并 / 速查模块 174

离线二维数点 / 速查模块 176

线段树优化建图 / 速查模块 178

Kruskal重构树 / 速查模块 182

最小割建模 / 速查模块 184

度数根号分治 / 速查模块 188

步长根号分治 / 速查模块 190

曼哈顿距离坐标旋转 / 速查模块 191

第三部分 算法思维与经典模型 194

动态规划 / 二状态树形背包 194

例题 1 Gym102992M Monster Hunter 194

例题 2 P2016 战略游戏 196

动态规划 / 二进制位贡献与Popcount计数 197

例题 1 Gym102832D Meaningless Sequence 197

例题 2 ABC356D Masked Popcount 199

例题 3 P4317 花神的数论题 200

动态规划 / 区间同色配对与涂色 202

例题 1 CF1132F Clear the String 202

例题 2 P4170 涂色 204

动态规划 / 区间强制对象分治 206

例题 1 P4766 Outer space invaders 206

例题 2 UVA10003 Cutting Sticks 208

动态规划 / 区间石子合并 211

例题 1 DP_N Slimes 211

例题 2 P1880 石子合并 213

动态规划 / 子树合并背包 215

例题 1 P2014 选课 215

例题 2 P1273 有线电视网 217

动态规划 / 括号计数与无歧义区间DP 219

例题 1 P7914 括号序列 219

例题 2 ABC312D Count Bracket Sequences 221
动态规划 / 换根连通块计数 222
例题 1 DP_V Subtree 222
例题 2 CF543D Road Improvement 224
动态规划 / 数位构造与不同数位限制 226
例题 1 CF1560F1 Nearest Beautiful Number easy 226
例题 2 CF1560F2 Nearest Beautiful Number hard 229
动态规划 / 数位背包 232
例题 1 P3188 梦幻岛宝珠 232
例题 2 P1048 采药 233
动态规划 / 树上最大连通块权和 235
例题 1 P1122 最大子树和 235
例题 2 CF1324F Maximum White Subtree 236
动态规划 / 树上行走与返回状态设计 238
例题 1 ABC361E Tree and Hamilton Path 2 238
例题 2 Gym102769K Kingdom’s Power 240
例题 3 Gym103470H Crystalfly 242
动态规划 / 树上距离分层计数 244
例题 1 CF1551F Equidistant Vertices 244
例题 2 CF161D Distance in Tree 246
动态规划 / 自相似序列奇偶分治 248
例题 1 QOJ12409 Bit Sequence 248
例题 2 ABC242D ABC Transform 250
动态规划 / 进位支付与有符号数位 252
例题 1 ABC231E Minimal payments 252
例题 2 ABC155E Payment 253
数据结构 / 启发式分治 255
例题 1 UVA1608 Non-boring sequences 255
例题 2 CF1156E Special Segments 257
字符串、线性代数与几何 / 极角排序 260
例题 1 CF598C Nearest vectors 260
例题 2 ABC139F Engines 263
转化、构造与计数 / 两互质面额不可表示数 267
例题 1 P3951 小凯的疑惑 267
例题 2 HDU1792 A New Change Problem 268

转化、构造与计数 / 前缀关系计数 270
例题 1 CF1398C Good Subarrays 270
例题 2 ABC105D Candy Distribution 271
转化、构造与计数 / 同余差与GCD 273
例题 1 CF1920C Partitioning the Array 273
例题 2 ABC109C Skip 274
转化、构造与计数 / 异或比较与最高不同位 276
例题 1 CF1720D1 Xor-Subsequence (easy version) 276
例题 2 CF276D Little Girl and Maximum XOR 277
转化、构造与计数 / 置换环分解 279
例题 1 ABC350C Sort 279
例题 2 ABC247F Cards 281
例题 3 ABC436E Minimum Swap 283
数论 / LCM单行求和 286
例题 1 SP5971 LCMSUM - LCM Sum 286
例题 2 P1891 疯狂LCM 288
数论 / LCM矩形与集合求和 290
例题 1 P1829 Crash的数字表格 290
例题 2 AGC038C LCMs 292
数论 / 互质集合与GCD分类计数 294
例题 1 CF803F Coprime Subsequences 294
例题 2 CF839D Winter is here 296
数论 / 卡特兰数与反射原理 298
例题 1 P1044 栈 298
例题 2 P1641 生成字符串 299

第四部分 混合与综合应用 301

数据结构 / DSU on Tree 301
例题 1 CF600E Lomsat gelral 301
例题 2 CF375D Tree and Queries 302
数据结构 / 启发式合并 303
例题 1 P3201 梦幻布丁 303
例题 2 CF600E Lomsat gelral 305
数据结构 / 离线二维数点 306
例题 1 P2163 园丁的烦恼 306

例题 2 P1972 HH的项链 307
数据结构 / 线段树优化建图 308
例题 1 CF786B Legacy 308
例题 2 P6348 Journeys 309
图论 / Kruskal重构树 310
例题 1 P2245 星际导航 310
例题 2 P1967 货车运输 311
图论 / 基环树最大权独立集 313
例题 1 P2607 骑士 313
例题 2 P1453 城市环路 316
图论 / 基环树最长路径与断环直径 320
例题 1 P4381 Island 320
例题 2 CF835F Roads in the Kingdom 324
图论 / 最小割建模 329
例题 1 P2774 方格取数问题 329
例题 2 P2762 太空飞行计划问题 330
例题 3 P4126 最小割边判定 331
动态规划 / DFS序依赖背包 333
例题 1 P2014 选课 DFS序版 333
例题 2 P2515 软件安装 335
动态规划 / 树上不相交给定路径 337
例题 1 HDU5293 Tree chain problem 337
例题 2 HDU4912 Paths on the tree 340
动态规划 / 祖先路径最小费用覆盖 344
例题 1 CF1675D Vertical Paths 344
例题 2 CF671D Roads in Yusland 345
字符串、线性代数与几何 / 度数根号分治 349
例题 1 ABC219G Propagation 349
例题 2 Typical90-083 Colorful Graph 350
字符串、线性代数与几何 / 步长根号分治 352
例题 1 CF1207F Remainder Problem 352
例题 2 CF797E Array Queries 353
转化、构造与计数 / 曼哈顿距离坐标旋转 354
例题 1 ABC178E Dist Max 354
例题 2 Typical90036 Max Manhattan Distance 354

例题 3 ABC437F Manhattan Christmas Tree 2 355

第五部分 附录与索引 357

常用代码片段与复杂度 357

常见边界与本版验证范围 357

知识点与旧版映射 357

题号、位置与原题网址 361

第二部分 算法模板速查

数据结构

数据结构 / 01Trie

类型与用途：模板型；非负整数的动态可重集合与最大异或查询；最高位B<=62。

复杂度与适用范围：增删/查询O(B)，空间O(D*B)，删除不回收节点。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 01Trie: 非负整数的动态可重集合与最大异或查询；最高位B<=62。
// 增删/查询O(B)，空间O(D*B)，删除不回收节点。
struct Trie {
    struct node {
        int32_t t[2] = {0, 0};
        int cnt = 0;
    };
    int B;
    vector<node> seg;
    Trie(int b = 32) {
        init(b);
    }
    void init(int b = 32) {
        assert(0 <= b && b <= 62);
        B = b;
        seg.assign(1, {});
    }
    void update(int x) {
        assert(x >= 0);
        int u = 0;
        seg[u].cnt++;
        for (int i = B; i >= 0; i--) {
            int ch = x >> i & 1;
            if (!seg[u].t[ch]) {
                int id = seg.size();
                seg.push_back({});
                seg[u].t[ch] = id;
            }
            u = seg[u].t[ch];
            seg[u].cnt++;
        }
    }
    int count(int x) {
        int u = 0;
        for (int i = B; i >= 0; i--) {
            int ch = x >> i & 1;
            if (!seg[u].t[ch])
                return 0;
            u = seg[u].t[ch];
        }
        return seg[u].cnt;
    }
```

```
    bool eras(int x) {
        if (!count(x))
            return false;
        int u = 0;
        seg[u].cnt--;
        for (int i = B; i >= 0; i--) {
            u = seg[u].t[x >> i & 1];
            seg[u].cnt--;
        }
        return true;
    }
    int qur(int x) {
        assert(seg[0].cnt > 0);
        int u = 0, ans = 0;
        for (int i = B; i >= 0; i--) {
            int ch = x >> i & 1, to = ch ^ 1;
            int v = seg[u].t[to];
            if (v && seg[v].cnt) {
                ans |= 1LL << i;
                u = v;
            } else
                u = seg[u].t[ch];
        }
        return ans;
    }
};

void solve() {
    int q;
    cin >> q;
    Trie tr(30);
    tr.update(0);
    while (q--) {
        char op;
        int x;
        cin >> op >> x;
        if (op == '+')
            tr.update(x);
        else if (op == '-')
            tr.eras(x);
        else
            cout << tr.qur(x) << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

update(x)：插入

eras(x)：删除一个值，不存在时返回false且不修改状态

qur(x)：最大x xor y

例题 1 CF706D Vasiliy’s Multiset

https://codeforces.com/problemset/problem/706/D

题意：初始有0的集合，支持增删和最大异或。

观察与建模：初始有0的集合，支持增删和最大异或。 Trie逐位优先走相反位，贪心取得最大异或值。

为何对应此算法：Trie逐位优先走相反位，贪心取得最大异或值。

初始化、组织数据与调用：init，update(0)；+用update，-用eras，?用qur。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(qB)时间、O(qB)空间。

复用模板：01Trie（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int q;
    cin >> q;
    Trie tr(30);
    tr.update(0);
    while (q--) {
        char op;
        int x;
        cin >> op >> x;
        if (op == '+')
            tr.update(x);
        else if (op == '-')
            tr.eras(x);
        else
            cout << tr.qur(x) << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P4551 最长异或路径

https://www.luogu.com.cn/problem/P4551

题意：树上找边权异或最大的路径。

观察与建模：树上找边权异或最大的路径。 两点路径异或等于根到两点的前缀异或之xor，将所有前缀放入Trie。

为何对应此算法：两点路径异或等于根到两点的前缀异或之xor，将所有前缀放入Trie。

初始化、组织数据与调用：DFS计算前缀；update每个前缀，再qur求最大。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(nB)时间、O(nB)空间。

复用模板：01Trie（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    vector<vector<pair<int, int>>> e(n + 1);
    for (int i = 1, u, v, w; i < n; i++) {
        cin >> u >> v >> w;
        e[u].push_back({v, w});
        e[v].push_back({u, w});
    }
    vector<int> d(n + 1);
    function<void(int, int)> dfs = [&](int u, int fa) {
        for (auto [v, w] : e[u])
            if (v != fa) {
                d[v] = d[u] ^ w;
                dfs(v, u);
            }
    };
    dfs(1, 0);
    Trie tr(30);
    for (int i = 1; i <= n; i++)
        tr.update(d[i]);
    int ans = 0;
    for (int i = 1; i <= n; i++)
        ans = max(ans, tr.qur(d[i]));
    cout << ans << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 删除先验证完整路径，修复原代码不存在元素时部分减计数的问题。
- 值必须落在0..2^(B+1)-1；B=62只支持非负long long。

个人资料来源：数据结构/01tire（支持删）.cpp

数据结构 / ST表

类型与用途：模板型；静态区间最小值与最大值；重叠两段要求合并幂等。

复杂度与适用范围：建表O(n log n)，查询O(1)，空间O(n log n)。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// ST表：静态区间最小值与最大值；重叠两段要求合并幂等。
// 建表O(n log n)，查询O(1)，空间O(n log n)。
struct ST {
    struct info {
        int mn, mx;
        friend info operator+(const info &l, const info &r) {
            return {min(l.mn, r.mn), max(l.mx, r.mx)};
        }
    };
    int n = 0;
```

```
    vector<int> lg;
    vector<vector<info>> st;
    void build(const vector<int> &a) {
        n = a.size() - 1;
        assert(n > 0);
        lg.assign(n + 1, 0);
        for (int i = 2; i <= n; i++)
            lg[i] = lg[i / 2] + 1;
        st.assign(lg[n] + 1, vector<info>(n + 1));
        for (int i = 1; i <= n; i++)
            st[0][i] = {a[i], a[i]};
        for (int j = 1; j <= lg[n]; j++)
            for (int i = 1; i + (1LL << j) - 1 <= n; i++)
                st[j][i] = st[j - 1][i] + st[j - 1][i + (1LL << (j - 1))];
    }
    info qur(int l, int r) {
        assert(1 <= l && l <= r && r <= n);
        int k = lg[r - l + 1];
        return st[k][l] + st[k][r - (1LL << k) + 1];
    }
};

void solve() {
    int n, q;
    cin >> n >> q;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    ST st;
    st.build(a);
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << st.qur(l, r).mx << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

build(a)：1-based数组；按实际n动态分配

qur(l,r)：闭区间，返回mn/mx

例题 1 P3865 ST表

https://www.luogu.com.cn/problem/P3865

题意：静态区间最大值。

观察与建模：静态区间最大值。 max幂等，重叠查询段不会重复影响答案。

为何对应此算法：max幂等，重叠查询段不会重复影响答案。

初始化、组织数据与调用：build(a)，qur(l,r).mx。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n log n+q)时间、O(n log n)空间。

复用模板：ST表（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, q;
    cin >> n >> q;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    ST st;
    st.build(a);
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << st.qur(l, r).mx << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P2880 Balanced Lineup

https://www.luogu.com.cn/problem/P2880

题意：每段牛身高的最大值与最小值之差。

观察与建模：每段牛身高的最大值与最小值之差。 一次info同时维护min/max，查询后相减。

为何对应此算法：一次info同时维护min/max，查询后相减。

初始化、组织数据与调用：qur返回info，输出mx-mn。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n log n+q)时间、O(n log n)空间。

复用模板：ST表（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, q;
    cin >> n >> q;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    ST st;
    st.build(a);
    while (q--) {
        int l, r;
        cin >> l >> r;
        auto v = st.qur(l, r);
        cout << v.mx - v.mn << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 不能直接用于求和；修改数组后重建。
- 从原int32_t定长320MiB表改为实际规模long long表；n很大仍须评估MLE。

个人资料来源：数据结构/ST表.cpp

数据结构 / 动态开点权值线段树

类型与用途：模板型；值域可重集合增删、区间次数、第k小/大；支持负值域。

复杂度与适用范围：操作O(1+log V)，空间O(1+D log V)，删除不回收节点。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 动态开点权值线段树：值域可重集合增删、区间次数、第k小/大；支持负值域。
// 操作O(1+log V)，空间O(1+D log V)，删除不回收节点。
struct WeightSegTree {
    struct info {
        int cnt = 0; // 出现次数；所有叶子的次数必须非负
        friend info operator+(const info &l, const info &r) {
            return {l.cnt + r.cnt};
        }
    };
    struct node {
        info val;
        int32_t lc = 0, rc = 0;
    };

    vector<node> seg;
    int root = 0;
    int L, R;

    WeightSegTree(int l = 1, int r = 1000000000) {
        init(l, r);
    }

    void init(int l, int r) {
        assert(l <= r);
        L = l, R = r;
        root = 0;
        seg.clear();
        seg.push_back({}); // seg[0] 表示空节点
    }

    int ls(int id) {
        return seg[id].lc;
    }
    int rs(int id) {
        return seg[id].rc;
    }

    int newnode() {
        assert(seg.size() < (size_t)INT32_MAX);
        seg.push_back({});
        return seg.size() - 1;
```

```
    }

    void update(int id) {
        seg[id].val = seg[ls(id)].val + seg[rs(id)].val;
    }

    // 单点修改：v 是 info 增量，{1} 插入，{-1} 删除
    int change(int id, int l, int r, int pos, const info &v) {
        if (!id) {
            assert(v.cnt > 0);
            id = newnode();
        }
        if (l == r) {
            assert((__int128)seg[id].val.cnt + v.cnt >= 0);
            assert((__int128)seg[id].val.cnt + v.cnt <= LLONG_MAX);
            seg[id].val = seg[id].val + v;
            return id;
        }
        int mid = l + ((__int128)r - l) / 2;
        if (pos <= mid) {
            int t = change(ls(id), l, mid, pos, v);
            seg[id].lc = t;
        } else {
            int t = change(rs(id), mid + 1, r, pos, v);
            seg[id].rc = t;
        }
        update(id);
        return id;
    }

    void change(int pos, const info &v) {
        assert(L <= pos && pos <= R && v.cnt != 0);
        assert((__int128)size() + v.cnt >= 0);
        assert((__int128)size() + v.cnt <= LLONG_MAX);
        root = change(root, L, R, pos, v);
    }

    // 查询值域 [ql, qr]，直接返回聚合信息 info
    info qur(int id, int l, int r, int ql, int qr) {
        if (!id || ql > r || qr < l)
            return {};
        if (ql <= l && r <= qr)
            return seg[id].val;
        int mid = l + ((__int128)r - l) / 2;
        return qur(ls(id), l, mid, ql, qr) + qur(rs(id), mid + 1, r, ql, qr);
    }

    info qur(int l, int r) {
        if (l > r)
            return {};
        return qur(root, L, R, l, r);
    }

    info all() {
        return seg[root].val;
    }

    // 线段树上二分：查询第 k 小（k 从 1 开始）
```

```
    int kth(int id, int l, int r, int k) {
        if (l == r)
            return l;
        int mid = l + ((__int128)r - l) / 2;
        int cnt = seg[ls(id)].val.cnt;
        if (k <= cnt)
            return kth(ls(id), l, mid, k);
        return kth(rs(id), mid + 1, r, k - cnt);
    }

    int kth(int k) {
        if (k <= 0 || k > seg[root].val.cnt)
            return -1;
        return kth(root, L, R, k);
    }

    int kth_max(int k) {
        if (k <= 0 || k > size())
            return -1;
        return kth(size() - k + 1);
    }

    int size() {
        return seg[root].val.cnt;
    }
};

void solve() {
    int q;
    cin >> q;
    WeightSegTree seg(0, 1000000000);
    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {
            int x;
            cin >> x;
            seg.change(x, {1});
        } else if (op == 2) {
            int x, c;
            cin >> x >> c;
            int del = min(c, seg.qur(x, x).cnt);
            if (del)
                seg.change(x, {-del}); // 没有该元素时不调用 change(x,{0})
        } else {
            // 原题保证此时非空；重复元素会占据多个排名。
            cout << seg.kth_max(1) - seg.kth(1) << '\n';
        }
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

init(L,R)：清空旧树

change(x,{c})：修改出现次数

qur(l,r).cnt：值域闭区间次数

kth(k)/kth_max(k)：1-based排名，非法返回-1

例题 1 ABC253C Max-Min Query

https://atcoder.jp/contests/abc253/tasks/abc253_c

题意：增删元素，查询最大值与最小值之差。

观察与建模：增删元素，查询最大值与最小值之差。 次数叶子可批量删除，最值是排名端点。

为何对应此算法：次数叶子可批量删除，最值是排名端点。

初始化、组织数据与调用：先查次数再change；kth(1)、kth_max(1)。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(Q log V)时间、O(D log V)空间。

复用模板：动态开点权值线段树（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int q;
    cin >> q;
    WeightSegTree seg(0, 1000000000);
    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {
            int x;
            cin >> x;
            seg.change(x, {1});
        } else if (op == 2) {
            int x, c;
            cin >> x >> c;
            int del = min(c, seg.qur(x, x).cnt);
            if (del)
                seg.change(x, {-del}); // 没有该元素时不调用 change(x,{0})
        } else {
            // 原题保证此时非空；重复元素会占据多个排名。
            cout << seg.kth_max(1) - seg.kth(1) << '\n';
        }
    }
}
```

验证：C++17组合程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P3369 普通平衡树

https://www.luogu.com.cn/problem/P3369

题意：可重集合的六种有序操作。

观察与建模：可重集合的六种有序操作。 次数前缀可转化为排名和严格前驱后继。

为何对应此算法：次数前缀可转化为排名和严格前驱后继。

初始化、组织数据与调用：change增删；qur计数；kth输出排名值。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(Q log V)时间、O(D log V)空间。

复用模板：动态开点权值线段树（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    const int L = -10000000, R = 10000000;
    WeightSegTree seg(L, R);
    while (n--) {
        int op, x;
        cin >> op >> x;
        if (op == 1)
            seg.change(x, {1});
        else if (op == 2)
            seg.change(x, {-1});
        else if (op == 3)
            cout << seg.qur(L, x - 1).cnt + 1 << '\n';
        else if (op == 4)
            cout << seg.kth(x) << '\n';
        else if (op == 5) {
            int cnt = seg.qur(L, x - 1).cnt;
            cout << seg.kth(cnt) << '\n';
        } else {
            int cnt = seg.qur(L, x).cnt;
            cout << seg.kth(cnt + 1) << '\n';
        }
        // 原题保证第k小、前驱、后继答案存在；删除须满足集合操作前提。
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 3 ABC241D Sequence Query

https://atcoder.jp/contests/abc241/tasks/abc241_d

题意：查询<=x的第k大或>=x的第k小。

观察与建模：查询<=x的第k大或>=x的第k小。 离散化1e18值域，将限制排名映射到全局排名。

为何对应此算法：离散化1e18值域，将限制排名映射到全局排名。

初始化、组织数据与调用：离线收集阈值；按原顺序change/qur/kth并映射回原值。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(Q log Q)时间、O(Q)空间。

复用模板：动态开点权值线段树（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int q;
    cin >> q;
    vector<array<int, 3>> ask(q);
    vector<int> a;
    for (auto &v : ask) {
        cin >> v[0] >> v[1];
        v[2] = 0;
        if (v[0] != 1)
```

```
            cin >> v[2];
        a.push_back(v[1]);
    }
    sort(a.begin(), a.end());
    a.erase(unique(a.begin(), a.end()), a.end());
    int m = a.size();
    WeightSegTree seg(1, m);
    for (auto [op, x, k] : ask) {
        int pos = lower_bound(a.begin(), a.end(), x) - a.begin() + 1;
        if (op == 1)
            seg.change(pos, {1});
        else if (op == 2) {
            int cnt = seg.qur(1, pos).cnt;
            if (cnt < k)
                cout << -1 << '\n';
            else
                cout << a[seg.kth(cnt - k + 1) - 1] << '\n';
        } else {
            int cnt = seg.qur(1, pos - 1).cnt;
            if (seg.size() - cnt < k)
                cout << -1 << '\n';
            else
                cout << a[seg.kth(cnt + k) - 1] << '\n';
        }
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 删除前保证足够数量。
- -1可能是合法值，先验证排名。
- 中点使用__int128；节点编号32位，计数64位。

个人资料来源：数据结构/线段树/权值线段树（动态开点基础版）.cpp

数据结构 / 势能线段树

类型与用途：模板型；非负数组区间开方、区间取模、单点赋值和区间和；用mx跳过不变节点。

复杂度与适用范围：开方每值只改变O(log log A)次；取模有效改变使值至少减半，累计O((n+m)log A log n)；查询O(log n)，空间O(n)。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 势能线段树：非负数组区间开方、区间取模、单点赋值和区间和；用mx跳过不变节点。
// 开方每值只改变O(log log A)次；取模有效改变使值至少减半，累计O((n+m)log A log n)；查询O(log
//  n)，空间O(n)。
struct SegTree {
    struct info {
        int sum = 0, mx = 0;
        friend info operator+(const info &l, const info &r) {
            return {l.sum + r.sum, max(l.mx, r.mx)};
        }
```

```
    };
    struct node {
        info val;
    };
    int n;
    vector<node> seg;
    int ls(int id) {
        return id << 1;
    }
    int rs(int id) {
        return id << 1 | 1;
    }
    void update(int id) {
        seg[id].val = seg[ls(id)].val + seg[rs(id)].val;
    }
    void build(int id, int l, int r, const vector<int> &a) {
        if (l == r) {
            seg[id].val = {a[l], a[l]};
            return;
        }
        int mid = (l + r) >> 1;
        build(ls(id), l, mid, a);
        build(rs(id), mid + 1, r, a);
        update(id);
    }
    void build(const vector<int> &a) {
        n = a.size() - 1;
        assert(n > 0);
        seg.assign(4 * n + 4, {});
        build(1, 1, n, a);
    }
    int isqrt(int x) {
        int y = sqrtl((long double)x);
        while ((__int128)y * y > x)
            --y;
        while ((__int128)(y + 1) * (y + 1) <= x)
            ++y;
        return y;
    }
    void update_sqrt(int id, int l, int r, int ql, int qr) {
        if (qr < l || r < ql || seg[id].val.mx <= 1)
            return;
        if (l == r) {
            int v = isqrt(seg[id].val.mx);
            seg[id].val = {v, v};
            return;
        }
        int mid = (l + r) >> 1;
        update_sqrt(ls(id), l, mid, ql, qr);
        update_sqrt(rs(id), mid + 1, r, ql, qr);
        update(id);
    }
    void modify_mod(int id, int l, int r, int ql, int qr, int x) {
        if (qr < l || r < ql || seg[id].val.mx < x)
            return;
        if (l == r) {
            int v = seg[id].val.mx % x;
            seg[id].val = {v, v};
            return;
        }
```

```
        int mid = (l + r) >> 1;
        modify_mod(ls(id), l, mid, ql, qr, x);
        modify_mod(rs(id), mid + 1, r, ql, qr, x);
        update(id);
    }
    void change(int id, int l, int r, int p, int x) {
        if (l == r) {
            seg[id].val = {x, x};
            return;
        }
        int mid = (l + r) >> 1;
        if (p <= mid)
            change(ls(id), l, mid, p, x);
        else
            change(rs(id), mid + 1, r, p, x);
        update(id);
    }
    info qur(int id, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr)
            return seg[id].val;
        int mid = (l + r) >> 1;
        if (qr <= mid)
            return qur(ls(id), l, mid, ql, qr);
        if (ql > mid)
            return qur(rs(id), mid + 1, r, ql, qr);
        return qur(ls(id), l, mid, ql, qr) + qur(rs(id), mid + 1, r, ql, qr);
    }
    void update_sqrt(int l, int r) {
        update_sqrt(1, 1, n, l, r);
    }
    void modify_mod(int l, int r, int x) {
        assert(x > 0);
        modify_mod(1, 1, n, l, r, x);
    }
    void change(int p, int x) {
        assert(x >= 0);
        change(1, 1, n, p, x);
    }
    info qur(int l, int r) {
        return qur(1, 1, n, l, r);
    }
};

void solve() {
    int n;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    SegTree seg;
    seg.build(a);
    int m;
    cin >> m;
    while (m--) {
        int op, l, r;
        cin >> op >> l >> r;
        if (l > r)
            swap(l, r);
        if (!op)
            seg.update_sqrt(l, r);
```

```
        else
            cout << seg.qur(l, r).sum << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

update_sqrt(l,r)：向下取整开方

modify_mod(l,r,x)：x>0，区间取模

change(p,x)：非负赋值

qur(l,r).sum：区间和

例题 1 P4145 花神游历各国

https://www.luogu.com.cn/problem/P4145

题意：区间开方或区间和。

观察与建模：区间开方或区间和。 max<=1的节点所有值已稳定，可停止递归。

为何对应此算法：max<=1的节点所有值已稳定，可停止递归。

初始化、组织数据与调用：op0调用update_sqrt，op1调用qur；先交换反向端点。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：累计O(n log log A log n+m log n)时间、O(n)空间。

复用模板：势能线段树（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    SegTree seg;
    seg.build(a);
    int m;
    cin >> m;
    while (m--) {
        int op, l, r;
        cin >> op >> l >> r;
        if (l > r)
            swap(l, r);
        if (!op)
            seg.update_sqrt(l, r);
        else
            cout << seg.qur(l, r).sum << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF438D The Child and Sequence

https://codeforces.com/problemset/problem/438/D

题意：区间和、区间取模、单点赋值。

观察与建模：区间和、区间取模、单点赋值。 a%x只有在a>=x时改变，并且改变后小于原值一半。

为何对应此算法：a%x只有在a>=x时改变，并且改变后小于原值一半。

初始化、组织数据与调用：modify_mod用区间max剪枝；赋值change恢复势能。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：摊还O((n+m)log A log n)时间、O(n)空间。

复用模板：势能线段树（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    SegTree seg;
    seg.build(a);
    while (m--) {
        int op, l, r, x;
        cin >> op;
        if (op == 3) {
            cin >> l >> x;
            seg.change(l, x);
        } else {
            cin >> l >> r;
            if (op == 1)
                cout << seg.qur(l, r).sum << '\n';
            else {
                cin >> x;
                seg.modify_mod(l, r, x);
            }
        }
    }
}
```

验证：C++17组合程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 用整数平方比较修正sqrt浮点误差。
- 不能将单次修改声称总是O(log n)；需势能摊还分析。

个人资料来源：数据结构/线段树/势能线段树.cpp

数据结构 / 区间加线段树

类型与用途：模板型；闭区间加、区间和、单点赋值；1-based。

复杂度与适用范围：建树O(n)，修改/查询O(log n)，空间O(n)。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 区间加线段树：闭区间加、区间和、单点赋值；1-based。
// 建树O(n)，修改/查询O(log n)，空间O(n)。
struct SegTree {
    struct info {
        int sum = 0, len = 0;
        friend info operator+(const info &l, const info &r) {
            return {l.sum + r.sum, l.len + r.len};
        }
    };
    struct tag {
        int add = 0;
    };
    struct node {
        info val;
        tag t;
    };
    int n = 0;
    vector<node> seg;
    SegTree(int n_ = 0) {
        init(n_);
    }
    void init(int n_) {
        n = n_;
        seg.assign(4 * n + 4, {});
    }
    int ls(int id) {
        return id << 1;
    }
    int rs(int id) {
        return id << 1 | 1;
    }
    void update(int id) {
        seg[id].val = seg[ls(id)].val + seg[rs(id)].val;
    }
    void settag(int id, tag t) {
        seg[id].val.sum += t.add * seg[id].val.len;
        seg[id].t.add += t.add;
    }
    void pushdown(int id) {
        if (seg[id].t.add) {
            settag(ls(id), seg[id].t);
            settag(rs(id), seg[id].t);
            seg[id].t.add = 0;
        }
    }
```

```
    void build(int id, int l, int r, const vector<int> &a) {
        if (l == r) {
            seg[id].val = {a[l], 1};
            return;
        }
        int mid = (l + r) >> 1;
        build(ls(id), l, mid, a);
        build(rs(id), mid + 1, r, a);
        update(id);
    }
    void modify(int id, int l, int r, int ql, int qr, tag t) {
        if (ql <= l && r <= qr) {
            settag(id, t);
            return;
        }
        pushdown(id);
        int mid = (l + r) >> 1;
        if (ql <= mid)
            modify(ls(id), l, mid, ql, qr, t);
        if (qr > mid)
            modify(rs(id), mid + 1, r, ql, qr, t);
        update(id);
    }
    int qur(int id, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr)
            return seg[id].val.sum;
        pushdown(id);
        int mid = (l + r) >> 1, ans = 0;
        if (ql <= mid)
            ans += qur(ls(id), l, mid, ql, qr);
        if (qr > mid)
            ans += qur(rs(id), mid + 1, r, ql, qr);
        return ans;
    }
    void change(int id, int l, int r, int pos, int val) {
        if (l == r) {
            seg[id].val.sum = val;
            seg[id].t = {};
            return;
        }
        pushdown(id);
        int mid = (l + r) >> 1;
        if (pos <= mid)
            change(ls(id), l, mid, pos, val);
        else
            change(rs(id), mid + 1, r, pos, val);
        update(id);
    }
    void build(const vector<int> &a) {
        init(a.size() - 1);
        assert(n > 0);
        build(1, 1, n, a);
    }
    void modify(int l, int r, tag t) {
        assert(1 <= l && l <= r && r <= n);
        modify(1, 1, n, l, r, t);
    }
    void change(int pos, int val) {
        assert(1 <= pos && pos <= n);
        change(1, 1, n, pos, val);
```

```
    }
    int qur(int l, int r) {
        if (l > r)
            return 0;
        assert(1 <= l && r <= n);
        return qur(1, 1, n, l, r);
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    SegTree seg;
    seg.build(a);
    while (m--) {
        int op, l, r, k;
        cin >> op >> l >> r;
        if (op == 1) {
            cin >> k;
            seg.modify(l, r, {k});
        } else
            cout << seg.qur(l, r) << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

modify(l,r,{x})：区间加x

change(pos,val)：赋值前下放标记

qur(l,r)：区间和

例题 1 P3372 线段树1

https://www.luogu.com.cn/problem/P3372

题意：区间加和区间求和。

观察与建模：区间加和区间求和。 标记可累加，区间和的变化等于增量乘长度。

为何对应此算法：标记可累加，区间和的变化等于增量乘长度。

初始化、组织数据与调用：build后将1操作映射modify，2操作映射qur。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O((n+m)log n)时间、O(n)空间。

复用模板：区间加线段树（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    SegTree seg;
    seg.build(a);
    while (m--) {
        int op, l, r, k;
        cin >> op >> l >> r;
        if (op == 1) {
            cin >> k;
            seg.modify(l, r, {k});
        } else
            cout << seg.qur(l, r) << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P3374 树状数组1

https://www.luogu.com.cn/problem/P3374

题意：单点增加，查询区间和。

观察与建模：单点增加，查询区间和。 单点是长度1的区间修改，直接复用懒标记板子。

为何对应此算法：单点是长度1的区间修改，直接复用懒标记板子。

初始化、组织数据与调用：modify(x,x,{k})；qur(x,y)。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O((n+m)log n)时间、O(n)空间。

复用模板：区间加线段树（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    SegTree seg;
    seg.build(a);
    while (m--) {
        int op, x, y;
        cin >> op >> x >> y;
        if (op == 1)
            seg.modify(x, x, {y});
        else
            cout << seg.qur(x, y) << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- t.add*len与总和不得溢出long long。
- 进入部分区间前pushdown，回溯update。

个人资料来源：数据结构/线段树/区间修改区间查询.cpp

数据结构 / 对顶堆与中位数

类型与用途：模板型；动态插入删除，查询下中位数与到中位数的绝对值和；允许重复。

复杂度与适用范围：增删O(log n)，mid/cost为O(1)，空间O(n)。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 对顶堆与中位数：动态插入删除，查询下中位数与到中位数的绝对值和；允许重复。
// 增删O(log n)，mid/cost为O(1)，空间O(n)。
struct Mid {
    multiset<int> L, R;
    int sumL = 0, sumR = 0;

    void init() {
        L.clear();
        R.clear();
        sumL = sumR = 0;
    }

    int size() const {
        return L.size() + R.size();
    }
    bool empty() const {
        return L.empty() && R.empty();
    }

    // 调整两个集合大小。插入时按 L 的最大值分类，保证大小调整后顺序仍正确。
    void balance() {
        while (L.size() > R.size() + 1) {
            auto it = prev(L.end());
            int x = *it;
            L.erase(it);
            sumL -= x;
            R.insert(x);
            sumR += x;
        }
        while (L.size() < R.size()) {
            auto it = R.begin();
            int x = *it;
            R.erase(it);
            sumR -= x;
            L.insert(x);
            sumL += x;
        }
    }

    // 插入一个 x，可有重复元素。
```

```
    void add(int x) {
        if (L.empty() || x <= *L.rbegin()) {
            L.insert(x);
            sumL += x;
        } else {
            R.insert(x);
            sumR += x;
        }
        balance();
    }

    // 删除一个值为 x 的元素（只删一次）；要求 x 已存在。
    void del(int x) {
        auto it = L.find(x);
        if (it != L.end()) {
            L.erase(it);
            sumL -= x;
        } else {
            it = R.find(x);
            assert(it != R.end());
            R.erase(it);
            sumR -= x;
        }
        balance();
    }

    // 当前下中位数；要求非空。
    int mid() const {
        assert(!L.empty());
        return *L.rbegin();
    }

    // min_x sum |a_i - x|，最优 x 可以取中位数。空集返回 0。
    int cost() const {
        if (empty())
            return 0;
        int x = mid();
        return x * (int)L.size() - sumL + sumR - x * (int)R.size();
    }
};

void solve() {
    int q;
    cin >> q;
    Mid st;
    int bsum = 0;
    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {
            int a, b;
            cin >> a >> b;
            st.add(a);
            bsum += b;
        } else
            cout << st.mid() << ' ' << st.cost() + bsum << '\n';
    }
}

```

```
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

add(x)：加入一个值

del(x)：删除一个已存在的值

mid()：非空集合下中位数

cost()：最小绝对偏差和，空集为0

例题 1 ABC127F Absolute Minima

https://atcoder.jp/contests/abc127/tasks/abc127_f

题意：不断加入|x-a|+b，查询使函数最小的最小x及最小值。

观察与建模：不断加入|x-a|+b，查询使函数最小的最小x及最小值。 最小x是下中位数；常数b单独累加。

为何对应此算法：最小x是下中位数；常数b单独累加。

初始化、组织数据与调用：add(a)，查询mid()和cost()+常数和。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(q log q)时间、O(q)空间。

复用模板：对顶堆与中位数（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int q;
    cin >> q;
    Mid st;
    int bsum = 0;
    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {
            int a, b;
            cin >> a >> b;
            st.add(a);
            bsum += b;
        } else
            cout << st.mid() << ' ' << st.cost() + bsum << '\n';
    }
}
```

验证：C++17组合程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P1168 中位数

https://www.luogu.com.cn/problem/P1168

题意：依次读入整数，对每个奇数长度前缀输出中位数。

观察与建模：依次读入整数，对每个奇数长度前缀输出中位数。 插入后保持两个集合的大小与顺序，中位数直接取L最大值。

为何对应此算法：插入后保持两个集合的大小与顺序，中位数直接取L最大值。

初始化、组织数据与调用：逐个add，奇数次插入后输出mid()。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n log n)时间、O(n)空间。

复用模板：对顶堆与中位数（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    Mid st;
    for (int i = 1, x; i <= n; i++) {
        cin >> x;
        st.add(x);
        if (i % 2)
            cout << st.mid() << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 3 LeetCode480 滑动窗口中位数

https://leetcode.cn/problems/sliding-window-median/description/

题意：返回每个长度k窗口的中位数；偶数窗口取中间两数平均。

观察与建模：返回每个长度k窗口的中位数；偶数窗口取中间两数平均。 滑窗加入新值、删除旧值，multiset可删任意一个重复值，无需额外惰性删除；偶数窗口组合L最大与R最小。

为何对应此算法：滑窗加入新值、删除旧值，multiset可删任意一个重复值，无需额外惰性删除；偶数窗口组合L最大与R最小。

初始化、组织数据与调用：原平台提交Solution::medianSlidingWindow，签名使用int32_t避免宏改变平台int类型；solve是本地n,k和数组适配器。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n log k)时间/O(k)维护空间，输出另需O(n)。

复用模板：对顶堆与中位数（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
struct Solution {
    vector<double> medianSlidingWindow(vector<int32_t> &nums, int32_t k) {
        Mid t;
        vector<double> ans;
        for (int i = 0; i < (int)nums.size(); i++) {
            t.add(nums[i]);
            if (i >= k)
                t.del(nums[i - k]);
            if (i + 1 >= k) {
                double z = t.mid();
                if (k % 2 == 0)
                    z = (z + (double)*t.R.begin()) / 2;
                ans.push_back(z);
            }
```

```
        }
        return ans;
    }
};
void solve() {
    int n;
    int32_t k;
    cin >> n >> k;
    vector<int32_t> a(n);
    for (auto &x : a)
        cin >> x;
    Solution s;
    auto ans = s.medianSlidingWindow(a, k);
    for (double x : ans)
        cout << setprecision(10) << x << ' ';
    cout << '\n';
}
```

验证：C++17组合程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- L大小等于R或多1，且max(L)<=min(R)。

- 和与代价不得溢出；删除只删除一个重复值。

个人资料来源：数据结构/对顶堆（multiset维护中位数）.cpp

数据结构 / 并查集

类型与用途：模板型；动态合并连通块，查询连通性与大小；编号0..n均可用。

复杂度与适用范围：初始化O(n)，按大小合并与路径压缩均摊O(alpha(n))，空间O(n)。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 并查集：动态合并连通块，查询连通性与大小；编号0..n均可用。
// 初始化O(n)，按大小合并与路径压缩均摊O(alpha(n))，空间O(n)。
struct DSU {
    vector<int> fid, sz;
    DSU(int n = 0) {
        init(n);
    }
    void init(int n) {
        fid.resize(n + 1);
        iota(fid.begin(), fid.end(), 0);
        sz.assign(n + 1, 1);
    }
    int root(int x) {
        return fid[x] == x ? x : fid[x] = root(fid[x]);
    }
    bool merge(int x, int y) {
        x = root(x);
        y = root(y);
        if (x == y)
            return false;
        if (sz[x] > sz[y])
            swap(x, y);
```

```
        fid[x] = y;
        sz[y] += sz[x];
        return true;
    }
    bool same(int x, int y) {
        return root(x) == root(y);
    }
    int size(int x) {
        return sz[root(x)];
    }
};

void solve() {
    int n, q;
    cin >> n >> q;
    DSU dsu(n);
    while (q--) {
        int op, u, v;
        cin >> op >> u >> v;
        if (!op)
            dsu.merge(u, v);
        else
            cout << dsu.same(u, v) << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

root(x)：沿用原名，压缩路径

merge(x,y)：返回是否成功合并

same(x,y)：判断同一连通块

size(x)：连通块大小

例题 1 practice2_a Disjoint Set Union

https://atcoder.jp/contests/practice2/tasks/practice2_a

题意：加无向边或询问两点是否连通。

观察与建模：加无向边或询问两点是否连通。 加边只会合并连通块，适用普通DSU。

为何对应此算法：加边只会合并连通块，适用普通DSU。

初始化、组织数据与调用：0操作merge，1操作same；题目点号0-based。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O((n+q)alpha(n))时间、O(n)空间。

复用模板：并查集（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, q;
    cin >> n >> q;
    DSU dsu(n);
    while (q--) {
        int op, u, v;
        cin >> op >> u >> v;
        if (!op)
            dsu.merge(u, v);
        else
            cout << dsu.same(u, v) << '\n';
    }
}
```

验证：C++17组合程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 ABC120D Decayed Bridges

https://atcoder.jp/contests/abc120/tasks/abc120_d

题意：按顺序删桥，输出每次删除后的不连通点对数。

观察与建模：按顺序删桥，输出每次删除后的不连通点对数。 逆序变成加边；合并大小a,b的块减少a*b对。

为何对应此算法：逆序变成加边；合并大小a,b的块减少a*b对。

初始化、组织数据与调用：从空图开始，先记答案，读取size后merge。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O((n+m)alpha(n))时间、O(n+m)空间。

复用模板：并查集（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<pair<int, int>> e(m);
    for (auto &[u, v] : e)
        cin >> u >> v;
    DSU dsu(n);
    vector<int> ans(m);
    int cur = n * (n - 1) / 2;
    for (int i = m - 1; i >= 0; i--) {
        ans[i] = cur;
        auto [u, v] = e[i];
        if (!dsu.same(u, v)) {
            cur -= dsu.size(u) * dsu.size(v);
            dsu.merge(u, v);
        }
    }
    for (int x : ans)
        cout << x << '\n';
}
```

验证：C++17组合程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 不支持删除边；删除类题优先考虑倒序离线。

- 若需要回滚，应另用不压缩路径的变体。

个人资料来源：数据结构/并查集/板子.cpp；数据结构/并查集/并查集.md

数据结构 / 普通线段树

类型与用途：模板型；单点赋值、区间最小值；数组1-based，查询为闭区间。

复杂度与适用范围：建树O(n)，修改/查询O(log n)，空间O(n)。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 普通线段树：单点赋值、区间最小值；数组1-based，查询为闭区间。
// 建树O(n)，修改/查询O(log n)，空间O(n)。
struct SegTree {
    struct info {
        int mn;
        friend info operator+(const info &l, const info &r) {
            return {min(l.mn, r.mn)};
        }
    };
    struct node {
        info val;
    };
    int n = 0;
    vector<node> seg;
    SegTree(int n_ = 0) {
        init(n_);
    }
    void init(int n_) {
        n = n_;
        seg.assign(4 * n + 4, {});
    }
    int ls(int id) {
        return id << 1;
    }
    int rs(int id) {
        return id << 1 | 1;
    }
    void update(int id) {
        seg[id].val = seg[ls(id)].val + seg[rs(id)].val;
    }
    void build(int id, int l, int r, const vector<int> &a) {
        if (l == r) {
            seg[id].val = {a[l]};
            return;
        }
        int mid = (l + r) >> 1;
        build(ls(id), l, mid, a);
        build(rs(id), mid + 1, r, a);
        update(id);
    }
    void change(int id, int l, int r, int pos, int val) {
        if (l == r) {
            seg[id].val = {val};
            return;
        }
```

```
        int mid = (l + r) >> 1;
        if (pos <= mid)
            change(ls(id), l, mid, pos, val);
        else
            change(rs(id), mid + 1, r, pos, val);
        update(id);
    }
    info qur(int id, int l, int r, int ql, int qr) {
        if (ql <= l && r <= qr)
            return seg[id].val;
        int mid = (l + r) >> 1;
        if (qr <= mid)
            return qur(ls(id), l, mid, ql, qr);
        if (ql > mid)
            return qur(rs(id), mid + 1, r, ql, qr);
        return qur(ls(id), l, mid, ql, qr) + qur(rs(id), mid + 1, r, ql, qr);
    }
    void build(const vector<int> &a) {
        init(a.size() - 1);
        assert(n > 0);
        build(1, 1, n, a);
    }
    void change(int pos, int val) {
        assert(1 <= pos && pos <= n);
        change(1, 1, n, pos, val);
    }
    info qur(int l, int r) {
        assert(1 <= l && l <= r && r <= n);
        return qur(1, 1, n, l, r);
    }
};

void solve() {
    int n, q;
    cin >> n >> q;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    SegTree seg;
    seg.build(a);
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << seg.qur(l, r).mn << (q ? ' ' : '\n');
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

build(a)：a[1..n]建树

change(pos,val)：单点赋值

qur(l,r).mn：闭区间最小值

例题 1 P1816 忠诚

https://www.luogu.com.cn/problem/P1816

题意：询问多段账目中的最小金额。

观察与建模：询问多段账目中的最小金额。 区间最小值满足结合律，建树后直接查询。

为何对应此算法：区间最小值满足结合律，建树后直接查询。

初始化、组织数据与调用：build(a)，每个区间qur(l,r).mn。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+q log n)时间、O(n)空间。

复用模板：普通线段树（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, q;
    cin >> n >> q;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    SegTree seg;
    seg.build(a);
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << seg.qur(l, r).mn << (q ? ' ' : '\n');
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P1198 最大数

https://www.luogu.com.cn/problem/P1198

题意：在线追加数，查询末尾L个数的最大值；追加依赖上次答案。

观察与建模：在线追加数，查询末尾L个数的最大值；追加依赖上次答案。 把追加视为预留数组上的单点赋值，查询变为后缀区间最大值。

为何对应此算法：把追加视为预留数组上的单点赋值，查询变为后缀区间最大值。

初始化、组织数据与调用：预留M个位置build；change(++len,x)，qur(len-L+1,len)。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

特殊修改：只将info的min合并改为max；成员mn沿用原名表示最大值。

```
// 合并信息的题目变体：
// 原
return {min(l.mn, r.mn)};
// 新
return {max(l.mn, r.mn)};
```

本题复杂度：O(M log M)时间、O(M)空间。

复用模板：普通线段树（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int m, d;
    cin >> m >> d;
    vector<int> a(m + 1, LLONG_MIN);
    SegTree seg;
    seg.build(a);
    int len = 0, last = 0;
    while (m--) {
        char op;
        int x;
        cin >> op >> x;
        if (op == 'A') {
            int v = ((__int128)x + last) % d;
            if (v < 0)
                v += d;
            seg.change(++len, v);
        } else {
            last = seg.qur(len - x + 1, len).mn;
            cout << last << '\n';
        }
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- n必须为正；不支持懒标记。
- 自定义info必须满足结合律；保留合并左右顺序。

个人资料来源：数据结构/线段树/区间查询单点修改.cpp

数据结构 / 权值线段树合并与分裂

类型与用途：模板型；同值域多棵可重集合树；破坏性合并、分裂出最大的k个。

复杂度与适用范围：插入/排名/分裂O(log V)，合并O(实际同时非空访问节点数)；空间O(累计新建节点)。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 权值线段树合并与分裂：同值域多棵可重集合树；破坏性合并、分裂出最大的k个。
// 插入/排名/分裂O(log V)，合并O(实际同时非空访问节点数)；空间O(累计新建节点)。
struct WeightSegForest {
    struct info {
        int cnt = 0; // 当前值域内的元素个数（包含重复）
    };

    friend info operator+(const info &l, const info &r) {
        return {l.cnt + r.cnt};
    }

    struct node {
        info val;
        int32_t lc = 0, rc = 0; // 动态开点，不能用 id<<1 找儿子
```

```
};

vector<node> seg = vector<node>(1); // seg[0] 是空节点，cnt = 0

inline int ls(int id) {
    return seg[id].lc;
}
inline int rs(int id) {
    return seg[id].rc;
}

int newnode() {
    seg.push_back({});
    return (int)seg.size() - 1;
}

void init(int reserve_nodes = 0) { // 多测时重置；reserve_nodes 是预估节点数
    seg.clear();
    if (reserve_nodes > 0)
        seg.reserve(reserve_nodes + 1);
    seg.push_back({});
}

void update(int id) {
    seg[id].val = seg[ls(id)].val + seg[rs(id)].val;
}

// 单点增减出现次数；pos 在 [l,r]，删除前须保证数量足够
// 返回可能新建的根，必须接收返回值
int change(int id, int l, int r, int pos, int v = 1) {
    assert(l <= pos && pos <= r);
    assert(v != 0);
    if (!id) {
        assert(v > 0);
        id = newnode();
    }
    if (l == r) {
        assert(seg[id].val.cnt + v >= 0);
        seg[id].val.cnt += v;
        return id;
    }
    int mid = l + ((__int128)r - l) / 2;
    if (pos <= mid) {
        seg[id].lc = change(ls(id), l, mid, pos, v);
    } else {
        seg[id].rc = change(rs(id), mid + 1, r, pos, v);
    }
    update(id);
    return id;
}

// 按值域查询元素个数，允许重数
int qur(int id, int l, int r, int ql, int qr) {
    if (!id || qr < l || r < ql)
        return 0;
    if (ql <= l && r <= qr)
        return seg[id].val.cnt;
    int mid = l + ((__int128)r - l) / 2;
    return qur(ls(id), l, mid, ql, qr) + qur(rs(id), mid + 1, r, ql, qr);
}
```

```

    // 查第 k 大（1-indexed），返回值域上的数；要求 1 <= k <= cnt[root]
    int kth(int id, int l, int r, int k) {
        assert(id && 1 <= k && k <= seg[id].val.cnt);
        if (l == r)
            return l;
        int mid = l + ((__int128)r - l) / 2;
        int cnt = seg[rs(id)].val.cnt;
        if (k <= cnt)
            return kth(rs(id), mid + 1, r, k);
        return kth(ls(id), l, mid, k - cnt);
    }

    // 将 y 并入 x。合并是破坏性的，旧的 x/y 不再作为独立的树使用。
    // 必须保证两棵树表示同一值域 [l,r]。
    int merge(int x, int y, int l, int r) {
        if (!x || !y)
            return x + y;
        if (l == r) {
            seg[x].val.cnt += seg[y].val.cnt;
            return x;
        }
        int mid = l + ((__int128)r - l) / 2;
        seg[x].lc = merge(ls(x), ls(y), l, mid);
        seg[x].rc = merge(rs(x), rs(y), mid + 1, r);
        update(x);
        return x;
    }

    // 分裂出最大的 k 个元素，返回 {剩余树的根，取出树的根}
    // 要求 0 <= k <= cnt[id]；相同值会在叶子按出现次数拆开。
    // 原根 id 在操作后不能再作为“未分裂前的树”使用。
    pair<int, int> split(int id, int l, int r, int k) {
        assert(0 <= k && k <= seg[id].val.cnt);
        if (!id || k == 0)
            return {id, 0};
        if (k == seg[id].val.cnt)
            return {0, id};

        int x = newnode(); // 分裂出来的新树在当前值域的根
        if (l == r) {
            seg[x].val.cnt = k;
            seg[id].val.cnt -= k;
            return {id, x};
        }

        int mid = l + ((__int128)r - l) / 2;
        int cnt = seg[rs(id)].val.cnt;
        if (k <= cnt) {
            // 较大的 k 个全部落在右子树
            auto [a, b] = split(rs(id), mid + 1, r, k);
            seg[id].rc = a;
            seg[x].rc = b;
        } else {
            // 整个右子树取走；剩余 k-cnt 个从左边拿
            seg[x].rc = rs(id);
            seg[id].rc = 0;
            auto [a, b] = split(ls(id), l, mid, k - cnt);
            seg[id].lc = a;
            seg[x].lc = b;
```

```
        }
        update(id);
        update(x);
        return {id, x};
    }

    int L, R;
    WeightSegForest(int l = 1, int r = 1000000000) {
        L = l;
        R = r;
        init();
    }
    int change(int root, int pos, int v) {
        return change(root, L, R, pos, v);
    }
    int qur(int root, int l, int r) {
        return l > r ? 0 : qur(root, L, R, l, r);
    }
    int size(int root) {
        return seg[root].val.cnt;
    }
    int kth(int root, int k) {
        return k <= 0 || k > size(root) ? -1 : kth(root, L, R, k);
    }
    int merge(int x, int y) {
        return merge(x, y, L, R);
    }
    pair<int, int> split(int root, int k) {
        return split(root, L, R, k);
    }
};

struct DSU {
    vector<int> fid, sz;
    DSU(int n = 0) {
        init(n);
    }
    void init(int n) {
        fid.resize(n + 1);
        iota(fid.begin(), fid.end(), 0);
        sz.assign(n + 1, 1);
    }
    int root(int x) {
        return fid[x] == x ? x : fid[x] = root(fid[x]);
    }
    bool merge(int x, int y) {
        x = root(x);
        y = root(y);
        if (x == y)
            return false;
        if (sz[x] > sz[y])
            swap(x, y);
        fid[x] = y;
        sz[y] += sz[x];
        return true;
    }
    bool same(int x, int y) {
        return root(x) == root(y);
    }
```

```
    int size(int x) {
        return sz[root(x)];
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    WeightSegForest seg(1, n);
    vector<int> root(m + 2);
    int tot = 1;
    for (int i = 1, c; i <= n; i++) {
        cin >> c;
        if (c)
            root[1] = seg.change(root[1], i, c);
    }
    while (m--) {
        int op, p;
        cin >> op >> p;
        if (op == 0) {
            int x, y;
            cin >> x >> y;
            int c = seg.qur(root[p], 1, x - 1);
            auto [low, high] = seg.split(root[p], seg.size(root[p]) - c);
            int hi = seg.qur(high, y + 1, n);
            auto [mid, upper] = seg.split(high, hi);
            root[p] = seg.merge(low, upper);
            root[++tot] = mid;
        } else if (op == 1) {
            int t;
            cin >> t;
            root[p] = seg.merge(root[p], root[t]);
            root[t] = 0;
        } else if (op == 2) {
            int x, q;
            cin >> x >> q;
            if (x)
                root[p] = seg.change(root[p], q, x);
        } else if (op == 3) {
            int x, y;
            cin >> x >> y;
            cout << seg.qur(root[p], x, y) << '\n';
        } else {
            int k;
            cin >> k;
            if (k <= 0 || k > seg.size(root[p]))
                cout << -1 << '\n';
            else
                cout << seg.kth(root[p], seg.size(root[p]) - k + 1) << '\n';
        }
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
```

```
    return 0;
}
```

change(root,pos,c)：返回新根，必须接收

merge(x,y)：返回合并根，旧y不能继续独立使用

split(root,k)：返回{剩余,最大的k个}

kth(root,k)：原接口第k大，非法返回-1

例题 1 P5494 线段树分裂

https://www.luogu.com.cn/problem/P5494

题意：维护多个集合，按值域拆分、合并、加频率和排名查询。

观察与建模：维护多个集合，按值域拆分、合并、加频率和排名查询。 按<x和>y的次数切排名，将中间值域移到新根。

为何对应此算法：按<x和>y的次数切排名，将中间值域移到新根。

初始化、组织数据与调用：两次split；merge两侧；所有变根操作接收返回值。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：分裂/更新/查询O(log n)，合并按节点交叠计；空间O((n+m)log n)。

复用模板：权值线段树合并与分裂（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    WeightSegForest seg(1, n);
    vector<int> root(m + 2);
    int tot = 1;
    for (int i = 1, c; i <= n; i++) {
        cin >> c;
        if (c)
            root[1] = seg.change(root[1], i, c);
    }
    while (m--) {
        int op, p;
        cin >> op >> p;
        if (op == 0) {
            int x, y;
            cin >> x >> y;
            int c = seg.qur(root[p], 1, x - 1);
            auto [low, high] = seg.split(root[p], seg.size(root[p]) - c);
            int hi = seg.qur(high, y + 1, n);
            auto [mid, upper] = seg.split(high, hi);
            root[p] = seg.merge(low, upper);
            root[++tot] = mid;
        } else if (op == 1) {
            int t;
            cin >> t;
            root[p] = seg.merge(root[p], root[t]);
            root[t] = 0;
        } else if (op == 2) {
            int x, q;
            cin >> x >> q;
            if (x)
                root[p] = seg.change(root[p], q, x);
```

```
        } else if (op == 3) {
            int x, y;
            cin >> x >> y;
            cout << seg.qur(root[p], x, y) << '\n';
        } else {
            int k;
            cin >> k;
            if (k <= 0 || k > seg.size(root[p]))
                cout << -1 << '\n';
            else
                cout << seg.kth(root[p], seg.size(root[p]) - k + 1) << '\n';
        }
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P3224 永无乡

https://www.luogu.com.cn/problem/P3224

题意：合并岛屿，查询连通块内重要度第k小的岛编号。

观察与建模：合并岛屿，查询连通块内重要度第k小的岛编号。 DSU维护块代表；每块一棵权值树，合并时破坏性merge。

为何对应此算法：DSU维护块代表；每块一棵权值树，合并时破坏性merge。

初始化、组织数据与调用：建立排名到编号映射；DSU合并前取两根，合并后挂到新代表。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：总合并访问量按节点计，查询O(log n)；空间O(n log n)。

复用模板：权值线段树合并与分裂（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    DSU dsu(n);
    WeightSegForest seg(1, n);
    vector<int> root(n + 1), id(n + 1);
    for (int i = 1, x; i <= n; i++) {
        cin >> x;
        id[x] = i;
        root[i] = seg.change(0, x, 1);
    }
    auto join = [&](int x, int y) {
        x = dsu.root(x);
        y = dsu.root(y);
        if (x == y)
            return;
        int r = seg.merge(root[x], root[y]);
        root[x] = root[y] = 0;
        dsu.merge(x, y);
        root[dsu.root(x)] = r;
    };
    while (m--) {
        int u, v;
        cin >> u >> v;
```

```
        join(u, v);
    }
    int q;
    cin >> q;
    while (q--) {
        char op;
        int x, y;
        cin >> op >> x >> y;
        if (op == 'B')
            join(x, y);
        else {
            int r = root[dsu.root(x)];
            if (y > seg.size(r))
                cout << -1 << '\n';
            else
                cout << id[seg.kth(r, seg.size(r) - y + 1)] << '\n';
        }
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 根之间不能共享节点；merge(x,x)非法。
- 第k小要转换为第size-k+1大。
- 按值域分裂由两次排名分裂实现；移走树之后及时清空旧根。

个人资料来源：数据结构/线段树/权值线段树（合并与分裂）.cpp

数据结构 / 树状数组

类型与用途：模板型；单点加、前缀和、区间和；非负频率时支持前缀排名二分。

复杂度与适用范围：初始化O(n)，修改/查询/select为O(log n)，空间O(n)。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 树状数组：单点加、前缀和、区间和；非负频率时支持前缀排名二分。
// 初始化O(n)，修改/查询/select为O(log n)，空间O(n)。
template <typename T> struct BIT {
    int n;
    vector<T> a;
    BIT(int n_ = 0) {
        init(n_);
    }
    void init(int n_) {
        n = n_;
        a.assign(n + 1, T{});
    }
    void add(int x, const T &v) {
        assert(1 <= x && x <= n);
        for (int i = x; i <= n; i += i & -i)
            a[i] += v;
    }
```

```
    T sum(int x) {
        assert(0 <= x && x <= n);
        T ans{};
        for (int i = x; i; i -= i & -i)
            ans += a[i];
        return ans;
    }
    T rangeSum(int l, int r) {
        return l > r ? T{} : sum(r) - sum(l - 1);
    }
    // 非负原数组：返回满足prefix(x)<=k的最大x；k>=0，结果0..n。
    int select(const T &k) {
        assert(k >= T{});
        int x = 0, step = 1;
        T cur{};
        while (step <= n / 2)
            step <<= 1;
        for (; step; step >>= 1)
            if (x + step <= n && cur + a[x + step] <= k) {
                x += step;
                cur += a[x];
            }
        return x;
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    BIT<int> bit(n);
    for (int i = 1, x; i <= n; i++) {
        cin >> x;
        bit.add(i, x);
    }
    while (m--) {
        int op, x, y;
        cin >> op >> x >> y;
        if (op == 1)
            bit.add(x, y);
        else
            cout << bit.rangeSum(x, y) << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

add(x,v)：下标1..n；禁止x=0

sum(x)：前缀[1,x]，sum(0)=0

rangeSum(l,r)：闭区间和

select(k)：最后一个前缀和<=k的下标；第k小可用select(k-1)+1

例题 1 P3374 树状数组1

https://www.luogu.com.cn/problem/P3374

题意：单点加、区间和。

观察与建模：单点加、区间和。 将区间和拆成两段前缀和之差。

为何对应此算法：将区间和拆成两段前缀和之差。

初始化、组织数据与调用：init(n)，依次add初值；操作add或rangeSum。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O((n+m)log n)时间、O(n)空间。

复用模板：树状数组（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    BIT<int> bit(n);
    for (int i = 1, x; i <= n; i++) {
        cin >> x;
        bit.add(i, x);
    }
    while (m--) {
        int op, x, y;
        cin >> op >> x >> y;
        if (op == 1)
            bit.add(x, y);
        else
            cout << bit.rangeSum(x, y) << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P1908 逆序对

https://www.luogu.com.cn/problem/P1908

题意：统计i<j且a[i]>a[j]的数对。

观察与建模：统计i<j且a[i]>a[j]的数对。 按下标插入频率，已出现数中大于当前值的数量就是新增逆序对。

为何对应此算法：按下标插入频率，已出现数中大于当前值的数量就是新增逆序对。

初始化、组织数据与调用：离散化；ans+=已插入数-sum(pos)，再add(pos,1)。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n log n)时间、O(n)空间。

复用模板：树状数组（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    vector<int> a(n), v;
    for (auto &x : a)
        cin >> x;
    v = a;
    sort(v.begin(), v.end());
    v.erase(unique(v.begin(), v.end()), v.end());
    BIT<int> bit(v.size());
    int ans = 0;
    for (int i = 0; i < n; i++) {
        int p = lower_bound(v.begin(), v.end(), a[i]) - v.begin() + 1;
        ans += i - bit.sum(p);
        bit.add(p, 1);
    }
    cout << ans << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 3 P1966 火柴排队

https://www.luogu.com.cn/problem/P1966

题意：两列高度互异的火柴，可相邻交换，先最小化平方距离，再求最少交换数。

观察与建模：两列高度互异的火柴，可相邻交换，先最小化平方距离，再求最少交换数。 同秩配对最小化平方差；把第一列每个秩映射到第二列对应位置，最少相邻交换就是该位置排列逆序数。

为何对应此算法：同秩配对最小化平方差；把第一列每个秩映射到第二列对应位置，最少相邻交换就是该位置排列逆序数。

初始化、组织数据与调用：排序两列(value,index)，构造位置排列，用BIT频率统计先前更大元素，结果模100000000-3。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n log n)时间/O(n)空间。

复用模板：树状数组（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    vector<pair<int, int>> a(n), b(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i].first;
        a[i].second = i;
    }
    for (int i = 0; i < n; i++) {
        cin >> b[i].first;
        b[i].second = i + 1;
    }
    sort(a.begin(), a.end());
    sort(b.begin(), b.end());
    vector<int> p(n);
    for (int i = 0; i < n; i++)
```

```
        p[a[i].second] = b[i].second;
    BIT<int> t(n);
    int ans = 0;
    for (int i = 0; i < n; i++) {
        ans = (ans + i - t.sum(p[i])) % (100000000 - 3);
        t.add(p[i], 1);
    }
    cout << ans << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 原select注释含义错误，现明确边界；不用浮点log2计算最高位。
- select仅用于非负频率，否则前缀不单调。

个人资料来源：数据结构/树状数组/封装.cpp；tirick/总结.md

数据结构 / 珂朵莉树

类型与用途：模板型；用 set 保存分段常数区间；区间赋值使区间数减少，支持逐段操作。

复杂度与适用范围：每次 O(log n+s)，s 为访问区间数；第k小另需 O(s log s)。最坏 O(nq)，空间 O(n)。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 珂朵莉树：用 set 保存分段常数区间；区间赋值使区间数减少，支持逐段操作。
// 每次 O(log n+s)，s 为访问区间数；第k小另需 O(s log s)。最坏 O(nq)，空间 O(n)。
struct ODT {
    struct node {
        int l, r;
        mutable int v;
        bool operator<(const node &o) const {
            return l < o.l;
        }
    };
    int n;
    set<node> odt;
    ODT(int n = 0, int v = 0) {
        init(n, v);
    }
    void init(int n_, int v = 0) {
        n = n_;
        odt.clear();
        if (n)
            odt.insert({1, n, v});
    }
    auto split(int x) {
        if (x == n + 1)
            return odt.end();
        assert(1 <= x && x <= n);
        auto it = odt.lower_bound({x, 0, 0});
        if (it != odt.end() && it->l == x)
```

```
            return it;
        --it;
        int l = it->l, r = it->r, v = it->v;
        odt.erase(it);
        odt.insert({l, x - 1, v});
        return odt.insert({x, r, v}).first;
    }
    void assign(int l, int r, int v) {
        auto itr = split(r + 1), itl = split(l);
        odt.erase(itl, itr);
        odt.insert({l, r, v});
    }
    void add(int l, int r, int v) {
        auto itr = split(r + 1), itl = split(l);
        for (; itl != itr; ++itl)
            itl->v += v;
    }
    int kth(int l, int r, int k) {
        auto itr = split(r + 1), itl = split(l);
        vector<pair<int, int>> a;
        for (; itl != itr; ++itl)
            a.push_back({itl->v, itl->r - itl->l + 1});
        sort(a.begin(), a.end());
        for (auto [v, c] : a) {
            if (k <= c)
                return v;
            k -= c;
        }
        assert(false);
        return 0;
    }
    int powmod(int a, int b, int p) {
        int r = 1 % p;
        a %= p;
        while (b) {
            if (b & 1)
                r = (__int128)r * a % p;
            a = (__int128)a * a % p;
            b >>= 1;
        }
        return r;
    }
    int qur(int l, int r, int k, int p) {
        auto itr = split(r + 1), itl = split(l);
        int ans = 0;
        for (; itl != itr; ++itl)
            ans = (ans + (__int128)(itl->r - itl->l + 1) * powmod(itl->v, k, p)) % p;
        return ans;
    }
};

void solve() {
    int n, m, seed, vmax;
    cin >> n >> m >> seed >> vmax;
    auto rnd = [&]() {
        int ret = seed;
        seed = (seed * 7 + 13) % 1000000007;
        return ret;
    };
    ODT t(n);
```

```
    for (int i = 1; i <= n; i++)
        t.assign(i, i, rnd() % vmax + 1);
    for (int i = 0; i < m; i++) {
        int op = rnd() % 4 + 1, l = rnd() % n + 1, r = rnd() % n + 1;
        if (l > r)
            swap(l, r);
        int x = op == 3 ? rnd() % (r - l + 1) + 1 : rnd() % vmax + 1;
        if (op == 1)
            t.add(l, r, x);
        if (op == 2)
            t.assign(l, r, x);
        if (op == 3)
            cout << t.kth(l, r, x) << '\n';
        if (op == 4) {
            int y = rnd() % vmax + 1;
            cout << t.qur(l, r, x, y) << '\n';
        }
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

init(n,v)：建立全值v的1..n区间

split(x)：在x处分割；n+1返回end

assign/add(l,r,v)：闭区间赋值或增加

kth/qur：第k小、幂次和模p

例题 1 CF896C Willem Chtholly and Seniorious

https://codeforces.com/contest/896/problem/C

题意：生成数组和四种随机操作，维护增加、赋值、第k小、幂和。

观察与建模：生成数组和四种随机操作，维护增加、赋值、第k小、幂和。 随机操作与大量赋值符合ODT常见使用条件；区间长度作为元素重数。

为何对应此算法：随机操作与大量赋值符合ODT常见使用条件；区间长度作为元素重数。

初始化、组织数据与调用：按原题顺序调用rng生成全部参数，依操作调用add、assign、kth、qur。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：最坏O(nm log n)，随机数据通常区间数较少；O(n)。

复用模板：珂朵莉树（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m, seed, vmax;
    cin >> n >> m >> seed >> vmax;
    auto rnd = [&]() {
        int ret = seed;
        seed = (seed * 7 + 13) % 1000000007;
```

```
        return ret;
    };
    ODT t(n);
    for (int i = 1; i <= n; i++)
        t.assign(i, i, rnd() % vmax + 1);
    for (int i = 0; i < m; i++) {
        int op = rnd() % 4 + 1, l = rnd() % n + 1, r = rnd() % n + 1;
        if (l > r)
            swap(l, r);
        int x = op == 3 ? rnd() % (r - l + 1) + 1 : rnd() % vmax + 1;
        if (op == 1)
            t.add(l, r, x);
        if (op == 2)
            t.assign(l, r, x);
        if (op == 3)
            cout << t.kth(l, r, x) << '\n';
        if (op == 4) {
            int y = rnd() % vmax + 1;
            cout << t.qur(l, r, x, y) << '\n';
        }
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P3740 贴海报

https://www.luogu.com.cn/problem/P3740

题意：海报依次覆盖整数墙面，求最终仍可见的海报数量。

观察与建模：海报依次覆盖整数墙面，求最终仍可见的海报数量。 每次覆盖就是区间赋值；最终扫描不为0的段收集编号。

为何对应此算法：每次覆盖就是区间赋值；最终扫描不为0的段收集编号。

初始化、组织数据与调用：init(墙长,0)，第i张assign(l,r,i)，遍历odt中的v。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：最坏O(nm)，O(n+m)空间；此题墙长小。

复用模板：珂朵莉树（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    ODT t(n);
    for (int i = 1; i <= m; i++) {
        int l, r;
        cin >> l >> r;
        t.assign(l, r, i);
    }
    set<int> vis;
    for (auto z : t.odt)
        if (z.v)
            vis.insert(z.v);
    cout << vis.size() << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 必须先 split(r+1) 再 split(l)，否则旧迭代器可能失效。
- 随机数据下常用，不保证普通对抗数据的高效；P3740只有覆盖操作可另用线段树。
- k必须在区间长度内，p>0；拆分不自动合并相邻等值段。

个人资料来源：数据结构/珂朵莉树.cpp

图论

图论 / 2-SAT

类型与用途：模板型；二元布尔变量OR子句与蕴含约束；内部独立Tarjan SCC。

复杂度与适用范围：O(变量数+子句数)时间与空间。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 2-SAT：二元布尔变量OR子句与蕴含约束；内部独立Tarjan SCC。
// O(变量数+子句数)时间与空间。
struct SCC {
    // 以下顶点表均由init按n+1项assign重建；顶点下标1..n，0号预留不用。
    // 内层邻接位置从0开始，init清空旧边；标记/时间戳/归属表清0。
    int n;   // 顶点数，n>=1；顶点编号为 1..n。
    int idx; // DFS 时钟，范围 [0,n]；每首次访问一个点加 1。
    int cnt; // SCC 数，范围 [0,n]；按弹栈先后编号。

    // e：二维变长邻接表；e[u][j] 为 u 的第 j 条出边终点。
    // u=1..n，j=0..e[u].size()-1；init 中 clear；允许重边、自环。
    vector<vector<int>> e;
    // dfn：一维；dfn[u] 为首次访问时间，0 表示未访问；init 清 0；有效值 1..n。
    vector<int> dfn;
    // low：一维；low[u] 为子树按 Tarjan 规则能追溯到的最早未封闭时间戳。
    // 下标 1..n；init 清 0；首次访问置 dfn[u]，随后取 min。
    vector<int> low;
    // ins：一维；ins[u] 为 u 是否仍在 Tarjan 栈中；init 清0，标记取0/1，入栈置 true，弹栈置
    //  false。
    vector<int> ins;
    // bel：一维；bel[u] 为 SCC 编号 1..cnt；init 清 0；0 表示尚未分配。
    vector<int> bel;
    // stk：一维栈，元素为顶点编号 1..n；init 弹空；最多存 n 个点。
    stack<int> stk;
    // scc：二维变长表；scc[c-1] 为 SCC c 的全部原图顶点；init clear；总成员数 n。
    vector<vector<int>> scc;

    // init(n_)：参数为顶点数；返回 void；清图和 Tarjan 状态。
    // 前提：n_>=1，下一步重新加边；首次初始化时间O(n_)；复用对象还需销毁旧图的邻接元素。
    void init(int n_) {
        n = n_;
        idx = cnt = 0;
        e.assign(n + 1, {});
        dfn.assign(n + 1, 0);
```

```
        low.assign(n + 1, 0);
        ins.assign(n + 1, 0);
        bel.assign(n + 1, 0);
        while (!stk.empty())
            stk.pop();
        scc.clear();
    }

    // dfs(u): u 为当前顶点；返回 void；递归求 SCC 并填写 bel、scc。
    // 前提：图已建好，dfn[u]==0；一次 DFS 为 O(所访问顶点+扫描边)，全图 O(n+m)。
    void dfs(int u) {
        dfn[u] = low[u] = ++idx;
        ins[u] = true;
        stk.push(u);
        for (int v : e[u]) { // v: 出边终点，范围 1..n。
            if (!dfn[v]) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (ins[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (dfn[u] == low[u]) {
            ++cnt;
            vector<int> c; // 一维临时点集，初始为空；存当前 SCC 顶点，大小至少 1。
            while (true) {
                int v = stk.top(); // v: 当前弹出顶点，1..n；最后一次必为 u。
                stk.pop();
                ins[v] = false;
                bel[v] = cnt;
                c.push_back(v);
                if (v == u)
                    break;
            }
            scc.push_back(c);
        }
    }

    // run(): 无参数，返回 void；遍历 DFS 森林，完成全图 SCC。
    // 前提：已 init 并加边，尚未运行；时间 O(n+m)，递归栈最坏 O(n)。
    void run() {
        for (int u = 1; u <= n; u++)
            if (!dfn[u])
                dfs(u);
    }

    // addedge(u,v): 参数为有向边起终点1..n；返回void；前提已init且尚未run。
    // 作用：只封装原PPT邻接表push_back；摊还时间O(1)。
    void addedge(int u, int v) {
        e[u].push_back(v);
    }
};
struct TwoSAT {
    // scc: 内部SCC对象，独立保存2*vars个命题点的邻接表、dfn/low/bel与点栈。
    // init_sat调用scc.init清图；调用前提2*vars不溢出int，不与外部SCC对象共享状态。
    SCC scc;

    int vars; // 布尔变量数，要求 1<=vars 且 2*vars不溢出int；Tarjan 的 n=2*vars。
    // ans: 一维，ans[i] 为变量 i 的值 0/1；下标 1..vars。
    // init_sat 以 vars+1 项清 0；求解失败时不可使用 ans。
```

```
    vector<int> ans;

    // id(i,value): 变量 i 的取值命题编号; 返回 1..2*vars 的 int。
    // 前提 i=1..vars, value=0/1; 作用: 统一映射; 时间 O(1)。
    int id(int i, int value) {
        return i + value * vars;
    }
    // neg(u): u 为命题点; 返回同变量相反取值的命题点; 前提 u=1..2*vars; O(1)。
    int neg(int u) {
        return u <= vars ? u + vars : u - vars;
    }
    // init_sat(k): k 为布尔变量数; 返回 void; 清 Tarjan 图并初始化答案。
    // 前提 k>=1且2*k+1不溢出int, 随后重新加子句; 时间 O(k)。
    void init_sat(int k) {
        vars = k;
        scc.init(2 * vars);
        ans.assign(vars + 1, 0);
    }
    // add_or(a,A,b,B): 加入 (变量a=A) OR (变量b=B); 返回 void。
    // 前提 a/b=1..vars, A/B=0/1, 已 init_sat; 摊还 O(1)。
    void add_or(int a, int A, int b, int B) {
        scc.e[id(a, A ^ 1)].push_back(id(b, B));
        scc.e[id(b, B ^ 1)].push_back(id(a, A));
    }
    // imply(u,v): 加入命题 u 推出 v, 及其逆否; 返回 void。
    // 前提 u/v=1..2*vars; 作用: 保证蕴含图成对建边; 摊还 O(1)。
    void imply(int u, int v) {
        scc.e[u].push_back(v);
        scc.e[neg(v)].push_back(neg(u));
    }
    // solve_sat(): 无参数; 返回是否可行; 成功时写 ans。
    // 前提尚未 run, 且使用第1.2节 Tarjan 的反拓扑 SCC 编号; 时间 O(vars+子句数)。
    bool solve_sat() {
        scc.run();
        for (int i = 1; i <= vars; i++) { // i: 布尔变量编号。
            if (scc.bel[id(i, 0)] == scc.bel[id(i, 1)])
                return false;
            ans[i] = scc.bel[id(i, 1)] < scc.bel[id(i, 0)];
        }
        return true;
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    TwoSAT sat;
    sat.init_sat(n);
    while (m--) {
        int a, A, b, B;
        cin >> a >> A >> b >> B;
        sat.add_or(a, A, b, B);
    }
    if (!sat.solve_sat()) {
        cout << "IMPOSSIBLE\n";
        return;
    }
    cout << "POSSIBLE\n";
    for (int i = 1; i <= n; i++)
        cout << sat.ans[i] << (i == n ? '\n' : ' ');
```

```
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

init_sat(n)：n个变量，1-based

add_or(a,A,b,B)：(a=A)或(b=B)

imply(u,v)：自动添加逆否边

solve_sat()：可行后ans保存0/1解

例题 1 P4782 2-SAT

https://www.luogu.com.cn/problem/P4782

题意：给OR子句求一个可行布尔赋值。

观察与建模：给OR子句求一个可行布尔赋值。 每条OR拆成两个蕴含，变量两取值不能在同一SCC。

为何对应此算法：每条OR拆成两个蕴含，变量两取值不能在同一SCC。

初始化、组织数据与调用：add_or建图，solve_sat后输出ans。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m)时间与空间。

复用模板：2-SAT（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    TwoSAT sat;
    sat.init_sat(n);
    while (m--) {
        int a, A, b, B;
        cin >> a >> A >> b >> B;
        sat.add_or(a, A, b, B);
    }
    if (!sat.solve_sat()) {
        cout << "IMPOSSIBLE\n";
        return;
    }
    cout << "POSSIBLE\n";
    for (int i = 1; i <= n; i++)
        cout << sat.ans[i] << (i == n ? '\n' : ' ');
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P4171 满汉全席

https://www.luogu.com.cn/problem/P4171

题意：每种材料做满式或汉式，满足每位评委至少喜欢一种菜。

观察与建模：每种材料做满式或汉式，满足每位评委至少喜欢一种菜。 评委条件是两个取值命题的OR子句。

为何对应此算法：评委条件是两个取值命题的OR子句。

初始化、组织数据与调用：h映射1、m映射0；每组init后add_or并判定。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(总变量+总条件)时间与空间。

复用模板：2-SAT（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int T;
    cin >> T;
    while (T--) {
        int n, m;
        cin >> n >> m;
        TwoSAT sat;
        sat.init_sat(n);
        while (m--) {
            char a, b;
            int x, y;
            cin >> a >> x >> b >> y;
            sat.add_or(x, a == 'h', y, b == 'h');
        }
        cout << (sat.solve_sat() ? "GOOD" : "BAD") << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 取值公式依赖Tarjan的反拓扑分量编号，不能直接替换为Kosaraju。
- 每次判定需重新初始化图；矛盾时ans无效。

个人资料来源：图/图论竞赛复习_严格课件注释版.md

图论 / DFS增广路匹配

类型与用途：模板型；保留原逐左点DFS增广写法；接口与HK统一，便于比较两种算法。

复杂度与适用范围：O(V_left*E)时间、O(V+E)空间。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// DFS增广路匹配：保留原逐左点DFS增广写法；接口与HK统一，便于比较两种算法。
// O(V_left*E)时间、O(V+E)空间。
struct HK {
    int n1, n2;
    vector<vector<int>> adj;
    vector<int> ml, mr, vis;
```

```
    void init(int n, int m) {
        n1 = n;
        n2 = m;
        adj.assign(n + 1, {});
        ml.assign(n + 1, 0);
        mr.assign(m + 1, 0);
        vis.assign(m + 1, 0);
    }
    void add(int u, int v) {
        adj[u].push_back(v);
    }
    bool dfs(int x) {
        for (int v : adj[x]) {
            if (vis[v])
                continue;
            vis[v] = 1;
            if (!mr[v] || dfs(mr[v])) {
                ml[x] = v;
                mr[v] = x;
                return true;
            }
        }
        return false;
    }
    int matching() {
        int ans = 0;
        for (int x = 1; x <= n1; x++)
            if (ml[x])
                ans++;
        for (int x = 1; x <= n1; x++)
            if (!ml[x]) {
                fill(vis.begin(), vis.end(), 0);
                if (dfs(x))
                    ans++;
            }
        return ans;
    }
};

void solve() {
    int n, m, q;
    cin >> n >> m >> q;
    HK hk;
    hk.init(n, m);
    while (q--) {
        int u, v;
        cin >> u >> v;
        hk.add(u, v);
    }
    cout << hk.matching() << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
```

```
}
```

init(n,m)：独立左右编号

add(u,v)：合法匹配边

matching()：每轮清vis再增广

例题 1 P3386 二分图最大匹配

https://www.luogu.com.cn/problem/P3386

题意：求左右两部图最大匹配数。

观察与建模：求左右两部图最大匹配数。 每点最多匹配一次，直接构造左到右的边。

为何对应此算法：每点最多匹配一次，直接构造左到右的边。

初始化、组织数据与调用：init(n,m)，add(u,v)，matching()。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(V_left*E)时间、O(V+E)空间。

复用模板：DFS增广路匹配（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m, q;
    cin >> n >> m >> q;
    HK hk;
    hk.init(n, m);
    while (q--) {
        int u, v;
        cin >> u >> v;
        hk.add(u, v);
    }
    cout << hk.matching() << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 practice2_d Maxflow

https://atcoder.jp/contests/practice2/tasks/practice2_d

题意：障碍格盘上放最多1x2骨牌，并输出方案。

观察与建模：障碍格盘上放最多1x2骨牌，并输出方案。 棋盘黑白染色后每个骨牌是一条二分图匹配边。

为何对应此算法：棋盘黑白染色后每个骨牌是一条二分图匹配边。

初始化、组织数据与调用：空白相邻格add；matching后从ml恢复箭头。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(V_left*E)时间、O(V+E)空间。

复用模板：DFS增广路匹配（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<string> a(n);
    for (auto &s : a)
        cin >> s;
    HK hk;
    hk.init(n * m, n * m);
    int dx[4] = {1, -1, 0, 0}, dy[4] = {0, 0, 1, -1};
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (a[i][j] == '.' && (i + j) % 2 == 0)
                for (int k = 0; k < 4; k++) {
                    int x = i + dx[k], y = j + dy[k];
                    if (0 <= x && x < n && 0 <= y && y < m && a[x][y] == '.')
                        hk.add(i * m + j + 1, x * m + y + 1);
                }
    cout << hk.matching() << '\n';
    for (int u = 1; u <= n * m; u++)
        if (hk.ml[u]) {
            int v = hk.ml[u], x = (u - 1) / m, y = (u - 1) % m, p = (v - 1) / m,
                q = (v - 1) % m;
            if (x == p) {
                a[x][min(y, q)] = '>';
                a[x][max(y, q)] = '<';
            } else {
                a[min(x, p)][y] = 'v';
                a[max(x, p)][y] = '^';
            }
        }
    for (auto &s : a)
        cout << s << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 每次未匹配左点增广前必须重置vis。
- 大图优先HK；这里保留简单递归作为小规模变体。

个人资料来源：图论/二分图匹配.cpp

图论 / Dinic最大流

类型与用途：模板型；整数非负容量网络最大流；保留head/dis/cur/e与递归DFS。

复杂度与适用范围：一般O(V^2 E)时间、O(V+E)空间；单位容量二分匹配有更强界。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// Dinic最大流：整数非负容量网络最大流；保留head/dis/cur/e与递归DFS。
// 一般O(V^2 E)时间、O(V+E)空间；单位容量二分匹配有更强界。
template <typename T> struct FlowGraph {
    int s, t; // 源/汇编号1..vtot，要求s!=t；init赋值。
    int vtot; // 顶点总数，>=1；包含全部辅助节点。
    int etot; // 已存残量边数，等于e.size()，始终偶数；边号须在int范围内；init清0。
    // head：一维vector，init分配vtot+1项，下标1..vtot；邻接链首边编号，-1为空；init置-1。
    vector<int> head;
    // dis：一维vector，init分配vtot+1项并清0；下标1..vtot；BFS层数，0未访问，源为1；DFS死点
    // 置-1；每次bfs清0。
    vector<int> dis;
    // cur：一维vector，init分配vtot+1项并置-1；下标1..vtot；当前弧下标或-1；每次bfs由head复
    // 制，DFS沿链推进。
    vector<int> cur;
    struct edge {
        int v;   // 边终点，1..vtot。
        int nxt; // 同起点的下一边编号，-1或0..etot-1；由addedge填写。
        T f;     // 当前残量，>=0；正向初始为输入容量，反向初始0。
    };
    // e：一维动态边表，下标0..etot-1；配对边i^1。
    // init执行e.clear并令etot=0；addedge成对push_back，不提前resize占用边号。
    vector<edge> e;

    // init(s_,t_,vtot_)：设置源汇与点数；返回void；清边数与邻接链。
    // 前提合法编号、s_!=t_，随后重新建图；首次时间O(vtot_)；重建还包括清除旧边。
    void init(int s_, int t_, int vtot_) {
        s = s_;
        t = t_;
        vtot = vtot_;
        etot = 0; // 多次建图必须清etot。
        head.assign(vtot + 1, -1);
        dis.assign(vtot + 1, 0);
        cur.assign(vtot + 1, -1);
        e.clear();
    }
    // addedge(u,v,f)：加入原始有向边及撤销边；返回void；两次push_back摊还时间O(1)。
    // 前提u/v=1..vtot，f>=0，边号及etot不溢出int；原容量需另存时由调用方记录。
    void addedge(int u, int v, T f) {
        e.push_back({v, head[u], f});
        head[u] = etot++;
        e.push_back({u, head[v], 0});
        head[v] = etot++;
    }
    // bfs()：无参数；返回残量图中汇是否可达；完成分层并重置cur。
    // 前提图已建好或上轮增广完成；时间O(vtot+etot)。
```

```
    bool bfs() {
        for (int i = 1; i <= vtot; i++) {
            dis[i] = 0;
            cur[i] = head[i];
        }
        queue<int> q; // 一维BFS队列，初始空；存顶点1..vtot，每点至多入队一次。
        q.push(s);
        dis[s] = 1;
        while (!q.empty()) {
            int u = q.front();
            q.pop();                                          // u：当前扩展顶点，1..vtot。
            for (int i = head[u]; i != -1; i = e[i].nxt) { // i：当前残量边编号。
                if (e[i].f > 0 && dis[e[i].v] == 0) {
                    int v = e[i].v; // v：当前可达边终点，1..vtot。
                    dis[v] = dis[u] + 1;
                    if (v == t)
                        return true;
                    q.push(v);
                }
            }
        }
        return false;
    }
    // dfs(u,m)：u当前点；m为本次允许送出的剩余流量，m>0；返回实际送出量T。
    // 作用：按分层图多路增广；前提bfs成功，cur已重置；一轮阻塞流一般O(VE)。
    T dfs(int u, T m) {
        if (u == t)
            return m;
        T flow = 0; // 本次从u成功送出的总量，0..初始m。
        for (int i = cur[u]; i != -1; cur[u] = i = e[i].nxt) {
            // i：当前弧；保留PPT的cur推进方式，送满本次额度时不前移。
            if (e[i].f > 0 && dis[e[i].v] == dis[u] + 1) {
                T f = dfs(e[i].v, min(m, e[i].f)); // 实际成功增广量，0..min(m,残量)。
                e[i].f -= f;
                e[i ^ 1].f += f;
                m -= f;
                flow += f;
                if (m == 0)
                    break;
            }
        }
        if (flow == 0)
            dis[u] = -1;
        return flow;
    }
    // dinic()：无参数；返回本次新增最大流T；作用：反复分层并求阻塞流。
    // 前提合法源汇、图已建好、总流量不溢出；一般时间O(Vnet^2*Enet)，空间O(Vnet+Enet)，额外递
    // 归栈O(Vnet)。
    T dinic() {
        T flow = 0; // 本次调用新增流量，init后首次调用即总最大流。
        while (bfs())
            flow += dfs(s, numeric_limits<T>::max());
        return flow;
    }
    vector<int> cut_side() {
        vector<int> vis(vtot + 1);
        queue<int> q;
        q.push(s);
        vis[s] = 1;
```

```
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int i = head[u]; i != -1; i = e[i].nxt)
                if (e[i].f > 0 && !vis[e[i].v]) {
                    vis[e[i].v] = 1;
                    q.push(e[i].v);
                }
        }
        return vis;
    }
};

void solve() {
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    FlowGraph<int> g;
    g.init(s, t, n);
    while (m--) {
        int u, v, c;
        cin >> u >> v >> c;
        g.addedge(u, v, c);
    }
    cout << g.dinic() << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

init(s,t,V)：清空所有旧残量，s!=t

addedge(u,v,c)：返回前etot是正向边号，反向边号i^1

dinic()：本次新增流量；首次init后即完整最大流

cut_side()：完整最大流后取源侧残量可达点

例题 1 P3376 网络最大流

https://www.luogu.com.cn/problem/P3376

题意：有向容量图求源到汇最大流。

观察与建模：有向容量图求源到汇最大流。 每条原边配反向撤销边，Dinic反复分层增广。

为何对应此算法：每条原边配反向撤销边，Dinic反复分层增广。

初始化、组织数据与调用：init(s,t,n)，addedge，dinic。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(V^2 E)时间、O(V+E)空间。

复用模板：Dinic最大流（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    FlowGraph<int> g;
    g.init(s, t, n);
    while (m--) {
        int u, v, c;
        cin >> u >> v >> c;
        g.addedge(u, v, c);
    }
    cout << g.dinic() << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P2763 试题库问题

https://www.luogu.com.cn/problem/P2763

题意：选题满足各类别需求，每道题最多用一次，输出分配。

观察与建模：选题满足各类别需求，每道题最多用一次，输出分配。 源到类别为需求，类别到可用题为1，题到汇为1。

为何对应此算法：源到类别为需求，类别到可用题为1，题到汇为1。

初始化、组织数据与调用：保存类别到题正向边号；总流达标后残量0的选择边为分配。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：一般O(V^2 E)时间、O(V+E)空间。

复用模板：Dinic最大流（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int k, n;
    cin >> k >> n;
    int s = k + n + 1, t = s + 1;
    FlowGraph<int> g;
    g.init(s, t, t);
    int need = 0;
    for (int c = 1, x; c <= k; c++) {
        cin >> x;
        need += x;
        g.addedge(s, c, x);
    }
    vector<array<int, 3>> chosen;
    for (int j = 1; j <= n; j++) {
        int p;
        cin >> p;
        while (p--) {
            int c;
            cin >> c;
            chosen.push_back({c, j, g.etot});
            g.addedge(c, k + j, 1);
        }
        g.addedge(k + j, t, 1);
    }
```

```
    if (g.dinic() != need) {
        cout << "No Solution!\n";
        return;
    }
    vector<vector<int>> ans(k + 1);
    for (auto [c, j, id] : chosen)
        if (g.e[id].f == 0)
            ans[c].push_back(j);
    for (int c = 1; c <= k; c++) {
        cout << c << ":";
        for (int j : ans[c])
            cout << ' ' << j;
        cout << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 原容量与残量不能混用；方案恢复要保存正向边号。
- 容量总和和流量累计不得溢出long long；极深增广路有递归栈风险。

个人资料来源：图/图论竞赛复习_严格课件注释版.md

图论 / Hopcroft-Karp匹配

类型与用途：模板型；二分图最大匹配；左右编号独立且从1开始，0表示未匹配。

复杂度与适用范围：O(E sqrt V)时间、O(V+E)空间。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// Hopcroft-Karp匹配：二分图最大匹配；左右编号独立且从1开始，0表示未匹配。
// O(E sqrt V)时间、O(V+E)空间。
struct HK {
    int n1, n2, shortest;
    vector<vector<int>> e;
    vector<int> ml, mr, dis, cur;

    void init(int _n1, int _n2) {
        n1 = _n1;
        n2 = _n2;
        e.assign(n1 + 1, {});
        ml.assign(n1 + 1, 0);
        mr.assign(n2 + 1, 0);
        dis.assign(n1 + 1, -1);
        cur.assign(n1 + 1, 0);
    }

    // 添加原图的边：左部 u -> 右部 v。
    void add(int u, int v) {
        e[u].push_back(v);
    }

```

```
    // 分层：从所有未匹配的左部点同时出发，求最短增广路长度。
    bool bfs() {
        queue<int> q;
        fill(dis.begin(), dis.end(), -1);
        shortest = LLONG_MAX;

        for (int i = 1; i <= n1; i++) {
            if (!ml[i]) {
                dis[i] = 0;
                q.push(i);
            }
        }

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            if (dis[u] >= shortest)
                continue;

            for (int v : e[u]) {
                int w = mr[v];
                if (!w) {
                    shortest = dis[u] + 1;
                } else if (dis[w] == -1) {
                    dis[w] = dis[u] + 1;
                    q.push(w);
                }
            }
        }
        return shortest != LLONG_MAX;
    }

    // 在当前分层图上递归 DFS，寻找最短增广路。
    // cur[u] 是当前弧，避免同一轮重复枚举无效边。
    bool dfs(int u) {
        for (int &i = cur[u]; i < (int)e[u].size(); i++) {
            int v = e[u][i];
            int w = mr[v];

            // v 未匹配，或它原来的搭档 w 能沿下一层找到新匹配。
            if ((!w && dis[u] + 1 == shortest) ||
                (w && dis[w] == dis[u] + 1 && dfs(w))) {
                ml[u] = v;
                mr[v] = u;
                return true;
            }
        }
        dis[u] = -1; // 当前 u 找不到增广路，剪枝
        return false;
    }

    // 返回当前图的最大匹配数；重复调用不会重复累计。
    int matching() {
        int ans = 0;
        for (int i = 1; i <= n1; i++)
            ans += (ml[i] != 0);
```

```
        while (bfs()) {
            fill(cur.begin(), cur.end(), 0);
            for (int i = 1; i <= n1; i++) {
                if (!ml[i] && dfs(i))
                    ans++;
            }
        }
        return ans;
    }
};

void solve() {
    int n, m, q;
    cin >> n >> m >> q;
    HK hk;
    hk.init(n, m);
    while (q--) {
        int u, v;
        cin >> u >> v;
        hk.add(u, v);
    }
    cout << hk.matching() << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

init(n1,n2)：清空图和匹配

add(u,v)：左部到右部的边

matching()：最大匹配数，重复调用不重复累计

ml/mr：匹配关系

例题 1 P3386 二分图最大匹配

https://www.luogu.com.cn/problem/P3386

题意：求左右两部图最大匹配数。

观察与建模：求左右两部图最大匹配数。 每点最多匹配一次，直接构造左到右的边。

为何对应此算法：每点最多匹配一次，直接构造左到右的边。

初始化、组织数据与调用：init(n,m)，add(u,v)，matching()。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：HK为O(E sqrt V)时间、O(V+E)空间。

复用模板：Hopcroft-Karp匹配（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m, q;
    cin >> n >> m >> q;
    HK hk;
    hk.init(n, m);
    while (q--) {
        int u, v;
        cin >> u >> v;
        hk.add(u, v);
    }
    cout << hk.matching() << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 practice2_d Maxflow

https://atcoder.jp/contests/practice2/tasks/practice2_d

题意：障碍格盘上放最多1x2骨牌，并输出方案。

观察与建模：障碍格盘上放最多1x2骨牌，并输出方案。 棋盘黑白染色后每个骨牌是一条二分图匹配边。

为何对应此算法：棋盘黑白染色后每个骨牌是一条二分图匹配边。

初始化、组织数据与调用：空白相邻格add；matching后从ml恢复箭头。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：HK为O(E sqrt V)时间、O(V+E)空间。

复用模板：Hopcroft-Karp匹配（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<string> a(n);
    for (auto &s : a)
        cin >> s;
    HK hk;
    hk.init(n * m, n * m);
    int dx[4] = {1, -1, 0, 0}, dy[4] = {0, 0, 1, -1};
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if (a[i][j] == '.' && (i + j) % 2 == 0)
                for (int k = 0; k < 4; k++) {
                    int x = i + dx[k], y = j + dy[k];
                    if (0 <= x && x < n && 0 <= y && y < m && a[x][y] == '.')
                        hk.add(i * m + j + 1, x * m + y + 1);
                }
    cout << hk.matching() << '\n';
    for (int u = 1; u <= n * m; u++)
        if (hk.ml[u]) {
            int v = hk.ml[u], x = (u - 1) / m, y = (u - 1) % m, p = (v - 1) / m,
                q = (v - 1) % m;
            if (x == p) {
                a[x][min(y, q)] = '>';
                a[x][max(y, q)] = '<';
```

```
            } else {
                a[min(x, p)][y] = 'v';
                a[max(x, p)][y] = '^';
            }
        }
    for (auto &s : a)
        cout << s << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 只用于二分图，不用于一般图匹配。
- 递归增广路深度最坏O(V)，极深数据有栈风险。

个人资料来源：图论/HK二分图最大匹配.cpp

图论 / Kosaraju强连通分量

类型与用途：模板型；有向图强连通分量划分与缩点；支持重边、自环及不连通图。

复杂度与适用范围：O(n+m)时间与空间，递归栈O(n)。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// Kosaraju强连通分量：有向图强连通分量划分与缩点；支持重边、自环及不连通图。
// O(n+m)时间与空间，递归栈O(n)。
struct Kosaraju {
    // 以下顶点表均由init按n+1项assign重建；顶点下标1..n，0号预留不用。
    // 内层邻接位置从0开始，init清空旧边；标记/时间戳/归属表清0。
    int n;   // 原图顶点数；编号 1..n。
    int cnt; // 已求 SCC 数，0..n；第二遍发现一个新分量时加 1。
    // e/erev：二维变长邻接表，分别存出边终点/入边起点；下标 1..n；init clear。
    // 每个邻接表的位置从 0 开始；允许重边、自环。
    vector<vector<int>> e, erev;
    // vis：一维访问标记，下标 1..n，值 0/1；init 清 0，两遍 DFS 之间必须再次清 0。
    vector<int> vis;
    // bel：一维 SCC 编号，下标 1..n，值 1..cnt；init 清 0，第二遍赋值。
    vector<int> bel;
    // out：一维退出序，out[j] 为第 j 个退出的顶点，j=0..n-1；init clear。
    vector<int> out;
    // c：一维当前 SCC 顶点表，每次 dfs2 前 clear；最多 n 项。
    vector<int> c;
    // scc：二维 SCC 表，scc[k-1] 存第 k 个分量；init clear；总成员数 n。
    vector<vector<int>> scc;

    // init(n_)：初始化点数与全部容器；返回 void；前提 n_>=1；首次初始化时间O(n_)；复用对象还
    // 需销毁旧图的邻接元素。
    void init(int n_) {
        n = n_;
        cnt = 0;
        out.clear();
        c.clear();
        scc.clear();
```

```
        e.assign(n + 1, {});
        erev.assign(n + 1, {});
        vis.assign(n + 1, 0);
        bel.assign(n + 1, 0);
    }
    // dfs(u)：原图 DFS；u 为未访问顶点；返回 void；退出时写入 out。
    // 前提：图已建好，vis[u]==0；整遍 O(n+m)。
    void dfs(int u) {
        vis[u] = 1;
        for (int v : e[u])
            if (!vis[v])
                dfs(v); // v：原图出边终点，1..n。
        out.push_back(u);
    }
    // dfs2(u)：反图 DFS；u 为未访问顶点；返回 void；收集一个 SCC 到 c。
    // 前提：第一遍已完成，vis 已清 0，cnt 已增加；整遍 O(n+m)。
    void dfs2(int u) {
        vis[u] = 1;
        bel[u] = cnt;
        c.push_back(u);
        for (int v : erev[u])
            if (!vis[v])
                dfs2(v); // v：反图邻点，1..n。
    }
    // run()：无参数，返回 void；完成原图退出序和反图分量划分；时间 O(n+m)。
    // 前提：已 init；每条 u->v 同时加入 e[u] 和 erev[v]；只运行一次。
    void run() {
        for (int u = 1; u <= n; u++)
            if (!vis[u])
                dfs(u);
        for (int u = 1; u <= n; u++)
            vis[u] = 0;
        reverse(out.begin(), out.end());
        for (int u : out)
            if (!vis[u]) {
                ++cnt;
                c.clear();
                dfs2(u);
                scc.push_back(c);
            }
    }

    // addedge(u,v)：参数为有向边起终点1..n；返回void；前提已init且尚未run。
    // 作用：同时建立原图与反图；每次摊还O(1)。
    void addedge(int u, int v) {
        e[u].push_back(v);
        erev[v].push_back(u);
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    Kosaraju scc;
    scc.init(n);
    vector<pair<int, int>> e(m);
    for (auto &[u, v] : e) {
        cin >> u >> v;
        scc.addedge(u, v);
    }
```

```
    scc.run();
    vector<int> out(scc.cnt + 1), sz(scc.cnt + 1);
    for (int u = 1; u <= n; u++)
        sz[scc.bel[u]]++;
    for (auto [u, v] : e)
        if (scc.bel[u] != scc.bel[v])
            out[scc.bel[u]]++;
    int ans = 0, count = 0;
    for (int c = 1; c <= scc.cnt; c++)
        if (!out[c]) {
            count++;
            ans = sz[c];
        }
    cout << (count == 1 ? ans : 0) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

init(n)：清空旧图

addedge(u,v)：有向边

run()：求SCC，每次建图仅运行一次

bel[u]/cnt/scc：归属编号、数量与分量成员

例题 1 P2341 受欢迎的牛

https://www.luogu.com.cn/problem/P2341

题意：找被所有点可达的点数。

观察与建模：找被所有点可达的点数。 缩点DAG唯一出度0分量中的所有点满足；多于一个则无解。

为何对应此算法：缩点DAG唯一出度0分量中的所有点满足；多于一个则无解。

初始化、组织数据与调用：run后枚举原边计算分量出度，检查唯一汇分量。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m)时间与空间。

复用模板：Kosaraju强连通分量（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    Kosaraju scc;
    scc.init(n);
    vector<pair<int, int>> e(m);
    for (auto &[u, v] : e) {
        cin >> u >> v;
        scc.addedge(u, v);
    }
    scc.run();
```

```
    vector<int> out(scc.cnt + 1), sz(scc.cnt + 1);
    for (int u = 1; u <= n; u++)
        sz[scc.bel[u]]++;
    for (auto [u, v] : e)
        if (scc.bel[u] != scc.bel[v])
            out[scc.bel[u]]++;
    int ans = 0, count = 0;
    for (int c = 1; c <= scc.cnt; c++)
        if (!out[c]) {
            count++;
            ans = sz[c];
        }
    cout << (count == 1 ? ans : 0) << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P3387 缩点

https://www.luogu.com.cn/problem/P3387

题意：有向图走一条路线，点权最多可取得多少；每点只计一次。

观察与建模：有向图走一条路线，点权最多可取得多少；每点只计一次。 正权SCC内可遍历全部点，缩为DAG做最长路。

为何对应此算法：正权SCC内可遍历全部点，缩为DAG做最长路。

初始化、组织数据与调用：run、汇总点权、建立分量边，用Kahn顺序转移dp。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m)时间与空间。

复用模板：Kosaraju强连通分量（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    Kosaraju scc;
    scc.init(n);
    vector<pair<int, int>> e(m);
    for (auto &[u, v] : e) {
        cin >> u >> v;
        scc.addedge(u, v);
    }
    scc.run();
    int k = scc.cnt;
    vector<int> w(k + 1), deg(k + 1);
    vector<vector<int>> g(k + 1);
    for (int i = 1; i <= n; i++)
        w[scc.bel[i]] += a[i];
    for (auto [u, v] : e) {
        u = scc.bel[u];
        v = scc.bel[v];
```

```
        if (u != v) {
            g[u].push_back(v);
            deg[v]++;
        }
    }
    queue<int> q;
    vector<int> dp = w;
    for (int c = 1; c <= k; c++)
        if (!deg[c])
            q.push(c);
    int ans = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        ans = max(ans, dp[u]);
        for (int v : g[u]) {
            dp[v] = max(dp[v], dp[u] + w[v]);
            if (!--deg[v])
                q.push(v);
        }
    }
    cout << ans << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- Tarjan只使用仍在栈中的点更新low；Kosaraju第二遍走反图。
- 两算法分量编号顺序不同，不把编号当统一拓扑序；应用显式拓扑排序。
- 深链递归有RE风险。

个人资料来源：图/图论竞赛复习_严格课件注释版.md；图论/SCC.md

图论 / Prim最小生成树

类型与用途：模板型；堆优化Prim无向图最小生成树；负边也可用；返回连通性与权值和。

复杂度与适用范围：O(m log m)堆操作上界，空间O(n+m)。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// Prim最小生成树：堆优化Prim无向图最小生成树；负边也可用；返回连通性与权值和。
// O(m log m)堆操作上界，空间O(n+m)。
struct Prim {
    struct edge {
        int v, w;
    };
    int n;
    vector<vector<edge>> e;
    void init(int n_) {
        n = n_;
        e.assign(n + 1, {});
    }
```

```
    void addedge(int u, int v, int w) {
        e[u].push_back({v, w});
        e[v].push_back({u, w});
    }
    pair<bool, int> run(int s = 1) {
        vector<int> vis(n + 1);
        priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int, int>>>
            q;
        q.push({0, s});
        int total = 0, count = 0;
        while (!q.empty()) {
            auto [w, u] = q.top();
            q.pop();
            if (vis[u])
                continue;
            vis[u] = 1;
            total += w;
            count++;
            for (auto [v, c] : e[u])
                if (!vis[v])
                    q.push({c, v});
        }
        return {count == n, total};
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    Prim g;
    g.init(n);
    while (m--) {
        int u, v, w;
        cin >> u >> v >> w;
        g.addedge(u, v, w);
    }
    auto [ok, ans] = g.run();
    if (ok)
        cout << ans << '\n';
    else
        cout << "orz\n";
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

addedge(u,v,w)：无向边

run(s)：返回{是否全图连通,生成树权重}

例题 1 P3366 最小生成树

https://www.luogu.com.cn/problem/P3366

题意：求MST，不连通输出orz。

观察与建模：求MST，不连通输出orz。 无向图跨割最小边贪心满足MST性质。

为何对应此算法：无向图跨割最小边贪心满足MST性质。

初始化、组织数据与调用：init、addedge、run，先判断bool。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(m log m)时间、O(n+m)空间。

复用模板：Prim最小生成树（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    Prim g;
    g.init(n);
    while (m--) {
        int u, v, w;
        cin >> u >> v >> w;
        g.addedge(u, v, w);
    }
    auto [ok, ans] = g.run();
    if (ok)
        cout << ans << '\n';
    else
        cout << "orz\n";
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 ABC065D Built

https://atcoder.jp/contests/abc065/tasks/arc076_b

题意：平面点间建路，代价min(|dx|,|dy|)，求最小总费用。

观察与建模：平面点间建路，代价min(|dx|,|dy|)，求最小总费用。 按x/y排序的相邻点边足以保留MST；把完全图稀疏化后运行Prim。

为何对应此算法：按x/y排序的相邻点边足以保留MST；把完全图稀疏化后运行Prim。

初始化、组织数据与调用：两种排序加相邻边，权值取真实min差；run。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n log n)时间、O(n)空间。

复用模板：Prim最小生成树（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    vector<array<int, 3>> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i][0] >> a[i][1];
        a[i][2] = i + 1;
    }
    Prim g;
    g.init(n);
```

```
    for (int dim = 0; dim < 2; dim++) {
        sort(a.begin(), a.end(), [&](auto x, auto y) { return x[dim] < y[dim]; });
        for (int i = 1; i < n; i++)
            g.addedge(a[i - 1][2], a[i][2],
                      min(llabs(a[i][0] - a[i - 1][0]), llabs(a[i][1] - a[i - 1][1])));
    }
    cout << g.run().second << '\n';
}
```

验证：C++17组合程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 不要把边权当成Dijkstra距离；每次选跨割的最小边。
- 不连通时返回的是源所在块生成树和，不能当全图MST。

个人资料来源：图论/prim.cpp

图论 / Tarjan强连通分量

类型与用途：模板型；有向图强连通分量划分与缩点；支持重边、自环及不连通图。

复杂度与适用范围：O(n+m)时间与空间，递归栈O(n)。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// Tarjan强连通分量：有向图强连通分量划分与缩点；支持重边、自环及不连通图。
// O(n+m)时间与空间，递归栈O(n)。
struct SCC {
    // 以下顶点表均由init按n+1项assign重建；顶点下标1..n，0号预留不用。
    // 内层邻接位置从0开始，init清空旧边；标记/时间戳/归属表清0。
    int n;   // 顶点数，n>=1；顶点编号为 1..n。
    int idx; // DFS 时钟，范围 [0,n]；每首次访问一个点加 1。
    int cnt; // SCC 数，范围 [0,n]；按弹栈先后编号。

    // e：二维变长邻接表；e[u][j] 为 u 的第 j 条出边终点。
    // u=1..n，j=0..e[u].size()-1；init 中 clear；允许重边、自环。
    vector<vector<int>> e;
    // dfn：一维；dfn[u] 为首次访问时间，0 表示未访问；init 清 0；有效值 1..n。
    vector<int> dfn;
    // low：一维；low[u] 为子树按 Tarjan 规则能追溯到的最早未封闭时间戳。
    // 下标 1..n；init 清 0；首次访问置 dfn[u]，随后取 min。
    vector<int> low;
    // ins：一维；ins[u] 为 u 是否仍在 Tarjan 栈中；init 清0，标记取0/1，入栈置 true，弹栈置
    //  false。
    vector<int> ins;
    // bel：一维；bel[u] 为 SCC 编号 1..cnt；init 清 0；0 表示尚未分配。
    vector<int> bel;
    // stk：一维栈，元素为顶点编号 1..n；init 弹空；最多存 n 个点。
    stack<int> stk;
    // scc：二维变长表；scc[c-1] 为 SCC c 的全部原图顶点；init clear；总成员数 n。
    vector<vector<int>> scc;

    // init(n_)：参数为顶点数；返回 void；清图和 Tarjan 状态。
    // 前提：n_>=1，下一步重新加边；首次初始化时间O(n_)；复用对象还需销毁旧图的邻接元素。
```

```
    void init(int n_) {
        n = n_;
        idx = cnt = 0;
        e.assign(n + 1, {});
        dfn.assign(n + 1, 0);
        low.assign(n + 1, 0);
        ins.assign(n + 1, 0);
        bel.assign(n + 1, 0);
        while (!stk.empty())
            stk.pop();
        scc.clear();
    }

    // dfs(u): u 为当前顶点；返回 void；递归求 SCC 并填写 bel、scc。
    // 前提：图已建好，dfn[u]==0；一次 DFS 为 O(所访问顶点+扫描边)，全图 O(n+m)。
    void dfs(int u) {
        dfn[u] = low[u] = ++idx;
        ins[u] = true;
        stk.push(u);
        for (int v : e[u]) { // v：出边终点，范围 1..n。
            if (!dfn[v]) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (ins[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (dfn[u] == low[u]) {
            ++cnt;
            vector<int> c; // 一维临时点集，初始为空；存当前 SCC 顶点，大小至少 1。
            while (true) {
                int v = stk.top(); // v：当前弹出顶点，1..n；最后一次必为 u。
                stk.pop();
                ins[v] = false;
                bel[v] = cnt;
                c.push_back(v);
                if (v == u)
                    break;
            }
            scc.push_back(c);
        }
    }

    // run(): 无参数，返回 void；遍历 DFS 森林，完成全图 SCC。
    // 前提：已 init 并加边，尚未运行；时间 O(n+m)，递归栈最坏 O(n)。
    void run() {
        for (int u = 1; u <= n; u++)
            if (!dfn[u])
                dfs(u);
    }

    // addedge(u,v)：参数为有向边起终点1..n；返回void；前提已init且尚未run。
    // 作用：只封装原PPT邻接表push_back；摊还时间O(1)。
    void addedge(int u, int v) {
        e[u].push_back(v);
    }
};

```

```cpp
void solve() {
    int n, m;
    cin >> n >> m;
    SCC scc;
    scc.init(n);
    vector<pair<int, int>> e(m);
    for (auto &[u, v] : e) {
        cin >> u >> v;
        scc.addedge(u, v);
    }
    scc.run();
    vector<int> out(scc.cnt + 1), sz(scc.cnt + 1);
    for (int u = 1; u <= n; u++)
        sz[scc.bel[u]]++;
    for (auto [u, v] : e)
        if (scc.bel[u] != scc.bel[v])
            out[scc.bel[u]]++;
    int ans = 0, count = 0;
    for (int c = 1; c <= scc.cnt; c++)
        if (!out[c]) {
            count++;
            ans = sz[c];
        }
    cout << (count == 1 ? ans : 0) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

init(n)：清空旧图

addedge(u,v)：有向边

run()：求SCC，每次建图仅运行一次

bel[u]/cnt/scc：归属编号、数量与分量成员

例题 1 P2341 受欢迎的牛

https://www.luogu.com.cn/problem/P2341

题意：找被所有点可达的点数。

观察与建模：找被所有点可达的点数。 缩点DAG唯一出度0分量中的所有点满足；多于一个则无解。

为何对应此算法：缩点DAG唯一出度0分量中的所有点满足；多于一个则无解。

初始化、组织数据与调用：run后枚举原边计算分量出度，检查唯一汇分量。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m)时间与空间。

复用模板：Tarjan强连通分量（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    SCC scc;
    scc.init(n);
    vector<pair<int, int>> e(m);
    for (auto &[u, v] : e) {
        cin >> u >> v;
        scc.addedge(u, v);
    }
    scc.run();
    vector<int> out(scc.cnt + 1), sz(scc.cnt + 1);
    for (int u = 1; u <= n; u++)
        sz[scc.bel[u]]++;
    for (auto [u, v] : e)
        if (scc.bel[u] != scc.bel[v])
            out[scc.bel[u]]++;
    int ans = 0, count = 0;
    for (int c = 1; c <= scc.cnt; c++)
        if (!out[c]) {
            count++;
            ans = sz[c];
        }
    cout << (count == 1 ? ans : 0) << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P3387 缩点

https://www.luogu.com.cn/problem/P3387

题意：有向图走一条路线，点权最多可取得多少；每点只计一次。

观察与建模：有向图走一条路线，点权最多可取得多少；每点只计一次。 正权SCC内可遍历全部点，缩为DAG做最长路。

为何对应此算法：正权SCC内可遍历全部点，缩为DAG做最长路。

初始化、组织数据与调用：run、汇总点权、建立分量边，用Kahn顺序转移dp。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m)时间与空间。

复用模板：Tarjan强连通分量（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    SCC scc;
    scc.init(n);
    vector<pair<int, int>> e(m);
```

```
    for (auto &[u, v] : e) {
        cin >> u >> v;
        scc.addedge(u, v);
    }
    scc.run();
    int k = scc.cnt;
    vector<int> w(k + 1), deg(k + 1);
    vector<vector<int>> g(k + 1);
    for (int i = 1; i <= n; i++)
        w[scc.bel[i]] += a[i];
    for (auto [u, v] : e) {
        u = scc.bel[u];
        v = scc.bel[v];
        if (u != v) {
            g[u].push_back(v);
            deg[v]++;
        }
    }
    queue<int> q;
    vector<int> dp = w;
    for (int c = 1; c <= k; c++)
        if (!deg[c])
            q.push(c);
    int ans = 0;
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        ans = max(ans, dp[u]);
        for (int v : g[u]) {
            dp[v] = max(dp[v], dp[u] + w[v]);
            if (!--deg[v])
                q.push(v);
        }
    }
    cout << ans << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- Tarjan只使用仍在栈中的点更新low；Kosaraju第二遍走反图。
- 两算法分量编号顺序不同，不把编号当统一拓扑序；应用显式拓扑排序。
- 深链递归有RE风险。

个人资料来源：图/图论竞赛复习_严格课件注释版.md；图论/SCC.md

图论 / 二分图染色

类型与用途：模板型；判断无向图是否可二染色，并统计每个连通块的两色数量。

复杂度与适用范围：O(n+m)时间与空间。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 二分图染色：判断无向图是否可二染色，并统计每个连通块的两色数量。
// O(n+m)时间与空间。
struct BipColor {
    int n;
    vector<vector<int>> e;
    vector<int> col;
    vector<array<int, 2>> cnt;
    void init(int n_) {
        n = n_;
        e.assign(n + 1, {});
        col.assign(n + 1, -1);
        cnt.clear();
    }
    void addedge(int u, int v) {
        e[u].push_back(v);
        e[v].push_back(u);
    }
    bool dfs(int u, int c, array<int, 2> &num) {
        col[u] = c;
        num[c]++;
        for (int v : e[u]) {
            if (col[v] == -1) {
                if (!dfs(v, c ^ 1, num))
                    return false;
            } else if (col[v] == c)
                return false;
        }
        return true;
    }
    bool run() {
        fill(col.begin(), col.end(), -1);
        cnt.clear();
        for (int u = 1; u <= n; u++)
            if (col[u] == -1) {
                array<int, 2> num = {0, 0};
                if (!dfs(u, 0, num))
                    return false;
                cnt.push_back(num);
            }
        return true;
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> a(m), b(m);
```

```
    for (int &x : a)
        cin >> x;
    for (int &x : b)
        cin >> x;
    BipColor g;
    g.init(n);
    for (int i = 0; i < m; i++)
        g.addedge(a[i], b[i]);
    cout << (g.run() ? "Yes" : "No") << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

init(n)：颜色-1代表未访问

addedge(u,v)：无向边

run()：不可染返回false；cnt记录各块两色数

例题 1 ABC327D Good Tuple Problem

https://atcoder.jp/contests/abc327/tasks/abc327_d

题意：给定成对约束X[Ai]!=X[Bi]，判断是否存在0/1赋值。

观察与建模：给定成对约束X[Ai]!=X[Bi]，判断是否存在0/1赋值。 每对不等约束就是边的两端必须不同色。

为何对应此算法：每对不等约束就是边的两端必须不同色。

初始化、组织数据与调用：分别读取A和B数组后addedge，run输出Yes/No。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m)时间与空间。

复用模板：二分图染色（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> a(m), b(m);
    for (int &x : a)
        cin >> x;
    for (int &x : b)
        cin >> x;
    BipColor g;
    g.init(n);
    for (int i = 0; i < m; i++)
        g.addedge(a[i], b[i]);
    cout << (g.run() ? "Yes" : "No") << '\n';
}
```

验证：C++17组合程序编译通过；3组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P1330 封锁阳光大学

https://www.luogu.com.cn/problem/P1330

题意：选择点封锁边，并要求相连点不能同时被选，最少选多少。

观察与建模：选择点封锁边，并要求相连点不能同时被选，最少选多少。 每条边恰有一端被选，每个块只能选二分图的一侧；取较小侧。

为何对应此算法：每条边恰有一端被选，每个块只能选二分图的一侧；取较小侧。

初始化、组织数据与调用：run判奇环，无解Impossible；累加min(cnt[0],cnt[1])。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m)时间与空间。

复用模板：二分图染色（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    BipColor g;
    g.init(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v);
    }
    if (!g.run()) {
        cout << "Impossible\n";
        return;
    }
    int ans = 0;
    for (auto c : g.cnt)
        ans += min(c[0], c[1]);
    cout << ans << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 遍历全部连通块；自环必然矛盾。
- 深链递归有RE风险，运行环境需足够栈。

个人资料来源：图论/二分图染色.cpp

图论 / 倍增LCA

类型与用途：模板型；有根树最近公共祖先与边数距离；1-based，0为虚祖先。

复杂度与适用范围：预处理O(n log n)，查询O(log n)，空间O(n log n)。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 倍增LCA：有根树最近公共祖先与边数距离；1-based，0为虚祖先。
// 预处理O(n log n)，查询O(log n)，空间O(n log n)。
struct LCA {
    int n, LOG;
    vector<vector<int>> e, fa;
    vector<int> dep;
    void init(int n_) {
        n = n_;
        LOG = 1;
        while ((1LL << LOG) <= n)
            LOG++;
        e.assign(n + 1, {});
        fa.assign(LOG, vector<int>(n + 1));
        dep.assign(n + 1, 0);
    }
    void addedge(int u, int v) {
        e[u].push_back(v);
        e[v].push_back(u);
    }
    void dfs(int u, int f) {
        fa[0][u] = f;
        dep[u] = dep[f] + 1;
        for (int j = 1; j < LOG; j++)
            fa[j][u] = fa[j - 1][fa[j - 1][u]];
        for (int v : e[u])
            if (v != f)
                dfs(v, u);
    }
    void build(int root = 1) {
        dfs(root, 0);
    }
    int lca(int u, int v) {
        if (dep[u] < dep[v])
            swap(u, v);
        int d = dep[u] - dep[v];
        for (int j = 0; j < LOG; j++)
            if (d >> j & 1)
                u = fa[j][u];
        if (u == v)
            return u;
        for (int j = LOG - 1; j >= 0; j--)
            if (fa[j][u] != fa[j][v]) {
                u = fa[j][u];
                v = fa[j][v];
            }
        return fa[0][u];
    }
```

```
    int distance(int u, int v) {
        int p = lca(u, v);
        return dep[u] + dep[v] - 2 * dep[p];
    }
};

void solve() {
    int n, q, s;
    cin >> n >> q >> s;
    LCA tr;
    tr.init(n);
    for (int i = 1, u, v; i < n; i++) {
        cin >> u >> v;
        tr.addedge(u, v);
    }
    tr.build(s);
    while (q--) {
        int u, v;
        cin >> u >> v;
        cout << tr.lca(u, v) << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

init(n)：按n选择倍增层数

build(root)：树上递归预处理

lca(u,v)：最近公共祖先

distance(u,v)：无权树距离

例题 1 P3379 最近公共祖先

https://www.luogu.com.cn/problem/P3379

题意：指定根，回答LCA。

观察与建模：指定根，回答LCA。 读入树并按指定根预处理倍增表。

为何对应此算法：读入树并按指定根预处理倍增表。

初始化、组织数据与调用：init、addedge、build(s)、lca。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O((n+q)log n)时间、O(n log n)空间。

复用模板：倍增LCA（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, q, s;
    cin >> n >> q >> s;
    LCA tr;
    tr.init(n);
    for (int i = 1, u, v; i < n; i++) {
        cin >> u >> v;
        tr.addedge(u, v);
    }
    tr.build(s);
    while (q--) {
        int u, v;
        cin >> u >> v;
        cout << tr.lca(u, v) << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF1328E Tree Queries

https://codeforces.com/problemset/problem/1328/E

题意：若干点各自至多向父移动一次，能否落在同一条根到某点路径上。

观察与建模：若干点各自至多向父移动一次，能否落在同一条根到某点路径上。 把点移到父后，所有点需成为最深点的祖先；用LCA判祖先。

为何对应此算法：把点移到父后，所有点需成为最深点的祖先；用LCA判祖先。

初始化、组织数据与调用：根的父替换为自身；取最深点v，检查lca(u,v)==u。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n log n+总查询点数log n)时间、O(n log n)空间。

复用模板：倍增LCA（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, q;
    cin >> n >> q;
    LCA tr;
    tr.init(n);
    for (int i = 1, u, v; i < n; i++) {
        cin >> u >> v;
        tr.addedge(u, v);
    }
    tr.build();
    while (q--) {
        int k;
        cin >> k;
        vector<int> a(k);
        int v = 1;
        for (int &u : a) {
            cin >> u;
            if (tr.dep[u] > tr.dep[v])
                v = u;
        }
```

```
        bool ok = true;
        for (int u : a) {
            u = u == 1 ? 1 : tr.fa[0][u];
            if (tr.lca(u, v) != u)
                ok = false;
        }
        cout << (ok ? "YES" : "NO") << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 仅在树上使用；图中环会导致递归失控。

- LOG包括足够层数；深链最大n=5e5存在栈风险。

个人资料来源：图论/倍增求lca.cpp

图论 / 割点

类型与用途：模板型；无向图割点与删点分块大小统计；根与非根分开判定。

复杂度与适用范围：O(n+m)时间与空间。

复用模板：割边（本册第二部分）

cut[u]：割点标记

parts[u]：low[v]>=dfn[u]的子树大小

sz[u]：DFS子树大小

例题 1 P3388 割点

https://www.luogu.com.cn/problem/P3388

题意：输出全部割点。

观察与建模：输出全部割点。 Tarjan low判定去点后有子树无法连回祖先。

为何对应此算法：Tarjan low判定去点后有子树无法连回祖先。

初始化、组织数据与调用：run遍历非连通图；按cut扫描。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m)时间与空间。

复用模板：割边（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    LowLink g;
    g.init(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v);
    }
    g.run();
    vector<int> ans;
```

```
    for (int u = 1; u <= n; u++)
        if (g.cut[u])
            ans.push_back(u);
    cout << ans.size() << '\n';
    for (int u : ans)
        cout << u << ' ';
    cout << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P3469 BLO-Blockade

https://www.luogu.com.cn/problem/P3469

题意：分别封锁每个点的所有边，统计不连通有序点对。

观察与建模：分别封锁每个点的所有边，统计不连通有序点对。 原图连通，删边后分块；总点对减去各块内部点对。

为何对应此算法：原图连通，删边后分块；总点对减去各块内部点对。

初始化、组织数据与调用：利用parts[u]与剩余块n-1-sum(parts)，再加2(n-1)。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m)时间与空间。

复用模板：割边（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    LowLink g;
    g.init(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v);
    }
    g.run();
    for (int u = 1; u <= n; u++) {
        int rest = n - 1, ans = 2 * (n - 1) + (n - 1) * (n - 1);
        for (int s : g.parts[u]) {
            rest -= s;
            ans -= s * s;
        }
        ans -= rest * rest;
        cout << ans << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 根需至少两个DFS儿子；非根条件为low[v]>=dfn[u]。
- BLO删除边后仍保留被隔离顶点，必须计入涉及该点的有序点对。

个人资料来源：图/图论竞赛复习_严格课件注释版.md

图论 / 割边

类型与用途：模板型；无向图桥检测，按唯一边号跳过父边；支持重边和自环。

复杂度与适用范围：O(n+m)时间与空间。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 割边：无向图桥检测，按唯一边号跳过父边；支持重边和自环。
// O(n+m)时间与空间。
struct LowLink {
    int n, idx, cnt;
    vector<vector<pair<int, int>>> e;
    vector<pair<int, int>> edges;
    vector<int> dfn, low, sz, cut, bridge, bel;
    vector<vector<int>> parts, bcc;
    void init(int n_) {
        n = n_;
        idx = cnt = 0;
        e.assign(n + 1, {});
        edges.clear();
        dfn.assign(n + 1, 0);
        low = dfn;
        sz = dfn;
        cut = dfn;
        bel = dfn;
        parts.assign(n + 1, {});
        bridge.clear();
        bcc.clear();
    }
    void addedge(int u, int v) {
        int id = edges.size();
        edges.push_back({u, v});
        bridge.push_back(0);
        e[u].push_back({v, id});
        e[v].push_back({u, id});
    }
    void dfs(int u, int id) {
        dfn[u] = low[u] = ++idx;
        sz[u] = 1;
        int ch = 0;
        for (auto [v, id2] : e[u]) {
            if (id2 == id)
                continue;
            if (!dfn[v]) {
                ch++;
                dfs(v, id2);
                sz[u] += sz[v];
                low[u] = min(low[u], low[v]);
                if (low[v] > dfn[u])
                    bridge[id2] = 1;
                if (low[v] >= dfn[u]) {
                    parts[u].push_back(sz[v]);
                    if (id != -1)
                        cut[u] = 1;
```

```
                }
            } else
                low[u] = min(low[u], dfn[v]);
        }
        if (id == -1 && ch >= 2)
            cut[u] = 1;
    }
    void paint(int u) {
        bel[u] = cnt;
        bcc.back().push_back(u);
        for (auto [v, id] : e[u])
            if (!bridge[id] && !bel[v])
                paint(v);
    }
    void run() {
        for (int u = 1; u <= n; u++)
            if (!dfn[u])
                dfs(u, -1);
        for (int u = 1; u <= n; u++)
            if (!bel[u]) {
                cnt++;
                bcc.push_back({});
                paint(u);
            }
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    LowLink g;
    g.init(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v);
    }
    g.run();
    vector<pair<int, int>> ans;
    for (int i = 0; i < (int)g.edges.size(); i++)
        if (g.bridge[i]) {
            auto [u, v] = g.edges[i];
            if (u > v)
                swap(u, v);
            ans.push_back({u, v});
        }
    sort(ans.begin(), ans.end());
    for (auto [u, v] : ans)
        cout << u << ' ' << v << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

addedge(u,v)：自动分配0-based边号

run()：dfs森林；bridge[id]标记桥

edges/e：原边端点与邻接表

例题 1 P1656 炸铁路

https://www.luogu.com.cn/problem/P1656

题意：输出删除后使图不连通的铁路。

观察与建模：输出删除后使图不连通的铁路。 这正是桥的定义。

为何对应此算法：这正是桥的定义。

初始化、组织数据与调用：run后筛bridge，端点排序后按字典序输出。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m+k log k)时间、O(n+m)空间。

复用模板：割边（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    LowLink g;
    g.init(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v);
    }
    g.run();
    vector<pair<int, int>> ans;
    for (int i = 0; i < (int)g.edges.size(); i++)
        if (g.bridge[i]) {
            auto [u, v] = g.edges[i];
            if (u > v)
                swap(u, v);
            ans.push_back({u, v});
        }
    sort(ans.begin(), ans.end());
    for (auto [u, v] : ans)
        cout << u << ' ' << v << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF118E Bertown Roads

https://codeforces.com/problemset/problem/118/E

题意：把连通无向图定向为强连通图。

观察与建模：把连通无向图定向为强连通图。 存在桥时不可能；无桥时DFS树边向下、返祖边向上可得强连通方向。

为何对应此算法：存在桥时不可能；无桥时DFS树边向下、返祖边向上可得强连通方向。

初始化、组织数据与调用：先run判桥；沿dfn更早到更晚识别树边，另一次DFS每边只定向一次。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m)时间与空间。

复用模板：割边（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    LowLink g;
    g.init(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v);
    }
    g.run();
    for (int b : g.bridge)
        if (b) {
            cout << 0 << '\n';
            return;
        }
    vector<int> vis(n + 1), used(g.edges.size());
    vector<pair<int, int>> ans(g.edges.size());
    function<void(int)> dfs = [&](int u) {
        vis[u] = 1;
        for (auto [v, id] : g.e[u])
            if (!used[id]) {
                used[id] = 1;
                ans[id] = {u, v};
                if (!vis[v])
                    dfs(v);
            }
    };
    dfs(1);
    for (auto [u, v] : ans)
        cout << u << ' ' << v << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 跳过父边号而非父节点，否则重边会误判。
- bridge判low[v]>dfn[u]，严格大于。
- 深链递归有栈风险。

个人资料来源：图/图论竞赛复习_严格课件注释版.md

图论 / 差分约束

类型与用途：模板型；一般上界差分约束用Bellman-Ford判负环；非正权上界特例提供SCC+DAG最小正值变体。

复杂度与适用范围：solve为O(nm)时间；minimum_positive为O(n+m)时间；空间O(n+m)。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 差分约束：一般上界差分约束用Bellman-Ford判负环；非正权上界特例提供SCC+DAG最小正值变体。
// solve为O(nm)时间；minimum_positive为O(n+m)时间；空间O(n+m)。
struct SCC {
    // 以下顶点表均由init按n+1项assign重建；顶点下标1..n，0号预留不用。
    // 内层邻接位置从0开始，init清空旧边；标记/时间戳/归属表清0。
    int n;   // 顶点数，n>=1；顶点编号为 1..n。
    int idx; // DFS 时钟，范围 [0,n]；每首次访问一个点加 1。
    int cnt; // SCC 数，范围 [0,n]；按弹栈先后编号。

    // e：二维变长邻接表；e[u][j] 为 u 的第 j 条出边终点。
    // u=1..n，j=0..e[u].size()-1；init 中 clear；允许重边、自环。
    vector<vector<int>> e;
    // dfn：一维；dfn[u] 为首次访问时间，0 表示未访问；init 清 0；有效值 1..n。
    vector<int> dfn;
    // low：一维；low[u] 为子树按 Tarjan 规则能追溯到的最早未封闭时间戳。
    // 下标 1..n；init 清 0；首次访问置 dfn[u]，随后取 min。
    vector<int> low;
    // ins：一维；ins[u] 为 u 是否仍在 Tarjan 栈中；init 清0，标记取0/1，入栈置 true，弹栈置
    //  false。
    vector<int> ins;
    // bel：一维；bel[u] 为 SCC 编号 1..cnt；init 清 0；0 表示尚未分配。
    vector<int> bel;
    // stk：一维栈，元素为顶点编号 1..n；init 弹空；最多存 n 个点。
    stack<int> stk;
    // scc：二维变长表；scc[c-1] 为 SCC c 的全部原图顶点；init clear；总成员数 n。
    vector<vector<int>> scc;

    // init(n_)：参数为顶点数；返回 void；清图和 Tarjan 状态。
    // 前提：n_>=1，下一步重新加边；首次初始化时间O(n_)；复用对象还需销毁旧图的邻接元素。
    void init(int n_) {
        n = n_;
        idx = cnt = 0;
        e.assign(n + 1, {});
        dfn.assign(n + 1, 0);
        low.assign(n + 1, 0);
        ins.assign(n + 1, 0);
        bel.assign(n + 1, 0);
        while (!stk.empty())
            stk.pop();
        scc.clear();
    }

    // dfs(u)：u 为当前顶点；返回 void；递归求 SCC 并填写 bel、scc。
    // 前提：图已建好，dfn[u]==0；一次 DFS 为 O(所访问顶点+扫描边)，全图 O(n+m)。
```

```
    void dfs(int u) {
        dfn[u] = low[u] = ++idx;
        ins[u] = true;
        stk.push(u);
        for (int v : e[u]) { // v：出边终点，范围 1..n。
            if (!dfn[v]) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (ins[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (dfn[u] == low[u]) {
            ++cnt;
            vector<int> c; // 一维临时点集，初始为空；存当前 SCC 顶点，大小至少 1。
            while (true) {
                int v = stk.top(); // v：当前弹出顶点，1..n；最后一次必为 u。
                stk.pop();
                ins[v] = false;
                bel[v] = cnt;
                c.push_back(v);
                if (v == u)
                    break;
            }
            scc.push_back(c);
        }
    }

    // run()：无参数，返回 void；遍历 DFS 森林，完成全图 SCC。
    // 前提：已 init 并加边，尚未运行；时间 O(n+m)，递归栈最坏 O(n)。
    void run() {
        for (int u = 1; u <= n; u++)
            if (!dfn[u])
                dfs(u);
    }

    // addedge(u,v)：参数为有向边起终点1..n；返回void；前提已init且尚未run。
    // 作用：只封装原PPT邻接表push_back；摊还时间O(1)。
    void addedge(int u, int v) {
        e[u].push_back(v);
    }
};

struct Difference {
    struct edge {
        int u, v, w;
    };
    int n;
    vector<edge> e;
    vector<int> x;
    void init(int n_) {
        n = n_;
        e.clear();
        x.assign(n + 1, 0);
    }
    // 加入x[v]-x[u]<=w，未知数1..n。
    void add_constraint(int u, int v, int w) {
        e.push_back({u, v, w});
    }
```

```
    bool solve() {
        fill(x.begin(), x.end(), 0);
        for (int i = 1; i <= n; i++) {
            bool changed = false;
            for (auto [u, v, w] : e)
                if (x[v] > x[u] + w) {
                    x[v] = x[u] + w;
                    changed = true;
                    if (i == n)
                        return false;
                }
            if (!changed)
                return true;
        }
        return true;
    }
    // 特例：所有w<=0；求a[i]>=1且a[v]>=a[u]-w的最小解，x=-a。
    bool minimum_positive() {
        SCC scc;
        scc.init(n);
        for (auto [u, v, w] : e) {
            assert(w <= 0);
            scc.addedge(u, v);
        }
        scc.run();
        int k = scc.cnt;
        vector<vector<pair<int, int>>> g(k + 1);
        vector<int> d(k + 1), ans(k + 1, 1);
        for (auto [u, v, w] : e) {
            u = scc.bel[u];
            v = scc.bel[v];
            if (u == v) {
                if (w < 0)
                    return false;
            } else {
                g[u].push_back({v, -w});
                d[v]++;
            }
        }
        queue<int> q;
        for (int u = 1; u <= k; u++)
            if (!d[u])
                q.push(u);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto [v, w] : g[u]) {
                ans[v] = max(ans[v], ans[u] + w);
                if (!--d[v])
                    q.push(v);
            }
        }
        for (int u = 1; u <= n; u++)
            x[u] = -ans[scc.bel[u]];
        return true;
    }
};

```

```
void solve() {
    int n, m;
    cin >> n >> m;
    Difference g;
    g.init(n);
    while (m--) {
        int c, d, y;
        cin >> c >> d >> y;
        g.add_constraint(d, c, y);
    }
    if (!g.solve()) {
        cout << "NO\n";
        return;
    }
    for (int i = 1; i <= n; i++)
        cout << g.x[i] << (i == n ? '\n' : ' ');
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

add_constraint(u,v,w)：x[v]-x[u]<=w

solve()：任意可行x或false

minimum_positive()：仅w<=0；返回x=-a且a[i]>=1的最小解

例题 1 P5960 差分约束

https://www.luogu.com.cn/problem/P5960

题意：求形如x[c]-x[c2]<=y的不等式组可行解。

观察与建模：求形如x[c]-x[c2]<=y的不等式组可行解。 边c2->c权y，用全零初值等价于超级源连到全部点。

为何对应此算法：边c2->c权y，用全零初值等价于超级源连到全部点。

初始化、组织数据与调用：add_constraint(c2,c,y)，solve后输出x。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(nm)时间、O(n+m)空间。

复用模板：差分约束（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    Difference g;
    g.init(n);
    while (m--) {
        int c, d, y;
        cin >> c >> d >> y;
        g.add_constraint(d, c, y);
```

```
    }
    if (!g.solve()) {
        cout << "NO\n";
        return;
    }
    for (int i = 1; i <= n; i++)
        cout << g.x[i] << (i == n ? '\n' : ' ');
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P3275 糖果

https://www.luogu.com.cn/problem/P3275

题意：每人至少1颗糖，满足五类大小关系，最少糖果总数。

观察与建模：每人至少1颗糖，满足五类大小关系，最少糖果总数。 取x=−糖果数后全部边权0或−1；同SCC负边意味着矛盾，缩点最长路求最小糖果。

为何对应此算法：取x=−糖果数后全部边权0或−1；同SCC负边意味着矛盾，缩点最长路求最小糖果。

初始化、组织数据与调用：映射五类约束后minimum_positive；输出-sum(x)。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m)时间与空间。

复用模板：差分约束（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    Difference g;
    g.init(n);
    while (m--) {
        int op, a, b;
        cin >> op >> a >> b;
        if (op == 1) {
            g.add_constraint(a, b, 0);
            g.add_constraint(b, a, 0);
        } else if (op == 2)
            g.add_constraint(a, b, -1);
        else if (op == 3)
            g.add_constraint(b, a, 0);
        else if (op == 4)
            g.add_constraint(b, a, -1);
        else
            g.add_constraint(a, b, 0);
    }
    if (!g.minimum_positive()) {
        cout << -1 << '\n';
        return;
    }
    int ans = 0;
    for (int i = 1; i <= n; i++)
        ans -= g.x[i];
    cout << ans << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 先写不等式再确定边方向；严格整数差用1而非浮点epsilon。
- 一般有正/负权时不能用SCC特例；大规模一般图Bellman-Ford会TLE。

个人资料来源：图论/差分约束.cpp；图/图论竞赛复习_严格课件注释版.md

图论 / 拓扑排序

类型与用途：模板型；Kahn拓扑序，保留原队列写法；可选最小堆得到字典序最小拓扑序。

复杂度与适用范围：队列O(n+m)，字典序O((n+m)log n)，空间O(n+m)。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 拓扑排序：Kahn拓扑序，保留原队列写法；可选最小堆得到字典序最小拓扑序。
// 队列O(n+m)，字典序O((n+m)log n)，空间O(n+m)。
struct Topo {
    int n;
    vector<vector<int>> edge;
    vector<int> d;
    void init(int n_) {
        n = n_;
        edge.assign(n + 1, {});
        d.assign(n + 1, 0);
    }
    void addedge(int u, int v) {
        edge[u].push_back(v);
        d[v]++;
    }
    vector<int> toposort(bool lex = false) {
        vector<int> deg = d, res;
        queue<int> q;
        priority_queue<int, vector<int>, greater<int>> pq;
        for (int i = 1; i <= n; i++)
            if (!deg[i]) {
                if (lex)
                    pq.push(i);
                else
                    q.push(i);
            }
        while (lex ? !pq.empty() : !q.empty()) {
            int u;
            if (lex) {
                u = pq.top();
                pq.pop();
            } else {
                u = q.front();
                q.pop();
            }
            res.push_back(u);
            for (int v : edge[u])
```

```
                if (!--deg[v]) {
                    if (lex)
                        pq.push(v);
                    else
                        q.push(v);
                }
        }
        return res;
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    Topo g;
    g.init(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v);
    }
    vector<int> dp(n + 1, 1);
    for (int u : g.toposort())
        for (int v : g.edge[u])
            dp[v] = max(dp[v], dp[u] + 1);
    for (int i = 1; i <= n; i++)
        cout << dp[i] << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

toposort(false)：返回拓扑序，少于n表示有环

toposort(true)：字典序最小

d：原始入度保留，重复调用不消耗

例题 1 P1137 旅行计划

https://www.luogu.com.cn/problem/P1137

题意：DAG中每个终点最多经过多少城市。

观察与建模：DAG中每个终点最多经过多少城市。 拓扑顺序做最长路，dp[v]=max(dp[v],dp[u]+1)。

为何对应此算法：拓扑顺序做最长路，dp[v]=max(dp[v],dp[u]+1)。

初始化、组织数据与调用：按toposort遍历，输出每点dp。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m)时间与空间。

复用模板：拓扑排序（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    Topo g;
    g.init(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v);
    }
    vector<int> dp(n + 1, 1);
    for (int u : g.toposort())
        for (int v : g.edge[u])
            dp[v] = max(dp[v], dp[u] + 1);
    for (int i = 1; i <= n; i++)
        cout << dp[i] << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 ABC223D Restricted Permutation

https://atcoder.jp/contests/abc223/tasks/abc223_d

题意：给先后约束，求字典序最小排列，无解-1。

观察与建模：给先后约束，求字典序最小排列，无解-1。 每步取当前所有零入度点中的最小点。

为何对应此算法：每步取当前所有零入度点中的最小点。

初始化、组织数据与调用：toposort(true)，长度<n判环。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O((n+m)log n)时间、O(n+m)空间。

复用模板：拓扑排序（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    Topo g;
    g.init(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v);
    }
    auto ans = g.toposort(true);
    if ((int)ans.size() != n) {
        cout << -1 << '\n';
        return;
    }
    for (int i = 0; i < n; i++)
        cout << ans[i] << (i + 1 == n ? '\n' : ' ');
}
```

验证：C++17组合程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 一般图不能强行假定DAG；必须检查返回长度。
- 多条重复边各自计入度，扫描时也各自减1。

个人资料来源：图论/拓扑.cpp

图论 / 无向欧拉路径

类型与用途：模板型；唯一边号标记、当前弧DFS，返回字典序最小无向欧拉路径。

复杂度与适用范围：O(m log m+n)时间、O(n+m)空间。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 无向欧拉路径：唯一边号标记、当前弧DFS，返回字典序最小无向欧拉路径。
// O(m log m+n)时间、O(n+m)空间。
struct Euler {
    struct edge {
        int v, id;
    };
    int n, m;
    vector<vector<edge>> adj;
    vector<int> deg, cur, vis, path;
    void init(int n_) {
        n = n_;
        m = 0;
        adj.assign(n + 1, {});
        deg.assign(n + 1, 0);
        cur = deg;
        vis.clear();
        path.clear();
    }
    void addedge(int u, int v) {
        adj[u].push_back({v, m});
        adj[v].push_back({u, m});
        deg[u]++;
        deg[v]++;
        vis.push_back(0);
        m++;
    }
    void dfs(int u) {
        for (int &i = cur[u]; i < (int)adj[u].size();) {
            auto [v, id] = adj[u][i++];
            if (vis[id])
                continue;
            vis[id] = 1;
            dfs(v);
        }
        path.push_back(u);
    }
    bool run() {
        int st = 0, odd = 0;
        for (int u = 1; u <= n; u++)
```

```
            if (deg[u] % 2) {
                odd++;
                if (!st)
                    st = u;
            }
        if (odd != 0 && odd != 2)
            return false;
        if (!st)
            for (int u = 1; u <= n; u++)
                if (deg[u]) {
                    st = u;
                    break;
                }
        if (!st)
            st = 1;
        for (int u = 1; u <= n; u++)
            sort(adj[u].begin(), adj[u].end(),
                 [](edge a, edge b) { return a.v < b.v; });
        fill(cur.begin(), cur.end(), 0);
        fill(vis.begin(), vis.end(), 0);
        path.clear();
        dfs(st);
        reverse(path.begin(), path.end());
        return (int)path.size() == m + 1;
    }
};

void solve() {
    int m;
    cin >> m;
    Euler g;
    g.init(500);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v);
    }
    g.run();
    for (int u : g.path)
        cout << u << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

addedge(u,v)：同一无向边的两向共享id

run()：检查奇度数量及全边覆盖

path：从最小可行起点出发

例题 1 P2731 Riding the Fences

https://www.luogu.com.cn/problem/P2731

题意：按字典序输出经过每条栅栏一次的路线。

观察与建模：按字典序输出经过每条栅栏一次的路线。 栅栏是无向多重边，选择最小奇度点或最小有边点出发。

为何对应此算法：栅栏是无向多重边，选择最小奇度点或最小有边点出发。

初始化、组织数据与调用：init(500)、addedge、run，按点逐行输出。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(m log m+500)时间、O(m+500)空间。

复用模板：无向欧拉路径（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int m;
    cin >> m;
    Euler g;
    g.init(500);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v);
    }
    g.run();
    for (int u : g.path)
        cout << u << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P1341 无序字母对

https://www.luogu.com.cn/problem/P1341

题意：把无序字母对拼成字典序最小字符串，每对用一次。

观察与建模：把无序字母对拼成字典序最小字符串，每对用一次。 字母是节点，字母对是无向边；字符ASCII顺序就是要求的字典序。

为何对应此算法：字母是节点，字母对是无向边；字符ASCII顺序就是要求的字典序。

初始化、组织数据与调用：字符号加1避免节点0，run后逐点输出字符。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(m log m+128)时间、O(m+128)空间。

复用模板：无向欧拉路径（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int m;
    cin >> m;
    Euler g;
    g.init(128);
    while (m--) {
        string s;
        cin >> s;
        g.addedge((unsigned char)s[0] + 1, (unsigned char)s[1] + 1);
    }
```

```
    if (!g.run()) {
        cout << "No Solution\n";
        return;
    }
    for (int u : g.path)
        cout << char(u - 1);
    cout << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 不能仅标记某一方向。
- 有边点须在同一个连通块；用路径长度验证。
- 深度可达m，保留递归并明确栈限制。

个人资料来源：图论/欧拉路径.md

图论 / 最小费用最大流

类型与用途：模板型；逐次最短增广；费用势使残量边非负，用Dijkstra取最便宜增广路，支持无负环的负费用初始网络。

复杂度与适用范围：初始非负费用免预处理；有负费用时O(VE)建势，之后O(A*E log V)，A为增广次数；O(V+E)空间。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 最小费用最大流：逐次最短增广；费用势使残量边非负，用Dijkstra取最便宜增广路，支持无负环的负费
// 用初始网络。
// 初始非负费用免预处理；有负费用时O(VE)建势，之后O(A*E log V)，A为增广次数；O(V+E)空间。
struct MinCostFlow {
    static constexpr int INF = 4000000000000000000LL;
    struct edge {
        int v, f, c;
    };
    int n;
    vector<edge> e;
    vector<vector<int>> g;
    vector<int> h, dis, pre;
    MinCostFlow(int n = 0) {
        init(n);
    }
    void init(int n_) {
        n = n_;
        e.clear();
        g.assign(n + 1, {});
        h.assign(n + 1, 0);
        dis = pre = h;
    }
    void add_edge(int u, int v, int f, int c) {
        int id = e.size();
        g[u].push_back(id);
        g[v].push_back(id ^ 1);
```

```
        e.push_back({v, f, c});
        e.push_back({u, 0, -c});
    }
    pair<int, int> run(int s, int t, int limit = INF) {
        assert(s != t);
        // 零势若已可行，无需Bellman-Ford；负费用时先建立全局可行势。
        fill(h.begin(), h.end(), 0);
        bool negative = false;
        for (auto z : e)
            if (z.f > 0 && z.c < 0)
                negative = true;
        if (negative)
            for (int k = 1; k <= n; k++) {
                bool changed = false;
                for (int u = 1; u <= n; u++)
                    for (int id : g[u]) {
                        auto z = e[id];
                        if (z.f > 0 && h[z.v] > h[u] + z.c)
                            h[z.v] = h[u] + z.c, changed = true;
                    }
                if (!changed)
                    break;
                assert(k < n);
            }
        int flow = 0, cost = 0;
        while (flow < limit) {
            fill(dis.begin(), dis.end(), INF);
            dis[s] = 0;
            priority_queue<pair<int, int>, vector<pair<int, int>>,
                           greater<pair<int, int>>>
                q;
            q.push({0, s});
            while (!q.empty()) {
                auto [d, u] = q.top();
                q.pop();
                if (d != dis[u])
                    continue;
                for (int id : g[u]) {
                    auto z = e[id];
                    if (z.f > 0 && dis[z.v] > d + z.c + h[u] - h[z.v]) {
                        dis[z.v] = d + z.c + h[u] - h[z.v];
                        pre[z.v] = id;
                        q.push({dis[z.v], z.v});
                    }
                }
            }
            if (dis[t] == INF)
                break;
            for (int u = 1; u <= n; u++)
                if (dis[u] < INF)
                    h[u] += dis[u];
            int f = limit - flow;
            for (int v = t; v != s; v = e[pre[v] ^ 1].v)
                f = min(f, e[pre[v]].f);
            for (int v = t; v != s; v = e[pre[v] ^ 1].v) {
                int id = pre[v];
                e[id].f -= f;
                e[id ^ 1].f += f;
                cost += f * e[id].c;
            }
```

```
            flow += f;
        }
        return {flow, cost};
    }
};

void solve() {
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    MinCostFlow g(n);
    while (m--) {
        int u, v, w, c;
        cin >> u >> v >> w >> c;
        g.add_edge(u, v, w, c);
    }
    auto [f, c] = g.run(s, t);
    cout << f << ' ' << c << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

add_edge(u,v,f,c)：容量f非负，费用c；自动配反向边

run(s,t,limit)：返回新增流量及新增最小费用，默认最大流

例题 1 P3381 最小费用最大流

https://www.luogu.com.cn/problem/P3381

题意：有向容量费用网络求最小费用最大流。

观察与建模：有向容量费用网络求最小费用最大流。 费用最短的残量增广路保证每个流量阶段费用最小。

为何对应此算法：费用最短的残量增广路保证每个流量阶段费用最小。

初始化、组织数据与调用：add_edge(u,v,w,c)，run(s,t)。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(AE log V)时间，O(V+E)空间；此题初始c>=0。

复用模板：最小费用最大流（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    MinCostFlow g(n);
    while (m--) {
        int u, v, w, c;
        cin >> u >> v >> w >> c;
        g.add_edge(u, v, w, c);
    }
    auto [f, c] = g.run(s, t);
    cout << f << ' ' << c << '\n';
```

```
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P4015 运输问题

https://www.luogu.com.cn/problem/P4015

题意：运输供需平衡的货物，求最小和最大费用。

观察与建模：运输供需平衡的货物，求最小和最大费用。 仓库到商店边的流量代表货运量，负费用后最小值对应最大费用。

为何对应此算法：仓库到商店边的流量代表货运量，负费用后最小值对应最大费用。

初始化、组织数据与调用：分别重建正费用、负费用网络；输出run的费用及负号恢复。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(VE+AE log V)时间、O(V+E)空间。

复用模板：最小费用最大流（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int m, n;
    cin >> m >> n;
    vector<int> a(m + 1), b(n + 1);
    int total = 0;
    for (int i = 1; i <= m; i++)
        cin >> a[i], total += a[i];
    for (int j = 1; j <= n; j++)
        cin >> b[j];
    vector<vector<int>> c(m + 1, vector<int>(n + 1));
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
            cin >> c[i][j];
    for (int sign : {1, -1}) {
        int s = m + n + 1, t = s + 1;
        MinCostFlow g(t);
        for (int i = 1; i <= m; i++) {
            g.add_edge(s, i, a[i], 0);
            for (int j = 1; j <= n; j++)
                g.add_edge(i, m + j, total, c[i][j] * sign);
        }
        for (int j = 1; j <= n; j++)
            g.add_edge(m + j, t, b[j], 0);
        cout << g.run(s, t).second * sign << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 初始正容量子图不允许负费用环；需要消负环的环流问题须另写算法。
- 最大费用可取相反费用重建网络，不能在同一已流过的网络直接换号。
- 容量*费用及累计费用必须在long long内。

个人资料来源：图/图论竞赛复习_严格课件注释版.md；图/图论竞赛复习_按课件整理.md

图论 / 有上下界可行流与最大流

类型与用途：模板型；先强制下界并记录节点净流入差，超级源汇补平衡；有源汇加t->s并在可行残量上继续增广。

复杂度与适用范围：一般Dinic O(V^2E)时间、O(V+E)空间。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 有上下界可行流与最大流：先强制下界并记录节点净流入差，超级源汇补平衡；有源汇加t->s并在可行残
// 量上继续增广。
// 一般Dinic O(V^2E)时间、O(V+E)空间。
template <typename T> struct FlowGraph {
    int s, t; // 源/汇编号1..vtot，要求s!=t；init赋值。
    int vtot; // 顶点总数，>=1；包含全部辅助节点。
    int etot; // 已存残量边数，等于e.size()，始终偶数；边号须在int范围内；init清0。
    // head：一维vector，init分配vtot+1项，下标1..vtot；邻接链首边编号，-1为空；init置-1。
    vector<int> head;
    // dis：一维vector，init分配vtot+1项并清0；下标1..vtot；BFS层数，0未访问，源为1；DFS死点
    // 置-1；每次bfs清0。
    vector<int> dis;
    // cur：一维vector，init分配vtot+1项并置-1；下标1..vtot；当前弧下标或-1；每次bfs由head复
    // 制，DFS沿链推进。
    vector<int> cur;
    struct edge {
        int v;   // 边终点，1..vtot。
        int nxt; // 同起点的下一边编号，-1或0..etot-1；由addedge填写。
        T f;     // 当前残量，>=0；正向初始为输入容量，反向初始0。
    };
    // e：一维动态边表，下标0..etot-1；配对边i^1。
    // init执行e.clear并令etot=0；addedge成对push_back，不提前resize占用边号。
    vector<edge> e;

    // init(s_,t_,vtot_)：设置源汇与点数；返回void；清边数与邻接链。
    // 前提合法编号、s_!=t_，随后重新建图；首次时间O(vtot_)；重建还包括清除旧边。
    void init(int s_, int t_, int vtot_) {
        s = s_;
        t = t_;
        vtot = vtot_;
        etot = 0; // 多次建图必须清etot。
        head.assign(vtot + 1, -1);
        dis.assign(vtot + 1, 0);
        cur.assign(vtot + 1, -1);
        e.clear();
    }
    // addedge(u,v,f)：加入原始有向边及撤销边；返回void；两次push_back摊还时间O(1)。
    // 前提u/v=1..vtot，f>=0，边号及etot不溢出int；原容量需另存时由调用方记录。
    void addedge(int u, int v, T f) {
        e.push_back({v, head[u], f});
        head[u] = etot++;
        e.push_back({u, head[v], 0});
        head[v] = etot++;
    }
    // bfs()：无参数；返回残量图中汇是否可达；完成分层并重置cur。
    // 前提图已建好或上轮增广完成；时间O(vtot+etot)。
```

```
    bool bfs() {
        for (int i = 1; i <= vtot; i++) {
            dis[i] = 0;
            cur[i] = head[i];
        }
        queue<int> q; // 一维BFS队列，初始空；存顶点1..vtot，每点至多入队一次。
        q.push(s);
        dis[s] = 1;
        while (!q.empty()) {
            int u = q.front();
            q.pop();                                          // u：当前扩展顶点，1..vtot。
            for (int i = head[u]; i != -1; i = e[i].nxt) { // i：当前残量边编号。
                if (e[i].f > 0 && dis[e[i].v] == 0) {
                    int v = e[i].v; // v：当前可达边终点，1..vtot。
                    dis[v] = dis[u] + 1;
                    if (v == t)
                        return true;
                    q.push(v);
                }
            }
        }
        return false;
    }
    // dfs(u,m)：u当前点；m为本次允许送出的剩余流量，m>0；返回实际送出量T。
    // 作用：按分层图多路增广；前提bfs成功，cur已重置；一轮阻塞流一般O(VE)。
    T dfs(int u, T m) {
        if (u == t)
            return m;
        T flow = 0; // 本次从u成功送出的总量，0..初始m。
        for (int i = cur[u]; i != -1; cur[u] = i = e[i].nxt) {
            // i：当前弧；保留PPT的cur推进方式，送满本次额度时不前移。
            if (e[i].f > 0 && dis[e[i].v] == dis[u] + 1) {
                T f = dfs(e[i].v, min(m, e[i].f)); // 实际成功增广量，0..min(m,残量)。
                e[i].f -= f;
                e[i ^ 1].f += f;
                m -= f;
                flow += f;
                if (m == 0)
                    break;
            }
        }
        if (flow == 0)
            dis[u] = -1;
        return flow;
    }
    // dinic()：无参数；返回本次新增最大流T；作用：反复分层并求阻塞流。
    // 前提合法源汇、图已建好、总流量不溢出；一般时间O(Vnet^2*Enet)，空间O(Vnet+Enet)，额外递
    // 归栈O(Vnet)。
    T dinic() {
        T flow = 0; // 本次调用新增流量，init后首次调用即总最大流。
        while (bfs())
            flow += dfs(s, numeric_limits<T>::max());
        return flow;
    }
    vector<int> cut_side() {
        vector<int> vis(vtot + 1);
        queue<int> q;
        q.push(s);
        vis[s] = 1;
```

```
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int i = head[u]; i != -1; i = e[i].nxt)
                if (e[i].f > 0 && !vis[e[i].v]) {
                    vis[e[i].v] = 1;
                    q.push(e[i].v);
                }
        }
        return vis;
    }
};
struct BoundedFlow {
    static constexpr int INF = 4000000000000000000LL;
    struct edge {
        int u, v, l, r, id;
    };
    int n, S, T;
    vector<int> balance;
    vector<edge> edges;
    FlowGraph<int> g;
    vector<int> aux;
    BoundedFlow(int n = 0) {
        init(n);
    }
    void init(int n_) {
        n = n_;
        S = n + 1;
        T = n + 2;
        balance.assign(n + 1, 0);
        edges.clear();
        aux.clear();
        g.init(S, T, T);
    }
    int add_edge(int u, int v, int l, int r) {
        assert(0 <= l && l <= r);
        int id = g.etot;
        g.addedge(u, v, r - l);
        balance[u] -= l;
        balance[v] += l;
        edges.push_back({u, v, l, r, id});
        return edges.size() - 1;
    }
    bool feasible() {
        int need = 0;
        for (int u = 1; u <= n; u++) {
            if (balance[u] > 0) {
                aux.push_back(g.etot);
                g.addedge(S, u, balance[u]);
                need += balance[u];
            }
            if (balance[u] < 0) {
                aux.push_back(g.etot);
                g.addedge(u, T, -balance[u]);
            }
        }
        return g.dinic() == need;
    }
```

```
    int value(int k) {
        auto z = edges[k];
        return z.r - g.e[z.id].f;
    }
    optional<int> maxflow(int s, int t) {
        int back = g.etot;
        g.addedge(t, s, INF);
        if (!feasible())
            return nullopt;
        int base = g.e[back ^ 1].f;
        g.e[back].f = g.e[back ^ 1].f = 0;
        for (int id : aux)
            g.e[id].f = g.e[id ^ 1].f = 0;
        g.s = s;
        g.t = t;
        return base + g.dinic();
    }
};

void solve() {
    int n;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    int s = n + 1, t = n + 2;
    BoundedFlow g(t);
    vector<array<int, 3>> choice;
    auto prime = [](int x) {
        if (x < 2)
            return false;
        for (int p = 2; p <= x / p; p++)
            if (x % p == 0)
                return false;
        return true;
    };
    for (int i = 1; i <= n; i++)
        if (a[i] & 1)
            g.add_edge(s, i, 2, 2);
        else
            g.add_edge(i, t, 2, 2);
    for (int i = 1; i <= n; i++)
        if (a[i] & 1)
            for (int j = 1; j <= n; j++)
                if (!(a[j] & 1) && prime(a[i] + a[j]))
                    choice.push_back({i, j, g.add_edge(i, j, 0, 1)});
    g.add_edge(t, s, 0, 2 * n);
    if (!g.feasible()) {
        cout << "Impossible\n";
        return;
    }
    vector<vector<int>> adj(n + 1), cycles;
    for (auto [u, v, k] : choice)
        if (g.value(k))
            adj[u].push_back(v), adj[v].push_back(u);
    vector<int> vis(n + 1);
    for (int i = 1; i <= n; i++)
        if (!vis[i]) {
            vector<int> cyc;
            int u = i, p = 0;
```

```
            do {
                cyc.push_back(u);
                vis[u] = 1;
                int v = adj[u][0] == p ? adj[u][1] : adj[u][0];
                p = u;
                u = v;
            } while (u != i);
            cycles.push_back(cyc);
        }
    cout << cycles.size() << '\n';
    for (auto c : cycles) {
        cout << c.size();
        for (int u : c)
            cout << ' ' << u;
        cout << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

add_edge(u,v,l,r)：记录0<=l<=流量<=r，返回原边序号

feasible()：无源汇环流判可行，只调用一次

maxflow(s,t)：有源汇非负最大流，nullopt表示不可行

value(k)：可行之后读取原第k条边实际流量

例题 1 CF510E Fox And Dinner

https://codeforces.com/contest/510/problem/E

题意：将年龄数排成若干长度至少3的圆桌，相邻和为素数。

观察与建模：将年龄数排成若干长度至少3的圆桌，相邻和为素数。 年龄>=2使素数边只可能连接奇偶；每点恰好两条选择边，转为上下界为2的二分图度约束。

为何对应此算法：年龄>=2使素数边只可能连接奇偶；每点恰好两条选择边，转为上下界为2的二分图度约束。

初始化、组织数据与调用：源到奇点[2,2]，素数边[0,1]，偶点到汇[2,2]，汇到源[0,2n]，feasible后value恢复环。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：建边O(n^2 sqrt A)，最大流一般O(V^2E)，O(n^2)空间。

复用模板：有上下界可行流与最大流（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
```

```
        cin >> a[i];
    int s = n + 1, t = n + 2;
    BoundedFlow g(t);
    vector<array<int, 3>> choice;
    auto prime = [](int x) {
        if (x < 2)
            return false;
        for (int p = 2; p <= x / p; p++)
            if (x % p == 0)
                return false;
        return true;
    };
    for (int i = 1; i <= n; i++)
        if (a[i] & 1)
            g.add_edge(s, i, 2, 2);
        else
            g.add_edge(i, t, 2, 2);
    for (int i = 1; i <= n; i++)
        if (a[i] & 1)
            for (int j = 1; j <= n; j++)
                if (!(a[j] & 1) && prime(a[i] + a[j]))
                    choice.push_back({i, j, g.add_edge(i, j, 0, 1)});
    g.add_edge(t, s, 0, 2 * n);
    if (!g.feasible()) {
        cout << "Impossible\n";
        return;
    }
    vector<vector<int>> adj(n + 1), cycles;
    for (auto [u, v, k] : choice)
        if (g.value(k))
            adj[u].push_back(v), adj[v].push_back(u);
    vector<int> vis(n + 1);
    for (int i = 1; i <= n; i++)
        if (!vis[i]) {
            vector<int> cyc;
            int u = i, p = 0;
            do {
                cyc.push_back(u);
                vis[u] = 1;
                int v = adj[u][0] == p ? adj[u][1] : adj[u][0];
                p = u;
                u = v;
            } while (u != i);
            cycles.push_back(cyc);
        }
    cout << cycles.size() << '\n';
    for (auto c : cycles) {
        cout << c.size();
        for (int u : c)
            cout << ' ' << u;
        cout << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P5192 Shoot the Bullet

https://www.luogu.com.cn/problem/P5192

题意：每天拍摄总量上限、每次对象数量区间、每位少女总需求下限，最大拍摄量。

观察与建模：每天拍摄总量上限、每次对象数量区间、每位少女总需求下限，最大拍摄量。 每项区间对应一天到少女的上下界边，少女到汇的下界代表需求，属于有源汇上下界最大流。

为何对应此算法：每项区间对应一天到少女的上下界边，少女到汇的下界代表需求，属于有源汇上下界最大流。

初始化、组织数据与调用：S->天[0,D]，天->少女[L,R]，少女->T[G,INF]，maxflow。重复少女边分别保留。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：一般O(V^2E)时间、O(V+E)空间。

复用模板：有上下界可行流与最大流（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    while (cin >> n >> m) {
        int s = n + m + 1, t = s + 1;
        BoundedFlow g(t);
        for (int j = 0; j < m; j++) {
            int G;
            cin >> G;
            g.add_edge(n + j + 1, t, G, 1000000000);
        }
        for (int i = 1; i <= n; i++) {
            int C, D;
            cin >> C >> D;
            g.add_edge(s, i, 0, D);
            while (C--) {
                int j, L, R;
                cin >> j >> L >> R;
                g.add_edge(i, n + j + 1, L, R);
            }
        }
        auto ans = g.maxflow(s, t);
        cout << (ans ? *ans : -1) << "\n\n";
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- balance正值接S->u，负值接u->T；容易接反。
- 第二次增广前必须同时清空所有辅助边的正反残量。
- INF需超过需求与可行总流，不能使用LLONG_MAX参与加法。

个人资料来源：图/图论竞赛复习_严格课件注释版.md；图/图论竞赛复习_按课件整理.md

图论 / 有向欧拉路径

类型与用途：模板型；Hierholzer递归删边与逆序；验证度条件和全边覆盖。

复杂度与适用范围：不排序O(n+m)，字典序排序O(m log m)，空间O(n+m)。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 有向欧拉路径：Hierholzer递归删边与逆序；验证度条件和全边覆盖。
// 不排序O(n+m)，字典序排序O(m log m)，空间O(n+m)。
struct Euler {
    int n, m;
    vector<vector<int>> adj;
    vector<int> in, out, path;
    void init(int n_) {
        n = n_;
        m = 0;
        adj.assign(n + 1, {});
        in.assign(n + 1, 0);
        out = in;
        path.clear();
    }
    void addedge(int u, int v) {
        adj[u].push_back(v);
        out[u]++;
        in[v]++;
        m++;
    }
    void dfs(int u) {
        while (!adj[u].empty()) {
            int v = adj[u].back();
            adj[u].pop_back();
            dfs(v);
        }
        path.push_back(u);
    }
    bool run(bool lex = true) {
        int st = 0, en = 0;
        for (int u = 1; u <= n; u++) {
            int d = out[u] - in[u];
            if (d == 1) {
                if (st)
                    return false;
                st = u;
            } else if (d == -1) {
                if (en)
                    return false;
                en = u;
            } else if (d)
                return false;
        }
        if (bool(st) != bool(en))
            return false;
        if (!st)
            for (int u = 1; u <= n; u++)
```

```
                if (out[u]) {
                    st = u;
                    break;
                }
        if (!st)
            st = 1;
        if (lex)
            for (int u = 1; u <= n; u++)
                sort(adj[u].begin(), adj[u].end(), greater<int>());
        path.clear();
        dfs(st);
        reverse(path.begin(), path.end());
        return (int)path.size() == m + 1;
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    Euler g;
    g.init(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v);
    }
    if (!g.run()) {
        cout << "No\n";
        return;
    }
    for (int u : g.path)
        cout << u << ' ';
    cout << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

addedge(u,v)：允许重复边与自环

run(lex)：破坏邻接表，得到path；失败不可使用

path：顺序路径点数m+1

例题 1 P7771 欧拉路径

https://www.luogu.com.cn/problem/P7771

题意：输出字典序最小有向欧拉路，没有则No。

观察与建模：输出字典序最小有向欧拉路，没有则No。 度数唯一决定非回路起点；降序邻接pop_back优先小点。

为何对应此算法：度数唯一决定非回路起点；降序邻接pop_back优先小点。

初始化、组织数据与调用：run(true)后输出path。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(m log m+n)时间、O(n+m)空间。

复用模板：有向欧拉路径（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    Euler g;
    g.init(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v);
    }
    if (!g.run()) {
        cout << "No\n";
        return;
    }
    for (int u : g.path)
        cout << u << ' ';
    cout << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF508D Tanya and Password

https://codeforces.com/contest/508/problem/D

题意：用三字符子串多重集还原原字符串。

观察与建模：用三字符子串多重集还原原字符串。 每个两字符串是节点，三字符串是前两位到后两位的有向边；所有子串恰用一次。

为何对应此算法：每个两字符串是节点，三字符串是前两位到后两位的有向边；所有子串恰用一次。

初始化、组织数据与调用：字母数字映射62进制节点；run(false)；输出首节点两位，再输出后续节点末位。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+62^2)时间与空间。

复用模板：有向欧拉路径（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    string ch = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz";
    auto cv = [&](char c) { return ch.find(c); };
    Euler g;
    g.init(62 * 62);
    while (n--) {
        string s;
        cin >> s;
        int u = cv(s[0]) * 62 + cv(s[1]) + 1, v = cv(s[1]) * 62 + cv(s[2]) + 1;
        g.addedge(u, v);
```

```
    }
    if (!g.run(false)) {
        cout << "NO\n";
        return;
    }
    cout << "YES\n";
    int first = g.path[0] - 1;
    cout << ch[first / 62] << ch[first % 62];
    for (int i = 1; i < (int)g.path.size(); i++)
        cout << ch[(g.path[i] - 1) % 62];
    cout << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 只判断出入度不足够，还要检查path.size()==m+1。
- 每边只pop一次；递归深度最坏m，有RE风险。

个人资料来源：图论/欧拉路径.md

图论 / 点双连通分量

类型与用途：模板型；Tarjan点栈构造点双；割点可属于多个块，孤立点单独成块。

复杂度与适用范围：O(n+m)时间与空间。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 点双连通分量：Tarjan点栈构造点双；割点可属于多个块，孤立点单独成块。
// O(n+m)时间与空间。
struct VDCC {
    int n, idx;                          // n：顶点数；idx：DFS 时钟。
    vector<vector<pair<int, int>>> e; // 二维，(v,id) 为邻点和唯一无向边编号。
    vector<int> dfn, low, cut,
        stk; // 一维：dfn/low 按顶点访问；cut[u] 割点标记；stk[j] 待归块顶点。
    vector<vector<int>> bcc; // 二维，bcc[k] 存第 k 个点双的原图顶点，k 从 0 开始。

    void init(int n_) {
        n = n_;
        idx = 0;
        e.assign(n + 1, {});
        dfn.assign(n + 1, 0);
        low.assign(n + 1, 0);
        cut.assign(n + 1, 0);
        stk.clear();
        bcc.clear();
    }
    void addedge(int u, int v, int id) {
        if (u == v)
            return;
        e[u].push_back({v, id});
        e[v].push_back({u, id});
    }
```

```
    void dfs(int u, int id) {
        dfn[u] = low[u] = ++idx;
        stk.push_back(u);
        int ch = 0; // u 的 DFS 树儿子数。
        if (e[u].empty()) {
            bcc.push_back({u});
            stk.pop_back();
            return;
        }
        for (auto [v, id2] : e[u]) {
            if (id2 == id)
                continue;
            if (!dfn[v]) {
                ++ch;
                dfs(v, id2);
                low[u] = min(low[u], low[v]);
                if (low[v] >= dfn[u]) {
                    if (id != -1)
                        cut[u] = 1;
                    vector<int> c; // 一维，本次弹出的块；u 作为公共边界另行加入。
                    while (true) {
                        int x = stk.back(); // x：从点栈弹出的块内顶点。
                        stk.pop_back();
                        c.push_back(x);
                        if (x == v)
                            break;
                    }
                    c.push_back(u);
                    bcc.push_back(c);
                }
            } else
                low[u] = min(low[u], dfn[v]);
        }
        if (id == -1 && ch >= 2)
            cut[u] = 1;
    }
    void run() {
        for (int u = 1; u <= n; u++)
            if (!dfn[u]) {
                dfs(u, -1);
                stk.clear(); // 本棵 DFS 树的根可能仍留在栈中。
            }
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    VDCC g;
    g.init(n);
    for (int id = 0; id < m; id++) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v, id);
    }
    g.run();
    cout << g.bcc.size() << '\n';
    for (auto &c : g.bcc) {
        cout << c.size();
        for (int u : c)
```

```
            cout << ' ' << u;
        cout << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

init(n)：清空状态

addedge(u,v,id)：无向边唯一编号，忽略自环

run()：得到bcc与cut

例题 1 P8435 点双连通分量

https://www.luogu.com.cn/problem/P8435

题意：输出所有点双。

观察与建模：输出所有点双。 直接调用点栈Tarjan。

为何对应此算法：直接调用点栈Tarjan。

初始化、组织数据与调用：边号0..m-1；run后输出bcc。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m)时间与空间。

复用模板：点双连通分量（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    VDCC g;
    g.init(n);
    for (int id = 0; id < m; id++) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v, id);
    }
    g.run();
    cout << g.bcc.size() << '\n';
    for (auto &c : g.bcc) {
        cout << c.size();
        for (int u : c)
            cout << ' ' << u;
        cout << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P3225 矿场搭建

https://www.luogu.com.cn/problem/P3225

题意：设置最少出口，保证任一矿点坍塌后仍可逃离，统计方案数。

观察与建模：设置最少出口，保证任一矿点坍塌后仍可逃离，统计方案数。 块无割点需2出口；叶子块有1割点需在非割点中选1出口。

为何对应此算法：块无割点需2出口；叶子块有1割点需在非割点中选1出口。

初始化、组织数据与调用：按边中出现的编号建图，run后逐bcc统计cut数并乘方案。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m)时间与空间，方案数按题目保证使用unsigned long long。

复用模板：点双连通分量（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int m, tc = 0;
    while (cin >> m && m) {
        vector<pair<int, int>> e(m);
        int n = 0;
        for (auto &[u, v] : e) {
            cin >> u >> v;
            n = max(n, max(u, v));
        }
        VDCC g;
        g.init(n);
        vector<int> present(n + 1);
        for (int i = 0; i < m; i++) {
            auto [u, v] = e[i];
            present[u] = present[v] = 1;
            g.addedge(u, v, i);
        }
        g.run();
        int need = 0;
        unsigned long long ways = 1;
        for (auto &c : g.bcc) {
            if (!present[c[0]])
                continue;
            int cuts = 0;
            for (int u : c)
                cuts += g.cut[u];
            int s = c.size();
            if (cuts == 0) {
                need += 2;
                ways *= 1ULL * s * (s - 1) / 2;
            } else if (cuts == 1) {
                need++;
                ways *= s - 1;
            }
        }
        cout << "Case " << ++tc << ": " << need << ' ' << ways << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- low[v]>=dfn[u]时弹到v，再把u加入块。
- 不能把点双编号当作每点唯一归属。

个人资料来源：图/图论竞赛复习_按课件整理.md

图论 / 边双连通分量

类型与用途：模板型；去掉所有桥后得到边双分量，缩点得到桥森林。

复杂度与适用范围：O(n+m)时间与空间。

复用模板：割边（本册第二部分）

run()：先判桥再paint非桥边

bel[u]：边双编号1..cnt

bcc：分量成员表

例题 1 P8436 边双连通分量

https://www.luogu.com.cn/problem/P8436

题意：输出边双数量及每个分量的点。

观察与建模：输出边双数量及每个分量的点。 桥分隔分量，删除桥后的连通块就是答案。

为何对应此算法：桥分隔分量，删除桥后的连通块就是答案。

初始化、组织数据与调用：run后输出bcc。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m)时间与空间。

复用模板：割边（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    LowLink g;
    g.init(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v);
    }
    g.run();
    cout << g.bcc.size() << '\n';
    for (auto &c : g.bcc) {
        cout << c.size();
        for (int u : c)
            cout << ' ' << u;
        cout << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P2860 Redundant Paths

https://www.luogu.com.cn/problem/P2860

题意：最少加边让任意两点有两条边不重叠路径。

观察与建模：最少加边让任意两点有两条边不重叠路径。 缩为桥树后，配对叶子可消除所有桥；答案ceil(叶子数/2)。

为何对应此算法：缩为桥树后，配对叶子可消除所有桥；答案ceil(叶子数/2)。

初始化、组织数据与调用：用原边与bel计算桥树度数，数度1点。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+m)时间与空间。

复用模板：割边（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    LowLink g;
    g.init(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        g.addedge(u, v);
    }
    g.run();
    vector<int> d(g.cnt + 1);
    for (auto [u, v] : g.edges)
        if (g.bel[u] != g.bel[v]) {
            d[g.bel[u]]++;
            d[g.bel[v]]++;
        }
    int leaf = 0;
    for (int i = 1; i <= g.cnt; i++)
        leaf += d[i] == 1;
    cout << (leaf + 1) / 2 << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 桥树不是原图生成树，分量内可能有很多环。
- Redundant Paths保证连通；不连通情况不能直接套叶子公式。

个人资料来源：图/图论竞赛复习_严格课件注释版.md

数论

数论 / BSGS离散对数

类型与用途：模板型；求a^x=b mod m的最小非负x；保留map大步小步思路，并扩展非互质约除。

复杂度与适用范围：O(sqrt m log m)时间、O(sqrt m)空间。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// BSGS离散对数：求a^x=b mod m的最小非负x；保留map大步小步思路，并扩展非互质约除。
// O(sqrt m log m)时间、O(sqrt m)空间。
struct Exgcd {
    int exgcd(int a, int b, int &x, int &y) {
        if (!b) {
            x = 1;
            y = 0;
            return a;
        }
        int g = exgcd(b, a % b, y, x);
        y -= a / b * x;
        return g;
    }
    int norm(int a, int m) {
        a %= m;
        return a < 0 ? a + m : a;
    }
    int inverse(int a, int m) {
        int x, y;
        int g = exgcd(norm(a, m), m, x, y);
        return g == 1 ? norm(x, m) : -1;
    }
    int linear(int a, int b, int m) {
        assert(m > 0);
        a = norm(a, m);
        b = norm(b, m);
        int x, y, g = exgcd(a, m, x, y);
        if (b % g)
            return -1;
        return norm((__int128)x * (b / g) % (m / g), m / g);
    }
};

struct BSGS {
    Exgcd ex;
    int powmod(int a, int b, int m) {
        int ans = 1 % m;
        for (a %= m; b; b >>= 1, a = (__int128)a * a % m)
            if (b & 1)
                ans = (__int128)ans * a % m;
        return ans;
    }
    int unit(int a, int b, int m) {
        if (m == 1)
            return 0;
        int t = sqrtl((long double)m) + 1;
        map<int, int> hs;
```

```
        int v = 1 % m;
        for (int j = 0; j < t; j++) {
            if (!hs.count(v))
                hs[v] = j;
            v = (__int128)v * a % m;
        }
        int factor = ex.inverse(powmod(a, t, m), m), cur = b;
        for (int q = 0; q <= t; q++) {
            if (hs.count(cur))
                return q * t + hs[cur];
            cur = (__int128)cur * factor % m;
        }
        return -1;
    }
    int solve(int a, int b, int m) {
        assert(m > 0);
        a = ex.norm(a, m);
        b = ex.norm(b, m);
        if (m == 1)
            return 0;
        int k = 1 % m, count = 0;
        for (int g = gcd(a, m); g > 1; g = gcd(a, m)) {
            if (b == k)
                return count;
            if (b % g)
                return -1;
            b /= g;
            m /= g;
            k = (__int128)k * (a / g) % m;
            count++;
        }
        int x = unit(a % m, (__int128)b * ex.inverse(k, m) % m, m);
        return x < 0 ? -1 : x + count;
    }
};

void solve() {
    int p, b, n;
    cin >> p >> b >> n;
    BSGS g;
    int ans = g.solve(b, n, p);
    if (ans < 0)
        cout << "no solution\n";
    else
        cout << ans << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

solve(a,b,m)：最小非负解；不存在-1

unit(a,b,m)：内部仅用于gcd(a,m)=1

例题 1 P3846 可爱的质数

https://www.luogu.com.cn/problem/P3846

题意：输入p,b,n，求bˆx=n mod p。

观察与建模：输入p,b,n，求bˆx=n mod p。 素数模下通常互质，直接用BSGS。

为何对应此算法：素数模下通常互质，直接用BSGS。

初始化、组织数据与调用：solve(b,n,p)，无解输出no solution。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(sqrt p log p)时间、O(sqrt p)空间。

复用模板：BSGS离散对数（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int p, b, n;
    cin >> p >> b >> n;
    BSGS g;
    int ans = g.solve(b, n, p);
    if (ans < 0)
        cout << "no solution\n";
    else
        cout << ans << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P2485 计算器

https://www.luogu.com.cn/problem/P2485

题意：三类操作分别为快速幂、线性同余、离散对数。

观察与建模：三类操作分别为快速幂、线性同余、离散对数。 对模式3用BSGS；其他模式调用板子内的powmod和Exgcd。

为何对应此算法：对模式3用BSGS；其他模式调用板子内的powmod和Exgcd。

初始化、组织数据与调用：按k选择powmod、ex.linear、solve；无解按题面字符串输出。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：模式3每组O(sqrt p log p)，其他O(log p)。

复用模板：BSGS离散对数（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int T, k;
    cin >> T >> k;
    BSGS g;
    while (T--) {
        int y, z, p;
        cin >> y >> z >> p;
        int ans = k == 1   ? g.powmod(y, z, p)
                  : k == 2 ? g.ex.linear(y, z, p)
                           : g.solve(y, z, p);
        if (ans < 0)
```

```
            cout << "Orz, I cannot find x!\n";
        else
            cout << ans << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 先处理x=0与m=1；普通BSGS不能裸用于非互质。
- 重复baby值保留最小指数以保证最小解。

个人资料来源：数论/数论8_ACM板子.md；数论/数论8_课程复习.md

数论 / Lucas定理

类型与用途：模板型；大n、小素数p的组合数模p；保留逐p进制位乘binom。

复杂度与适用范围：预处理O(p)，单问O(log_p n)，空间O(p)。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// Lucas定理：大n、小素数p的组合数模p；保留逐p进制位乘binom。
// 预处理O(p)，单问O(log_p n)，空间O(p)。
struct Comb {
    int mod;
    vector<int> fac, fnv;
    int powmod(int a, int b) {
        int ans = 1 % mod;
        for (a %= mod; b; b >>= 1, a = (__int128)a * a % mod)
            if (b & 1)
                ans = (__int128)ans * a % mod;
        return ans;
    }
    void init(int n, int p = 1000000007) {
        assert(p > 1 && n < p);
        mod = p;
        fac.assign(n + 1, 1);
        fnv.assign(n + 1, 1);
        for (int i = 1; i <= n; i++)
            fac[i] = fac[i - 1] * i % mod;
        fnv[n] = powmod(fac[n], mod - 2);
        for (int i = n; i >= 1; i--)
            fnv[i - 1] = fnv[i] * i % mod;
    }
    int binom(int n, int m) {
        if (m < 0 || m > n)
            return 0;
        assert(n < (int)fac.size());
        return fac[n] * fnv[m] % mod * fnv[n - m] % mod;
    }
};

```

```
struct Lucas {
    int p;
    Comb c;
    void init(int p_) {
        p = p_;
        c.init(p - 1, p);
    }
    int binom(int n, int m) {
        if (m < 0 || m > n)
            return 0;
        int ans = 1;
        while (n || m) {
            ans = ans * c.binom(n % p, m % p) % p;
            n /= p;
            m /= p;
        }
        return ans;
    }
    int powmod(int a, int b) {
        return c.powmod(a, b);
    }
};

void solve() {
    int T;
    cin >> T;
    while (T--) {
        int n, m, p;
        cin >> n >> m >> p;
        Lucas c;
        c.init(p);
        cout << c.binom(n + m, n) << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

init(p)：素数p，按p实际分配

binom(n,m)：按p进制分解

powmod(a,b)：同模数快速幂

例题 1 P3807 Lucas定理

https://www.luogu.com.cn/problem/P3807

题意：多组求C(n+m,n) mod p。

观察与建模：多组求C(n+m,n) mod p。 n可大于p，逐位Lucas降为小组合数。

为何对应此算法：n可大于p，逐位Lucas降为小组合数。

初始化、组织数据与调用：每个p重新init，binom(n+m,n)。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度： 每组O(p+log_p(n+m))时间、O(p)空间。

复用模板：Lucas定理（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int T;
    cin >> T;
    while (T--) {
        int n, m, p;
        cin >> n >> m >> p;
        Lucas c;
        c.init(p);
        cout << c.binom(n + m, n) << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P1313 计算系数

https://www.luogu.com.cn/problem/P1313

题意： 求(ax+by)^k中x^n*y^m的系数模10007。

观察与建模： 求(ax+by)^k中x^n*y^m的系数模10007。 二项式定理给C(k,n)*a^n*b^m；Lucas负责组合数部分。

为何对应此算法： 二项式定理给C(k,n)*a^n*b^m；Lucas负责组合数部分。

初始化、组织数据与调用： init(10007)，binom与powmod组合。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度： O(10007+log k)时间、O(10007)空间。

复用模板：Lucas定理（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int a, b, k, n, m;
    cin >> a >> b >> k >> n >> m;
    Lucas c;
    c.init(10007);
    int ans = c.binom(k, n) * c.powmod(a, n) % 10007 * c.powmod(b, m) % 10007;
    cout << ans << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- p必须素数；p很大时O(p)预处理会MLE/TLE。
- n+m不得溢出long long。

个人资料来源：数论/数论6_ACM板子.md；数论/数论6_课程复习.md

数论 / 中国剩余定理

类型与用途：模板型；逐式合并同余，兼容两两互质CRT与非互质扩展CRT。

复杂度与适用范围：n条同余约O(n log M)时间，额外O(log M)递归空间。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 中国剩余定理：逐式合并同余，兼容两两互质CRT与非互质扩展CRT。
// n条同余约O(n log M)时间，额外O(log M)递归空间。
struct Exgcd {
    int exgcd(int a, int b, int &x, int &y) {
        if (!b) {
            x = 1;
            y = 0;
            return a;
        }
        int g = exgcd(b, a % b, y, x);
        y -= a / b * x;
        return g;
    }
    int norm(int a, int m) {
        a %= m;
        return a < 0 ? a + m : a;
    }
    int inverse(int a, int m) {
        int x, y;
        int g = exgcd(norm(a, m), m, x, y);
        return g == 1 ? norm(x, m) : -1;
    }
    int linear(int a, int b, int m) {
        assert(m > 0);
        a = norm(a, m);
        b = norm(b, m);
        int x, y, g = exgcd(a, m, x, y);
        if (b % g)
            return -1;
        return norm((__int128)x * (b / g) % (m / g), m / g);
    }
};

struct CRT {
    Exgcd ex;
    int ans = 0, mod = 1;
    void init() {
        ans = 0;
        mod = 1;
    }
    bool merge(int r, int m) {
        assert(m > 0);
        r = ex.norm(r, m);
        int x, y, g = ex.exgcd(mod, m, x, y);
        int delta = r - ans;
        if (delta % g)
            return false;
```

```
        int q = m / g;
        int k = ex.norm((__int128)(delta / g) * x % q, q);
        __int128 nm = (__int128)mod * q;
        assert(nm <= LLONG_MAX);
        ans = ((__int128)ans + (__int128)mod * k) % nm;
        mod = nm;
        return true;
    }
};

void solve() {
    int n;
    cin >> n;
    CRT crt;
    while (n--) {
        int m, r;
        cin >> m >> r;
        crt.merge(r, m);
    }
    cout << crt.ans << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

init(): 初始x=0 mod1

merge(r,m): 合并x=r mod m，矛盾返回false

ans/mod: 最小非负解与合并模数

例题 1 P1495 曹冲养猪

https://www.luogu.com.cn/problem/P1495

题意：给若干除数和余数，求满足条件的最小猪数。

观察与建模：给若干除数和余数，求满足条件的最小猪数。 各同余两两互质，是普通CRT。

为何对应此算法：各同余两两互质，是普通CRT。

初始化、组织数据与调用：读入m,r后merge(r,m)，输出ans。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n log M)时间。

复用模板：中国剩余定理（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    CRT crt;
    while (n--) {
        int m, r;
        cin >> m >> r;
```

```
        crt.merge(r, m);
    }
    cout << crt.ans << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P4777 扩展中国剩余定理

https://www.luogu.com.cn/problem/P4777

题意：合并不保证互质的同余方程。

观察与建模：合并不保证互质的同余方程。 差值必须被gcd整除；新模数取lcm而非乘积。

为何对应此算法：差值必须被gcd整除；新模数取lcm而非乘积。

初始化、组织数据与调用：逐式merge，按题面存在解约定输出ans。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n log M)时间。

复用模板：中国剩余定理（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    CRT crt;
    bool ok = true;
    while (n--) {
        int m, r;
        cin >> m >> r;
        if (ok)
            ok = crt.merge(r, m);
    }
    cout << (ok ? crt.ans : -1) << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 最小公倍数须在long long内；中间乘积用__int128。
- 原CRT仅互质，本版merge增加gcd整除判断，保留exgcd解法。

个人资料来源：数论/CRT（中国剩余定理）.cpp

数论 / 原根

类型与用途：模板型；寻找最小原根并列举全部原根；从原素数版扩展至2、4、pˆk、2pˆk。

复杂度与适用范围：试除O(sqrt m)，找根O(G*omega(phi)*log m)，枚举O(phi(m)log phi(m))及排序，空间O(phi(m))。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 原根：寻找最小原根并列举全部原根；从原素数版扩展至2、4、p^k、2p^k。
// 试除O(sqrt m)，找根O(G*omega(phi)*log m)，枚举O(phi(m)log phi(m))及排序，空间O(phi(m))。
struct PrimitiveRoot {
    int phi(int n) {
        int ans = n;
        for (int p = 2; p <= n / p; p++)
            if (n % p == 0) {
                ans = ans / p * (p - 1);
                while (n % p == 0)
                    n /= p;
            }
        if (n > 1)
            ans = ans / n * (n - 1);
        return ans;
    }
    int powmod(int a, int b, int m) {
        int ans = 1 % m;
        for (a %= m; b; b >>= 1, a = (__int128)a * a % m)
            if (b & 1)
                ans = (__int128)ans * a % m;
        return ans;
    }
    bool exists(int m) {
        if (m == 2 || m == 4)
            return true;
        if (m < 2)
            return false;
        if (m % 2 == 0)
            m /= 2;
        if (m % 2 == 0)
            return false;
        int p = 0;
        for (int i = 3; i <= m / i; i += 2)
            if (m % i == 0) {
                p = i;
                break;
            }
        if (!p)
            p = m;
        while (m % p == 0)
            m /= p;
        return m == 1;
    }
    int find(int m) {
        if (!exists(m))
            return -1;
```

```
        int ph = phi(m), q = ph;
        vector<int> pf;
        for (int p = 2; p <= q / p; p++)
            if (q % p == 0) {
                pf.push_back(p);
                while (q % p == 0)
                    q /= p;
            }
        if (q > 1)
            pf.push_back(q);
        for (int g = 1; g < m; g++) {
            if (gcd(g, m) != 1)
                continue;
            bool ok = true;
            for (int p : pf)
                if (powmod(g, ph / p, m) == 1) {
                    ok = false;
                    break;
                }
            if (ok)
                return g;
        }
        return -1;
    }
    vector<int> all(int m) {
        int g = find(m);
        if (g == -1)
            return {};
        int ph = phi(m), x = 1;
        vector<int> ans;
        for (int k = 1; k <= ph; k++) {
            x = (__int128)x * g % m;
            if (gcd(k, ph) == 1)
                ans.push_back(x);
        }
        sort(ans.begin(), ans.end());
        return ans;
    }
};

void solve() {
    int T;
    cin >> T;
    PrimitiveRoot p;
    while (T--) {
        int n, d;
        cin >> n >> d;
        auto a = p.all(n);
        cout << a.size() << '\n';
        for (int i = d - 1; i < (int)a.size(); i += d)
            cout << a[i] << ' ';
        cout << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
```

```
    while (t--)
        solve();
    return 0;
}
```

find(m)：不存在返回-1

all(m)：升序全部原根

exists(m)：按原根存在条件判断

例题 1 P6091 原根

https://www.luogu.com.cn/problem/P6091

题意：求全部原根数量，并输出排序后每第d个原根。

观察与建模：求全部原根数量，并输出排序后每第d个原根。 求一原根后，互质指数生成全部原根。

为何对应此算法：求一原根后，互质指数生成全部原根。

初始化、组织数据与调用：all(n)，输出size与下标d-1,2d-1,...。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：按模板找根与枚举复杂度。

复用模板：原根（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int T;
    cin >> T;
    PrimitiveRoot p;
    while (T--) {
        int n, d;
        cin >> n >> d;
        auto a = p.all(n);
        cout << a.size() << '\n';
        for (int i = d - 1; i < (int)a.size(); i += d)
            cout << a[i] << ' ';
        cout << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF284A Cows and Primitive Roots

https://codeforces.com/problemset/problem/284/A

题意：素数p下有多少个原根。

观察与建模：素数p下有多少个原根。 枚举所有原根并计数；p小，能展示判定与构造接口。

为何对应此算法：枚举所有原根并计数；p小，能展示判定与构造接口。

初始化、组织数据与调用：all(p).size()；也可用phi(phi(p))验证数量。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(p log p+找根耗时)时间、O(p)空间。

复用模板：原根（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int p;
    cin >> p;
    PrimitiveRoot g;
    cout << g.all(p).size() << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 非所有模数都有原根。
- 原根个数是phi(phi(m))；枚举需数据规模允许。

个人资料来源：数论/数论8_ACM板子.md；数论/数论8_课程复习.md

数论 / 扩展Lucas

类型与用途：模板型；合数模组合数，按质数幂去除阶乘质因子后CRT；单质数幂<=2e6。

复杂度与适用范围：预处理O(sqrt M+Σp^e)，每问O(Σlog_p n*log(p^e))，空间O(Σp^e)。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 扩展Lucas：合数模组合数，按质数幂去除阶乘质因子后CRT；单质数幂<=2e6。
// 预处理O(sqrt M+Σp^e)，每问O(Σlog_p n*log(p^e))，空间O(Σp^e)。
struct Exgcd {
    int exgcd(int a, int b, int &x, int &y) {
        if (!b) {
            x = 1;
            y = 0;
            return a;
        }
        int g = exgcd(b, a % b, y, x);
        y -= a / b * x;
        return g;
    }
    int norm(int a, int m) {
        a %= m;
        return a < 0 ? a + m : a;
    }
    int inverse(int a, int m) {
        int x, y;
        int g = exgcd(norm(a, m), m, x, y);
        return g == 1 ? norm(x, m) : -1;
    }
    int linear(int a, int b, int m) {
        assert(m > 0);
        a = norm(a, m);
        b = norm(b, m);
        int x, y, g = exgcd(a, m, x, y);
        if (b % g)
            return -1;
        return norm((__int128)x * (b / g) % (m / g), m / g);
```

```
    }
};

struct ExLucas {
    struct node {
        int p, pe;
        vector<int> fac;
    };
    int mod;
    vector<node> parts;
    Exgcd ex;
    int powmod(int a, int b, int m) {
        int ans = 1 % m;
        for (a %= m; b; b >>= 1, a = (__int128)a * a % m)
            if (b & 1)
                ans = (__int128)ans * a % m;
        return ans;
    }
    void init(int m) {
        assert(m >= 1);
        mod = m;
        parts.clear();
        int q = m;
        for (int p = 2; p <= q / p; p++)
            if (q % p == 0) {
                int pe = 1;
                while (q % p == 0) {
                    q /= p;
                    pe *= p;
                }
                parts.push_back({p, pe, {}});
            }
        if (q > 1)
            parts.push_back({q, q, {}});
        for (auto &v : parts) {
            assert(v.pe <= 2000000);
            v.fac.assign(v.pe + 1, 1);
            for (int i = 1; i <= v.pe; i++)
                v.fac[i] = v.fac[i - 1] * (i % v.p ? i : 1) % v.pe;
        }
    }
    int valuation(int n, int p) {
        int ans = 0;
        while (n) {
            n /= p;
            ans += n;
        }
        return ans;
    }
    int fact(int n, const node &v) {
        if (!n)
            return 1;
        return powmod(v.fac[v.pe], n / v.pe, v.pe) * v.fac[n % v.pe] % v.pe *
               fact(n / v.p, v) % v.pe;
    }
    int binom(int n, int m) {
        if (m < 0 || m > n || mod == 1)
            return 0;
        int ans = 0;
```

```
        for (auto &v : parts) {
            int exp = valuation(n, v.p) - valuation(m, v.p) - valuation(n - m, v.p);
            int a = fact(n, v), b = fact(m, v) * fact(n - m, v) % v.pe;
            int val = a * ex.inverse(b, v.pe) % v.pe * powmod(v.p, exp, v.pe) % v.pe;
            int M = mod / v.pe;
            ans = (ans + (__int128)val * M * ex.inverse(M, v.pe)) % mod;
        }
        return ans;
    }
};

void solve() {
    int n, m, p;
    cin >> n >> m >> p;
    ExLucas c;
    c.init(p);
    cout << c.binom(n, m) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

init(M)：允许M=1；分解并预处理非p因子阶乘

binom(n,m)：合数模组合数

例题 1 P4720 扩展Lucas

https://www.luogu.com.cn/problem/P4720

题意：求C(n,m)模给定合数p。

观察与建模：求C(n,m)模给定合数p。 普通逆阶乘不适用；分解为质数幂计算后CRT。

为何对应此算法：普通逆阶乘不适用；分解为质数幂计算后CRT。

初始化、组织数据与调用：init(p)，binom(n,m)。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：按模板复杂度。

复用模板：扩展Lucas（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m, p;
    cin >> n >> m >> p;
    ExLucas c;
    c.init(p);
    cout << c.binom(n, m) << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P2183 礼物

https://www.luogu.com.cn/problem/P2183

题意：n件不同礼物分给m人，每人指定件数，模合数p求方案。

观察与建模：n件不同礼物分给m人，每人指定件数，模合数p求方案。 多项式系数可写成依次选人的组合数连乘；同一个模数复用预处理。

为何对应此算法：多项式系数可写成依次选人的组合数连乘；同一个模数复用预处理。

初始化、组织数据与调用：先判断需求总和<=n；逐人binom(remaining,wi)后扣减。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：预处理一次，加m次组合数查询；空间Σp^e。

复用模板：扩展Lucas（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int p, n, m;
    cin >> p >> n >> m;
    vector<int> w(m);
    int sum = 0;
    for (int &x : w) {
        cin >> x;
        sum += x;
    }
    if (sum > n) {
        cout << "Impossible\n";
        return;
    }
    ExLucas c;
    c.init(p);
    int ans = 1 % p;
    for (int x : w) {
        ans = (__int128)ans * c.binom(n, x) % p;
        n -= x;
    }
    cout << ans << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 阶乘单位部分才能求逆；质因子指数单独乘p^exp。
- 大质数幂不能直接开fac；明确O(p^e)限制。
- 重建时parts清空，修复固定大数组与负指数处理风险。

个人资料来源：数论/数论6_ACM板子.md；数论/数论6_课程复习.md

数论 / 扩展欧几里得

类型与用途：模板型；求Bezout系数、模逆元与线性同余最小非负解；模数为正。

复杂度与适用范围：O(log m)时间与递归空间。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 扩展欧几里得：求Bezout系数、模逆元与线性同余最小非负解；模数为正。
// O(log m)时间与递归空间。
struct Exgcd {
    int exgcd(int a, int b, int &x, int &y) {
        if (!b) {
            x = 1;
            y = 0;
            return a;
        }
        int g = exgcd(b, a % b, y, x);
        y -= a / b * x;
        return g;
    }
    int norm(int a, int m) {
        a %= m;
        return a < 0 ? a + m : a;
    }
    int inverse(int a, int m) {
        int x, y;
        int g = exgcd(norm(a, m), m, x, y);
        return g == 1 ? norm(x, m) : -1;
    }
    int linear(int a, int b, int m) {
        assert(m > 0);
        a = norm(a, m);
        b = norm(b, m);
        int x, y, g = exgcd(a, m, x, y);
        if (b % g)
            return -1;
        return norm((__int128)x * (b / g) % (m / g), m / g);
    }
};

void solve() {
    int a, b;
    cin >> a >> b;
    Exgcd ex;
    int x = ex.inverse(a, b);
    cout << (x == 0 ? b : x) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
```

```
        solve();
    return 0;
}
```

exgcd(a,b,x,y)：非负a,b，返回gcd并使ax+by=gcd

inverse(a,m)：不存在返回-1

linear(a,b,m)：ax=b mod m，返回最小非负解或-1

例题 1 P1082 同余方程

https://www.luogu.com.cn/problem/P1082

题意：求ax=1 mod b的最小正整数解。

观察与建模：求ax=1 mod b的最小正整数解。 题目互质，扩展欧几里得系数给逆元。

为何对应此算法：题目互质，扩展欧几里得系数给逆元。

初始化、组织数据与调用：inverse(a,b)，将0按正解约定改成b。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(log b)时间与空间。

复用模板：扩展欧几里得（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int a, b;
    cin >> a >> b;
    Exgcd ex;
    int x = ex.inverse(a, b);
    cout << (x == 0 ? b : x) << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 ABC186E Throne

https://atcoder.jp/contests/abc186/tasks/abc186_e

题意：从位置S每次走K，最少几次回到模N的0点。

观察与建模：从位置S每次走K，最少几次回到模N的0点。 K*x=-S mod N，gcd整除目标才有解。

为何对应此算法：K*x=-S mod N，gcd整除目标才有解。

初始化、组织数据与调用：每组linear(K,-S,N)。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(T log N)时间、O(log N)空间。

复用模板：扩展欧几里得（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int T;
    cin >> T;
    Exgcd ex;
    while (T--) {
        int n, s, k;
        cin >> n >> s >> k;
```

```
        cout << ex.linear(k, -s, n) << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 取模后需归一化负余数。
- 系数递推与模数合并需评估溢出；乘积取模用__int128。

个人资料来源：数论/exgcd.md

数论 / 扩展欧拉与指数塔

类型与用途：模板型；固定2的无限幂塔，以及正整数数组区间幂塔模m；通过phi链与真实阈值判定。

复杂度与适用范围：phi试除按链各层求和；缓存后每问按O(log m)级链深递归及截断幂比较，空间为链深。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 扩展欧拉与指数塔：固定2的无限幂塔，以及正整数数组区间幂塔模m；通过phi链与真实阈值判定。
// phi试除按链各层求和；缓存后每问按O(log m)级链深递归及截断幂比较，空间为链深。
struct PowerTower {
    vector<int> a;
    map<int, int> phi_cache;
    int phi(int m) {
        if (phi_cache.count(m))
            return phi_cache[m];
        int ans = m, q = m;
        for (int p = 2; p <= q / p; p++)
            if (q % p == 0) {
                ans = ans / p * (p - 1);
                while (q % p == 0)
                    q /= p;
            }
        if (q > 1)
            ans = ans / q * (q - 1);
        return phi_cache[m] = ans;
    }
    int powmod(int x, int e, int m) {
        int ans = 1 % m;
        for (x %= m; e; e >>= 1, x = (__int128)x * x % m)
            if (e & 1)
                ans = (__int128)ans * x % m;
        return ans;
    }
    int calc(int m) {
        if (m == 1)
            return 0;
        int p = phi(m);
        return powmod(2, calc(p) + p, m);
    }
```

```
    void init(const vector<int> &v) {
        a = v;
        for (int i = 1; i < (int)a.size(); i++)
            assert(a[i] >= 1);
    }
    // 返回min(真实幂塔,limit)，只用于阈值判断；避免用已取模指数判断大小。
    int cap(int l, int r, int limit) {
        if (limit <= 1)
            return limit;
        if (l == r)
            return min(a[l], limit);
        if (a[l] == 1)
            return 1;
        int need = 0;
        __int128 val = 1;
        while (val < limit) {
            val *= a[l];
            need++;
        }
        int e = cap(l + 1, r, need);
        val = 1;
        while (e-- && val < limit)
            val *= a[l];
        return val >= limit ? limit : (int)val;
    }
    int qur(int l, int r, int m) {
        if (m == 1)
            return 0;
        if (l == r)
            return a[l] % m;
        int p = phi(m);
        int e = qur(l + 1, r, p);
        if (cap(l + 1, r, p) >= p)
            e += p;
        return powmod(a[l], e, m);
    }
};

void solve() {
    int T;
    cin >> T;
    PowerTower tr;
    while (T--) {
        int p;
        cin >> p;
        cout << tr.calc(p) << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

calc(m)：无限2指数塔

init(a)：1-based且a[i]>=1

qur(l,r,m)：右结合a[l]^(a[l+1]^...)

cap(l,r,limit)：判断真实值是否达到limit

例题 1 P4139 上帝与集合的正确用法

https://www.luogu.com.cn/problem/P4139

题意：多组无限2幂塔模p。

观察与建模：多组无限2幂塔模p。 固定2的无限塔沿phi链递归，指数足够大可加phi。

为何对应此算法：固定2的无限塔沿phi链递归，指数足够大可加phi。

初始化、组织数据与调用：每组calc(p)，缓存phi可复用。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：各phi链试除与快速幂之和。

复用模板：扩展欧拉与指数塔（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int T;
    cin >> T;
    PowerTower tr;
    while (T--) {
        int p;
        cin >> p;
        cout << tr.calc(p) << '\n';
    }
}
```

验证：C++17组合程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF906D Power Tower

https://codeforces.com/problemset/problem/906/D

题意：数组上给区间，求该区间指数塔模固定m。

观察与建模：数组上给区间，求该区间指数塔模固定m。 区间长度可大，但phi链很短；底数为1时cap直接结束。

为何对应此算法：区间长度可大，但phi链很短；底数为1时cap直接结束。

初始化、组织数据与调用：init(a)，每问qur(l,r,m)。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：预处理phi链，查询沿短链与阈值递归；空间O(n+log m)。

复用模板：扩展欧拉与指数塔（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    PowerTower tr;
    tr.init(a);
```

```
    int q;
    cin >> q;
    while (q--) {
        int l, r;
        cin >> l >> r;
        cout << tr.qur(l, r, m) << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 非互质时只有指数足够大才加phi，不能无条件加。
- 不要从取模后的指数推断真实大小；cap独立判断。
- 本版约定m<=1e9，正底数，0^0不属于适用范围。

个人资料来源：数论/数论7_ACM板子.md；数论/数论7_课程复习.md

数论 / 整除分块

类型与用途：模板型；相同floor(n/i)的连续区间合并；同时支持两个商不变的分块。

复杂度与适用范围：单商/双商分块O(sqrt n+sqrt m)，额外O(1)空间。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 整除分块：相同floor(n/i)的连续区间合并；同时支持两个商不变的分块。
// 单商/双商分块O(sqrt n+sqrt m)，额外O(1)空间。
struct FloorBlock {
    template <class F> void run(int n, F calc) {
        for (int l = 1, r; l <= n; l = r + 1) {
            int q = n / l;
            r = n / q;
            calc(l, r, q);
        }
    }
    template <class F> void run(int n, int m, F calc) {
        for (int l = 1, r; l <= min(n, m); l = r + 1) {
            int a = n / l, b = m / l;
            r = min(n / a, m / b);
            calc(l, r, a, b);
        }
    }
    int count_divisors_sum(int n) {
        int ans = 0;
        run(n, [&](int l, int r, int q) { ans += (r - l + 1) * q; });
        return ans;
    }
    __int128 divisor_sum(int n) {
        __int128 ans = 0;
        if (n <= 0)
            return 0;
```

```
        run(n, [&](int l, int r, int q) {
            ans += (__int128)(l + r) * (r - l + 1) / 2 * q;
        });
        return ans;
    }
    void print(__int128 x) {
        if (x < 0) {
            cout << '-';
            x = -x;
        }
        if (x >= 10)
            print(x / 10);
        cout << char('0' + x % 10);
    }
};

void solve() {
    int n;
    cin >> n;
    FloorBlock f;
    cout << f.count_divisors_sum(n) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

run(n,calc)：回调(l,r,n/l)

run(n,m,calc)：同时商值不变

divisor_sum(n)：1..n的约数和总和，返回宽整数

例题 1 P1403 约数研究

https://www.luogu.com.cn/problem/P1403

题意：求1..n每个数的约数个数之和。

观察与建模：求1..n每个数的约数个数之和。 交换求和得到sum floor(n/d)，按相同商合并。

为何对应此算法：交换求和得到sum floor(n/d)，按相同商合并。

初始化、组织数据与调用：count_divisors_sum(n)。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(sqrt n)时间、O(1)空间。

复用模板：整除分块（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    FloorBlock f;
    cout << f.count_divisors_sum(n) << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P2424 约数和

https://www.luogu.com.cn/problem/P2424

题意：求区间[x,y]每个数的约数和之和。

观察与建模：求区间[x,y]每个数的约数和之和。 前缀=Σ d*floor(n/d)，块内d的和用等差数列。

为何对应此算法：前缀=Σ d*floor(n/d)，块内d的和用等差数列。

初始化、组织数据与调用：divisor_sum(y)-divisor_sum(x-1)，print宽整数。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(sqrt y)时间、O(1)额外空间。

复用模板：整除分块（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int x, y;
    cin >> x >> y;
    FloorBlock f;
    f.print(f.divisor_sum(y) - f.divisor_sum(x - 1));
    cout << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 右端点n/(n/l)，循环更新到r+1。
- n接近LLONG_MAX时r+1溢出；本题使用n<=2e9。

个人资料来源：数论/数论分块.cpp；数论/数论分块.md

数论 / 素数模组合数

类型与用途：模板型；阶乘与逆阶乘计算组合数；p为素数，预处理范围n<p。

复杂度与适用范围：预处理O(n+log p)，每问O(1)，空间O(n)。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 素数模组合数：阶乘与逆阶乘计算组合数；p为素数，预处理范围n<p。
// 预处理O(n+log p)，每问O(1)，空间O(n)。
struct Comb {
    int mod;
    vector<int> fac, fnv;
    int powmod(int a, int b) {
        int ans = 1 % mod;
        for (a %= mod; b; b >>= 1, a = (__int128)a * a % mod)
            if (b & 1)
                ans = (__int128)ans * a % mod;
        return ans;
```

```
    }
    void init(int n, int p = 1000000007) {
        assert(p > 1 && n < p);
        mod = p;
        fac.assign(n + 1, 1);
        fnv.assign(n + 1, 1);
        for (int i = 1; i <= n; i++)
            fac[i] = fac[i - 1] * i % mod;
        fnv[n] = powmod(fac[n], mod - 2);
        for (int i = n; i >= 1; i--)
            fnv[i - 1] = fnv[i] * i % mod;
    }
    int binom(int n, int m) {
        if (m < 0 || m > n)
            return 0;
        assert(n < (int)fac.size());
        return fac[n] * fnv[m] % mod * fnv[n - m] % mod;
    }
};

void solve() {
    int w, h;
    cin >> w >> h;
    Comb c;
    c.init(w + h - 2);
    cout << c.binom(w + h - 2, w - 1) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

init(n,p)：要求素数p且n<p

binom(n,m)：非法m返回0

powmod(a,b)：b非负

例题 1 ABC034C 経路

https://atcoder.jp/contests/abc034/tasks/abc034_c

题意：W乘H网格从左上向右下仅向右或下走，求路径数。

观察与建模：W乘H网格从左上向右下仅向右或下走，求路径数。 总步数W+H-2，选W-1步向右。

为何对应此算法：总步数W+H-2，选W-1步向右。

初始化、组织数据与调用：init(W+H-2)，binom(total,W-1)。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(W+H)时间与空间。

复用模板：素数模组合数（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int w, h;
    cin >> w >> h;
    Comb c;
    c.init(w + h - 2);
    cout << c.binom(w + h - 2, w - 1) << '\n';
}
```

验证：C++17组合程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 ABC145D Knight

https://atcoder.jp/contests/abc145/tasks/abc145_d

题意：每步(1,2)或(2,1)，求走到(X,Y)方案数。

观察与建模：每步(1,2)或(2,1)，求走到(X,Y)方案数。 求两种步数a=(2Y-X)/3、b=(2X-Y)/3，非负整数时组合数计排列。

为何对应此算法：求两种步数a=(2Y-X)/3、b=(2X-Y)/3，非负整数时组合数计排列。

初始化、组织数据与调用：检查整除与非负；init(a+b)，binom(a+b,a)。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(X+Y)时间与空间上界。

复用模板：素数模组合数（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int x, y;
    cin >> x >> y;
    int a = 2 * y - x, b = 2 * x - y;
    if (a < 0 || b < 0 || a % 3 || b % 3) {
        cout << 0 << '\n';
        return;
    }
    a /= 3;
    b /= 3;
    Comb c;
    c.init(a + b);
    cout << c.binom(a + b, a) << '\n';
}
```

验证：C++17组合程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 跨过模数后阶乘为0，不能求逆，应使用Lucas。
- 普通乘积在本例mod1e9+7内安全；powmod宽整数支持。

个人资料来源：数论/组合数.cpp

数论 / 莫比乌斯反演与线性筛

类型与用途：模板型；线性筛mu与phi；利用互质指示函数和整除分块统计数对；保留原稿模2^32反演接口。

复杂度与适用范围：筛O(N)时间/空间；coprime O(sqrt(min(a,b)))；invert32 O(N log N)。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 莫比乌斯反演与线性筛：线性筛mu与phi；利用互质指示函数和整除分块统计数对；保留原稿模2^32反演接口。
// 筛O(N)时间/空间；coprime O(sqrt(min(a,b)))；invert32 O(N log N)。
struct Mobius {
    int n;
    vector<int> p, pr, mu, smu, phi;
    Mobius(int n = 0) {
        init(n);
    }
    void init(int n_) {
        n = n_;
        p.assign(n + 1, 0);
        mu = smu = phi = p;
        pr.clear();
        if (n)
            mu[1] = phi[1] = 1;
        for (int i = 2; i <= n; i++) {
            if (!p[i])
                p[i] = i, pr.push_back(i), mu[i] = -1, phi[i] = i - 1;
            for (int x : pr) {
                if (x > n / i)
                    break;
                p[i * x] = x;
                if (i % x == 0) {
                    phi[i * x] = phi[i] * x;
                    break;
                }
                mu[i * x] = -mu[i];
                phi[i * x] = phi[i] * (x - 1);
            }
        }
        for (int i = 1; i <= n; i++)
            smu[i] = smu[i - 1] + mu[i];
    }
    int coprime(int a, int b) {
        if (a > b)
            swap(a, b);
        assert(a <= n);
        int ans = 0;
        for (int l = 1, r; l <= a; l = r + 1) {
            int x = a / l, y = b / l;
            r = min(a / x, b / y);
            ans += (smu[r] - smu[l - 1]) * x * y;
        }
        return ans;
    }
```

```
    vector<uint32_t> invert32(const vector<uint32_t> &f) {
        assert((int)f.size() == n + 1);
        vector<uint32_t> g(n + 1);
        for (int d = 1; d <= n; d++)
            for (int k = 1; k <= n / d; k++)
                g[d * k] += f[d] * (uint32_t)mu[k];
        return g;
    }
};

void solve() {
    int T;
    cin >> T;
    vector<array<int, 3>> q(T);
    int N = 0;
    for (auto &z : q) {
        cin >> z[0] >> z[1] >> z[2];
        N = max(N, min(z[0] / z[2], z[1] / z[2]));
    }
    Mobius t(N);
    for (auto z : q)
        cout << t.coprime(z[0] / z[2], z[1] / z[2]) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

init(N)：筛1..N的最小素因子、mu、phi及mu前缀和

coprime(a,b)：1<=x<=a、1<=y<=b且gcd=1的有序数对

invert32(f)：已知f(n)=Σg(d)，在模2^32环内反演g

例题 1 P2522 ZAP-Queries

https://www.luogu.com.cn/problem/P2522

题意：统计1..a和1..b中gcd(x,y)=d的数对。

观察与建模：统计1..a和1..b中gcd(x,y)=d的数对。 同时除以d，转为互质矩形计数。

为何对应此算法：同时除以d，转为互质矩形计数。

初始化、组织数据与调用：离线读多组以求最大min(a/d,b/d)，init后coprime。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(N+T sqrt N)时间、O(N+T)空间。

复用模板：莫比乌斯反演与线性筛（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int T;
    cin >> T;
    vector<array<int, 3>> q(T);
    int N = 0;
```

```
    for (auto &z : q) {
        cin >> z[0] >> z[1] >> z[2];
        N = max(N, min(z[0] / z[2], z[1] / z[2]));
    }
    Mobius t(N);
    for (auto z : q)
        cout << t.coprime(z[0] / z[2], z[1] / z[2]) << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P3455 Problem b

https://www.luogu.com.cn/problem/P3455

题意：统计两个闭区间内gcd=k的数对。

观察与建模：统计两个闭区间内gcd=k的数对。 将区间容斥为四个前缀矩形，再缩放为互质计数。

为何对应此算法：将区间容斥为四个前缀矩形，再缩放为互质计数。

初始化、组织数据与调用：读取a,b,c,d,k，计算F(b,d)-F(a-1,d)-F(b,c-1)+F(a-1,c-1)。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(N+T sqrt N)时间、O(N+T)空间。

复用模板：莫比乌斯反演与线性筛（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int T;
    cin >> T;
    vector<array<int, 5>> q(T);
    int N = 0;
    for (auto &z : q) {
        for (int &x : z)
            cin >> x;
        N = max(N, min(z[1] / z[4], z[3] / z[4]));
    }
    Mobius t(N);
    for (auto z : q) {
        auto F = [&](int a, int b) { return t.coprime(a / z[4], b / z[4]); };
        cout << F(z[1], z[3]) - F(z[0] - 1, z[3]) - F(z[1], z[2] - 1) +
                    F(z[0] - 1, z[2] - 1)
             << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- mu用有符号整数，原rng题unsigned运算用uint32_t保留32位溢出语义。
- 数对结果要求a*b不超long long；任一边为0返回0。
- 分块取两个商共同右端点。

个人资料来源：数论/求mo函数.cpp；数论/数论9_ACM板子.md

动态规划

动态规划 / 数位DP与有限状态

类型与用途：模板型；把前缀必要信息编码为自动机状态，统一处理tight、前导零和区间前缀差；包含原数数三转移。

复杂度与适用范围：O(L*S*10)时间、O(S)滚动空间；L是位数，S是状态数。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 数位DP与有限状态：把前缀必要信息编码为自动机状态，统一处理tight、前导零和区间前缀差；包含原数
// 数三转移。
// O(L*S*10)时间、O(S)滚动空间；L是位数，S是状态数。
struct DigitDP {
    int S, initial, mod;
    vector<array<int, 10>> go;
    vector<int> accept;
    DigitDP(int S = 1, int mod = 1000000007) {
        init(S, mod);
    }
    void init(int s, int m) {
        S = s;
        mod = m;
        initial = 0;
        go.assign(S, {});
        for (auto &a : go)
            a.fill(-1);
        accept.assign(S, 0);
    }
    int plus(int a, int b) {
        return mod ? (a + b) % mod : a + b;
    }
    int qur(const string &bound) {
        vector<array<int, 2>> dp(S), tmp(S);
        dp[initial][1] = 1;
        for (char c : bound) {
            for (auto &z : tmp)
                z = {0, 0};
            for (int s = 0; s < S; s++)
                for (int tight = 0; tight < 2; tight++)
                    if (dp[s][tight]) {
                        int hi = tight ? c - '0' : 9;
                        for (int d = 0; d <= hi; d++) {
                            int v = go[s][d];
                            if (v < 0)
                                continue;
                            int nt = tight && d == hi;
                            tmp[v][nt] = plus(tmp[v][nt], dp[s][tight]);
                        }
                    }
            dp.swap(tmp);
        }
        int ans = 0;
        for (int s = 0; s < S; s++)
            if (accept[s])
                ans = plus(ans, plus(dp[s][0], dp[s][1]));
```

```
        return ans;
    }
    int qur(int x) {
        return x < 0 ? 0 : qur(to_string(x));
    }
    void windy() {
        init(11, 0);
        for (int d = 0; d < 10; d++)
            go[0][d] = d ? d + 1 : 0;
        for (int last = 0; last < 10; last++)
            for (int d = 0; d < 10; d++)
                if (abs(last - d) >= 2)
                    go[last + 1][d] = d + 1;
        for (int s = 1; s <= 10; s++)
            accept[s] = 1;
    }
    void digit_sum(int D, int p) {
        init(D, p);
        for (int s = 0; s < D; s++)
            for (int d = 0; d < 10; d++)
                go[s][d] = (s + d) % D;
        accept[0] = 1;
    }
    // 状态0是尚无有效数位；其余编码(exist,last,inc)，inc=1..3。
    void increasing_three() {
        init(1 + 2 * 10 * 3, 0);
        auto id = [](int e, int last, int inc) {
            return 1 + e * 30 + last * 3 + inc - 1;
        };
        go[0][0] = 0;
        for (int d = 1; d < 10; d++)
            go[0][d] = id(0, d, 1);
        for (int e = 0; e < 2; e++)
            for (int last = 0; last < 10; last++)
                for (int inc = 1; inc <= 3; inc++) {
                    int s = id(e, last, inc);
                    accept[s] = e;
                    for (int d = 0; d < 10; d++) {
                        int k = d > last ? min(3LL, inc + 1) : 1;
                        go[s][d] = id(e || k == 3, d, k);
                    }
                }
    }
};

void solve() {
    int L, R;
    cin >> L >> R;
    DigitDP t;
    t.windy();
    cout << t.qur(R) - t.qur(L - 1) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
```

```
    return 0;
}
```

init(S,mod)：mod=0为精确计数；自定义go与accept，-1为禁止转移

windy/digit_sum/increasing_three：配置相邻差、数位和、连续三位递增状态

qur(bound)：统计0..bound，包括是否接受0由状态配置决定

例题 1 P2657 windy数

https://www.luogu.com.cn/problem/P2657

题意：统计区间内相邻十进制数位之差至少2的正整数。

观察与规模：L,R≤2×10^9，先处理F(L-1)负上界；前导0不能参与相邻差检查，数字0本身不接受。 上界可有上万位，枚举每个数不可能；合法性由相邻数字或数位和决定。

朴素思路：直接从1枚举到上界，即使检查单个数很快也无法完成。记住整个前缀又会有指数个状态。

关键突破：只保留前缀对未来的影响，另加tight表示当前前缀是否仍等于上界。

状态与建模：dp[s][tight]为已处理位置、自动机状态s及紧贴标记的前缀数。Windy状态0表示未开始，其余表示末位；Digit Sum状态是数位和模D。

本题映射：统计区间内相邻十进制数位之差至少2的正整数。 状态仅需前一有效数位，前导零有独立状态。 windy配置，然后qur(R)-qur(L-1)。

转移与正确性：枚举当前位d≤hi，经go[s][d]转移至v，非法转移-1跳过；新tight=旧tight且d等于当前上界位。每个数字串对应唯一逐位路径。

初始化与顺序：空前缀dp[initial][1]=1，逐位正序并清空tmp。Windy前导0留在未开始状态，末位状态才接受；Digit Sum允许全0，最后减1去掉数字0。区间用F(R)-F(L-1)。

优化与限制：时间O(位数×状态数×10)，空间O(状态数)。不取模时答案须能放入64位；相邻差限制的状态不能把未开始和末位0混为一谈。

本题复杂度：O(位数*110)时间、O(11)空间。

复用模板：数位DP与有限状态（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int L, R;
    cin >> L >> R;
    DigitDP t;
    t.windy();
    cout << t.qur(R) - t.qur(L - 1) << '\n';
}
```

启发式总结：当历史可以压成有限信息，就尝试状态机；状态是否足够要通过两个同状态前缀的后续合法性完全相同来检验。课堂连续三位递增另记末位、连续长度和是否已出现。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 DP_S Digit Sum

https://atcoder.jp/contests/dp/tasks/dp_s

题意：1..K中数位和能被D整除的数字个数，K可达10000位。

观察与规模：K可达10000位，D≤100；应将数位和只保留模D，不能把上界转成64位整数。 上界可有上万位，枚举每个数不可能；合法性由相邻数字或数位和决定。

朴素思路：直接从1枚举到上界，即使检查单个数很快也无法完成。记住整个前缀又会有指数个状态。

关键突破：只保留前缀对未来的影响，另加tight表示当前前缀是否仍等于上界。

状态与建模：dp[s][tight]为已处理位置、自动机状态s及紧贴标记的前缀数。Windy状态0表示未开始，其余表示末位；Digit Sum状态是数位和模D。

本题映射：1..K中数位和能被D整除的数字个数，K可达10000位。 数位和只需模D的余数，滚动状态避免把巨大上界转整数。 digit_sum(D,1e9+7)，qur(K)后减去0。

转移与正确性：枚举当前位d≤hi，经go[s][d]转移至v，非法转移-1跳过；新tight=旧tight且d等于当前上界位。每个数字串对应唯一逐位路径。

初始化与顺序：空前缀dp[initial][1]=1，逐位正序并清空tmp。Windy前导0留在未开始状态，末位状态才接受；Digit Sum允许全0，最后减1去掉数字0。区间用F(R)-F(L-1)。

优化与限制：时间O(位数×状态数×10)，空间O(状态数)。不取模时答案须能放入64位；相邻差限制的状态不能把未开始和末位0混为一谈。

本题复杂度：O(|K|*D*10)时间、O(D)空间。

复用模板：数位DP与有限状态（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    string K;
    int D;
    cin >> K >> D;
    DigitDP t;
    t.digit_sum(D, 1000000007);
    cout << (t.qur(K) - 1 + 1000000007) % 1000000007 << '\n';
}
```

启发式总结：当历史可以压成有限信息，就尝试状态机；状态是否足够要通过两个同状态前缀的后续合法性完全相同来检验。课堂连续三位递增另记末位、连续长度和是否已出现。

验证：C++17组合程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 数数三不把前导0当有效位；digit_sum接受0，例题需减去。
- mod=0仅在总计数不会溢出long long时使用。
- 原框架的x+1与当前含上界API不同，区间统一qur(R)-qur(L-1)。

当历史可以压成有限信息，就尝试状态机；状态是否足够要通过两个同状态前缀的后续合法性完全相同来检验。课堂连续三位递增另记末位、连续长度和是否已出现。

个人资料来源：dp/数位动态规划进阶_比赛板子.md

字符串、线性代数与几何

字符串、线性代数与几何 / KMP字符串匹配

类型与用途：模板型；前缀函数失配跳转在线性时间查找全部匹配；数字序列与字符串共用接口。

复杂度与适用范围：O(n+m)时间、O(m+匹配数)空间。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// KMP字符串匹配：前缀函数失配跳转在线性时间查找全部匹配；数字序列与字符串共用接口。
// O(n+m)时间、O(m+匹配数)空间。
struct KMP {
    vector<int> nxt, pat;
    void init(const vector<int> &a) {
        pat = a;
        int n = a.size();
        nxt.assign(n, 0);
        for (int i = 1, j = 0; i < n; i++) {
            while (j && a[i] != a[j])
                j = nxt[j - 1];
            if (a[i] == a[j])
                j++;
            nxt[i] = j;
        }
    }
    void init(const string &s) {
        init(vector<int>(s.begin(), s.end()));
    }
    vector<int> find(const vector<int> &a) {
        vector<int> ans;
        int m = pat.size();
        if (!m) {
            for (int i = 0; i <= (int)a.size(); i++)
                ans.push_back(i + 1);
            return ans;
        }
        for (int i = 0, j = 0; i < (int)a.size(); i++) {
            while (j && a[i] != pat[j])
                j = nxt[j - 1];
            if (a[i] == pat[j])
                j++;
            if (j == m)
                ans.push_back(i - m + 2), j = nxt[j - 1];
        }
        return ans;
    }
    vector<int> find(const string &s) {
        return find(vector<int>(s.begin(), s.end()));
    }
};

void solve() {
    string s, p;
    cin >> s >> p;
    KMP t;
    t.init(p);
```

```
    for (int x : t.find(s))
        cout << x << '\n';
    for (int i = 0; i < (int)p.size(); i++)
        cout << t.nxt[i] << " \n"[i + 1 == (int)p.size()];
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

init(pattern)：建立模式前缀函数nxt，0基数组

find(text)：返回全部1基匹配起点，支持重叠匹配

例题 1 P3375 KMP

https://www.luogu.com.cn/problem/P3375

题意：输出模式串所有出现位置及每个前缀的最长border。

观察与建模：输出模式串所有出现位置及每个前缀的最长border。 重复扫描通过前缀函数复用已知匹配长度。

为何对应此算法：重复扫描通过前缀函数复用已知匹配长度。

初始化、组织数据与调用：init(s2)，find(s1)，依次输出nxt。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(|s1|+|s2|)时间、O(|s2|+匹配数)空间。

复用模板：KMP字符串匹配（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    string s, p;
    cin >> s >> p;
    KMP t;
    t.init(p);
    for (int x : t.find(s))
        cout << x << '\n';
    for (int i = 0; i < (int)p.size(); i++)
        cout << t.nxt[i] << " \n"[i + 1 == (int)p.size()];
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF471D MUH and Cube Walls

https://codeforces.com/contest/471/problem/D

题意：允许整体上下平移，统计目标高度序列出现次数。

观察与建模：允许整体上下平移，统计目标高度序列出现次数。 两序列可整体平移等价相邻差相同，转成整数序列匹配。

为何对应此算法：两序列可整体平移等价相邻差相同，转成整数序列匹配。

初始化、组织数据与调用：w=1直接答n；否则差分后init和find。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n+w)时间与O(n+w)空间。

复用模板：KMP字符串匹配（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, w;
    cin >> n >> w;
    vector<int> a(n), b(w);
    for (int &x : a)
        cin >> x;
    for (int &x : b)
        cin >> x;
    if (w == 1) {
        cout << n << '\n';
        return;
    }
    vector<int> x, y;
    for (int i = 1; i < n; i++)
        x.push_back(a[i] - a[i - 1]);
    for (int i = 1; i < w; i++)
        y.push_back(b[i] - b[i - 1]);
    KMP t;
    t.init(y);
    cout << t.find(x).size() << '\n';
}
```

验证：C++17组合程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 匹配后回退到nxt[m-1]，不能直接置0，否则漏重叠。
- 空模式按接口定义匹配n+1个边界；实际竞赛例题模式非空。
- 差分会出现负数，不能用char保存数字模式。

个人资料来源：杂/kmp.cpp

字符串、线性代数与几何 / 点直线与线段几何

类型与用途：模板型；保留P向量运算与相交、投影、反射、距离等接口，使用long double并处理退化线段。

复杂度与适用范围：单次运算O(1)时间/空间。

完整 C++17 板子

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 点直线与线段几何：保留P向量运算与相交、投影、反射、距离等接口，使用long double并处理退化线段
// 。
// 单次运算O(1)时间/空间。
using db = long double;
const db EPS = 1e-9L;
```

```
int sign(db x) {
    return x < -EPS ? -1 : x > EPS ? 1 : 0;
}
struct P {
    db x = 0, y = 0;
    P() {}
    P(db x, db y) : x(x), y(y) {}
    P operator+(P p) const {
        return {x + p.x, y + p.y};
    }
    P operator-(P p) const {
        return {x - p.x, y - p.y};
    }
    P operator*(db k) const {
        return {x * k, y * k};
    }
    P operator/(db k) const {
        return {x / k, y / k};
    }
    bool operator<(P p) const {
        return x != p.x ? x < p.x : y < p.y;
    }
    db dot(P p) const {
        return x * p.x + y * p.y;
    }
    db det(P p) const {
        return x * p.y - y * p.x;
    }
    db abs2() const {
        return dot(*this);
    }
    db abs() const {
        return sqrtl(abs2());
    }
    db distTo(P p) const {
        return (*this - p).abs();
    }
    db alpha() const {
        return atan2l(y, x);
    }
    P rot90() const {
        return {-y, x};
    }
    P rot(db a) const {
        return {x * cosl(a) - y * sinl(a), x * sinl(a) + y * cosl(a)};
    }
    P unit() const {
        assert(abs2() > 0);
        return *this / abs();
    }
    int quad() const {
        return y > 0 || (y == 0 && x >= 0);
    }
    void read() {
        cin >> x >> y;
    }
};
```

```
struct Geometry {
    db cross(P a, P b, P c) {
        return (b - a).det(c - a);
    }
    bool intersect(db a, db b, db c, db d) {
        if (a > b)
            swap(a, b);
        if (c > d)
            swap(c, d);
        return sign(b - c) >= 0 && sign(d - a) >= 0;
    }
    bool onSeg(P a, P b, P p) {
        return !sign(cross(a, b, p)) && sign((p - a).dot(p - b)) <= 0;
    }
    bool onSeg_strict(P a, P b, P p) {
        return !sign(cross(a, b, p)) && sign((p - a).dot(p - b)) < 0;
    }
    bool isSS(P a, P b, P c, P d) {
        return intersect(a.x, b.x, c.x, d.x) && intersect(a.y, b.y, c.y, d.y) &&
               sign(cross(a, b, c)) * sign(cross(a, b, d)) <= 0 &&
               sign(cross(c, d, a)) * sign(cross(c, d, b)) <= 0;
    }
    bool isSS_strict(P a, P b, P c, P d) {
        return sign(cross(a, b, c)) * sign(cross(a, b, d)) < 0 &&
               sign(cross(c, d, a)) * sign(cross(c, d, b)) < 0;
    }
    bool chkLL(P a, P b, P c, P d) {
        return sign((b - a).det(d - c)) != 0;
    }
    P isLL(P a, P b, P c, P d) {
        assert(chkLL(a, b, c, d));
        return a + (b - a) * ((c - a).det(d - c) / (b - a).det(d - c));
    }
    P proj(P a, P b, P p) {
        P v = b - a;
        assert(v.abs2() > 0);
        return a + v * (v.dot(p - a) / v.abs2());
    }
    P reflect(P a, P b, P p) {
        return proj(a, b, p) * 2 - p;
    }
    db nearest(P a, P b, P p) {
        if ((b - a).abs2() == 0)
            return p.distTo(a);
        P v = b - a;
        db t = clamp(v.dot(p - a) / v.abs2(), 0.0L, 1.0L);
        return p.distTo(a + v * t);
    }
    db disSS(P a, P b, P c, P d) {
        if (isSS(a, b, c, d))
            return 0;
        return min(
            {nearest(a, b, c), nearest(a, b, d), nearest(c, d, a), nearest(c, d, b)});
    }
};

void solve() {
    P a, b;
    a.read();
    b.read();
```

```
    Geometry t;
    int q;
    cin >> q;
    cout << fixed << setprecision(10);
    while (q--) {
        P p;
        p.read();
        P h = t.proj(a, b, p);
        cout << h.x << ' ' << h.y << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

P::dot/det/rot/unit：点积叉积、旋转和单位向量

isSS/isSS_strict/onSeg：含边界与严格相交/在线段判定

chkLL/isLL/proj/reflect：非平行直线交点、投影、反射

nearest/disSS：点段或段段距离，支持点状线段

例题 1 CGL_1_A Projection

https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_1_A

题意：求多点到给定直线的垂足。

观察与建模：求多点到给定直线的垂足。 点在直线方向上的正交分量由点积给出。

为何对应此算法：点在直线方向上的正交分量由点积给出。

初始化、组织数据与调用：读取a,b，每点proj后输出x,y。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(q)时间、O(1)额外空间。

复用模板：点直线与线段几何（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    P a, b;
    a.read();
    b.read();
    Geometry t;
    int q;
    cin >> q;
    cout << fixed << setprecision(10);
    while (q--) {
        P p;
        p.read();
        P h = t.proj(a, b, p);
        cout << h.x << ' ' << h.y << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CGL_1_B Reflection

https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_1_B

题意：求点关于直线的镜像。

观察与建模：求点关于直线的镜像。 垂足是点与镜像的中点，镜像=2*投影-原点。

为何对应此算法：垂足是点与镜像的中点，镜像=2*投影-原点。

初始化、组织数据与调用：调用reflect(a,b,p)。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(q)时间、O(1)额外空间。

复用模板：点直线与线段几何（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    P a, b;
    a.read();
    b.read();
    Geometry t;
    int q;
    cin >> q;
    cout << fixed << setprecision(10);
    while (q--) {
        P p;
        p.read();
        P h = t.reflect(a, b, p);
        cout << h.x << ' ' << h.y << '\n';
    }
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- sort用精确字典序；EPS比较不满足严格弱序，不能用于排序。
- proj要求a!=b，isLL要求非平行；nearest单独处理a==b。
- EPS按数据尺度调整；整数精确几何可改用__int128叉积。

个人资料来源：杂/点线.cpp

字符串、线性代数与几何 / 线性基

类型与用途：模板型；GF(2)消元维护可达异或空间，支持最大值、可表示判定、第k小及最大权线性无关集。

复杂度与适用范围：insert/exist/qur O(B)，kth O(Bˆ2)，空间O(B)，B=63。

完整 C++17 板子

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 线性基：GF(2)消元维护可达异或空间，支持最大值、可表示判定、第k小及最大权线性无关集。
// insert/exist/qur O(B)，kth O(B^2)，空间O(B)，B=63。
struct LinearBasis {
    static const int B = 62;
    array<int, B + 1> p{};
    bool zero = false;
    void init() {
        p.fill(0);
        zero = false;
    }
    bool insert(int x) {
        assert(x >= 0);
        for (int i = B; i >= 0; i--)
            if ((x >> i) & 1) {
                if (!p[i]) {
                    p[i] = x;
                    return true;
                }
                x ^= p[i];
            }
        zero = true;
        return false;
    }
    bool exist(int x) {
        for (int i = B; i >= 0; i--)
            if ((x >> i) & 1)
                x ^= p[i];
        return x == 0;
    }
    int qur(int x = 0) {
        for (int i = B; i >= 0; i--)
            x = max(x, x ^ p[i]);
        return x;
    }
    int rank() {
        int ans = 0;
        for (int x : p)
            ans += x != 0;
        return ans;
    }
    optional<int> kth(uint64_t k, bool nonempty = true) {
        auto b = p;
        for (int i = 0; i <= B; i++)
            if (b[i])
                for (int j = i + 1; j <= B; j++)
                    if ((b[j] >> i) & 1)
```

```
                        b[j] ^= b[i];
        vector<int> a;
        for (int x : b)
            if (x)
                a.push_back(x);
        // nonempty: k为1基；有非空零组合时，第1小为0。
        if (nonempty) {
            if (!k)
                return nullopt;
            if (zero)
                --k;
        }
        if (k >= (1ULL << a.size()))
            return nullopt;
        int ans = 0;
        for (int i = 0; i < (int)a.size(); i++)
            if ((k >> i) & 1)
                ans ^= a[i];
        return ans;
    }
};

void solve() {
    int n;
    cin >> n;
    LinearBasis t;
    while (n--) {
        int x;
        cin >> x;
        t.insert(x);
    }
    cout << t.qur() << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

insert(x)：独立时加入并返回true；相关时记录非空零组合

exist/qur(x)：判可表示；在x基础上求最大异或

kth(k,nonempty)：默认非空子集的1基第k小；nonempty=false则含空集且k为0基

例题 1 P3812 线性基

https://www.luogu.com.cn/problem/P3812

题意：从给定整数中选任意子集，使异或和最大。

观察与建模：从给定整数中选任意子集，使异或和最大。 所有子集异或构成向量空间，逐位贪心最大化。

为何对应此算法：所有子集异或构成向量空间，逐位贪心最大化。

初始化、组织数据与调用：逐个insert，输出qur()。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(nB)时间、O(B)空间。

复用模板：线性基（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    LinearBasis t;
    while (n--) {
        int x;
        cin >> x;
        t.insert(x);
    }
    cout << t.qur() << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P4151 最大XOR和路径

https://www.luogu.com.cn/problem/P4151

题意：无向连通图求1到n的最大异或权路径，可重复边点。

观察与建模：无向连通图求1到n的最大异或权路径，可重复边点。 任意路径异或为一条树路径值加环空间组合；每条非树边产生环异或。

为何对应此算法：任意路径异或为一条树路径值加环空间组合；每条非树边产生环异或。

初始化、组织数据与调用：DFS计算dis并insert(dis[u]^dis[v]^w)，最后qur(dis[n])。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O((n+m)B)时间、O(n+m+B)空间。

复用模板：线性基（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<vector<pair<int, int>>> g(n + 1);
    while (m--) {
        int u, v, w;
        cin >> u >> v >> w;
        g[u].push_back({v, w});
        g[v].push_back({u, w});
    }
    vector<int> vis(n + 1), dis(n + 1);
    LinearBasis t;
    function<void(int)> dfs = [&](int u) {
        vis[u] = 1;
        for (auto [v, w] : g[u])
            if (!vis[v]) {
                dis[v] = dis[u] ^ w;
                dfs(v);
            } else
                t.insert(dis[u] ^ dis[v] ^ w);
    };
    dfs(1);
    cout << t.qur(dis[n]) << '\n';
```

```
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 3 P4570 元素

https://www.luogu.com.cn/problem/P4570

题意：选择异或线性无关的矿石，使魔力总和最大。

观察与建模：选择异或线性无关的矿石，使魔力总和最大。 向量独立集构成拟阵；按魔力降序贪心，insert返回是否保持独立。

为何对应此算法：向量独立集构成拟阵；按魔力降序贪心，insert返回是否保持独立。

初始化、组织数据与调用：按Magic排序，只有insert成功才累加Magic。

具体调用顺序及变量对应见下面solve；区间开闭、值域和无解约定遵循本节接口，不能只凭函数名替换其他模板。

本题复杂度：O(n log n+nB)时间、O(n+B)空间。

复用模板：线性基（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    vector<pair<int, int>> a(n);
    for (auto &[v, x] : a)
        cin >> x >> v;
    sort(a.rbegin(), a.rend());
    LinearBasis t;
    int ans = 0;
    for (auto [v, x] : a)
        if (t.insert(x))
            ans += v;
    cout << ans << '\n';
}
```

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 输入限制0<=x<2^63，位移用1ULL避免第63位符号溢出。
- 相关向量使0可由非空子集构成；空集与非空的排名含义必须分清。
- 第k小先消成低位无干扰的基，原按低位组合但未消元的写法不保证顺序。

个人资料来源：杂/线性基.md

综合应用的可复用模块

以下板子在第四部分使用；这里只保存一次定义，建模推导与例题集中放在第四部分。

DSU on Tree / 速查模块

轻子树算完清空、重子树保留，以单一统计容器回答所有子树询问。

O(n log n)次增删；总时间再乘增删与询问成本；O(n)树结构空间。

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// DSU on Tree：轻子树算完清空、重子树保留，以单一统计容器回答所有子树询问。
// O(n log n)次增删；总时间再乘增删与询问成本；O(n)树结构空间。
struct DSUOnTree {
    int n, tot = 0;
    vector<vector<int>> adj;
    vector<int> fa, siz, son, l, r, seq;
    DSUOnTree(int n = 0) {
        init(n);
    }
    void init(int n_) {
        n = n_;
        tot = 0;
        adj.assign(n + 1, {});
        fa.assign(n + 1, 0);
        siz = son = l = r = fa;
        seq.assign(n + 1, 0);
    }
    void add_edge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void dfs1(int u, int p) {
        fa[u] = p;
        siz[u] = 1;
        l[u] = ++tot;
        seq[tot] = u;
        for (int v : adj[u])
            if (v != p) {
                dfs1(v, u);
                siz[u] += siz[v];
                if (siz[v] > siz[son[u]])
                    son[u] = v;
            }
        r[u] = tot;
    }
    template <class Change, class Answer>
    void dfs2(int u, bool keep, Change &change, Answer &answer) {
        for (int v : adj[u])
            if (v != fa[u] && v != son[u])
                dfs2(v, false, change, answer);
        if (son[u])
            dfs2(son[u], true, change, answer);
        for (int v : adj[u])
            if (v != fa[u] && v != son[u])
                for (int i = l[v]; i <= r[v]; i++)
```

```
                    change(seq[i], 1);
        change(u, 1);
        answer(u);
        if (!keep)
            for (int i = l[u]; i <= r[u]; i++)
                change(seq[i], -1);
    }
    template <class Change, class Answer>
    void run(Change change, Answer answer, int root = 1) {
        tot = 0;
        fill(son.begin(), son.end(), 0);
        dfs1(root, 0);
        dfs2(root, false, change, answer);
    }
};

void solve() {
    int n;
    cin >> n;
    vector<int> c(n + 1), ans(n + 1), cnt(n + 1), fc(n + 1), sum(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> c[i];
    DSUOnTree t(n);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        t.add_edge(u, v);
    }
    int mx = 0;
    auto change = [&](int u, int d) {
        int x = c[u], k = cnt[x];
        if (k)
            fc[k]--, sum[k] -= x;
        k = cnt[x] += d;
        if (k)
            fc[k]++, sum[k] += x;
        mx = max(mx, k);
        while (mx && !fc[mx])
            mx--;
    };
    auto answer = [&](int u) { ans[u] = sum[mx]; };
    t.run(change, answer);
    for (int i = 1; i <= n; i++)
        cout << ans[i] << " \n"[i == n];
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

add_edge：加入无向树边

run(change,answer,root)：change(u,+1/-1)增删，answer(u)读取完整子树状态

启发式合并 / 速查模块

每次枚举小容器合入大容器；保留颜色桶句柄交换与map统计两个原资料模型。

位置桶总O(n log n+q)，map树上合并O(n log^2n)；空间O(n+颜色数)。

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 启发式合并：每次枚举小容器合入大容器；保留颜色桶句柄交换与map统计两个原资料模型。
// 位置桶总O(n log n+q)，map树上合并O(n log^2n)；空间O(n+颜色数)。
struct SmallMerge {
    int n, segments;
    vector<int> a, id;
    vector<vector<int>> pos;
    SmallMerge(int n = 0) {
        init(n);
    }
    void init(int n_) {
        n = n_;
        a.assign(n + 2, 0);
        pos.assign(n + 1, {});
        id.clear();
        segments = 0;
    }
    void build(const vector<int> &col, int kinds) {
        id.resize(kinds + 1);
        iota(id.begin(), id.end(), 0);
        pos.resize(kinds + 1);
        for (int i = 1; i <= n; i++) {
            a[i] = col[i];
            pos[a[i]].push_back(i);
            segments += (i == 1 || a[i] != a[i - 1]);
        }
    }
    void change(int x, int y) {
        if (x == y)
            return;
        if (pos[id[x]].size() > pos[id[y]].size())
            swap(id[x], id[y]);
        int u = id[x], v = id[y];
        for (int p : pos[u]) {
            if (p > 1)
                segments -= (a[p] != a[p - 1]);
            if (p < n)
                segments -= (a[p] != a[p + 1]);
            a[p] = v;
            if (p > 1)
                segments += (a[p] != a[p - 1]);
            if (p < n)
                segments += (a[p] != a[p + 1]);
            pos[v].push_back(p);
        }
        vector<int>().swap(pos[u]);
    }
    // 另一种容器：小map并入大map，维护最大出现次数的颜色编号和。
    struct info {
        map<int, int> cnt;
        int mx = 0, sum = 0;
```

```
    };
    static void insert(info &s, int c, int k) {
        int x = (s.cnt[c] += k);
        if (x > s.mx)
            s.mx = x, s.sum = c;
        else if (x == s.mx)
            s.sum += c;
    }
    static void merge(info &a, info &b) {
        if (a.cnt.size() < b.cnt.size())
            swap(a, b);
        for (auto [c, k] : b.cnt)
            insert(a, c, k);
        b = info();
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1), all;
    for (int i = 1; i <= n; i++)
        cin >> a[i], all.push_back(a[i]);
    struct op {
        int t, x, y;
    };
    vector<op> q(m);
    for (auto &z : q) {
        cin >> z.t;
        if (z.t == 1) {
            cin >> z.x >> z.y;
            all.push_back(z.x);
            all.push_back(z.y);
        }
    }
    sort(all.begin(), all.end());
    all.erase(unique(all.begin(), all.end()), all.end());
    auto id = [&](int x) {
        return lower_bound(all.begin(), all.end(), x) - all.begin() + 1;
    };
    for (int i = 1; i <= n; i++)
        a[i] = id(a[i]);
    SmallMerge t(n);
    t.build(a, all.size());
    for (auto z : q)
        if (z.t == 1)
            t.change(id(z.x), id(z.y));
        else
            cout << t.segments << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

build(col,kinds)：建立颜色位置桶，颜色先离散化为1..kinds

change(x,y)：把逻辑颜色x全部变y，交换物理桶保证小并大

info/insert/merge：map颜色频率统计：合并后mx及sum为众数频率与颜色和

离线二维数点 / 速查模块

按x扫描，树状数组维护y；四次前缀容斥回答闭矩形计数，允许重复点。

O((n+q)log(n+q))时间、O(n+q)空间。

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 离线二维数点：按x扫描，树状数组维护y；四次前缀容斥回答闭矩形计数，允许重复点。
// O((n+q)log(n+q))时间、O(n+q)空间。
struct RectangleCounter {
    struct point {
        int x, y;
    };
    struct query {
        int l, b, r, t;
    };
    vector<point> a;
    vector<query> q;
    void init() {
        a.clear();
        q.clear();
    }
    void add(int x, int y) {
        a.push_back({x, y});
    }
    void ask(int l, int b, int r, int t) {
        assert(l <= r && b <= t);
        q.push_back({l, b, r, t});
    }
    vector<int> run() {
        struct event {
            int x, y, id, sgn;
            bool strictx, stricty;
        };
        vector<event> e;
        vector<int> ys, ans(q.size());
        for (auto p : a)
            ys.push_back(p.y);
        for (int i = 0; i < (int)q.size(); i++) {
            auto z = q[i];
            e.push_back({z.r, z.t, i, 1, false, false});
            e.push_back({z.l, z.t, i, -1, true, false});
            e.push_back({z.r, z.b, i, -1, false, true});
            e.push_back({z.l, z.b, i, 1, true, true});
        }
        sort(ys.begin(), ys.end());
        ys.erase(unique(ys.begin(), ys.end()), ys.end());
        sort(a.begin(), a.end(), [](point p, point q) { return p.x < q.x; });
        sort(e.begin(), e.end(), [](event p, event q) {
            if (p.x != q.x)
                return p.x < q.x;
            return p.strictx > q.strictx;
        });
```

```
        vector<int> c(ys.size() + 1);
        int j = 0;
        for (auto z : e) {
            while (j < (int)a.size() &&
                   (a[j].x < z.x || (!z.strictx && a[j].x == z.x))) {
                int k = lower_bound(ys.begin(), ys.end(), a[j].y) - ys.begin() + 1;
                for (; k < (int)c.size(); k += k & -k)
                    c[k]++;
                j++;
            }
            int k = (z.stricty ? lower_bound(ys.begin(), ys.end(), z.y)
                               : upper_bound(ys.begin(), ys.end(), z.y)) -
                    ys.begin(),
                s = 0;
            for (; k; k -= k & -k)
                s += c[k];
            ans[z.id] += z.sgn * s;
        }
        return ans;
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    RectangleCounter t;
    while (n--) {
        int x, y;
        cin >> x >> y;
        t.add(x, y);
    }
    while (m--) {
        int a, b, c, d;
        cin >> a >> b >> c >> d;
        t.ask(a, b, c, d);
    }
    for (int x : t.run())
        cout << x << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

add(x,y)：加入一个点，可重复

ask(l,b,r,t)：加入闭矩形询问

run()：返回询问顺序的计数

线段树优化建图 / 速查模块

点到区间、区间到点建有向边；两棵树方向相反，原点共享叶子。

建图O(n+q log n)，非负最短路O((n+q log n)log(n+q))，空间O(n+q log n)。

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 线段树优化建图：点到区间、区间到点建有向边；两棵树方向相反，原点共享叶子。
// 建图O(n+q log n)，非负最短路O((n+q log n)log(n+q))，空间O(n+q log n)。
const int INF = 4000000000000000000LL;
using pii = pair<int, int>;
struct SegGraph {
    int n;
    int tot;

    // out：点 -> 区间，父 -> 子
    // in ：区间 -> 点，子 -> 父
    vector<int> out, in;

    struct edge {
        int to, w;
    };

    vector<vector<edge>> adj;

    inline int ls(int id) {
        return id << 1;
    }

    inline int rs(int id) {
        return id << 1 | 1;
    }

    SegGraph(int _n) {
        n = _n;
        tot = n;

        out.resize(4 * n + 10);
        in.resize(4 * n + 10);

        // 总节点约 3n
        adj.resize(4 * n + 10);

        build_out(1, 1, n);
        build_in(1, 1, n);
    }

    // 普通建边
    void add_edge(int u, int v, int w) {
        if ((int)adj.size() <= max(u, v))
            adj.resize(max(u, v) + 1);
        adj[u].push_back({v, w});
    }

    /*
        点 -> 区间

```

```
        out 树:

              父
             /  \
            ↓    ↓
           左    右
    */
    void build_out(int id, int l, int r) {
        if (l == r) {
            out[id] = l;
            return;
        }

        out[id] = ++tot;

        int mid = (l + r) >> 1;

        build_out(ls(id), l, mid);
        build_out(rs(id), mid + 1, r);

        add_edge(out[id], out[ls(id)], 0);
        add_edge(out[id], out[rs(id)], 0);
    }

    /*
        区间 -> 点

        in 树:

           左    右
            ↓    ↓
              父
    */
    void build_in(int id, int l, int r) {
        if (l == r) {
            in[id] = l;
            return;
        }

        in[id] = ++tot;

        int mid = (l + r) >> 1;

        build_in(ls(id), l, mid);
        build_in(rs(id), mid + 1, r);

        add_edge(in[ls(id)], in[id], 0);
        add_edge(in[rs(id)], in[id], 0);
    }

    /*
        v -> [ql, qr]
    */
    void add_out(int id, int l, int r, int ql, int qr, int v, int w) {
        if (ql <= l && r <= qr) {
            add_edge(v, out[id], w);
            return;
        }

        int mid = (l + r) >> 1;
```

```
115
116         if (ql <= mid)
117             add_out(ls(id), l, mid, ql, qr, v, w);
118
119         if (qr > mid)
120             add_out(rs(id), mid + 1, r, ql, qr, v, w);
121     }
122
123     // 外部调用版本
124     void add_out(int v, int l, int r, int w) {
125         add_out(1, 1, n, l, r, v, w);
126     }
127
128     /*
129         [ql, qr] -> v
130     */
131     void add_in(int id, int l, int r, int ql, int qr, int v, int w) {
132         if (ql <= l && r <= qr) {
133             add_edge(in[id], v, w);
134             return;
135         }
136
137         int mid = (l + r) >> 1;
138
139         if (ql <= mid)
140             add_in(ls(id), l, mid, ql, qr, v, w);
141
142         if (qr > mid)
143             add_in(rs(id), mid + 1, r, ql, qr, v, w);
144     }
145
146     // 外部调用版本
147     void add_in(int l, int r, int v, int w) {
148         add_in(1, 1, n, l, r, v, w);
149     }
150
151     /*
152         Dijkstra
153     */
154     vector<int> dijkstra(int s) {
155         vector<int> dis(tot + 1, INF);
156         vector<bool> vis(tot + 1);
157
158         priority_queue<pii, vector<pii>, greater<pii>> pq;
159
160         dis[s] = 0;
161         pq.push({0, s});
162
163         while (!pq.empty()) {
164             auto [d, u] = pq.top();
165             pq.pop();
166
167             if (vis[u])
168                 continue;
169             vis[u] = true;
170
171             for (auto [v, w] : adj[u]) {
172                 if (dis[v] > d + w) {
173                     dis[v] = d + w;
174                     pq.push({dis[v], v});
```

```
                }
            }
        }

        return dis;
    }
};

void solve() {
    int n, q, s;
    cin >> n >> q >> s;
    SegGraph sg(n);
    while (q--) {
        int op, v, l, r, w;
        cin >> op;
        if (op == 1) {
            cin >> v >> l >> w;
            sg.add_edge(v, l, w);
        } else {
            cin >> v >> l >> r >> w;
            if (op == 2)
                sg.add_out(v, l, r, w);
            else
                sg.add_in(l, r, v, w);
        }
    }
    auto d = sg.dijkstra(s);
    for (int i = 1; i <= n; i++)
        cout << (d[i] == INF ? -1 : d[i]) << (i == n ? '\n' : ' ');
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

add_edge(u,v,w)：普通边，允许新增辅助点

add_out(v,l,r,w)：点到区间

add_in(l,r,v,w)：区间到点

dijkstra(s)：非负边权，返回距离

Kruskal重构树 / 速查模块

按权值排序建立连通块合并历史；支持不连通森林与LCA瓶颈查询。

排序O(m log m)，仅路径压缩的并查集按O((n+m)log n)保守估计，倍增O(n log n)；空间O(n log n+m)。

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// Kruskal重构树：按权值排序建立连通块合并历史；支持不连通森林与LCA瓶颈查询。
// 排序O(m log m)，建树与倍增O(n log n+m alpha(n))的通常界；当前重构树代表递归find可有长链，
// 空间O(n log n+m)。
struct KruskalTree {
    struct edge {
        int u, v, w;
    };
    int n, tot, LOG;
    vector<int> fa, val, dep, bel;
    vector<vector<int>> tr, up;
    vector<edge> e;
    void init(int n_) {
        n = n_;
        tot = n;
        e.clear();
        fa.resize(2 * n + 1);
        iota(fa.begin(), fa.end(), 0);
        val.assign(2 * n + 1, 0);
        tr.assign(2 * n + 1, {});
        dep.assign(2 * n + 1, 0);
        bel.assign(2 * n + 1, 0);
        LOG = 1;
        while ((1LL << LOG) <= 2 * n)
            LOG++;
        up.assign(LOG, vector<int>(2 * n + 1));
    }
    int find(int x) {
        return fa[x] == x ? x : fa[x] = find(fa[x]);
    }
    void addedge(int u, int v, int w) {
        e.push_back({u, v, w});
    }
    void dfs(int u, int f, int component) {
        up[0][u] = f;
        dep[u] = dep[f] + 1;
        bel[u] = component;
        for (int j = 1; j < LOG; j++)
            up[j][u] = up[j - 1][up[j - 1][u]];
        for (int v : tr[u])
            dfs(v, u, component);
    }
    void build(bool maximum = false) {
        sort(e.begin(), e.end(),
             [&](edge a, edge b) { return maximum ? a.w > b.w : a.w < b.w; });
        for (auto [u, v, w] : e) {
            int x = find(u), y = find(v);
            if (x == y)
                continue;
            ++tot;
            val[tot] = w;
```

```
            tr[tot] = {x, y};
            fa[x] = fa[y] = tot;
        }
        for (int u = 1; u <= tot; u++)
            if (fa[u] == u)
                dfs(u, 0, u);
    }
    int lca(int u, int v) {
        if (bel[u] != bel[v])
            return 0;
        if (dep[u] < dep[v])
            swap(u, v);
        int d = dep[u] - dep[v];
        for (int j = 0; j < LOG; j++)
            if (d >> j & 1)
                u = up[j][u];
        if (u == v)
            return u;
        for (int j = LOG - 1; j >= 0; j--)
            if (up[j][u] != up[j][v]) {
                u = up[j][u];
                v = up[j][v];
            }
        return up[0][u];
    }
    int qur(int u, int v) {
        int p = lca(u, v);
        return p ? val[p] : -1;
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    KruskalTree tr;
    tr.init(n);
    while (m--) {
        int u, v, w;
        cin >> u >> v >> w;
        tr.addedge(u, v, w);
    }
    tr.build(false);
    int q;
    cin >> q;
    while (q--) {
        int u, v;
        cin >> u >> v;
        int p = tr.lca(u, v);
        if (!p)
            cout << "impossible\n";
        else
            cout << tr.val[p] << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
```

```
    while (t--)
        solve();
    return 0;
}
```

build(false)：升序：最小化路径最大边

build(true)：降序：最大化路径最小边

lca(u,v)：不连通返回0

qur(u,v)：LCA权值，不连通-1，同点约定0

最小割建模 / 速查模块

复用Dinic，支持最大权闭合子图、二元选择与二分图最大权独立集。

最大流一般O(V^2 E)，源侧恢复O(V+E)；空间O(V+E)。

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 最小割建模：复用Dinic，支持最大权闭合子图、二元选择与二分图最大权独立集。
// 最大流一般O(V^2 E)，源侧恢复O(V+E)；空间O(V+E)。
template <typename T> struct FlowGraph {
    int s, t; // 源/汇编号1..vtot，要求s!=t；init赋值。
    int vtot; // 顶点总数，>=1；包含全部辅助节点。
    int etot; // 已存残量边数，等于e.size()，始终偶数；边号须在int范围内；init清0。
    // head：一维vector，init分配vtot+1项，下标1..vtot；邻接链首边编号，-1为空；init置-1。
    vector<int> head;
    // dis：一维vector，init分配vtot+1项并清0；下标1..vtot；BFS层数，0未访问，源为1；DFS死点
    // 置-1；每次bfs清0。
    vector<int> dis;
    // cur：一维vector，init分配vtot+1项并置-1；下标1..vtot；当前弧下标或-1；每次bfs由head复
    // 制，DFS沿链推进。
    vector<int> cur;
    struct edge {
        int v;   // 边终点，1..vtot。
        int nxt; // 同起点的下一边编号，-1或0..etot-1；由addedge填写。
        T f;     // 当前残量，>=0；正向初始为输入容量，反向初始0。
    };
    // e：一维动态边表，下标0..etot-1；配对边i^1。
    // init执行e.clear并令etot=0；addedge成对push_back，不提前resize占用边号。
    vector<edge> e;

    // init(s_,t_,vtot_)：设置源汇与点数；返回void；清边数与邻接链。
    // 前提合法编号、s_!=t_，随后重新建图；首次时间O(vtot_)；重建还包括清除旧边。
    void init(int s_, int t_, int vtot_) {
        s = s_;
        t = t_;
        vtot = vtot_;
        etot = 0; // 多次建图必须清etot。
        head.assign(vtot + 1, -1);
        dis.assign(vtot + 1, 0);
        cur.assign(vtot + 1, -1);
        e.clear();
    }
    // addedge(u,v,f)：加入原始有向边及撤销边；返回void；两次push_back摊还时间O(1)。
    // 前提u/v=1..vtot，f>=0，边号及etot不溢出int；原容量需另存时由调用方记录。
```

```
    void addedge(int u, int v, T f) {
        e.push_back({v, head[u], f});
        head[u] = etot++;
        e.push_back({u, head[v], 0});
        head[v] = etot++;
    }
    // bfs(): 无参数；返回残量图中汇是否可达；完成分层并重置cur。
    // 前提图已建好或上轮增广完成；时间O(vtot+etot)。
    bool bfs() {
        for (int i = 1; i <= vtot; i++) {
            dis[i] = 0;
            cur[i] = head[i];
        }
        queue<int> q; // 一维BFS队列，初始空；存顶点1..vtot，每点至多入队一次。
        q.push(s);
        dis[s] = 1;
        while (!q.empty()) {
            int u = q.front();
            q.pop();                                          // u: 当前扩展顶点，1..vtot。
            for (int i = head[u]; i != -1; i = e[i].nxt) { // i: 当前残量边编号。
                if (e[i].f > 0 && dis[e[i].v] == 0) {
                    int v = e[i].v; // v: 当前可达边终点，1..vtot。
                    dis[v] = dis[u] + 1;
                    if (v == t)
                        return true;
                    q.push(v);
                }
            }
        }
        return false;
    }
    // dfs(u,m): u当前点；m为本次允许送出的剩余流量，m>0；返回实际送出量T。
    // 作用：按分层图多路增广；前提bfs成功，cur已重置；一轮阻塞流一般O(VE)。
    T dfs(int u, T m) {
        if (u == t)
            return m;
        T flow = 0; // 本次从u成功送出的总量，0..初始m。
        for (int i = cur[u]; i != -1; cur[u] = i = e[i].nxt) {
            // i: 当前弧；保留PPT的cur推进方式，送满本次额度时不前移。
            if (e[i].f > 0 && dis[e[i].v] == dis[u] + 1) {
                T f = dfs(e[i].v, min(m, e[i].f)); // 实际成功增广量，0..min(m,残量)。
                e[i].f -= f;
                e[i ^ 1].f += f;
                m -= f;
                flow += f;
                if (m == 0)
                    break;
            }
        }
        if (flow == 0)
            dis[u] = -1;
        return flow;
    }
    // dinic(): 无参数；返回本次新增最大流T；作用：反复分层并求阻塞流。
    // 前提合法源汇、图已建好、总流量不溢出；一般时间O(Vnet^2*Enet)，空间O(Vnet+Enet)，额外递
    // 归栈O(Vnet)。
    T dinic() {
        T flow = 0; // 本次调用新增流量，init后首次调用即总最大流。
        while (bfs())
            flow += dfs(s, numeric_limits<T>::max());
```

```
        return flow;
    }
    vector<int> cut_side() {
        vector<int> vis(vtot + 1);
        queue<int> q;
        q.push(s);
        vis[s] = 1;
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int i = head[u]; i != -1; i = e[i].nxt)
                if (e[i].f > 0 && !vis[e[i].v]) {
                    vis[e[i].v] = 1;
                    q.push(e[i].v);
                }
        }
        return vis;
    }
};
struct SCC {
    // 以下顶点表均由init按n+1项assign重建；顶点下标1..n，0号预留不用。
    // 内层邻接位置从0开始，init清空旧边；标记/时间戳/归属表清0。
    int n;   // 顶点数，n>=1；顶点编号为 1..n。
    int idx; // DFS 时钟，范围 [0,n]；每首次访问一个点加 1。
    int cnt; // SCC 数，范围 [0,n]；按弹栈先后编号。

    // e：二维变长邻接表；e[u][j] 为 u 的第 j 条出边终点。
    // u=1..n，j=0..e[u].size()-1；init 中 clear；允许重边、自环。
    vector<vector<int>> e;
    // dfn：一维；dfn[u] 为首次访问时间，0 表示未访问；init 清 0；有效值 1..n。
    vector<int> dfn;
    // low：一维；low[u] 为子树按 Tarjan 规则能追溯到的最早未封闭时间戳。
    // 下标 1..n；init 清 0；首次访问置 dfn[u]，随后取 min。
    vector<int> low;
    // ins：一维；ins[u] 为 u 是否仍在 Tarjan 栈中；init 清0，标记取0/1，入栈置 true，弹栈置
    //  false。
    vector<int> ins;
    // bel：一维；bel[u] 为 SCC 编号 1..cnt；init 清 0；0 表示尚未分配。
    vector<int> bel;
    // stk：一维栈，元素为顶点编号 1..n；init 弹空；最多存 n 个点。
    stack<int> stk;
    // scc：二维变长表；scc[c-1] 为 SCC c 的全部原图顶点；init clear；总成员数 n。
    vector<vector<int>> scc;

    // init(n_)：参数为顶点数；返回 void；清图和 Tarjan 状态。
    // 前提：n_>=1，下一步重新加边；首次初始化时间O(n_)；复用对象还需销毁旧图的邻接元素。
    void init(int n_) {
        n = n_;
        idx = cnt = 0;
        e.assign(n + 1, {});
        dfn.assign(n + 1, 0);
        low.assign(n + 1, 0);
        ins.assign(n + 1, 0);
        bel.assign(n + 1, 0);
        while (!stk.empty())
            stk.pop();
        scc.clear();
    }

    // dfs(u)：u 为当前顶点；返回 void；递归求 SCC 并填写 bel、scc。
```

```
    // 前提：图已建好，dfn[u]==0；一次 DFS 为 O(所访问顶点+扫描边)，全图 O(n+m)。
    void dfs(int u) {
        dfn[u] = low[u] = ++idx;
        ins[u] = true;
        stk.push(u);
        for (int v : e[u]) { // v：出边终点，范围 1..n。
            if (!dfn[v]) {
                dfs(v);
                low[u] = min(low[u], low[v]);
            } else if (ins[v]) {
                low[u] = min(low[u], dfn[v]);
            }
        }
        if (dfn[u] == low[u]) {
            ++cnt;
            vector<int> c; // 一维临时点集，初始为空；存当前 SCC 顶点，大小至少 1。
            while (true) {
                int v = stk.top(); // v：当前弹出顶点，1..n；最后一次必为 u。
                stk.pop();
                ins[v] = false;
                bel[v] = cnt;
                c.push_back(v);
                if (v == u)
                    break;
            }
            scc.push_back(c);
        }
    }

    // run()：无参数，返回 void；遍历 DFS 森林，完成全图 SCC。
    // 前提：已 init 并加边，尚未运行；时间 O(n+m)，递归栈最坏 O(n)。
    void run() {
        for (int u = 1; u <= n; u++)
            if (!dfn[u])
                dfs(u);
    }

    // addedge(u,v)：参数为有向边起终点1..n；返回void；前提已init且尚未run。
    // 作用：只封装原PPT邻接表push_back；摊还时间O(1)。
    void addedge(int u, int v) {
        e[u].push_back(v);
    }
};

void solve() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> a(n, vector<int>(m));
    int total = 0;
    for (auto &row : a)
        for (int &x : row) {
            cin >> x;
            total += x;
        }
    int s = n * m + 1, t = s + 1;
    FlowGraph<int> g;
    g.init(s, t, t);
    int dx[4] = {1, -1, 0, 0}, dy[4] = {0, 0, 1, -1};
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < m; j++) {
            int u = i * m + j + 1;
            if ((i + j) % 2 == 0) {
                g.addedge(s, u, a[i][j]);
                for (int k = 0; k < 4; k++) {
                    int x = i + dx[k], y = j + dy[k];
                    if (0 <= x && x < n && 0 <= y && y < m)
                        g.addedge(u, x * m + y + 1, total + 1);
                }
            } else
                g.addedge(u, t, a[i][j]);
        }
    cout << total - g.dinic() << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

dinic()：最大流值等于最小割

cut_side()：1为源侧，0为汇侧

addedge(u,v,INF)：禁止把u源侧、v汇侧；INF须超过有限割上界

度数根号分治 / 速查模块

小度点广播直接修改邻居，大度点广播记录时间戳；查询从相邻大点恢复最后一次颜色。

预处理O(n+m)，每次O(B+m/B)，B约sqrt(m)；空间O(n+m)。

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 度数根号分治：小度点广播直接修改邻居，大度点广播记录时间戳；查询从相邻大点恢复最后一次颜色。
// 预处理O(n+m)，每次O(B+m/B)，B约sqrt(m)；空间O(n+m)。
struct DegreeDivide {
    int n, stamp = 0, B;
    vector<vector<int>> g, heavy;
    vector<int> big, col, tim, lazy, ltim;
    DegreeDivide(int n = 0) {
        init(n);
    }
    void init(int n_) {
        n = n_;
        stamp = 0;
        g.assign(n + 1, {});
        heavy = g;
        big.assign(n + 1, 0);
        col = tim = lazy = ltim = big;
    }
    void add_edge(int u, int v) {
        g[u].push_back(v);
        g[v].push_back(u);
    }
```

```
    void build(const vector<int> &a) {
        col = a;
        int m = 0;
        for (int i = 1; i <= n; i++)
            m += g[i].size();
        B = sqrt(max(1LL, m)) + 1;
        for (int i = 1; i <= n; i++)
            big[i] = (int)g[i].size() >= B;
        for (int i = 1; i <= n; i++)
            for (int v : g[i])
                if (big[v])
                    heavy[i].push_back(v);
    }
    int qur(int u) {
        for (int v : heavy[u])
            if (ltim[v] > tim[u])
                col[u] = lazy[v], tim[u] = ltim[v];
        return col[u];
    }
    void change(int u, int c, bool self = false) {
        ++stamp;
        if (self)
            col[u] = c, tim[u] = stamp;
        if (big[u])
            lazy[u] = c, ltim[u] = stamp;
        else
            for (int v : g[u])
                col[v] = c, tim[v] = stamp;
    }
};

void solve() {
    int n, m, q;
    cin >> n >> m >> q;
    DegreeDivide t(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        t.add_edge(u, v);
    }
    vector<int> a(n + 1);
    iota(a.begin(), a.end(), 0);
    t.build(a);
    while (q--) {
        int x;
        cin >> x;
        int c = t.qur(x);
        t.change(x, c);
    }
    for (int i = 1; i <= n; i++)
        cout << t.qur(i) << " \n"[i == n];
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
```

```
    return 0;
}
```

add_edge/build(a)：静态无向图，a为1基初始颜色

qur(u)：读取含懒广播影响的当前颜色

change(u,c,self)：向所有邻居广播c；self=true也更新自己

步长根号分治 / 速查模块

小步长预维护/预处理，大步长暴力跳跃；收录动态模下标和与静态跳跃两个模型。

动态修改O(B)、查询O(1)或O(n/B)，空间O(n+B^2)；跳跃预处理O(nB)、空间O(nB)，B约sqrt(n)。

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 步长根号分治：小步长预维护/预处理，大步长暴力跳跃；收录动态模下标和与静态跳跃两个模型。
// 动态修改O(B)、查询O(1)或O(n/B)，空间O(n+B^2)；跳跃预处理O(nB)、空间O(nB)，B约sqrt(n)。
struct SqrtDivide {
    int n, B;
    vector<int> a;
    vector<vector<int>> tag;
    vector<vector<int32_t>> jump;
    SqrtDivide(int n = 0) {
        init(n);
    }
    void init(int n_) {
        n = n_;
        B = sqrt(max(1LL, n)) + 1;
        a.assign(n + 1, 0);
        tag.assign(B, {});
        for (int i = 1; i < B; i++)
            tag[i].assign(i, 0);
        jump.clear();
    }
    void change(int x, int v) {
        assert(1 <= x && x <= n);
        a[x] += v;
        for (int k = 1; k < B; k++)
            tag[k][x % k] += v;
    }
    int qur(int k, int r) {
        assert(k > 0 && 0 <= r && r < k);
        if (k < B)
            return tag[k][r];
        int ans = 0;
        for (int i = r ? r : k; i <= n; i += k)
            ans += a[i];
        return ans;
    }
    void build_jump(const vector<int> &v) {
        a = v;
        jump.assign(B, vector<int32_t>(n + 2));
        for (int k = 1; k < B; k++)
            for (int i = n; i >= 1; i--) {
                int j = i + a[i] + k;
                jump[k][i] = 1 + (j <= n ? jump[k][j] : 0);
            }
    }
```

```
    int jump_count(int p, int k) {
        if (k < B)
            return jump[k][p];
        int ans = 0;
        while (p <= n)
            ans++, p += a[p] + k;
        return ans;
    }
};

void solve() {
    int q;
    cin >> q;
    SqrtDivide t(500000);
    while (q--) {
        int op, x, y;
        cin >> op >> x >> y;
        if (op == 1)
            t.change(x, y);
        else
            cout << t.qur(x, y) << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

change/qur：动态单点加与下标模k等于r求和

build_jump/jump_count：静态正数数组跳到p+a[p]+k，求越界前次数

曼哈顿距离坐标旋转 / 速查模块

用u=x+y,v=x-y把平面曼哈顿距离变为旋转坐标的切比雪夫距离；最大距离只需两维极值。

add/qur/diameter O(1)；建n点O(n)时间/O(1)额外空间。

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 曼哈顿距离坐标旋转：用u=x+y,v=x-y把平面曼哈顿距离变为旋转坐标的切比雪夫距离；最大距离只需两
// 维极值。
// add/qur/diameter O(1)；建n点O(n)时间/O(1)额外空间。
struct Manhattan {
    int mn1 = LLONG_MAX, mx1 = LLONG_MIN, mn2 = LLONG_MAX, mx2 = LLONG_MIN;
    void init() {
        mn1 = mn2 = LLONG_MAX;
        mx1 = mx2 = LLONG_MIN;
    }
    void add(int x, int y) {
        int u = x + y, v = x - y;
        mn1 = min(mn1, u);
        mx1 = max(mx1, u);
```

```
        mn2 = min(mn2, v);
        mx2 = max(mx2, v);
    }
    int diameter() {
        assert(mn1 != LLONG_MAX);
        return max(mx1 - mn1, mx2 - mn2);
    }
    int qur(int x, int y) {
        assert(mn1 != LLONG_MAX);
        int u = x + y, v = x - y;
        return max({llabs(u - mn1), llabs(u - mx1), llabs(v - mn2), llabs(v - mx2)});
    }
};

struct ManhattanSeg {
    int n;
    vector<Manhattan> seg;
    ManhattanSeg(int n = 0) : n(n), seg(4 * n + 4) {}
    Manhattan merge(Manhattan a, Manhattan b) {
        a.mn1 = min(a.mn1, b.mn1);
        a.mx1 = max(a.mx1, b.mx1);
        a.mn2 = min(a.mn2, b.mn2);
        a.mx2 = max(a.mx2, b.mx2);
        return a;
    }
    void change(int u, int l, int r, int p, int x, int y) {
        if (l == r) {
            seg[u].init();
            seg[u].add(x, y);
            return;
        }
        int m = (l + r) >> 1;
        if (p <= m)
            change(u << 1, l, m, p, x, y);
        else
            change(u << 1 | 1, m + 1, r, p, x, y);
        seg[u] = merge(seg[u << 1], seg[u << 1 | 1]);
    }
    Manhattan qur(int u, int l, int r, int L, int R) {
        if (L <= l && r <= R)
            return seg[u];
        int m = (l + r) >> 1;
        Manhattan ans;
        if (L <= m)
            ans = merge(ans, qur(u << 1, l, m, L, R));
        if (R > m)
            ans = merge(ans, qur(u << 1 | 1, m + 1, r, L, R));
        return ans;
    }
    void change(int p, int x, int y) {
        change(1, 1, n, p, x, y);
    }
    Manhattan qur(int l, int r) {
        return qur(1, 1, n, l, r);
    }
};

void solve() {
    int n;
    cin >> n;
```

```
    Manhattan t;
    while (n--) {
        int x, y;
        cin >> x >> y;
        t.add(x, y);
    }
    cout << t.diameter() << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

add(x,y)：加入点，维护两维极值

diameter()：已有点中最大曼哈顿距离

qur(x,y)：询问到已有点的最大曼哈顿距离

第三部分 算法思维与经典模型

动态规划：模型与推导

动态规划 / 二状态树形背包

类型与用途：思维型；每点选0/1状态，价值来自点费用和父子状态费用；背包维记录状态1点数，覆盖原Monster Hunter模型。

复杂度与适用范围：按真实子树规模合并O(n^2)时间；保存全部节点DP最坏O(n^2)空间。

例题 1 Gym102992M Monster Hunter

https://codeforces.com/gym/102992/problem/M

题意：有限魔法可无成本杀任意怪，普通杀怪受父亲和存活孩子血量影响，求每个魔法次数的最小总力量。

观察与规模：n≤2000，施法次数k从0到n全部输出；父子都普通时子怪物多一次hp，点费和额外边费各付一次。 父子选择存在相互作用，且要按使用特殊操作数量输出最小代价；一个数量维无法判断边的费用。

朴素思路：枚举施法/选择节点子集指数级；只记用了k次，会把父亲是否施法的信息丢掉。

关键突破：只需把当前根的二值状态作为子树与外界的接口，父子之间的额外代价留到合并时付。

状态与建模：dp[u][k][a]为u子树特殊操作恰用k次、u状态a的最小费用。cost[u][a]是点费用，edge_cost[v][a][b]是父u和子v状态对应的边费用。

本题映射：有限魔法可无成本杀任意怪，普通杀怪受父亲和存活孩子血量影响，求每个魔法次数的最小总力量。 先施放所有魔法；普通点自己的hp付一次，只有父子均普通时，子hp还会在父亲被杀时再付一次。 普通cost=hp、魔法cost=0；edge_cost[v][0][0]=hp[v]，run后输出全部k。

转移与正确性：tmp[i+j][a]=min(dp[u][i][a]+dp[v][j][b]+edge_cost[v][a][b])，枚举a,b∈{0,1}。Monster Hunter普通击杀点费hp；仅父子均普通时额外再付子hp，施法计入k且点费0。顶点覆盖把父子均不选设INF。

初始化与顺序：初始dp[u][0][0]=cost[u][0]，dp[u][1][1]=cost[u][1]，其余INF。后序逐孩子合并，tmp置INF，禁止从无效状态做加法。

优化与限制：子树数量裁剪后总体O(n^2)时间/空间上界。Monster Hunter必须按k输出，保留数量表；P2016仅求最少哨兵，下一例压成经典两状态O(n)，无需额外数量维。

本题复杂度：O(n^2)时间/空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

static constexpr int INF = 4000000000000000000LL;
int n;
vector<vector<int>> son;
vector<array<int, 2>> cost;
vector<array<array<int, 2>, 2>> edge_cost;
vector<vector<array<int, 2>>> dp;

void init(int n_) {
    n = n_;
    son.assign(n + 1, {});
    cost.assign(n + 1, {0, 0});
```

```
    edge_cost.resize(n + 1);
    for (auto &a : edge_cost)
        for (auto &b : a)
            b = {0, 0};
    dp.assign(n + 1, {});
}
void add_edge(int u, int v) {
    son[u].push_back(v);
}
void dfs(int u) {
    dp[u] = {{cost[u][0], INF}, {INF, cost[u][1]}};
    for (int v : son[u]) {
        dfs(v);
        int A = dp[u].size() - 1, B = dp[v].size() - 1;
        vector<array<int, 2>> tmp(A + B + 1, {INF, INF});
        for (int i = 0; i <= A; i++)
            for (int j = 0; j <= B; j++)
                for (int a = 0; a < 2; a++)
                    for (int b = 0; b < 2; b++)
                        if (dp[u][i][a] < INF && dp[v][j][b] < INF &&
                            edge_cost[v][a][b] < INF)
                            tmp[i + j][a] =
                                min(tmp[i + j][a],
                                    dp[u][i][a] + dp[v][j][b] + edge_cost[v][a][b]);
        dp[u].swap(tmp);
    }
}
vector<int> run(int root = 1) {
    dfs(root);
    vector<int> ans(n + 1, INF);
    for (int k = 0; k < (int)dp[root].size(); k++)
        ans[k] = min(dp[root][k][0], dp[root][k][1]);
    return ans;
}

void init_model(int n = 0) {
    init(n);
}

void solve() {
    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        ::init_model(n);
        for (int i = 2; i <= n; i++) {
            int p;
            cin >> p;
            ::add_edge(p, i);
        }
        for (int i = 1; i <= n; i++) {
            int hp;
            cin >> hp;
            ::cost[i] = {hp, 0};
            ::edge_cost[i][0][0] = hp;
        }
        auto ans = ::run();
        for (int k = 0; k <= n; k++)
            cout << ans[k] << " \n"[k == n];
```

```
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：两层状态分别回答“用了多少资源”和“外界要知道根是什么状态”。边上兼容性用禁止状态或额外费用表达，比口头假设更容易检验。

验证：C++17独立程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P2016 战略游戏

https://www.luogu.com.cn/problem/P2016

题意：树上选最少士兵，使每条边至少一个端点被选。

观察与规模：n≤1500，树边只给一次但方向任意，求覆盖每条边的最少哨兵；没有要求输出每个选点数量。

朴素思路：枚举哨兵点集2^n；给每条未覆盖边随便选一端，不保证最少点数。

关键突破：一条父子边是否覆盖只与两个端点是否选有关，根的二值状态就是对子树外的充分接口。

状态与建模：dp[u][0]为u不选时覆盖其子树内部全部边的最少点数；dp[u][1]为u选时的相同目标。此直接题解省去Monster Hunter的数量维。

转移与正确性：u不选时每个孩子v必须选，dp[u][0]=Σdp[v][1]；u选时边已覆盖，孩子可取两态最小，dp[u][1]=1+Σmin(dp[v][0],dp[v][1])。孩子之间无边，所以独立相加。

初始化与顺序：叶子两态为0和1；建立无向图，以0为根后序DFS，跳过父亲，最后取根的两态最小。单点树答案0。

优化与限制：O(n)时间/空间。输入0基原样使用，不把任意输入方向当父子方向。若图有环，父亲跳边不够且独立相加失效。

本题复杂度：O(n)时间、O(n)邻接、状态及递归空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

void solve() {
    int n;
    cin >> n;
    vector<vector<int>> e(n);
    for (int i = 0; i < n; i++) {
        int u, k;
        cin >> u >> k;
        while (k--) {
            int v;
            cin >> v;
            e[u].push_back(v);
```

```
            e[v].push_back(u);
        }
    }
    vector<array<int, 2>> dp(n);
    function<void(int, int)> dfs = [&](int u, int fa) {
        dp[u] = {0, 1};
        for (int v : e[u])
            if (v != fa) {
                dfs(v, u);
                dp[u][0] += dp[v][1];
                dp[u][1] += min(dp[v][0], dp[v][1]);
            }
    };
    dfs(0, -1);
    cout << min(dp[0][0], dp[0][1]) << '\n';
}
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：只在题目需要按资源数量输出时才增加背包维度；同一个二值树模型可以从一般数量DP压到两状态。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 原Monster Hunter已把hp[v]包含在子DP中，父状态0子状态0只额外加hp[v]，不能再加2hp[v]。
- 所有边和点费用累计须远小于INF；不可达先判断再相加。
- 边费用依赖父子状态，不能把状态维省略。

两层状态分别回答“用了多少资源”和“外界要知道根是什么状态”。边上兼容性用禁止状态或额外费用表达，比口头假设更容易检验。

个人资料来源：dp/树形动态规划进阶_比赛板子.md

动态规划 / 二进制位贡献与Popcount计数

类型与用途：思维型；位独立贡献的乘法原理：原Meaningless Sequence加权和、按掩码统计1位次数、按popcount分类。

复杂度与适用范围：weighted O(L)时间/空间；masked O(log n)时间/O(1)空间；distribution O(log^2 n)时间/空间。

例题 1 Gym102832D Meaningless Sequence

https://codeforces.com/gym/102832/problem/D

题意：按原递推定义序列，求0..N的项和，N用二进制给出。

观察与规模：N为二进制字符串，目标Σc^popcount(i)；分支权重相乘而非置位数相加，答案包含0及N。上界很大，目标由置位数、某些位的贡献或置位数的乘积决定；不能按数枚举。

朴素思路：枚举0..N逐个popcount是O(N)。按位统计时若不分“首次小于上界”的位置，会重复或遗漏数。

关键突破：每个小于N的数有唯一首次小于位置；此后后缀自由。独立一位的0/1权重为1/c，所以m位自由后缀权重和为(1+c)^m。

状态与建模：weighted的pre是固定前缀权重，ans累计已经小于上界的分支。distribution的ans[k]计数[0,N]恰有k个1的数；masked按位统计[0,N]该位为1的个数。

本题映射：按原递推定义序列，求0..N的项和，N用二进制给出。 递推可化为c^popcount；首次低于上界后，剩余自由位贡献(1+c)^剩余位。 PopcountDP(1e9+7).weighted(N,c)。

转移与正确性：上界某位为1时可放0，加入pre×(1+c)^剩余位，再沿等于分支令pre乘c。分布版加入C(剩余位,k)并累积已见1数。按位周期为2^(b+1)，整段各有2^b个1，余段再算超出前半的长度。

初始化与顺序：weighted从高位到低位，末尾加N本身。组合表C[0][0]=1，Pascal递推；分布最后加N所在桶。周期长度与N+1用__int128，避免60位附近加法溢出。

优化与限制：weighted O(位数)，distribution O(位数^2)，masked O(位数)。P4317对每类k做k^cnt[k]后相乘，0只有数字0并被排除。指数是精确计数，不能先模答案模数。

本题复杂度：O(|N|)时间与空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int mod;

int weighted(string s, int c) {
    int n = s.size();
    c %= mod;
    vector<int> pw(n + 1, 1);
    for (int i = 1; i <= n; i++)
        pw[i] = pw[i - 1] * (c + 1) % mod;
    int pre = 1, ans = 0;
    for (int i = 0; i < n; i++)
        if (s[i] == '1') {
            ans = (ans + pre * pw[n - i - 1]) % mod;
            pre = pre * c % mod;
        }
    return (ans + pre) % mod;
}

void init_model(int mod_ = 1000000007) {
    mod = mod_;
}

void solve() {
    string N;
    int c;
    cin >> N >> c;
    ::init_model();
    cout << ::weighted(N, c) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
```

```
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先问贡献可否按位线性拆开；乘积则按相同贡献分组。前缀分类和周期统计是两种不同工具，不能把所有位强行套同一个DP。

验证：C++17独立程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 ABC356D Masked Popcount

https://atcoder.jp/contests/abc356/tasks/abc356_d

题意：求0..N每个数与M按位与后的popcount总和。

观察与规模：N,M<2^60，求Σpopcount(i&M)；只对M的置位累加周期贡献，不能使用(c+1)^长度公式。上界很大，目标由置位数、某些位的贡献或置位数的乘积决定；不能按数枚举。

朴素思路：枚举0..N逐个popcount是O(N)。按位统计时若不分“首次小于上界”的位置，会重复或遗漏数。

关键突破：每个小于N的数有唯一首次小于位置；此后后缀自由。独立一位的0/1权重为1/c，所以m位自由后缀权重和为(1+c)^m。

状态与建模：weighted的pre是固定前缀权重，ans累计已经小于上界的分支。distribution的ans[k]计数[0,N]恰有k个1的数；masked按位统计[0,N]该位为1的个数。

本题映射：求0..N每个数与M按位与后的popcount总和。 拆成M中每个1位出现次数；该位0/1按周期交替。 PopcountDP(998244353).masked(N,M)。

转移与正确性：上界某位为1时可放0，加入pre×(1+c)^剩余位，再沿等于分支令pre乘c。分布版加入C(剩余位,k)并累积已见1数。按位周期为2^(b+1)，整段各有2^b个1，余段再算超出前半的长度。

初始化与顺序：weighted从高位到低位，末尾加N本身。组合表C[0][0]=1，Pascal递推；分布最后加N所在桶。周期长度与N+1用__int128，避免60位附近加法溢出。

优化与限制：weighted O(位数)，distribution O(位数^2)，masked O(位数)。P4317对每类k做k^cnt[k]后相乘，0只有数字0并被排除。指数是精确计数，不能先模答案模数。

本题复杂度：O(log N)时间、O(1)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int mod;

int masked(int n, int mask) {
    int ans = 0;
    for (int b = 0; b <= 60; b++)
        if ((mask >> b) & 1) {
            __int128 bit = (__int128)1 << b, period = bit * 2, len = (__int128)n + 1;
            __int128 c = (len / period) * bit + max((__int128)0, len % period - bit);
            ans = (ans + c % mod) % mod;
        }
    return ans;
}

```

```
void init_model(int mod_ = 1000000007) {
    mod = mod_;
}

void solve() {
    int N, M;
    cin >> N >> M;
    ::init_model(998244353);
    cout << ::masked(N, M) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先问贡献可否按位线性拆开；乘积则按相同贡献分组。前缀分类和周期统计是两种不同工具，不能把所有位强行套同一个DP。

验证：C++17独立程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 3 P4317 花神的数论题

https://www.luogu.com.cn/problem/P4317

题意：将1..N每个数的二进制1的个数相乘，模10000007。

观察与规模：N≤10^15，先求每种popcount的精确个数，再快速幂求乘积；模数10000007，不能错误使用费马缩指数。 上界很大，目标由置位数、某些位的贡献或置位数的乘积决定；不能按数枚举。

朴素思路：枚举0..N逐个popcount是O(N)。按位统计时若不分“首次小于上界”的位置，会重复或遗漏数。

关键突破：每个小于N的数有唯一首次小于位置；此后后缀自由。独立一位的0/1权重为1/c，所以m位自由后缀权重和为(1+c)^m。

状态与建模：weighted的pre是固定前缀权重，ans累计已经小于上界的分支。distribution的ans[k]计数[0,N]恰有k个1的数；masked按位统计[0,N]该位为1的个数。

本题映射：将1..N每个数的二进制1的个数相乘，模10000007。 按popcount=k分组，每类贡献k^计数；需要精确计数作为指数。 distribution(N)，从k=1累乘powmod(k,cnt[k])。

转移与正确性：上界某位为1时可放0，加入pre×(1+c)^剩余位，再沿等于分支令pre乘c。分布版加入C(剩余位,k)并累积已见1数。按位周期为2^(b+1)，整段各有2^b个1，余段再算超出前半的长度。

初始化与顺序：weighted从高位到低位，末尾加N本身。组合表C[0][0]=1，Pascal递推；分布最后加N所在桶。周期长度与N+1用__int128，避免60位附近加法溢出。

优化与限制：weighted O(位数)，distribution O(位数^2)，masked O(位数)。P4317对每类k做k^cnt[k]后相乘，0只有数字0并被排除。指数是精确计数，不能先模答案模数。

本题复杂度：O(log^2 N)时间/空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int mod;

vector<int> distribution(int n) {
    string s;
    do {
        s.push_back('0' + (n & 1));
        n >>= 1;
    } while (n);
    reverse(s.begin(), s.end());
    int L = s.size();
    vector<vector<int>> C(L + 1, vector<int>(L + 1));
    C[0][0] = 1;
    for (int i = 1; i <= L; i++) {
        C[i][0] = C[i][i] = 1;
        for (int j = 1; j < i; j++)
            C[i][j] = C[i - 1][j - 1] + C[i - 1][j];
    }
    vector<int> ans(L + 1);
    int ones = 0;
    for (int i = 0; i < L; i++)
        if (s[i] == '1') {
            for (int k = 0; k <= L - i - 1; k++)
                ans[ones + k] += C[L - i - 1][k];
            ones++;
        }
    ans[ones]++;
    return ans;
}
int powmod(int a, int b) {
    int r = 1;
    while (b) {
        if (b & 1)
            r = (__int128)r * a % mod;
        a = (__int128)a * a % mod;
        b >>= 1;
    }
    return r;
}

void init_model(int mod_ = 1000000007) {
    mod = mod_;
}

void solve() {
    int N;
    cin >> N;
    ::init_model(10000007);
    auto cnt = ::distribution(N);
    int ans = 1;
    for (int k = 1; k < (int)cnt.size(); k++)
        ans = (__int128)ans * ::powmod(k, cnt[k]) % ::mod;
    cout << ans << '\n';
}

```

```
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先问贡献可否按位线性拆开；乘积则按相同贡献分组。前缀分类和周期统计是两种不同工具，不能把所有位强行套同一个DP。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 前导0在二进制统计中不改变popcount。
- c=0时0^0按序列a0=1处理，weighted仍正确。
- distribution输出精确指数，不对mod取模；指数模数也不能随意用答案模数。

先问贡献可否按位线性拆开；乘积则按相同贡献分组。前缀分类和周期统计是两种不同工具，不能把所有位强行套同一个DP。

个人资料来源：dp/数位动态规划进阶_比赛板子.md

动态规划 / 区间同色配对与涂色

类型与用途：思维型；压缩相邻同色后，配对相同端点节省操作；收录原Cities最大节省状态与清空/涂色的次数接口。

复杂度与适用范围：O(n^3)最坏时间、O(n^2)空间；每色最多f次时O(fn^2)。

例题 1 CF1132F Clear the String

https://codeforces.com/contest/1132/problem/F

题意：每次删除一段相同字符，求清空字符串最少次数。

观察与规模：每次只能删除当前连续同色段，配对两块时先删其间内容；不能一次删散布的所有同色字符。 相邻相同字符可压成一个块；两题分别删除同色段、覆盖涂色，操作历史看似不同。

朴素思路：逐次枚举所有段和所有操作序列是指数级；总是删除最长段会破坏后续同色块合并。

关键突破：把同色块放在同一次操作中可以节省一次。选最左块，枚举与它配对的后续同色块，中间必须先解决。

状态与建模：代码dp[l][r]是区间能节省的操作数，答案为块数-dp[l][n]；这与直接定义最少操作次数等价，不能把dp误读为次数。

本题映射：每次删除一段相同字符，求清空字符串最少次数。 先清空两个同色位置中间内容，使两端可共同删除，节省一次。 字符转整数，init后clear_min。

转移与正确性：不配对时继承dp[l+1][r]；配到同色位置x时为dp[l+1][x-1]+dp[x][r]+1。内部先消除/覆盖，再共享端点操作。所有方案按最左块是否与后面共享分类。

初始化与顺序：空区间及单块节省0；l从右向左，r从l向右，确保更靠右的起点已完成。nxt链只访问同色位置。

优化与限制：压缩相邻重复，预建同色后继；最坏仍O(n^3)/O(n^2)。涂色的覆盖顺序须按删除方案逆序理解。课堂把所有段变成一种颜色可少一次，但CF1114D的染色规则不同，不可混用。

本题复杂度：O(n^3)时间、O(n^2)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

vector<int> a, nxt;
vector<vector<int>> dp;
void init(const vector<int> &v) {
    a = {0};
    for (int x : v)
        if (a.size() == 1 || a.back() != x)
            a.push_back(x);
    int n = a.size() - 1;
    nxt.assign(n + 2, n + 1);
    map<int, int> pos;
    for (int i = n; i >= 1; i--) {
        if (pos.count(a[i]))
            nxt[i] = pos[a[i]];
        pos[a[i]] = i;
    }
    dp.assign(n + 2, vector<int>(n + 2));
    for (int l = n; l >= 1; l--)
        for (int r = l; r <= n; r++) {
            dp[l][r] = dp[l + 1][r];
            for (int x = nxt[l]; x <= r; x = nxt[x])
                dp[l][r] = max(dp[l][r], dp[l + 1][x - 1] + dp[x][r] + 1);
        }
}
int clear_min() {
    int n = a.size() - 1;
    return n - dp[1][n];
}
int uniform_min() {
    return max(0LL, clear_min() - 1);
}

void init_model() {
}

void solve() {
    int n;
    string s;
    cin >> n >> s;
    ::init_model();
    ::init(vector<int>(s.begin(), s.end()));
    cout << ::clear_min() << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：区间内相同端点可能共享代价时，尝试把一端固定，枚举下一次同色配对；必须证明中间段可独立处理。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P4170 涂色

https://www.luogu.com.cn/problem/P4170

题意：每次将连续区间涂同一颜色，可覆盖旧颜色，形成目标串最少次数。

观察与规模：覆盖绘画的执行顺序与删除同色段相反；压缩后也必须保留颜色相对顺序。 相邻相同字符可压成一个块；两题分别删除同色段、覆盖涂色，操作历史看似不同。

朴素思路：逐次枚举所有段和所有操作序列是指数级；总是删除最长段会破坏后续同色块合并。

关键突破：把同色块放在同一次操作中可以节省一次。选最左块，枚举与它配对的后续同色块，中间必须先解决。

状态与建模：代码dp[l][r]是区间能节省的操作数，答案为块数-dp[1][n]；这与直接定义最少操作次数等价，不能把dp误读为次数。

本题映射：每次将连续区间涂同一颜色，可覆盖旧颜色，形成目标串最少次数。 一次同色涂色可先跨过中间区间，再补涂中间；与端点同色配对的省操作转移一致。 目标颜色串转整数，init后clear_min。

转移与正确性：不配对时继承dp[l+1][r]；配到同色位置x时为dp[l+1][x-1]+dp[x][r]+1。内部先消除/覆盖，再共享端点操作。所有方案按最左块是否与后面共享分类。

初始化与顺序：空区间及单块节省0；l从右向左，r从l向右，确保更靠右的起点已完成。nxt链只访问同色位置。

优化与限制：压缩相邻重复，预建同色后继；最坏仍O(n^3)/O(n^2)。涂色的覆盖顺序须按删除方案逆序理解。课堂把所有段变成一种颜色可少一次，但CF1114D的染色规则不同，不可混用。

本题复杂度：O(n^3)时间、O(n^2)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

vector<int> a, nxt;
vector<vector<int>> dp;
void init(const vector<int> &v) {
    a = {0};
    for (int x : v)
        if (a.size() == 1 || a.back() != x)
            a.push_back(x);
    int n = a.size() - 1;
    nxt.assign(n + 2, n + 1);
    map<int, int> pos;
    for (int i = n; i >= 1; i--) {
        if (pos.count(a[i]))
            nxt[i] = pos[a[i]];
        pos[a[i]] = i;
    }
    dp.assign(n + 2, vector<int>(n + 2));
    for (int l = n; l >= 1; l--)
```

```
        for (int r = l; r <= n; r++) {
            dp[l][r] = dp[l + 1][r];
            for (int x = nxt[l]; x <= r; x = nxt[x])
                dp[l][r] = max(dp[l][r], dp[l + 1][x - 1] + dp[x][r] + 1);
        }
}
int clear_min() {
    int n = a.size() - 1;
    return n - dp[1][n];
}
int uniform_min() {
    return max(0LL, clear_min() - 1);
}

void init_model() {
}

void solve() {
    string s;
    cin >> s;
    ::init_model();
    ::init(vector<int>(s.begin(), s.end()));
    cout << ::clear_min() << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：区间内相同端点可能共享代价时，尝试把一端固定，枚举下一次同色配对；必须证明中间段可独立处理。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 配对后右子区间必须从x开始，而非x+1，因为x仍能继续配对。
- 原读入循环缺读n并改写n导致易错，整理版先读后压缩。
- 固定一个起点的Flood Fill是另一模型，不能直接假定uniform_min适用。

区间内相同端点可能共享代价时，尝试把一端固定，枚举下一次同色配对；必须证明中间段可独立处理。

个人资料来源：dp/区间动态规划进阶_比赛板子.md

动态规划 / 区间强制对象分治

类型与用途：思维型；选择一次事件作为区间分界，付当前费用后，左右不再互相影响。敌人模型由最远对象限定合法分界；切木棍模型由当前区间长度决定费用。

复杂度与适用范围：n为敌人数、K≤2n为端点数：O(K^2(n+K))时间/O(K^2+n)空间；切木棍q切点：O(q^3)时间/O(q^2)空间。

例题 1 P4766 Outer space invaders

https://www.luogu.com.cn/problem/P4766

题意：敌人在各时间区间出现，一次半径R的冲击波可击中当前距离≤R的敌人，费用R，求全灭最小费用。

观察与规模：n≤300且攻击可在时间窗口内选择时刻；选最强对象不是贪心直接删它，而是枚举其负责攻击的时刻再划分左右。 切木棍按第一刀分治；Aliens在区间内有必须消灭的最强对象，其时间可选但强度确定。

朴素思路：任意枚举攻击/切割顺序是阶乘级；每次选最低成本操作没有保证。

关键突破：选择当前区间必须负责的一次操作，让左右未处理对象互不影响。Enemies取完整落在区间内的最强敌人，其攻击时刻只能落在其窗口。

状态与建模：dp[l][r]为端点离散化后当前区间的最小代价；切割模式表示尚需完成的切点[l,r]，敌人模式表示窗口完全在区间中的敌人。

本题映射：敌人在各时间区间出现，一次半径R的冲击波可击中当前距离≤R的敌人，费用R，求全灭最小费用。 先考虑最远敌人被杀时刻k；该击可消灭所有跨k敌人，余下左右期限区间独立。 读T及期限a,b和距离d，init_enemies，run。

转移与正确性：枚举强制操作位置k：dfs(l,k-1)+dfs(k+1,r)+cost。最强攻击可代替所有跨越k且强度较小的攻击，左右剩余敌人窗口互不跨k；第一刀后左右木棍完全独立。

初始化与顺序：空区间0，无敌人区间0；memo用-1表示未算，非负答案。单点窗口仍要攻击，不能把l==r直接当0。切点必须严格位于木棍内且互异。

优化与限制：敌人n小，状态O(n^2)，每个扫描敌人及枚举时刻，O(n^3)量级；木棍O(q^3)/O(q^2)。离散化只需各敌人窗口端点，区间内部变化不影响可被覆盖集合。

本题复杂度：O(n^3)时间/O(n^2)空间，离散时间点≤2n。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

struct info {
    int l, r, w;
};
static constexpr int INF = 4000000000000000000LL;
int n, mode;
vector<int> x;
vector<info> a;
vector<vector<int>> dp;
void init_enemies(vector<info> p) {
    mode = 0;
    a = p;
    x.clear();
```

```
    for (auto z : a) {
        x.push_back(z.l);
        x.push_back(z.r);
    }
    sort(x.begin(), x.end());
    x.erase(unique(x.begin(), x.end()), x.end());
    n = x.size();
    for (auto &z : a) {
        z.l = lower_bound(x.begin(), x.end(), z.l) - x.begin();
        z.r = lower_bound(x.begin(), x.end(), z.r) - x.begin();
    }
    dp.assign(n, vector<int>(n, -1));
}
void init_cuts(int length, vector<int> cuts) {
    mode = 1;
    x = cuts;
    sort(x.begin(), x.end());
    assert(adjacent_find(x.begin(), x.end()) == x.end());
    x.insert(x.begin(), 0);
    x.push_back(length);
    n = cuts.size();
    dp.assign(n, vector<int>(n, -1));
}
int dfs(int l, int r) {
    if (l > r)
        return 0;
    int &ans = dp[l][r];
    if (ans != -1)
        return ans;
    int L = l, R = r, cost = 0;
    if (mode == 0) {
        int pos = -1;
        for (int i = 0; i < (int)a.size(); i++)
            if (a[i].l >= l && a[i].r <= r && (pos == -1 || a[i].w > a[pos].w))
                pos = i;
        if (pos == -1)
            return ans = 0;
        L = a[pos].l;
        R = a[pos].r;
        cost = a[pos].w;
    } else
        cost = x[r + 2] - x[l];
    ans = INF;
    for (int k = L; k <= R; k++)
        ans = min(ans, dfs(l, k - 1) + dfs(k + 1, r) + cost);
    return ans;
}
int run() {
    return n ? dfs(0, n - 1) : 0;
}

void init_model() {
}

void solve() {
    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
```

```
        vector<::info> a(n);
        for (auto &z : a)
            cin >> z.l >> z.r >> z.w;
        ::init_model();
        ::init_enemies(a);
        cout << ::run() << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：区间DP不只有“最后一步”，也可找不可回避的最强/第一操作；左右独立必须由问题性质证明。

验证：C++17独立程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 UVA10003 Cutting Sticks

https://onlinejudge.org/external/100/10003.pdf

题意：在给定位置切断木棍，每次费用为正在切的那段长度，选切割顺序使总费用最小。

观察与规模：切点最多50，先固定第一刀，成本为当前木棍长度x[r+2]-x[l]；不是切点数量，也不是原木总长。 切木棍按第一刀分治；Aliens在区间内有必须消灭的最强对象，其时间可选但强度确定。

朴素思路：任意枚举攻击/切割顺序是阶乘级；每次选最低成本操作没有保证。

关键突破：选择当前区间必须负责的一次操作，让左右未处理对象互不影响。Enemies取完整落在区间内的最强敌人，其攻击时刻只能落在其窗口。

状态与建模：dp[l][r]为端点离散化后当前区间的最小代价；切割模式表示尚需完成的切点[l,r]，敌人模式表示窗口完全在区间中的敌人。

本题映射：在给定位置切断木棍，每次费用为正在切的那段长度，选切割顺序使总费用最小。 把第一次切割k作为分界，左右后续独立。沿用相同dfs分界框架，将费用替换为两侧边界长度，允许分界为全部待切点。 循环读length直到0，读q和切点，init_cuts，run，输出指定英文句式。

转移与正确性：枚举强制操作位置k：dfs(l,k-1)+dfs(k+1,r)+cost。最强攻击可代替所有跨越k且强度较小的攻击，左右剩余敌人窗口互不跨k；第一刀后左右木棍完全独立。

初始化与顺序：空区间0，无敌人区间0；memo用-1表示未算，非负答案。单点窗口仍要攻击，不能把l==r直接当0。切点必须严格位于木棍内且互异。

优化与限制：敌人n小，状态O(n^2)，每个扫描敌人及枚举时刻，O(n^3)量级；木棍O(q^3)/O(q^2)。离散化只需各敌人窗口端点，区间内部变化不影响可被覆盖集合。

本题复杂度：O(q^3)时间/O(q^2)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

struct info {
    int l, r, w;
};
static constexpr int INF = 4000000000000000000LL;
int n, mode;
vector<int> x;
vector<info> a;
vector<vector<int>> dp;
void init_enemies(vector<info> p) {
    mode = 0;
    a = p;
    x.clear();
    for (auto z : a) {
        x.push_back(z.l);
        x.push_back(z.r);
    }
    sort(x.begin(), x.end());
    x.erase(unique(x.begin(), x.end()), x.end());
    n = x.size();
    for (auto &z : a) {
        z.l = lower_bound(x.begin(), x.end(), z.l) - x.begin();
        z.r = lower_bound(x.begin(), x.end(), z.r) - x.begin();
    }
    dp.assign(n, vector<int>(n, -1));
}
void init_cuts(int length, vector<int> cuts) {
    mode = 1;
    x = cuts;
    sort(x.begin(), x.end());
    assert(adjacent_find(x.begin(), x.end()) == x.end());
    x.insert(x.begin(), 0);
    x.push_back(length);
    n = cuts.size();
    dp.assign(n, vector<int>(n, -1));
}
int dfs(int l, int r) {
    if (l > r)
        return 0;
    int &ans = dp[l][r];
    if (ans != -1)
        return ans;
    int L = l, R = r, cost = 0;
    if (mode == 0) {
        int pos = -1;
        for (int i = 0; i < (int)a.size(); i++)
            if (a[i].l >= l && a[i].r <= r && (pos == -1 || a[i].w > a[pos].w))
                pos = i;
        if (pos == -1)
            return ans = 0;
        L = a[pos].l;
        R = a[pos].r;
        cost = a[pos].w;
    } else
        cost = x[r + 2] - x[l];
```

```
    ans = INF;
    for (int k = L; k <= R; k++)
        ans = min(ans, dfs(l, k - 1) + dfs(k + 1, r) + cost);
    return ans;
}
int run() {
    return n ? dfs(0, n - 1) : 0;
}

void init_model() {
}

void solve() {
    int length;
    while (cin >> length && length) {
        int q;
        cin >> q;
        vector<int> cuts(q);
        for (int &x : cuts)
            cin >> x;
        ::init_model();
        ::init_cuts(length, cuts);
        cout << "The minimum cutting is " << ::run() << ".\n";
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：区间DP不只有“最后一步”，也可找不可回避的最强/第一操作；左右独立必须由问题性质证明。

验证：C++17独立程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 左右必须递归[l,k-1]和[k+1,r]。原OCR版闭区间solve(l,k)+solve(k,r)可能无限递归，且l==r仍可能存在敌人。
- 敌人中跨过所选k的对象全部被此最远对象所需的一击消灭，剩余仅完全位于左右的期限区间。
- 压缩仅保留期限端点即可，总有最优射击时刻可移动到某端点。
- 切木棍的费用取两侧已切边界距离，和敵人最大距离属于同一分界递推的两种初始化；接口显式区分，不能混用。

区间DP不只有“最后一步”，也可找不可回避的最强/第一操作；左右独立必须由问题性质证明。

个人资料来源：dp/区间动态规划进阶_比赛板子.md

动态规划 / 区间石子合并

类型与用途：思维型；相邻堆合并的区间DP；保留一次合并L..R堆三维状态，并提供二元合并与环形最大/最小版本。

复杂度与适用范围：binary O(n^3)时间/O(n^2)空间；arity O(n^3R)时间/O(n^3)空间，n<=100。

例题 1 DP_N Slimes

https://atcoder.jp/contests/dp/tasks/dp_n

题意：相邻两堆合并，总代价最小。

观察与规模：n≤400、a_i≤10^9，O(n^3)符合规模，答案及前缀和用64位。最后输出dp[1][n]。 相邻合并保留原顺序，最终每堆对应一个连续区间；n为几百，区间三重枚举可行。

朴素思路：直接枚举合并顺序会产生大量重复的中间区间。贪心每次合并最小相邻和没有一般正确性。

关键突破：倒着看最后一次合并：它必把[1,k]和[k+1,r]两堆合成一堆；最后成本只依赖区间总和。

状态与建模：dp[l][r]是把区间合成一堆的最小成本。边界两堆的内部历史不影响下一次合并，所以端点足够。

本题映射：相邻两堆合并，总代价最小。 最后一次合并将区间分为左右两堆，枚举最后断点。 读a后binary(a)。

转移与正确性：dp[l][r]=min_k(dp[l][k]+dp[k+1][r]+sum(l,r))。枚举最后分界不重不漏；求最大值时把min改max。环形复制数组，枚举长度n的窗口，表示最后保留的断点。

初始化与顺序：dp[i][i]=0；最小值其余置INF，最大值置-INF。按长度从2到n计算，左右子区间必已完成。环形只计算长度不超过n。

优化与限制：前缀和把每次区间求和降至O(1)，整体O(n^3)/O(n^2)。课堂L到R堆合并要增加剩余堆数k：先拼出k堆，再付区间和变为1堆；不能直接套二叉转移。

本题复杂度：O(n^3)时间、O(n^2)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

static constexpr int INF = 4000000000000000000LL;
int binary(const vector<int> &a, bool circle = false, bool maximum = false) {
    int n = a.size(), N = circle ? 2 * n : n;
    vector<int> s(N + 1);
    for (int i = 1; i <= N; i++)
        s[i] = s[i - 1] + a[(i - 1) % n];
    vector<vector<int>> dp(N + 2, vector<int>(N + 2));
    for (int len = 2; len <= n; len++)
        for (int l = 1; l + len - 1 <= N; l++) {
            int r = l + len - 1;
            dp[l][r] = maximum ? -INF : INF;
            for (int k = l; k < r; k++) {
                int val = dp[l][k] + dp[k + 1][r] + s[r] - s[l - 1];
                if (maximum)
                    dp[l][r] = max(dp[l][r], val);
                else
                    dp[l][r] = min(dp[l][r], val);
            }
        }
```

```
    int ans = maximum ? -INF : INF;
    for (int l = 1; l <= (circle ? n : 1); l++)
        if (maximum)
            ans = max(ans, dp[l][l + n - 1]);
        else
            ans = min(ans, dp[l][l + n - 1]);
    return ans;
}
int arity(const vector<int> &a, int L, int R) {
    assert(2 <= L && L <= R);
    int n = a.size();
    vector<int> s(n + 1);
    for (int i = 1; i <= n; i++)
        s[i] = s[i - 1] + a[i - 1];
    vector<vector<vector<int>>> dp(n + 2,
                                   vector<vector<int>>(n + 2, vector<int>(n + 1, INF)));
    for (int i = 1; i <= n; i++)
        dp[i][i][1] = 0;
    for (int len = 2; len <= n; len++)
        for (int l = 1; l + len - 1 <= n; l++) {
            int r = l + len - 1;
            for (int k = 2; k <= min(R, len); k++)
                for (int mid = l; mid < r; mid++)
                    if (dp[l][mid][1] < INF && dp[mid + 1][r][k - 1] < INF)
                        dp[l][r][k] =
                            min(dp[l][r][k], dp[l][mid][1] + dp[mid + 1][r][k - 1]);
            for (int k = L; k <= min(R, len); k++)
                if (dp[l][r][k] < INF)
                    dp[l][r][1] = min(dp[l][r][1], dp[l][r][k] + s[r] - s[l - 1]);
        }
    return n ? dp[1][n][1] : 0;
}

void init_model() {
}

void solve() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a)
        cin >> x;
    ::init_model();
    cout << ::binary(a) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：出现相邻对象合并，先问最后一步分成几段、成本是否仅由整段决定。环形断开后窗口长度必须固定；多叉合并需要明确可达的堆数。

验证：C++17独立程序编译通过；5组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P1880 石子合并

https://www.luogu.com.cn/problem/P1880

题意：圆周相邻两堆合并，求最小和最大总代价。

观察与规模：环形起点自由，复制两遍后枚举恰好n个数的窗口；最小和最大分别计算，不能用长度2n的结果。 相邻合并保留原顺序，最终每堆对应一个连续区间；n为几百，区间三重枚举可行。

朴素思路：直接枚举合并顺序会产生大量重复的中间区间。贪心每次合并最小相邻和没有一般正确性。

关键突破：倒着看最后一次合并：它必把[l,k]和[k+1,r]两堆合成一堆；最后成本只依赖区间总和。

状态与建模：dp[l][r]是把区间合成一堆的最小成本。边界两堆的内部历史不影响下一次合并，所以端点足够。

本题映射：圆周相邻两堆合并，求最小和最大总代价。 复制数组后枚举长度n区间，覆盖全部断环位置；状态合并改min/max。 binary(a,true,false)与binary(a,true,true)。

转移与正确性：dp[l][r]=min_k(dp[l][k]+dp[k+1][r]+sum(l,r))。枚举最后分界不重不漏；求最大值时把min改max。环形复制数组，枚举长度n的窗口，表示最后保留的断点。

初始化与顺序：dp[i][i]=0；最小值其余置INF，最大值置-INF。按长度从2到n计算，左右子区间必已完成。环形只计算长度不超过n。

优化与限制：前缀和把每次区间求和降至O(1)，整体O(n^3)/O(n^2)。课堂L到R堆合并要增加剩余堆数k：先拼出k堆，再付区间和变为1堆；不能直接套二叉转移。

本题复杂度：O(n^3)时间、O(n^2)空间。

完整函数题解

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

static constexpr int INF = 4000000000000000000LL;
int binary(const vector<int> &a, bool circle = false, bool maximum = false) {
    int n = a.size(), N = circle ? 2 * n : n;
    vector<int> s(N + 1);
    for (int i = 1; i <= N; i++)
        s[i] = s[i - 1] + a[(i - 1) % n];
    vector<vector<int>> dp(N + 2, vector<int>(N + 2));
    for (int len = 2; len <= n; len++)
        for (int l = 1; l + len - 1 <= N; l++) {
            int r = l + len - 1;
            dp[l][r] = maximum ? -INF : INF;
            for (int k = l; k < r; k++) {
                int val = dp[l][k] + dp[k + 1][r] + s[r] - s[l - 1];
                if (maximum)
                    dp[l][r] = max(dp[l][r], val);
                else
                    dp[l][r] = min(dp[l][r], val);
            }
        }
    int ans = maximum ? -INF : INF;
    for (int l = 1; l <= (circle ? n : 1); l++)
        if (maximum)
            ans = max(ans, dp[l][l + n - 1]);
        else
            ans = min(ans, dp[l][l + n - 1]);
    return ans;
```

```
}
int arity(const vector<int> &a, int L, int R) {
    assert(2 <= L && L <= R);
    int n = a.size();
    vector<int> s(n + 1);
    for (int i = 1; i <= n; i++)
        s[i] = s[i - 1] + a[i - 1];
    vector<vector<vector<int>>> dp(n + 2,
                                   vector<vector<int>>(n + 2, vector<int>(n + 1, INF)));
    for (int i = 1; i <= n; i++)
        dp[i][i][1] = 0;
    for (int len = 2; len <= n; len++)
        for (int l = 1; l + len - 1 <= n; l++) {
            int r = l + len - 1;
            for (int k = 2; k <= min(R, len); k++)
                for (int mid = l; mid < r; mid++)
                    if (dp[l][mid][1] < INF && dp[mid + 1][r][k - 1] < INF)
                        dp[l][r][k] =
                            min(dp[l][r][k], dp[l][mid][1] + dp[mid + 1][r][k - 1]);
            for (int k = L; k <= min(R, len); k++)
                if (dp[l][r][k] < INF)
                    dp[l][r][1] = min(dp[l][r][1], dp[l][r][k] + s[r] - s[l - 1]);
        }
    return n ? dp[1][n][1] : 0;
}

void init_model() {
}

void solve() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a)
        cin >> x;
    ::init_model();
    cout << ::binary(a, true) << '\n' << ::binary(a, true, true) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：出现相邻对象合并，先问最后一步分成几段、成本是否仅由整段决定。环形断开后窗口长度必须固定；多叉合并需要明确可达的堆数。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 单堆费用0；三维k不能超过区间长度。
- DP_N n=400必须用binary，不可使用O(n^4)的课堂多元版本。
- INF状态不要直接相加；费用和须小于INF。

出现相邻对象合并，先问最后一步分成几段、成本是否仅由整段决定。环形断开后窗口长度必须固定；多叉合并需要明确可达的堆数。

个人资料来源：dp/区间动态规划进阶_比赛板子.md

动态规划 / 子树合并背包

类型与用途：思维型；孩子背包卷积合并，固定点数连通选点；用户计数变体可把内部节点权重置0并扣启用边费用。

复杂度与适用范围：单位点数或叶子计数模型，限制循环到实际子树规模，总O(nM)，完整M=n时O(n^2)；空间O(nM)。

例题 1 P2014 选课

https://www.luogu.com.cn/problem/P2014

题意：恰好选m门课并满足先修依赖，使学分最大。

观察与规模：恰选m门课且每门至多一个先修；虚根0不计数量，必须允许森林但不能丢掉恰好数量约束。 选择孩子需要先选择父节点，或叶子带来收益但启用分支要付边费；数量上限m提供背包维度。

朴素思路：枚举节点子集指数级；普通物品背包不记录依赖，会选到父亲未选的孩子。

关键突破：将每个孩子子树视为一组方案，逐个做分组背包；只有取孩子非空方案才付该边费用。

状态与建模：dp[u][k]为u子树选择数量/叶子数恰为k的最大净收益；非空状态包含u(内部网络节点重量为0)，k=0允许整棵子树不选。

本题映射：恰好选m门课并满足先修依赖，使学分最大。 无先修课程挂在虚根0，虚根w=0不计课程数；每个真实课程w=1。 add_edge(先修,i)，run(0)，qur(0,m)。

转移与正确性：合并时保留原方案(不选v)，或tmp[i+j]=max(tmp[i+j],dp[u][i]+dp[v][j]-cost)，j≥1。P2014每课程重1、虚根重0；P1273内部点重0、叶子重1，边费用减一次。

初始化与顺序：先仅含u的状态dp[u][w[u]]=a[u]，其余-INF。处理孩子前该状态不允许绕过u；所有孩子合并后才补dp[u][0]=0。tmp独立缓冲防止复用同一孩子。

优化与限制：按真实子树可达数量裁剪数组并截到m；一般上界O(nm^2)，总量受树结构约束可更小。忽略边费或在j=0仍扣边费均错误。

本题复杂度：O(nm)时间/空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

static constexpr int INF = 4000000000000000000LL;
int n, M;
vector<vector<pair<int, int>>> son;
vector<int> a, w;
vector<vector<int>> dp;

void init(int n_, int m) {
    n = n_;
    M = m;
    son.assign(n + 1, {});
    a.assign(n + 1, 0);
    w.assign(n + 1, 1);
    dp.assign(n + 1, {});
```

```
}
void add_edge(int u, int v, int cost = 0) {
    son[u].push_back({v, cost});
}
void dfs(int u) {
    dp[u].assign(min(M, w[u]) + 1, -INF);
    if (w[u] <= M)
        dp[u][w[u]] = a[u];
    for (auto [v, cost] : son[u]) {
        dfs(v);
        int A = dp[u].size() - 1, B = dp[v].size() - 1;
        vector<int> tmp(min(M, A + B) + 1, -INF);
        for (int i = 0; i <= A; i++)
            if (dp[u][i] > -INF) {
                tmp[i] = max(tmp[i], dp[u][i]);
                for (int j = 1; j <= B && i + j <= M; j++)
                    if (dp[v][j] > -INF)
                        tmp[i + j] = max(tmp[i + j], dp[u][i] + dp[v][j] - cost);
            }
        dp[u].swap(tmp);
    }
    dp[u][0] = max(dp[u][0], 0LL);
}
void run(int root) {
    dfs(root);
}
int qur(int u, int k) {
    return k < (int)dp[u].size() ? dp[u][k] : -INF;
}

void init_model(int n = 0, int M = 0) {
    init(n, M);
}

void solve() {
    int n, m;
    cin >> n >> m;
    ::init_model(n, m);
    ::w[0] = 0;
    for (int i = 1; i <= n; i++) {
        int p;
        cin >> p >> ::a[i];
        ::add_edge(p, i);
    }
    ::run(0);
    cout << ::qur(0, m) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：树形背包先明确背包维度计什么，再区别“完全不选子树”和“选根但数量0”。零重量内部点不能简单等同于没有选。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P1273 有线电视网

https://www.luogu.com.cn/problem/P1273

题意：选择尽量多用户，在用户收益扣转播边费用后不亏本。

观察与规模：只把叶子算作用户，内部点重量0；同一连接边被多用户共用只付一次，答案是净收益≥0的最大用户数。 选择孩子需要先选择父节点，或叶子带来收益但启用分支要付边费；数量上限m提供背包维度。

朴素思路：枚举节点子集指数级；普通物品背包不记录依赖，会选到父亲未选的孩子。

关键突破：将每个孩子子树视为一组方案，逐个做分组背包；只有取孩子非空方案才付该边费用。

状态与建模：dp[u][k]为u子树选择数量/叶子数恰为k的最大净收益；非空状态包含u(内部网络节点重量为0)，k=0允许整棵子树不选。

本题映射：选择尽量多用户，在用户收益扣转播边费用后不亏本。 只有叶子计用户数；某孩子子树接入了至少一个用户时才支付该边费用。 内部节点w=0，叶子w=1且a=收益，add_edge含费用；从大到小找dp根[k]>=0。

转移与正确性：合并时保留原方案(不选v)，或tmp[i+j]=max(tmp[i+j],dp[u][i]+dp[v][j]-cost)，j≥1。P2014每课程重1、虚根重0；P1273内部点重0、叶子重1，边费用减一次。

初始化与顺序：先仅含u的状态dp[u][w[u]]=a[u]，其余-INF。处理孩子前该状态不允许绕过u；所有孩子合并后才补dp[u][0]=0。tmp独立缓冲防止复用同一孩子。

优化与限制：按真实子树可达数量裁剪数组并截到m；一般上界O(nm^2)，总量受树结构约束可更小。忽略边费或在j=0仍扣边费均错误。

本题复杂度：O(nm)时间/空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

static constexpr int INF = 4000000000000000000LL;
int n, M;
vector<vector<pair<int, int>>> son;
vector<int> a, w;
vector<vector<int>> dp;

void init(int n_, int m) {
    n = n_;
    M = m;
    son.assign(n + 1, {});
    a.assign(n + 1, 0);
    w.assign(n + 1, 1);
    dp.assign(n + 1, {});
}
void add_edge(int u, int v, int cost = 0) {
    son[u].push_back({v, cost});
}
void dfs(int u) {
    dp[u].assign(min(M, w[u]) + 1, -INF);
    if (w[u] <= M)
        dp[u][w[u]] = a[u];
```

```
    for (auto [v, cost] : son[u]) {
        dfs(v);
        int A = dp[u].size() - 1, B = dp[v].size() - 1;
        vector<int> tmp(min(M, A + B) + 1, -INF);
        for (int i = 0; i <= A; i++)
            if (dp[u][i] > -INF) {
                tmp[i] = max(tmp[i], dp[u][i]);
                for (int j = 1; j <= B && i + j <= M; j++)
                    if (dp[v][j] > -INF)
                        tmp[i + j] = max(tmp[i + j], dp[u][i] + dp[v][j] - cost);
            }
        dp[u].swap(tmp);
    }
    dp[u][0] = max(dp[u][0], 0LL);
}
void run(int root) {
    dfs(root);
}
int qur(int u, int k) {
    return k < (int)dp[u].size() ? dp[u][k] : -INF;
}

void init_model(int n = 0, int M = 0) {
    init(n, M);
}

void solve() {
    int n, m;
    cin >> n >> m;
    ::init_model(n, m);
    for (int u = 1; u <= n - m; u++) {
        ::w[u] = 0;
        int k;
        cin >> k;
        while (k--) {
            int v, c;
            cin >> v >> c;
            ::add_edge(u, v, c);
        }
    }
    for (int u = n - m + 1; u <= n; u++)
        cin >> ::a[u];
    ::run(1);
    for (int k = m; k >= 0; k--)
        if (::qur(1, k) >= 0) {
            cout << k << '\n';
            break;
        }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：树形背包先明确背包维度计什么，再区别“完全不选子树”和“选根但数量0”。零重量内部点不能简单等同于没有选。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 不可达初始化-INF，不能用0假装可达。
- 有先修关系必须使父节点被选才能合并孩子；最后提供空集0状态供父亲跳过。
- 任意非单位w不再有O(nM)规模证明，应改用DFS序依赖背包。

树形背包先明确背包维度计什么，再区别“完全不选子树”和“选根但数量0”。零重量内部点不能简单等同于没有选。

个人资料来源：dp/树形动态规划进阶_比赛板子.md

动态规划 / 括号计数与无歧义区间DP

类型与用途：思维型；普通通配括号用余额状态；超级括号用首个完整括号块唯一分解，避免拼接计数重复。

复杂度与适用范围：normal O(n^2)时间/O(n)空间；super O(n^3)时间/O(n^2)空间。

例题 1 P7914 括号序列

https://www.luogu.com.cn/problem/P7914

题意：将?替换为括号或星号，使超级括号序列符合规范。

观察与规模：n≤500，问号替换为括号或星号；题面不允许最外星号段，故答案取f而非left/right；k限制每段星号。 普通括号只要求前缀余额非负且总余额0；超级括号另有连续星号上限及递归语法。

朴素思路：枚举问号替换指数级；超级括号若任意切分合法串，会把同一字符串按多种括号分组重复计数。

关键突破：普通括号的未来只依赖当前余额。超级括号用最左完整括号块及其右侧余串确定唯一分解。

状态与建模：normal中f[d]是已处理前缀余额为d的方案数。super中f是合法串，wrap是一个最外括号包裹的块，left/right是星号串接合法串，star是长度不超过k的全星号段。

本题映射：将?替换为括号或星号，使超级括号序列符合规范。 规则是嵌套加合法块拼接，需区间计数与无歧义分解，不能套普通括号余额。 BracketDP(1e9+7).super(s,k)。

转移与正确性：普通'('令d加1，')'仅在d>0令d减1，问号取两者。超级wrap内只能是空、S、A、SA、AS，不能凭空加SAS；f按第一个wrap及后面的A或SA连接，唯一首块保证无重计。

初始化与顺序：普通f[0]=1，每个字符用清空的g转移，最后取f[0]。super先标star，空内层贡献1；按区间长度递增，非法类型默认0。

优化与限制：普通滚动数组O(n^2)/O(n)。超级枚举分界O(n^3)/O(n^2)；星号长度限制必须同时用于left/right。计数取模，禁止用“方案存在”代替方案数相乘。

本题复杂度：O(n^3)时间、O(n^2)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int mod;

```

```
int super(string s, int k) {
    int n = s.size();
    s = " " + s;
    vector<vector<int>> f(n + 2, vector<int>(n + 2)), wrap = f, left = f, right = f,
                                                        star = f;
    for (int l = 1; l <= n; l++)
        for (int r = l; r <= min(n, l + k - 1); r++) {
            if (s[r] != '*' && s[r] != '?')
                break;
            star[l][r] = 1;
        }
    auto add = [&](int &x, int y) { x = (x + y) % mod; };
    for (int len = 1; len <= n; len++)
        for (int l = 1; l + len - 1 <= n; l++) {
            int r = l + len - 1;
            if (len >= 2 && (s[l] == '(' || s[l] == '?') &&
                (s[r] == ')' || s[r] == '?')) {
                int val = len == 2 ? 1 : star[l + 1][r - 1];
                if (len > 2) {
                    add(val, f[l + 1][r - 1]);
                    add(val, left[l + 1][r - 1]);
                    add(val, right[l + 1][r - 1]);
                }
                wrap[l][r] = val;
            }
            f[l][r] = wrap[l][r];
            for (int mid = l + 1; mid < r; mid++)
                add(f[l][r], wrap[l][mid] * (f[mid + 1][r] + left[mid + 1][r]) % mod);
            for (int mid = l; mid < r && mid - l + 1 <= k; mid++)
                if (star[l][mid])
                    add(left[l][r], f[mid + 1][r]);
            for (int mid = r - 1; mid >= l && r - mid <= k; mid--)
                if (star[mid + 1][r])
                    add(right[l][r], f[l][mid]);
        }
    return n ? f[1][n] : 0;
}

void init_model(int mod_ = 1000000007) {
    mod = mod_;
}

void solve() {
    int n, k;
    string s;
    cin >> n >> k >> s;
    ::init_model();
    cout << ::super(s, k) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：语法计数先寻找唯一分解，再写状态。不同合法状态若有交集，简单相加就会重计；单括号余额模型不能直接解决递归星号语法。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 ABC312D Count Bracket Sequences

https://atcoder.jp/contests/abc312/tasks/abc312_d

题意：无星号情况下，将?替换成括号并计数。

观察与规模：n≤3000且只含括号和问号，没有星号语法，余额为0且不曾负即充分；mod=998244353。 普通括号只要求前缀余额非负且总余额0；超级括号另有连续星号上限及递归语法。

朴素思路：枚举问号替换指数级；超级括号若任意切分合法串，会把同一字符串按多种括号分组重复计数。

关键突破：普通括号的未来只依赖当前余额。超级括号用最左完整括号块及其右侧余串确定唯一分解。

状态与建模：normal中f[d]是已处理前缀余额为d的方案数。super中f是合法串，wrap是一个最外括号包裹的块，left/right是星号串接合法串，star是长度不超过k的全星号段。

本题映射：无星号情况下，将?替换成括号并计数。 无星号退化为普通平衡括号，n=3000必须用余额滚动DP。 BracketDP(998244353).normal(s)。

转移与正确性：普通'('令d加1，')'仅在d>0令d减1，问号取两者。超级wrap内只能是空、S、A、SA、AS，不能凭空加SAS；f按第一个wrap及后面的A或SA连接，唯一首块保证无重计。

初始化与顺序：普通f[0]=1，每个字符用清空的g转移，最后取f[0]。super先标star，空内层贡献1；按区间长度递增，非法类型默认0。

优化与限制：普通滚动数组O(n^2)/O(n)。超级枚举分界O(n^3)/O(n^2)；星号长度限制必须同时用于left/right。计数取模，禁止用“方案存在”代替方案数相乘。

本题复杂度：O(n^2)时间、O(n)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int mod;

int normal(string s) {
    int n = s.size();
    vector<int> f(n + 2), g(n + 2);
    f[0] = 1;
    for (char c : s) {
        fill(g.begin(), g.end(), 0);
        for (int d = 0; d <= n; d++)
            if (f[d]) {
                if (c == '(' || c == '?')
                    g[d + 1] = (g[d + 1] + f[d]) % mod;
                if (d && (c == ')' || c == '?'))
                    g[d - 1] = (g[d - 1] + f[d]) % mod;
            }
        f.swap(g);
    }
    return f[0];
}

```

```
void init_model(int mod_ = 1000000007) {
    mod = mod_;
}

void solve() {
    string s;
    cin >> s;
    ::init_model(998244353);
    cout << ::normal(s) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：语法计数先寻找唯一分解，再写状态。不同合法状态若有交集，简单相加就会重计；单括号余额模型不能直接解决递归星号语法。

验证：C++17独立程序编译通过；3组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 超级序列不接受空串、外侧星号或(SAS)；不能只看括号余额。
- 以第一个原始括号块wrap分解，不能任意枚举两个合法块拼接，否则重复计数。
- 原两状态笔记遗漏嵌套单侧星段；整理版显式维护left/right。

语法计数先寻找唯一分解，再写状态。不同合法状态若有交集，简单相加就会重计；单括号余额模型不能直接解决递归星号语法。

个人资料来源：dp/区间动态规划进阶_比赛板子.md

动态规划 / 换根连通块计数

类型与用途：思维型；计算每点被包含的连通点集数量；前后缀乘积移除一个孩子，支持非素数模。

复杂度与适用范围：O(n)时间与空间，mod<=1e9+7。

例题 1 DP_V Subtree

https://atcoder.jp/contests/dp/tasks/dp_v

题意：给每点求该点为黑且全部黑点连通的染色数。

观察与规模：1<M≤10^9且不保证素数，前后缀不能换成逆元；选择的黑点连通，必须包含当前被询问的点。 对每个点分别询问含它的合法连通选择；模数可能合数，不能依赖乘法逆元。

朴素思路：每个点重新根化做一次树DP是O(n^2)；将总乘积除以孩子因子在因子为0或模数非素数时失效。

关键突破：树的每个邻接方向独立，每个方向要么不选，要么选一个与u连通的方案。

状态与建模：dp[u]为向下包含u的方案数；pd[u]为父方向必须连接父端点的方案数，不含“全部不选”选项。ans[u]=dp[u]×(pd[u]+1)。

本题映射：给每点求该点为黑且全部黑点连通的染色数。 黑点集合就是包含指定点的连通点集，每个孩子选或不选。 读mod和树，run，逐行输出ans。

转移与正确性：dp[u]=Π(dp[v]+1)。给孩子v的pd[v]=(pd[u]+1)×其余孩子因子乘积；pre/suf提供排除单个孩子的乘积。独立方向相乘、不选加1，连通性使分解唯一。

初始化与顺序：叶子dp=1表示只选自己；根pd=0表示父方向不存在。先后序算dp，再先序算pd；pre[0]=suf[m]=1(mod M)。

优化与限制：前后缀将每点所有孩子的排除乘积总计降至O(度数)，整体O(n)/O(n)。CF543D把坏边集合沿根路径最多一条解释为边界切断，等价选择含根的连通核心。

本题复杂度：O(n)时间与空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n, mod;
vector<vector<int>> son;
vector<int> dp, pd, ans;

void init(int n_, int m) {
    n = n_;
    mod = m;
    son.assign(n + 1, {});
    dp.assign(n + 1, 0);
    pd = ans = dp;
}
void add_edge(int u, int v) {
    son[u].push_back(v);
    son[v].push_back(u);
}
void dfs(int u, int p) {
    dp[u] = 1 % mod;
    for (int v : son[u])
        if (v != p) {
            dfs(v, u);
            dp[u] = dp[u] * (dp[v] + 1) % mod;
        }
}
void dfs2(int u, int p) {
    vector<int> a;
    for (int v : son[u])
        if (v != p)
            a.push_back(v);
    int m = a.size();
    vector<int> pre(m + 1, 1 % mod), suf(m + 1, 1 % mod);
    for (int i = 0; i < m; i++)
        pre[i + 1] = pre[i] * (dp[a[i]] + 1) % mod;
    for (int i = m - 1; i >= 0; i--)
        suf[i] = suf[i + 1] * (dp[a[i]] + 1) % mod;
    ans[u] = dp[u] * (pd[u] + 1) % mod;
    for (int i = 0; i < m; i++) {
        int v = a[i];
        pd[v] = pre[i] * suf[i + 1] % mod * (pd[u] + 1) % mod;
        dfs2(v, u);
    }
}
```

```
void run(int root = 1) {
    dfs(root, 0);
    pd[root] = 0;
    dfs2(root, 0);
}

void init_model(int n = 0, int m = 1000000007) {
    init(n, m);
}

void solve() {
    int n, m;
    cin >> n >> m;
    ::init_model(n, m);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        ::add_edge(u, v);
    }
    ::run();
    for (int i = 1; i <= n; i++)
        cout << ::ans[i] << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：多次“排除一个贡献”的转移优先考虑前后缀；乘法有零或模数合数时尤其不能用除法偷懒。

验证：C++17独立程序编译通过；4组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF543D Road Improvement

https://codeforces.com/contest/543/problem/D

题意：对每个首都，统计满足各路径最多一条未修路的道路修复方式。

观察与规模：某些边坏，要求从指定根到任一点至多一条坏边；坏边是根连通核心的边界，核心外全部边必须好，故与含根连通子树双射。 对每个点分别询问含它的合法连通选择；模数可能合数，不能依赖乘法逆元。

朴素思路：每个点重新根化做一次树DP是O(n^2)；将总乘积除以孩子因子在因子为0或模数非素数时失效。

关键突破：树的每个邻接方向独立，每个方向要么不选，要么选一个与u连通的方案。

状态与建模：dp[u]为向下包含u的方案数；pd[u]为父方向必须连接父端点的方案数，不含“全部不选”选项。ans[u]=dp[u]×(pd[u]+1)。

本题映射：对每个首都，统计满足各路径最多一条未修路的道路修复方式。 以首都为根时，若父边未修，整棵子树内所有边必须修；否则子树继续有dp种选择，所以同样是(dp[v]+1)乘积。 输入2..n父节点，add_edge，run；输出ans。

转移与正确性：dp[u]=Π(dp[v]+1)。给孩子v的pd[v]=(pd[u]+1)×其余孩子因子乘积；pre/suf提供排除单个孩子的乘积。独立方向相乘、不选加1，连通性使分解唯一。

初始化与顺序：叶子dp=1表示只选自己；根pd=0表示父方向不存在。先后序算dp，再先序算pd；pre[0]=suf[m]=1(mod M)。

优化与限制：前后缀将每点所有孩子的排除乘积总计降至O(度数)，整体O(n)/O(n)。CF543D把坏边集合沿根路径最多一条解释为边界切断，等价选择含根的连通核心。

本题复杂度：O(n)时间与空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n, mod;
vector<vector<int>> son;
vector<int> dp, pd, ans;

void init(int n_, int m) {
    n = n_;
    mod = m;
    son.assign(n + 1, {});
    dp.assign(n + 1, 0);
    pd = ans = dp;
}
void add_edge(int u, int v) {
    son[u].push_back(v);
    son[v].push_back(u);
}
void dfs(int u, int p) {
    dp[u] = 1 % mod;
    for (int v : son[u])
        if (v != p) {
            dfs(v, u);
            dp[u] = dp[u] * (dp[v] + 1) % mod;
        }
}
void dfs2(int u, int p) {
    vector<int> a;
    for (int v : son[u])
        if (v != p)
            a.push_back(v);
    int m = a.size();
    vector<int> pre(m + 1, 1 % mod), suf(m + 1, 1 % mod);
    for (int i = 0; i < m; i++)
        pre[i + 1] = pre[i] * (dp[a[i]] + 1) % mod;
    for (int i = m - 1; i >= 0; i--)
        suf[i] = suf[i + 1] * (dp[a[i]] + 1) % mod;
    ans[u] = dp[u] * (pd[u] + 1) % mod;
    for (int i = 0; i < m; i++) {
        int v = a[i];
        pd[v] = pre[i] * suf[i + 1] % mod * (pd[u] + 1) % mod;
        dfs2(v, u);
    }
}
void run(int root = 1) {
    dfs(root, 0);
    pd[root] = 0;
    dfs2(root, 0);
}
```

```
51
52 void init_model(int n = 0, int m = 1000000007) {
53     init(n, m);
54 }
55
56 void solve() {
57     int n;
58     cin >> n;
59     ::init_model(n);
60     for (int i = 2; i <= n; i++) {
61         int p;
62         cin >> p;
63         ::add_edge(p, i);
64     }
65     ::run();
66     for (int i = 1; i <= n; i++)
67         cout << ::ans[i] << " \n"[i == n];
68 }
69
70 signed main() {
71     ios::sync_with_stdio(false);
72     cin.tie(nullptr);
73     int t = 1;
74     // cin >> t;
75     while (t--)
76         solve();
77     return 0;
78 }
```

启发式总结：多次“排除一个贡献”的转移优先考虑前后缀；乘法有零或模数合数时尤其不能用除法偷懒。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 不能用逆元除去孩子贡献，非素数模或零贡献时不成立。
- 局部前后缀数组不会被递归覆盖；原共享数组需先算所有孩子再递归。
- 乘法两次取模，避免三个因子直接相乘溢出。

多次“排除一个贡献”的转移优先考虑前后缀；乘法有零或模数合数时尤其不能用除法偷懒。

个人资料来源：dp/树形动态规划进阶_比赛板子.md

动态规划 / 数位构造与不同数位限制

类型与用途：思维型；按字典序DFS构造>=n的最小数；收录至多k种与至少k种数位两个原资料变体。

复杂度与适用范围：at_most O(L*10)；at_least最多O(Lˆ2*1024*10)时间、O(L*1024)空间。

例题 1 CF1560F1 Nearest Beautiful Number easy

https://codeforces.com/contest/1560/problem/F1

题意：k<=2时构造不小于n的最近美丽数。

观察与规模：k≤2，可枚举允许数字集合验证小难度性质；与F2是同一问题的两档限制，讲解应比较值域而非声称模型不同。 目标是最小的≥n且不同数字数≤k的数，数字只有10种；F1限制k≤2，F2允许更多。

朴素思路：从n逐个加一可能跨越很长区间。局部选最小数字但不检查后缀可行性，也可能走进死路。

关键突破：同长度数的大小等于字典序；逐位从小到大试选，首次成功的完整串就是最小解。

状态与建模：dfs(pos,larger,cnt)记录位置、是否已大于下界、已用数字数；vis的十个计数记录哪些数字已出现，并保证回溯撤销正确。

本题映射：k<=2时构造不小于n的最近美丽数。 按数位升序找第一个可完成方案就是最小合法数。 每组at_most(to_string(n),k)。

转移与正确性：若larger=false，本位不能小于n对应位；新数字令cnt加1，超过k跳过。递归失败撤销vis和ans，成功立即返回；字典序遍历的第一条可行路径最优。

初始化与顺序：初始pos=0、larger=false、cnt=0、vis清0。首位至少1；末位后返回成功。相同长度总有全9解(k≥1)，因此不需要额外加一位。

优化与限制：F1也可枚举1或2种允许数字，再贪心找同长度最小合法数；F2采用回溯，严格前缀唯一，一旦变大，总可用已有数字补后缀。课堂至少k种要用(pos,mask,larger)可行性记忆化，并可能增加长度。

本题复杂度：每组O(L*10)时间、O(L+10)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

string bound, ans;
int k;
array<int, 10> vis{};
bool dfs(int pos, bool larger, int cnt) {
    if (pos == (int)bound.size())
        return true;
    int low = larger ? 0 : bound[pos] - '0';
    if (!pos)
        low = max(1LL, low);
    for (int d = low; d <= 9; d++) {
        int nc = cnt + (vis[d] == 0);
        if (nc > k)
            continue;
        vis[d]++;
        ans.push_back('0' + d);
        if (dfs(pos + 1, larger || d > bound[pos] - '0', nc))
            return true;
        ans.pop_back();
        vis[d]--;
    }
    return false;
}
string at_most(string n, int k_) {
    assert(1 <= k_ && k_ <= 10);
    k = k_;
    bound = n;
    ans.clear();
    vis.fill(0);
    bool ok = dfs(0, false, 0);
    assert(ok);
    return ans;
}
string at_least(string n, int k_) {
    assert(1 <= k_ && k_ <= 10);
    k = k_;
```

```
    for (int len = n.size(); len <= max((int)n.size(), k); len++) {
        bound = len == (int)n.size() ? n : "1" + string(len - 1, '0');
        ans.clear();
        vector<vector<array<int32_t, 2>>> memo(
            len + 1, vector<array<int32_t, 2>>(1024, {-1, -1}));
        function<bool(int, int, bool)> can = [&](int pos, int mask,
                                                 bool larger) -> bool {
            if (pos == len)
                return __builtin_popcount((unsigned)mask) >= k;
            auto &res = memo[pos][mask][larger];
            if (res != -1)
                return res != 0;
            int low = larger ? 0 : bound[pos] - '0';
            if (!pos)
                low = max(1LL, low);
            for (int d = low; d <= 9; d++)
                if (can(pos + 1, mask | (1 << d), larger || d > bound[pos] - '0'))
                    return res = 1;
            return res = 0;
        };
        if (!can(0, 0, false))
            continue;
        int mask = 0;
        bool larger = false;
        for (int pos = 0; pos < len; pos++) {
            int low = larger ? 0 : bound[pos] - '0';
            if (!pos)
                low = max(1LL, low);
            for (int d = low; d <= 9; d++)
                if (can(pos + 1, mask | (1 << d), larger || d > bound[pos] - '0')) {
                    ans.push_back('0' + d);
                    mask |= 1 << d;
                    larger |= d > bound[pos] - '0';
                    break;
                }
        }
        return ans;
    }
    return "";
}

void init_model() {
}

void solve() {
    int T;
    cin >> T;
    ::init_model();
    while (T--) {
        string n;
        int k;
        cin >> n >> k;
        cout << ::at_most(n, k) << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
```

```
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：最小构造常分为“枚举下一位”和“判断后缀可行”。至少与至多限制不可互换；禁止首位0、撤销计数和比较边界都是证明的一部分。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF1560F2 Nearest Beautiful Number hard

https://codeforces.com/contest/1560/problem/F2

题意：推广为k<=10的最小合法数。

观察与规模：k最多10；复用按字典序回溯，若不同数位限制不变，变大后填已用最小数字总可完成，避免指数回溯。 目标是最小的≥n且不同数字数≤k的数，数字只有10种；F1限制k≤2，F2允许更多。

朴素思路：从n逐个加一可能跨越很长区间。局部选最小数字但不检查后缀可行性，也可能走进死路。

关键突破：同长度数的大小等于字典序；逐位从小到大试选，首次成功的完整串就是最小解。

状态与建模：dfs(pos,larger,cnt)记录位置、是否已大于下界、已用数字数；vis的十个计数记录哪些数字已出现，并保证回溯撤销正确。

本题映射：推广为k<=10的最小合法数。 同一剪枝仍成立，无需枚举10位数字集合全部1024种。 沿用at_most；这是原题的困难版，注明与F1为同题两档。

转移与正确性：若larger=false，本位不能小于n对应位；新数字令cnt加1，超过k跳过。递归失败撤销vis和ans，成功立即返回；字典序遍历的第一条可行路径最优。

初始化与顺序：初始pos=0、larger=false、cnt=0、vis清0。首位至少1；末位后返回成功。相同长度总有全9解(k≥1)，因此不需要额外加一位。

优化与限制：F1也可枚举1或2种允许数字，再贪心找同长度最小合法数；F2采用回溯，严格前缀唯一，一旦变大，总可用已有数字补后缀。课堂至少k种要用(pos,mask,larger)可行性记忆化，并可能增加长度。

本题复杂度：每组O(L*10)时间、O(L+10)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

string bound, ans;
int k;
array<int, 10> vis{};
bool dfs(int pos, bool larger, int cnt) {
    if (pos == (int)bound.size())
        return true;
    int low = larger ? 0 : bound[pos] - '0';
    if (!pos)
        low = max(1LL, low);
    for (int d = low; d <= 9; d++) {
        int nc = cnt + (vis[d] == 0);
        if (nc > k)
            continue;
        vis[d]++;
```

```
        ans.push_back('0' + d);
        if (dfs(pos + 1, larger || d > bound[pos] - '0', nc))
            return true;
        ans.pop_back();
        vis[d]--;
    }
    return false;
}
string at_most(string n, int k_) {
    assert(1 <= k_ && k_ <= 10);
    k = k_;
    bound = n;
    ans.clear();
    vis.fill(0);
    bool ok = dfs(0, false, 0);
    assert(ok);
    return ans;
}
string at_least(string n, int k_) {
    assert(1 <= k_ && k_ <= 10);
    k = k_;
    for (int len = n.size(); len <= max((int)n.size(), k); len++) {
        bound = len == (int)n.size() ? n : "1" + string(len - 1, '0');
        ans.clear();
        vector<vector<array<int32_t, 2>>> memo(
            len + 1, vector<array<int32_t, 2>>(1024, {-1, -1}));
        function<bool(int, int, bool)> can = [&](int pos, int mask,
                                                 bool larger) -> bool {
            if (pos == len)
                return __builtin_popcount((unsigned)mask) >= k;
            auto &res = memo[pos][mask][larger];
            if (res != -1)
                return res != 0;
            int low = larger ? 0 : bound[pos] - '0';
            if (!pos)
                low = max(1LL, low);
            for (int d = low; d <= 9; d++)
                if (can(pos + 1, mask | (1 << d), larger || d > bound[pos] - '0'))
                    return res = 1;
            return res = 0;
        };
        if (!can(0, 0, false))
            continue;
        int mask = 0;
        bool larger = false;
        for (int pos = 0; pos < len; pos++) {
            int low = larger ? 0 : bound[pos] - '0';
            if (!pos)
                low = max(1LL, low);
            for (int d = low; d <= 9; d++)
                if (can(pos + 1, mask | (1 << d), larger || d > bound[pos] - '0')) {
                    ans.push_back('0' + d);
                    mask |= 1 << d;
                    larger |= d > bound[pos] - '0';
                    break;
                }
        }
        return ans;
    }
    return "";
```

```
}

void init_model() {
}

void solve() {
    int T;
    cin >> T;
    ::init_model();
    while (T--) {
        string n;
        int k;
        cin >> n >> k;
        cout << ::at_most(n, k) << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：最小构造常分为“枚举下一位”和“判断后缀可行”。至少与至多限制不可互换；禁止首位0、撤销计数和比较边界都是证明的一部分。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- vis必须用计数整数，bool++/--错误且C++17禁止bool++。
- 至少k种不保证只增加一位，要一直尝试到max(原长度,k)。
- 至多k种的非tight合法前缀总可填已有数字，因此只在tight路径回溯。

最小构造常分为“枚举下一位”和“判断后缀可行”。至少与至多限制不可互换；禁止首位0、撤销计数和比较边界都是证明的一部分。

个人资料来源：dp/数位动态规划进阶_比赛板子.md

动态规划 / 数位背包

类型与用途：思维型；体积拆成奇数*2^b，高位到低位下放剩余容量，用奇数部分总和截断状态。

复杂度与适用范围：O((63+n)*LIM)时间、O(n+LIM)空间；LIM=min(W,Σ奇数部分)。

例题 1 P3188 梦幻岛宝珠

https://www.luogu.com.cn/problem/P3188

题意：重量很大但w=a*2^b且a<=10的0/1背包。

观察与规模：重量奇数部分≤10，容量大但lim≤10n；输入以n=W=-1结束，每一组必须清空物品。 P1048容量小可直接01背包；P3188容量大但重量能分解为小奇数×2^b，这是额外结构。

朴素思路：容量为W的O(nW)背包在W很大时不可行；把重量简单除2会丢掉奇数和容量的低位。

关键突破：按重量最低置位分组，从容量高位向低位处理；可用余额随下一位变成2j+bit。

状态与建模：f[j]是已处理更高重量组后，在当前计量单位下至少还能容纳j单位的最大价值；过大的余额统一截到lim=所有奇数部分之和，更多余额已无法影响选择。

本题映射：重量很大但w=a*2^b且a<=10的0/1背包。 Σ奇数部分<=10n，使容量与2^30脱钩。 每组建立对象add全部物品，qur(W)，读到-1 -1终止。

转移与正确性：先将高位余额j映射到min(lim,2j+bit)，再处理本组物品：若原来可容纳j+w，则选它后余额j、价值加v。j正序，因此来源j+w还未被本件更新，实现01选择。

初始化与顺序：f[0]=0，其余-INF；从b=62到0，每层g清空。重量必须正，ctz(0)未定义。每组每件的余额循环正序，不能照搬消耗容量写法的倒序。

优化与限制：复杂度O(63×lim+n×lim)，lim依赖奇数部分总和，不仅依赖位数；一般大整数重量不满足小lim就会TLE/MLE。P1048是比较用基础模型，其直接O(TM)更简单。

本题复杂度：O((63+n)*10n)时间、O(n)量级状态空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

static constexpr int INF = 4000000000000000000LL;
vector<pair<int, int>> a;
void init() {
    a.clear();
}
void add(int w, int v) {
    assert(w > 0);
    a.push_back({w, v});
}
int qur(int W) {
    assert(W >= 0);
    vector<vector<pair<int, int>>> item(63);
    int lim = 0;
    for (auto [w, v] : a) {
        int b = __builtin_ctzll((unsigned long long)w);
        int odd = w >> b;
        item[b].push_back({odd, v});
        lim += odd;
    }
    lim = min(lim, W);
```

```
    vector<int> f(lim + 1, -INF), g(lim + 1, -INF);
    f[0] = 0;
    for (int b = 62; b >= 0; b--) {
        fill(g.begin(), g.end(), -INF);
        int bit = (W >> b) & 1;
        for (int j = 0; j <= lim; j++)
            if (f[j] > -INF)
                g[min(lim, 2 * j + bit)] = max(g[min(lim, 2 * j + bit)], f[j]);
        for (auto [w, v] : item[b])
            for (int j = 0; j + w <= lim; j++)
                if (g[j + w] > -INF)
                    g[j] = max(g[j], g[j + w] + v);
        f.swap(g);
    }
    return *max_element(f.begin(), f.end());
}

void init_model() {
    init();
}

void solve() {
    int n, W;
    while (cin >> n >> W && n != -1) {
        ::init_model();
        while (n--) {
            int w, v;
            cin >> w >> v;
            ::add(w, v);
        }
        cout << ::qur(W) << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：容量大时先观察重量结构，寻找尺度变化后仍很小的状态。截断状态必须证明未来最大需求上界，不可仅因为空间不足就截断。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P1048 采药

https://www.luogu.com.cn/problem/P1048

题意：每株草药只能采一次，限定时间内最大收益。

观察与规模：T≤1000、M≤100；每种药材只能采一次，时间和价值给出，正是有限容量01选择。

朴素思路：枚举全部子集2^M不可行；只选单位时间价值最高的药材不一定能填好剩余时间。

关键突破：按药材逐个决策，只需记录已使用时间或允许时间，不需记住具体已选集合。

状态与建模：处理前i件后dp[j]为总时间不超过j的最大价值；i维由外层循环隐含。

转移与正确性：不选第i件继承dp[j]；选它来源为旧dp[j-w]+v，二者取max。以最后一件是否选分类，覆盖全部合法子集。

初始化与顺序：空选择价值0，所有dp初0；每件按j从T倒到w，使来源仍是未选当前件的旧层，防止变成完全背包。

优化与限制：O(TM)时间、O(T)空间。这份基础题解采用普通背包，P3188再利用小奇数部分和数位缩小大容量状态。

本题复杂度：O(TM)时间、O(T)空间。

完整函数题解

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

void solve() {
    int T, M;
    cin >> T >> M;
    vector<int> dp(T + 1);
    while (M--) {
        int w, v;
        cin >> w >> v;
        for (int j = T; j >= w; j--)
            dp[j] = max(dp[j], dp[j - w] + v);
    }
    cout << dp[T] << '\n';
}
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先估算容量DP能否直接通过；仅在容量确实太大并有重量结构时，再考虑数位背包。滚动数组方向取决于是否允许重复取件。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 剩余容量语义的0/1转移必须j从小到大，反向会重复选同一物品。
- 只有奇数部分和小才适合大容量；普通采药例题展示可用但传统背包更直接。
- 不能对0重量调用ctzll；价值和须在long long内。

容量大时先观察重量结构，寻找尺度变化后仍很小的状态。截断状态必须证明未来最大需求上界，不可仅因为空间不足就截断。

个人资料来源：dp/数位动态规划进阶_比赛板子.md

动态规划 / 树上最大连通块权和

类型与用途：思维型；含根的最优连通块权和为自身加所有正的子树贡献；二次DFS给每点换根答案。

复杂度与适用范围：O(n)时间与空间。

例题 1 P1122 最大子树和

https://www.luogu.com.cn/problem/P1122

题意：修剪树后保留非空连通子树使权和最大。

观察与规模：取任意非空连通块，所有点负时取最大单点；答案取全部f的最大值而非根的ans。 要求连通，树上通往每个分支的连接唯一；点权可负，空集不能当最终非空答案。

朴素思路：枚举所有连通点集指数级；直接取全部正点可能不连通，连接负点有代价。

关键突破：固定必须包含u后，各孩子分支独立：贡献负就完全放弃，贡献正就接上。

状态与建模：f[u]为u子树内包含u的最大连通块权和。ans[u]为整棵树中包含u的最大值；up是父亲方向的最优可接贡献。

本题映射：修剪树后保留非空连通子树使权和最大。 任何连通块在固定根下有唯一最高点；取所有f的最大值。 写入a、add_edge、run，然后max f[1..n]。

转移与正确性：f[u]=a[u]+Σmax(0,f[v])。换根ans[u]=f[u]+max(0,up)，传给v的是ans[u]-max(0,f[v])，排除v自己的贡献，避免重复。

初始化与顺序：叶子f=a，第一遍子树后序；第二遍父到子传up，根up=0。全局最优P1122取max f，CF1324F对白/黑赋+1/-1再输出各ans。

优化与限制：两遍O(n)/O(n)，长链递归深度O(n)。全负时最大f仍是某个负点，不能把答案初始化为0后忽略非空要求。

本题复杂度：O(n)时间/空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n;
vector<vector<int>> son;
vector<int> a, f, ans;

void init(int n_) {
    n = n_;
    son.assign(n + 1, {});
    a.assign(n + 1, 0);
    f = ans = a;
}
void add_edge(int u, int v) {
    son[u].push_back(v);
    son[v].push_back(u);
}
void dfs(int u, int p) {
    f[u] = a[u];
    for (int v : son[u])
        if (v != p) {
            dfs(v, u);
            f[u] += max(0LL, f[v]);
```

```
        }
}
void dfs2(int u, int p, int up) {
    ans[u] = f[u] + max(0LL, up);
    for (int v : son[u])
        if (v != p)
            dfs2(v, u, ans[u] - max(0LL, f[v]));
}
void run(int root = 1) {
    dfs(root, 0);
    dfs2(root, 0, 0);
}

void init_model(int n = 0) {
    init(n);
}

void solve() {
    int n;
    cin >> n;
    ::init_model(n);
    for (int i = 1; i <= n; i++)
        cin >> ::a[i];
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        ::add_edge(u, v);
    }
    ::run();
    cout << *max_element(::f.begin() + 1, ::f.end()) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：树上连通选择常用“必须含当前根”消除耦合，再把不选孩子解释为贡献0；换根必须显式扣除目标孩子这一侧。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF1324F Maximum White Subtree

https://codeforces.com/contest/1324/problem/F

题意：对每点求包含它的连通块中白数减黑数的最大值。

观察与规模：每点都要输出含该点的最优连通块，白/黑点映射+1/-1，第二遍换根必不可少。 要求连通，树上通往每个分支的连接唯一；点权可负，空集不能当最终非空答案。

朴素思路：枚举所有连通点集指数级；直接取全部正点可能不连通，连接负点有代价。

关键突破：固定必须包含u后，各孩子分支独立：贡献负就完全放弃，贡献正就接上。

状态与建模：f[u]为u子树内包含u的最大连通块权和。ans[u]为整棵树中包含u的最大值；up是父亲方向的最优可接贡献。

本题映射：对每点求包含它的连通块中白数减黑数的最大值。 白点权+1，黑点权-1；换根把子树答案扩展为整树答案。 把输入颜色转为2*a-1，run后输出ans。

转移与正确性：f[u]=a[u]+Σmax(0,f[v])。换根ans[u]=f[u]+max(0,up)，传给v的是ans[u]-max(0,f[v])，排除v自己的贡献，避免重复。

初始化与顺序：叶子f=a，第一遍子树后序；第二遍父到子传up，根up=0。全局最优P1122取max f，CF1324F对白/黑赋+1/-1再输出各ans。

优化与限制：两遍O(n)/O(n)，长链递归深度O(n)。全负时最大f仍是某个负点，不能把答案初始化为0后忽略非空要求。

本题复杂度：O(n)时间/空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n;
vector<vector<int>> son;
vector<int> a, f, ans;

void init(int n_) {
    n = n_;
    son.assign(n + 1, {});
    a.assign(n + 1, 0);
    f = ans = a;
}
void add_edge(int u, int v) {
    son[u].push_back(v);
    son[v].push_back(u);
}
void dfs(int u, int p) {
    f[u] = a[u];
    for (int v : son[u])
        if (v != p) {
            dfs(v, u);
            f[u] += max(0LL, f[v]);
        }
}
void dfs2(int u, int p, int up) {
    ans[u] = f[u] + max(0LL, up);
    for (int v : son[u])
        if (v != p)
            dfs2(v, u, ans[u] - max(0LL, f[v]));
}
void run(int root = 1) {
    dfs(root, 0);
    dfs2(root, 0, 0);
}

void init_model(int n = 0) {
    init(n);
}

void solve() {
    int n;
    cin >> n;
```

```
    ::init_model(n);
    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        ::a[i] = 2 * x - 1;
    }
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        ::add_edge(u, v);
    }
    ::run();
    for (int i = 1; i <= n; i++)
        cout << ::ans[i] << " \n"[i == n];
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：树上连通选择常用“必须含当前根”消除耦合，再把不选孩子解释为贡献0；换根必须显式扣除目标孩子这一侧。

验证：C++17独立程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 不能把全负树答案初始化成0，题目要求非空。
- 换根必须扣去当前孩子的正贡献，再传给该孩子。
- 链树递归O(n)深度，注意评测栈限制。

树上连通选择常用“必须含当前根”消除耦合，再把不选孩子解释为贡献0；换根必须显式扣除目标孩子这一侧。

个人资料来源：dp/树形动态规划进阶_比赛板子.md

动态规划 / 树上行走与返回状态设计

类型与用途：思维型；从自由起终点遍历，到多军队，再到短倒计时：按题目约束重新推导子树与外界的状态接口。

复杂度与适用范围：三个例题均O(n)时间、O(n)空间；递归代码长链栈风险另计。

例题 1 ABC361E Tree and Hamilton Path 2

https://atcoder.jp/contests/abc361/tasks/abc361_e

题意：带正权树，任选起点和终点，访问所有顶点至少一次，求最短总路程。

观察与规模：N≤2×10^5、边权≤10^9，任意起终点；每边最少一次/两次的下界给出2Σw-diameter，并能沿直径实现。 三题都在树上行走，却分别允许任意起终点、从根新增军队、进入父点后触发短倒计时。限制变化会改变状态，不能共享一条转移。

朴素思路：枚举行走序列既有重复绕行又有多个行动者，不能作为大树算法；单一“子树最优”没有说明是否返回，无法与父节点连接。

关键突破：先用最简单的自由端点行走建立闭合/开放概念，再增加从根出发费用或局部时间条件。

状态与建模：ABC361E下行最长距离f[u]及两条最长分支描述直径。Kingdom dp[u][0]是一支已到u的军队遍历已合并部分再回u的代价，不新增军队，dp[u][1]是开放方案，包含可能新军从根到u的启动费用depth[u]。Crystalfly的f/g/h分别是自身已损失的自由探索、立即收自身再自由探索、收自身并限制先短返回的方案。

本题映射：带正权树，任选起点和终点，访问所有顶点至少一次，求最短总路程。 非起终点路径的边至少走两次，路径边至少一次；选择最长路径省下最多重复代价。 后序DFS维护每点两条最长下行分支，得到直径；输出2*sum-diameter。

转移与正确性：ABC每条不在起终点路径上的边至少两次，路径边一次，答案2Σw-diameter。Kingdom合并孩子：闭合加dp[v][0]+2；开放取旧开+子开、旧开+子闭+2、旧闭+子开三种。Crystalfly的基准sum=Σf[v]，h=a[u]+sum，f可先直接进入一个孩子拿a[v]，或短返孩子v贡献h[v]-f[v]后进入另一t=3孩子拿a[w]，g=f+a[u]。

初始化与顺序：ABC叶子最长分支0，后序更新全局直径。Kingdom初始闭0、开depth；原题fa[i]<i，逆编号正好后序。Crystalfly叶子f=0、g=h=a；子树后序，维护t=3孩子最大/次大a，排除第一孩子。第一孩子可为t=1，不能错误要求也为3。

优化与限制：三题均O(n)/O(n)，ABC边权和用64位；Kingdom需要原题父编号性质，任意编号应先求后序；Crystalfly只适用于t∈{1,2,3}短计时规则，任意长倒计时必须记录更多时间信息。

本题复杂度：O(n)时间、O(n)邻接与递归空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

void solve() {
    int n, sum = 0, diameter = 0;
    cin >> n;
    vector<vector<pair<int, int>>> e(n + 1);
    vector<int> f(n + 1);
    for (int i = 1; i < n; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        e[u].push_back({v, w});
        e[v].push_back({u, w});
        sum += w;
    }
    function<void(int, int)> dfs = [&](int u, int fa) {
        int mx1 = 0, mx2 = 0;
        for (auto [v, w] : e[u])
            if (v != fa) {
                dfs(v, u);
                int z = f[v] + w;
                if (z >= mx1) {
                    mx2 = mx1;
                    mx1 = z;
                } else
                    mx2 = max(mx2, z);
            }
        f[u] = mx1;
        diameter = max(diameter, mx1 + mx2);
    };
    dfs(1, 0);
```

```
    cout << 2 * sum - diameter << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先问返回接口、端点自由度、计时何时开始。不要把直径公式用于多军队题，也不要把任意子树最优当成能一条边返回的方案。

验证：C++17独立程序编译通过；2组登记运行通过；未在线提交；来源：已读取AtCoder原站题面，核实n≤2e5、正边权≤1e9及两个样例

例题 2 Gym102769K Kingdom’s Power

https://codeforces.com/gym/102769/problem/K

题意：根有无限军队，每秒指定一支沿边移动一步，最少多久访问全树。

观察与规模：每步只能移动一个军队，军队可持续从根进入；题面fa[i]<i，逆编号合并省邻接表；开放方案的新军启动费用是depth[u]。 三题都在树上行走，却分别允许任意起终点、从根新增军队、进入父点后触发短倒计时。限制变化会改变状态，不能共享一条转移。

朴素思路：枚举行走序列既有重复绕行又有多个行动者，不能作为大树算法；单一“子树最优”没有说明是否返回，无法与父节点连接。

关键突破：先用最简单的自由端点行走建立闭合/开放概念，再增加从根出发费用或局部时间条件。

状态与建模：ABC361E下行最长距离f[u]及两条最长分支描述直径。Kingdom dp[u][0]是一支已到u的军队遍历已合并部分再回u的代价，不新增军队，dp[u][1]是开放方案，包含可能新军从根到u的启动费用depth[u]。Crystalfly的f/g/h分别是自身已损失的自由探索、立即收自身再自由探索、收自身并限制先短返回的方案。

本题映射：根有无限军队，每秒指定一支沿边移动一步，最少多久访问全树。 单兵闭合分支每条边往返付2；开放分支可以接已有军队或另从根派兵，三种合并择优。 读T及每组n、父数组，run，输出Case #x: ans。

转移与正确性：ABC每条不在起终点路径上的边至少两次，路径边一次，答案2Σw-diameter。Kingdom合并孩子：闭合加dp[v][0]+2；开放取旧开+子开、旧开+子闭+2、旧闭+子开三种。Crystalfly的基准sum=Σf[v]，h=a[u]+sum，f可先直接进入一个孩子拿a[v]，或短返孩子v贡献h[v]-f[v]后进入另一t=3孩子拿a[w]，g=f+a[u]。

初始化与顺序：ABC叶子最长分支0，后序更新全局直径。Kingdom初始闭0、开depth；原题fa[i]<i，逆编号正好后序。Crystalfly叶子f=0、g=h=a；子树后序，维护t=3孩子最大/次大a，排除第一孩子。第一孩子可为t=1，不能错误要求也为3。

优化与限制：三题均O(n)/O(n)，ABC边权和用64位；Kingdom需要原题父编号性质，任意编号应先求后序；Crystalfly只适用于t∈{1,2,3}短计时规则，任意长倒计时必须记录更多时间信息。

本题复杂度：O(n)时间/O(n)空间。

完整函数题解

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n;
vector<int> fa, dep;
vector<array<int, 2>> dp;

void init(int n_) {
    n = n_;
    fa.assign(n + 1, 0);
    dep = fa;
    dp.assign(n + 1, {0, 0});
}
int run() {
    for (int u = 1; u <= n; u++) {
        if (u > 1) {
            assert(1 <= fa[u] && fa[u] < u);
            dep[u] = dep[fa[u]] + 1;
        }
        dp[u] = {0, dep[u]};
    }
    for (int v = n; v >= 2; v--) {
        int u = fa[v];
        dp[u][1] =
            min({dp[u][1] + dp[v][1], dp[u][1] + dp[v][0] + 2, dp[u][0] + dp[v][1]});
        dp[u][0] += dp[v][0] + 2;
    }
    return min(dp[1][0], dp[1][1]);
}

void init_model(int n = 0) {
    init(n);
}

void solve() {
    int T;
    cin >> T;
    for (int tc = 1; tc <= T; tc++) {
        int n;
        cin >> n;
        ::init_model(n);
        for (int u = 2; u <= n; u++)
            cin >> ::fa[u];
        cout << "Case #" << tc << ": " << ::run() << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先问返回接口、端点自由度、计时何时开始。不要把直径公式用于多军队题，也不要把任意子树最优当成能一条边返回的方案。

验证：C++17独立程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 3 Gym103470H Crystalfly

https://codeforces.com/gym/103470/problem/H

题意：在树上行走抓晶蝶，邻点晶蝶被首次惊动后延迟1..3秒消失，求最大收益。

观察与规模：进入父节点触发孩子的逃逸计时；t只有1、2、3。先进入t=1的孩子立即捕获仍合法，第二孩子需允许第3步进入，故只有它必须t=3。 三题都在树上行走，却分别允许任意起终点、从根新增军队、进入父点后触发短倒计时。限制变化会改变状态，不能共享一条转移。

朴素思路：枚举行走序列既有重复绕行又有多个行动者，不能作为大树算法；单一“子树最优”没有说明是否返回，无法与父节点连接。

关键突破：先用最简单的自由端点行走建立闭合/开放概念，再增加从根出发费用或局部时间条件。

状态与建模：ABC361E下行最长距离f[u]及两条最长分支描述直径。Kingdom dp[u][0]是一支已到u的军队遍历已合并部分再回u的代价，不新增军队，dp[u][1]是开放方案，包含可能新军从根到u的启动费用depth[u]。Crystalfly的f/g/h分别是自身已损失的自由探索、立即收自身再自由探索、收自身并限制先短返回的方案。

本题映射：在树上行走抓晶蝶，邻点晶蝶被首次惊动后延迟1..3秒消失，求最大收益。 t≤3使一个父亲处最多及时取得两个子点收益；枚举立即返回的第一子点，配不同的t=3第二子点，保留深入与后续失收益遍历两种贡献。 读T、n、a、t、边，调用run(1)。

转移与正确性：ABC每条不在起终点路径上的边至少两次，路径边一次，答案2Σw-diameter。Kingdom合并孩子：闭合加dp[v][0]+2；开放取旧开+子开、旧开+子闭+2、旧闭+子开三种。Crystalfly的基准sum=Σf[v]，h=a[u]+sum，f可先直接进入一个孩子拿a[v]，或短返孩子v贡献h[v]-f[v]后进入另一t=3孩子拿a[w]，g=f+a[u]。

初始化与顺序：ABC叶子最长分支0，后序更新全局直径。Kingdom初始闭0、开depth；原题fa[i]<i，逆编号正好后序。Crystalfly叶子f=0、g=h=a；子树后序，维护t=3孩子最大/次大a，排除第一孩子。第一孩子可为t=1，不能错误要求也为3。

优化与限制：三题均O(n)/O(n)，ABC边权和用64位；Kingdom需要原题父编号性质，任意编号应先求后序；Crystalfly只适用于t∈{1,2,3}短计时规则，任意长倒计时必须记录更多时间信息。

本题复杂度：O(n)时间/O(n)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n;
vector<vector<int>> e;
vector<int> a, t, f, g, h;

void init(int n_) {
    n = n_;
    e.assign(n + 1, {});
    a.assign(n + 1, 0);
    t = a;
    f = a;
    g = a;
```

```
    h = a;
}
void add_edge(int u, int v) {
    e[u].push_back(v);
    e[v].push_back(u);
}
void dfs(int u, int fa) {
    int sum = 0, best = 0;
    pair<int, int> mx1 = {-1, 0}, mx2 = {-1, 0};
    for (int v : e[u])
        if (v != fa) {
            dfs(v, u);
            sum += f[v];
            best = max(best, a[v]);
            if (t[v] == 3) {
                pair<int, int> z = {a[v], v};
                if (z > mx1) {
                    mx2 = mx1;
                    mx1 = z;
                } else if (z > mx2)
                    mx2 = z;
            }
        }
    h[u] = a[u] + sum;
    f[u] = sum + best;
    for (int v : e[u])
        if (v != fa) {
            int other = (mx1.second == v ? mx2.first : mx1.first);
            if (other >= 0)
                f[u] = max(f[u], sum + h[v] - f[v] + other);
        }
    g[u] = f[u] + a[u];
}
int run(int root = 1) {
    dfs(root, 0);
    return g[root];
}

void init_model(int n = 0) {
    init(n);
}

void solve() {
    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        ::init_model(n);
        for (int u = 1; u <= n; u++)
            cin >> ::a[u];
        for (int u = 1; u <= n; u++)
            cin >> ::t[u];
        for (int i = 1; i < n; i++) {
            int u, v;
            cin >> u >> v;
            ::add_edge(u, v);
        }
        cout << ::run() << '\n';
    }
```

```
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先问返回接口、端点自由度、计时何时开始。不要把直径公式用于多军队题，也不要把任意子树最优当成能一条边返回的方案。

验证：C++17独立程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 自由端点、从根新军、捕获倒计时三种规则不能共享同一转移。

- Crystalfly第一孩子可为t=1，仅第二孩子须t=3；Kingdom逆编号依赖fa[i]<i。

先问返回接口、端点自由度、计时何时开始。不要把直径公式用于多军队题，也不要把任意子树最优当成能一条边返回的方案。

个人资料来源：dp/树形动态规划进阶_比赛板子.md

动态规划 / 树上距离分层计数

类型与用途：思维型；用深度层节点数合并统计；原等距点集按中心各分支选点，固定距离数对用子树深度卷积。

复杂度与适用范围：equidistant O(n^2k)时间/O(nk)空间，适用n<=100；pairs O(nK)时间/空间。

例题 1 CF1551F Equidistant Vertices

https://codeforces.com/contest/1551/problem/F

题意：计数树上大小k的两两等距点集。

观察与规模：n≤100，对k≥3枚举公共中心；同一分支不能取两个同深度点，否则它们间距离小于2d。CF161D只计固定距离点对；CF1551F要求k点任意两点等距，k≥3时出现唯一公共中心。

朴素思路：枚举k点集合是指数级；枚举所有点对路径会到O(n^2)。

关键突破：路径在LCA汇合，点对只在首次合并所属两个不同分支时统计；k≥3等距点必须在中心不同分支且深度相同。

状态与建模：点对f[u][d]为已合并分支中距离u为d的点数。中心模型dp[d][j]为当前中心、深度d上从已处理的不同分支各选一人，合计j人的方案数。

本题映射：计数树上大小k的两两等距点集。 k>=3时必有唯一中心，所有选点距中心相同且来自不同分支。 多测，每组建树后equidistant(k)。

转移与正确性：合并孩子v先加Σf[u][d]×f[v][K-d-1]，再把f[v][d-1]并入f[u][d]，从而每对只在LCA计一次。中心模型按孩子计cnt[d]，dp[d][j]+=dp[d][j-1]×cnt[d]，每分支最多一人。

初始化与顺序：点对f[u][0]=1，子树后序。中心每个d的dp[d][0]=1，j倒序防止同分支多选。k=1直接n，k=2所有点对均满足，不能套唯一中心法。

优化与限制：固定距离O(nK)/O(nK)，计数表用32位，答案乘积转64位。等距模型枚举中心O(n^2k)，只用于原题的小n；不得对大树照抄。

本题复杂度：O(n^2k)时间、O(nk)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n, mod = 1000000007;
vector<vector<int>> g;

void init(int n_) {
    n = n_;
    g.assign(n + 1, {});
}
void add_edge(int u, int v) {
    g[u].push_back(v);
    g[v].push_back(u);
}
void dfs(int u, int p, int d, vector<int> &cnt) {
    if (d >= (int)cnt.size())
        return;
    cnt[d]++;
    for (int v : g[u])
        if (v != p)
            dfs(v, u, d + 1, cnt);
}
int equidistant(int k) {
    if (k == 1)
        return n;
    if (k == 2)
        return n * (n - 1) / 2 % mod;
    int ans = 0;
    for (int u = 1; u <= n; u++) {
        vector<vector<int>> dp(n + 1, vector<int>(k + 1));
        for (auto &z : dp)
            z[0] = 1;
        for (int v : g[u]) {
            vector<int> cnt(n + 1);
            dfs(v, u, 1, cnt);
            for (int d = 1; d <= n; d++)
                if (cnt[d])
                    for (int j = k; j >= 1; j--)
                        dp[d][j] = (dp[d][j] + dp[d][j - 1] * cnt[d]) % mod;
        }
        for (int d = 1; d <= n; d++)
            ans = (ans + dp[d][k]) % mod;
    }
    return ans;
}

void init_model(int n = 0) {
    init(n);
}

void solve() {
    int T;
    cin >> T;
    while (T--) {
        int n, k;
        cin >> n >> k;
        ::init_model(n);
```

```
        for (int i = 1; i < n; i++) {
            int u, v;
            cin >> u >> v;
            ::add_edge(u, v);
        }
        cout << ::equidistant(k) << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：树上距离约束先寻找路径交点/中心，再考虑按深度分层。k=2与k≥3的几何性质不同，必须单独证明特殊分支。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF161D Distance in Tree

https://codeforces.com/contest/161/problem/D

题意：统计距离恰好K的无序点对。

观察与规模：n≤50000、K≤500，只求点对距离K，先统计跨分支贡献再合并，防止同分支点被当成不同侧。 CF161D只计固定距离点对；CF1551F要求k点任意两点等距，k≥3时出现唯一公共中心。

朴素思路：枚举k点集合是指数级；枚举所有点对路径会到O(n^2)。

关键突破：路径在LCA汇合，点对只在首次合并所属两个不同分支时统计；k≥3等距点必须在中心不同分支且深度相同。

状态与建模：点对f[u][d]为已合并分支中距离u为d的点数。中心模型dp[d][j]为当前中心、深度d上从已处理的不同分支各选一人，合计j人的方案数。

本题映射：统计距离恰好K的无序点对。 每对在LCA合并两条不同子树深度时唯一计入，深度和+1=K。 建树后pairs(K)。

转移与正确性：合并孩子v先加Σf[u][d]×f[v][K-d-1]，再把f[v][d-1]并入f[u][d]，从而每对只在LCA计一次。中心模型按孩子计cnt[d]，dp[d][j]+=dp[d][j-1]×cnt[d]，每分支最多一人。

初始化与顺序：点对f[u][0]=1，子树后序。中心每个d的dp[d][0]=1，j倒序防止同分支多选。k=1直接n，k=2所有点对均满足，不能套唯一中心法。

优化与限制：固定距离O(nK)/O(nK)，计数表用32位，答案乘积转64位。等距模型枚举中心O(n^2k)，只用于原题的小n；不得对大树照抄。

本题复杂度：O(nK)时间、O(nK)空间。

完整函数题解

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n, mod = 1000000007;
vector<vector<int>> g;

void init(int n_) {
    n = n_;
    g.assign(n + 1, {});
}
void add_edge(int u, int v) {
    g[u].push_back(v);
    g[v].push_back(u);
}

int pairs(int K) {
    vector<vector<int32_t>> f(n + 1, vector<int32_t>(K + 1));
    int ans = 0;
    function<void(int, int)> work = [&](int u, int p) {
        f[u][0] = 1;
        for (int v : g[u])
            if (v != p) {
                work(v, u);
                for (int d = 0; d < K; d++)
                    ans += (int)f[u][d] * f[v][K - d - 1];
                for (int d = 1; d <= K; d++)
                    f[u][d] += f[v][d - 1];
            }
    };
    work(1, 0);
    return ans;
}

void init_model(int n = 0) {
    init(n);
}

void solve() {
    int n, K;
    cin >> n >> K;
    ::init_model(n);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        ::add_edge(u, v);
    }
    cout << ::pairs(K) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
```

```
59 }
```

启发式总结：树上距离约束先寻找路径交点/中心，再考虑按深度分层。k=2与k≥3的几何性质不同，必须单独证明特殊分支。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- k=2任意点对均等距，必须单独处理；k>=3才有唯一顶点中心。
- 同中心不同分支才能选同距离点；每个分支至多选一个。
- pairs深度计数int32_t减少nK内存，乘法显式转long long。

树上距离约束先寻找路径交点/中心，再考虑按深度分层。k=2与k≥3的几何性质不同，必须单独证明特殊分支。

个人资料来源：dp/树形动态规划进阶_比赛板子.md

动态规划 / 自相似序列奇偶分治

类型与用途：思维型；利用f(2x)、f(2x+1)与f(x)的关系缩小下标；包含Thue-Morse模式计数与三字符替换查询变体。

复杂度与适用范围：calc的上界与模式不断折半，记忆化状态约O(log^2 L)，每状态O(m)；transform O(min(t,log k))时间/栈空间。

例题 1 QOJ12409 Bit Sequence

https://qoj.ac/problem/12409

题意：统计0..L中，popcount(x+i)奇偶序列与给定模式相符的x个数。

观察与规模：匹配0/1限制串的起点上界L；两种奇偶对齐的父层串不同，(L,s)才是完整记忆化键。 序列反复替换后长度指数增长，查询下标/匹配前缀却只用很少信息。

朴素思路：真正展开t轮无法存储；按长度线性递归会被大t卡住。

关键突破：下标除2回到父字符，奇偶决定当前是哪个孩子；连续限制也可按起点奇偶缩回上一层。

状态与建模：ABC242D transform(s,t,k)是第t轮0基第k位字母编号。QOJ calc(L,s)是满足限制串s、起点0..L的合法起点数，memo键为完整(L,s)，不能只存长度。

本题映射：统计0..L中，popcount(x+i)奇偶序列与给定模式相符的x个数。 Thue-Morse的偶数位置继承，奇数位置取反；起点奇偶两类各自将模式压半。 每组init清memo，整数数组转01字符串，calc(L,s)。

转移与正确性：ABC每回上一层(k/2)，字母增量1+k%2(mod3)。QOJ按起点偶/奇两类：检查对应相邻对是否相反，取父层代表位；奇起点要补由首字符反推的前导位，再用(L-1)/2。

初始化与顺序：ABC t=0直接原串；k=0后直接累加t(mod3)，使递归深度由log k控制。QOJ空限制返回L+1，L<0返回0，单字符用全局计数性质；每组清memo。

优化与限制：查询无需生成序列。边界t大于log k必须快速停止；QOJ两种对齐方式不是同一种抽样，不能遗漏补出的首位。

本题复杂度：记忆化约O(m log^2 L)时间，上界状态空间同阶。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

map<pair<int, string>, int> memo;
void init() {
    memo.clear();
}
int calc(int L, const string &s) {
    if (L < 0)
        return 0;
    if (s.empty())
        return L + 1;
    if (L == 0) {
        for (int i = 0; i < (int)s.size(); i++)
            if ((__builtin_popcountll((unsigned long long)i) & 1) != s[i] - '0')
                return 0;
        return 1;
    }
    if (s.size() == 1) {
        int len = L + 1, half = len / 2;
        return half + (len % 2 &&
                       (__builtin_popcountll((unsigned long long)L) & 1) == s[0] - '0');
    }
    auto key = make_pair(L, s);
    if (memo.count(key))
        return memo[key];
    int n = s.size(), res = 0;
    bool ok = true;
    for (int i = 0; i + 1 < n; i += 2)
        if (s[i] == s[i + 1])
            ok = false;
    if (ok) {
        string t;
        for (int i = 0; i < n; i += 2)
            t.push_back(s[i]);
        res += calc(L / 2, t);
    }
    ok = true;
    for (int i = 1; i + 1 < n; i += 2)
        if (s[i] == s[i + 1])
            ok = false;
    if (ok && L >= 1) {
        string t;
        t.push_back(s[0] == '0' ? '1' : '0');
        for (int i = 1; i < n; i += 2)
            t.push_back(s[i]);
        res += calc((L - 1) / 2, t);
    }
    return memo[key] = res;
}

void init_model() {
}

void solve() {
    int T;
    cin >> T;
```

```
    ::init_model();
    while (T--) {
        int m, L;
        cin >> m >> L;
        string s;
        while (m--) {
            int x;
            cin >> x;
            s.push_back('0' + x);
        }
        ::init();
        cout << ::calc(L, s) << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：自相似问题先从一个最终位置追踪祖先，或按对齐余数分组。状态应保留被缩小后仍影响合法性的短限制串。

验证：C++17独立程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 ABC242D ABC Transform

https://atcoder.jp/contests/abc242/tasks/abc242_d

题意：ABC字符同时替换t轮后，查询第k个字符，t可能很大。

观察与规模：t≤10^18、k≤10^18；当k归零可直接把余下t步累加mod3，不能递归t层。 序列反复替换后长度指数增长，查询下标/匹配前缀却只用很少信息。

朴素思路：真正展开t轮无法存储；按长度线性递归会被大t卡住。

关键突破：下标除2回到父字符，奇偶决定当前是哪个孩子；连续限制也可按起点奇偶缩回上一层。

状态与建模：ABC242D transform(s,t,k)是第t轮0基第k位字母编号。QOJ calc(L,s)是满足限制串s、起点0..L的合法起点数，memo键为完整(L,s)，不能只存长度。

本题映射：ABC字符同时替换t轮后，查询第k个字符，t可能很大。 每个字符生成左右两子字符，0基位置的奇偶决定对子字符编号增加1或2；逆向回父位置。 将k转0基，transform(S,t,k-1)，返回值+A。

转移与正确性：ABC每回上一层(k/2)，字母增量1+k%2(mod3)。QOJ按起点偶/奇两类：检查对应相邻对是否相反，取父层代表位；奇起点要补由首字符反推的前导位，再用(L-1)/2。

初始化与顺序：ABC t=0直接原串；k=0后直接累加t(mod3)，使递归深度由log k控制。QOJ空限制返回L+1，L<0返回0，单字符用全局计数性质；每组清memo。

优化与限制：查询无需生成序列。边界t大于log k必须快速停止；QOJ两种对齐方式不是同一种抽样，不能遗漏补出的首位。

本题复杂度：每询问O(min(t,log k))时间与递归空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

map<pair<int, string>, int> memo;

int transform(const string &s, int t, int k) {
    if (t == 0)
        return s[k] - 'A';
    if (k == 0)
        return (s[0] - 'A' + t % 3) % 3;
    return (transform(s, t - 1, k / 2) + 1 + k % 2) % 3;
}

void init_model() {
}

void solve() {
    string s;
    int q;
    cin >> s >> q;
    ::init_model();
    while (q--) {
        int time, k;
        cin >> time >> k;
        cout << char('A' + ::transform(s, time, k - 1)) << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：自相似问题先从一个最终位置追踪祖先，或按对齐余数分组。状态应保留被缩小后仍影响合法性的短限制串。

验证：C++17独立程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 原calc在L=0或模式长1时可能无限递归，显式加入终止情形。
- 奇数起点的补位必须转换字符0/1；不要对ASCII直接产生非数字字符。
- transform是三字符变体，不能把mod2的翻转直接换成同一题输入。

自相似问题先从一个最终位置追踪祖先，或按对齐余数分组。状态应保留被缩小后仍影响合法性的短限制串。

个人资料来源：dp/数位动态规划进阶_比赛板子.md

动态规划 / 进位支付与有符号数位

类型与用途：思维型；每位选择直接支付或向更高币值进位并找零；二进制base=2就是原资料最少±2^k。

复杂度与适用范围：幂倍币制O(n)个实际金额状态、map查找O(log n)；固定进制O(L)时间/O(1)空间。

例题 1 ABC231E Minimal payments

https://atcoder.jp/contests/abc231/tasks/abc231_e

题意：币值组成整倍数链，买X元物品时支付和找零总币数最少。

观察与规模：每种币是前种的整数倍，X≤10^18；dfs中单位逐层缩小，只比较相邻两个商。 各面额逐级整除，允许付款并找零；单纯尽量用最大面额不一定省枚数。

朴素思路：枚举每种币的正负使用数量无界；仅向下取整遗漏“多付一张大币再找零”。

关键突破：在相邻面额比为base的层次，低位余数r只需考虑付r张或收base-r张并进位。

状态与建模：dfs(i,x)是在第i种面额单位下支付x的最少总币数。更小面额已经决定，未来仅与剩余商及进位相关。十进制实现f/g分别表示当前前缀直接结清/结清前缀加1的最小成本。

本题映射：币值组成整倍数链，买X元物品时支付和找零总币数最少。 低位余数只有向下支付和向上进位两种有效选择。 读升序币值调用qur(a,X)。

转移与正确性：递归取min(r+dfs(i+1,q)，base-r+dfs(i+1,q+1))，r=0无需向上付。数字法nf=min(f+d,g+10-d)，ng=min(f+d+1,g+9-d)，分别对应新前缀及新前缀加1。

初始化与顺序：递归最后面额只能用x张；每次qur清空memo。数字法空前缀f=0、g=1，按字符串从高位向低位更新，最终输出f。

优化与限制：相邻整除保证只有相邻的两个商需要保留；任意面额系统不成立。字符串避免大金额溢出；递归按层数下降，记忆化合并重复的(i,x)。

本题复杂度：O(n log n)时间、O(n)状态空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

vector<int> a;
map<pair<int, int>, int> memo;
int dfs(int i, int x) {
    if (i + 1 == (int)a.size())
        return x;
    auto key = make_pair(i, x);
    if (memo.count(key))
        return memo[key];
    int base = a[i + 1] / a[i], r = x % base, q = x / base;
    int ans = r + dfs(i + 1, q);
    if (r)
        ans = min(ans, base - r + dfs(i + 1, q + 1));
    return memo[key] = ans;
}
int qur(const vector<int> &coins, int x) {
    a = coins;
    assert(!a.empty() && a[0] == 1);
    for (int i = 1; i < (int)a.size(); i++)
        assert(a[i] > a[i - 1] && a[i] % a[i - 1] == 0);
    memo.clear();
```

```
    return dfs(0, x);
}

void init_model() {
}

void solve() {
    int n, X;
    cin >> n >> X;
    vector<int> a(n);
    for (int &x : a)
        cin >> x;
    ::init_model();
    cout << ::qur(a, X) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：可找零的数位问题要同时保留“当前结清”和“借一位/进一位”，先用一位base进制算式推导，再压成常数状态。

验证：C++17独立程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 ABC155E Payment

https://atcoder.jp/contests/abc155/tasks/abc155_e

题意：币值为10的幂，巨大十进制金额最少支付及找零币数。

观察与规模：金额可长达10^6位，用字符串并仅保留f/g两个数，O(长度)；返回硬币和支付硬币都计数。 各面额逐级整除，允许付款并找零；单纯尽量用最大面额不一定省枚数。

朴素思路：枚举每种币的正负使用数量无界；仅向下取整遗漏“多付一张大币再找零”。

关键突破：在相邻面额比为base的层次，低位余数r只需考虑付r张或收base-r张并进位。

状态与建模：dfs(i,x)是在第i种面额单位下支付x的最少总币数。更小面额已经决定，未来仅与剩余商及进位相关。十进制实现f/g分别表示当前前缀直接结清/结清前缀加1的最小成本。

本题映射：币值为10的幂，巨大十进制金额最少支付及找零币数。 不能转long long，直接滚动“当前前缀付精确值/付前缀+1”的两个状态。 调用digits(N,10)。

转移与正确性：递归取min(r+dfs(i+1,q), base-r+dfs(i+1,q+1))，r=0无需向上付。数字法nf=min(f+d,g+10-d)，ng=min(f+d+1,g+9-d)，分别对应新前缀及新前缀加1。

初始化与顺序：递归最后面额只能用x张；每次qur清空memo。数字法空前缀f=0、g=1，按字符串从高位向低位更新，最终输出f。

优化与限制：相邻整除保证只有相邻的两个商需要保留；任意面额系统不成立。字符串避免大金额溢出；递归按层数下降，记忆化合并重复的(i,x)。

本题复杂度：O(|N|)时间、O(1)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

vector<int> a;
map<pair<int, int>, int> memo;

int digits(string s, int base) {
    int f = 0, g = 1;
    for (char c : s) {
        int d = c - '0';
        int nf = min(f + d, g + base - d), ng = min(f + d + 1, g + base - 1 - d);
        f = nf;
        g = ng;
    }
    return f;
}

void init_model() {
}

void solve() {
    string N;
    cin >> N;
    ::init_model();
    cout << ::digits(N, 10) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：可找零的数位问题要同时保留“当前结清”和“借一位/进一位”，先用一位base进制算式推导，再压成常数状态。

验证：C++17独立程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 二进制由题目不限制最高幂时可以进到额外一位，未态允许一次大币找零。
- 原作业仅有口述伪代码，整理版明确了进位状态和收尾。
- 一般币制不满足整倍数时不能套此贪心转移。

可找零的数位问题要同时保留“当前结清”和“借一位/进一位”，先用一位base进制算式推导，再压成常数状态。

个人资料来源：dp/数位动态规划进阶_比赛板子.md

数据结构：模型与推导

数据结构 / 启发式分治

类型与用途：思维型；允许划分不均衡，但寻找分界或统计只扫描短侧；保留双端唯一元素判定，增加最大值分界的短侧枚举。

复杂度与适用范围：non_boring O(n log n)时间/O(n)空间；maximum_pairs O(n log n)时间/空间（ST预处理）。

例题 1 UVA1608 Non-boring sequences

https://onlinejudge.org/external/16/1608.pdf

题意：判定任意子区间是否都有只出现一次的值。

观察与规模：判定任意子区间是否都有只出现一次的值。 选当前区间唯一值为分界，含分界的所有区间已合法，只递归左右。 分界未必均衡；如果每层只扫描短侧，仍可控制总工作。

朴素思路：从左到右找分界在极端数组每次扫描大段，退化O(n^2)。区间最大值跨段对双枚举也会平方。

关键突破：Non-boring从两端同时找区间唯一值；Special Segments以最大值分界，跨界仅枚举短侧并用值反查位置。

状态与建模：pre/nxt判断某元素在当前区间唯一：pre[i]<l且nxt[i]>r。排列题pos[value]给唯一位置，ST提供当前区间argmax。

本题映射：判定任意子区间是否都有只出现一次的值。 选当前区间唯一值为分界，含分界的所有区间已合法，只递归左右。 init后non_boring，多测输出non-boring或boring。

转移与正确性：若找到唯一元素，其覆盖的全部含它区间合法，只递归两侧；若没有，则当前区间已是反例。排列以最大值m分界，跨界端点满足另一值=a[m]-已知值，只需检查其pos是否落另一侧。

初始化与顺序：pre初0、nxt初n+1；空区间Non-boring合法，单点排列贡献0。递归扫描双端，分界后递归，不需改成手写栈。

优化与限制：短侧元素每次被扫描后所属区间至少减半，总O(nlog n)；ST占O(nlog n)，递归仍可能深O(n)而导致栈RE。非排列有重复值时pos唯一性证明失效。

本题复杂度：O(n log n)时间、O(n)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n;
vector<int> a, pre, nxt;
void init(const vector<int> &v) {
    a = v;
    n = a.size() - 1;
    pre.assign(n + 1, 0);
    nxt.assign(n + 1, n + 1);
    map<int, int> last;
    for (int i = 1; i <= n; i++) {
        pre[i] = last[a[i]];
        if (pre[i])
            nxt[pre[i]] = i;
        last[a[i]] = i;
    }
}
```

```
bool dfs(int l, int r) {
    if (l > r)
        return true;
    for (int x = l, y = r; x <= y; x++, y--) {
        if (pre[x] < l && nxt[x] > r)
            return dfs(l, x - 1) && dfs(x + 1, r);
        if (pre[y] < l && nxt[y] > r)
            return dfs(l, y - 1) && dfs(y + 1, r);
    }
    return false;
}
bool non_boring() {
    return dfs(1, n);
}
int maximum_pairs() {
    vector<int> pos(n + 1), lg(n + 1);
    for (int i = 1; i <= n; i++) {
        assert(1 <= a[i] && a[i] <= n && !pos[a[i]]);
        pos[a[i]] = i;
    }
    for (int i = 2; i <= n; i++)
        lg[i] = lg[i / 2] + 1;
    vector<vector<int32_t>> st(lg[n] + 1, vector<int32_t>(n + 1));
    for (int i = 1; i <= n; i++)
        st[0][i] = i;
    auto better = [&](int i, int j) { return a[i] > a[j] ? i : j; };
    for (int k = 1; k < (int)st.size(); k++)
        for (int i = 1; i + (1 << k) - 1 <= n; i++)
            st[k][i] = better(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
    auto argmax = [&](int l, int r) {
        int k = lg[r - l + 1];
        return better(st[k][l], st[k][r - (1 << k) + 1]);
    };
    function<int(int, int)> work = [&](int l, int r) {
        if (l >= r)
            return 0LL;
        int m = argmax(l, r), ans = work(l, m - 1) + work(m + 1, r);
        if (m - l < r - m) {
            for (int i = l; i < m; i++) {
                int x = a[m] - a[i];
                if (x > 0 && pos[x] > m && pos[x] <= r)
                    ans++;
            }
        } else
            for (int j = m + 1; j <= r; j++) {
                int x = a[m] - a[j];
                if (x > 0 && pos[x] >= l && pos[x] < m)
                    ans++;
            }
        return ans;
    };
    return work(1, n);
}

void init_model() {
}

void solve() {
    int T;
    cin >> T;
```

```
    while (T--) {
        int n;
        cin >> n;
        vector<int> a(n + 1);
        for (int i = 1; i <= n; i++)
            cin >> a[i];
        ::init_model();
        ::init(a);
        cout << (::non_boring() ? "non-boring" : "boring") << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：时间与递归深度分别证明：短侧代价可控不代表栈浅。出现不均衡分治应问能否把代价只记在小的一侧。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF1156E Special Segments

https://codeforces.com/contest/1156/problem/E

题意：排列中计数两端和等于区间最大值的区间。

观察与规模：排列中计数两端和等于区间最大值的区间。 以区间最大值m分界，跨分界的两端只枚举短侧，另一值及位置由最大值减已知值确定。 分界未必均衡；如果每层只扫描短侧，仍可控制总工作。

朴素思路：从左到右找分界在极端数组每次扫描大段，退化O(n^2)。区间最大值跨段对双枚举也会平方。

关键突破：Non-boring从两端同时找区间唯一值；Special Segments以最大值分界，跨界仅枚举短侧并用值反查位置。

状态与建模：pre/nxt判断某元素在当前区间唯一：pre[i]<l且nxt[i]>r。排列题pos[value]给唯一位置，ST提供当前区间argmax。

本题映射：排列中计数两端和等于区间最大值的区间。 以区间最大值m分界，跨分界的两端只枚举短侧，另一值及位置由最大值减已知值确定。 init排列，maximum_pairs。

转移与正确性：若找到唯一元素，其覆盖的全部含它区间合法，只递归两侧；若没有，则当前区间已是反例。排列以最大值m分界，跨界端点满足另一值=a[m]-已知值，只需检查其pos是否落另一侧。

初始化与顺序：pre初0、nxt初n+1；空区间Non-boring合法，单点排列贡献0。递归扫描双端，分界后递归，不需改成手写栈。

优化与限制：短侧元素每次被扫描后所属区间至少减半，总O(nlog n)；ST占O(nlog n)，递归仍可能深O(n)而导致栈RE。非排列有重复值时pos唯一性证明失效。

本题复杂度：O(n log n)时间/空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n;
vector<int> a, pre, nxt;
void init(const vector<int> &v) {
    a = v;
    n = a.size() - 1;
    pre.assign(n + 1, 0);
    nxt.assign(n + 1, n + 1);
    map<int, int> last;
    for (int i = 1; i <= n; i++) {
        pre[i] = last[a[i]];
        if (pre[i])
            nxt[pre[i]] = i;
        last[a[i]] = i;
    }
}
bool dfs(int l, int r) {
    if (l > r)
        return true;
    for (int x = l, y = r; x <= y; x++, y--) {
        if (pre[x] < l && nxt[x] > r)
            return dfs(l, x - 1) && dfs(x + 1, r);
        if (pre[y] < l && nxt[y] > r)
            return dfs(l, y - 1) && dfs(y + 1, r);
    }
    return false;
}
bool non_boring() {
    return dfs(1, n);
}
int maximum_pairs() {
    vector<int> pos(n + 1), lg(n + 1);
    for (int i = 1; i <= n; i++) {
        assert(1 <= a[i] && a[i] <= n && !pos[a[i]]);
        pos[a[i]] = i;
    }
    for (int i = 2; i <= n; i++)
        lg[i] = lg[i / 2] + 1;
    vector<vector<int32_t>> st(lg[n] + 1, vector<int32_t>(n + 1));
    for (int i = 1; i <= n; i++)
        st[0][i] = i;
    auto better = [&](int i, int j) { return a[i] > a[j] ? i : j; };
    for (int k = 1; k < (int)st.size(); k++)
        for (int i = 1; i + (1 << k) - 1 <= n; i++)
            st[k][i] = better(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
    auto argmax = [&](int l, int r) {
        int k = lg[r - l + 1];
        return better(st[k][l], st[k][r - (1 << k) + 1]);
    };
    function<int(int, int)> work = [&](int l, int r) {
        if (l >= r)
            return 0LL;
        int m = argmax(l, r), ans = work(l, m - 1) + work(m + 1, r);
```

```
        if (m - l < r - m) {
            for (int i = l; i < m; i++) {
                int x = a[m] - a[i];
                if (x > 0 && pos[x] > m && pos[x] <= r)
                    ans++;
            }
        } else
            for (int j = m + 1; j <= r; j++) {
                int x = a[m] - a[j];
                if (x > 0 && pos[x] >= l && pos[x] < m)
                    ans++;
            }
        return ans;
    };
    return work(1, n);
}

void init_model() {
}

void solve() {
    int n;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    ::init_model();
    ::init(a);
    cout << ::maximum_pairs() << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：时间与递归深度分别证明：短侧代价可控不代表栈浅。出现不均衡分治应问能否把代价只记在小的一侧。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 从单侧扫描唯一元素会退化为O(n^2)。
- 枚举短侧并用值到位置映射查另一端，不能同时暴力枚举两侧。
- 递归可能是链，深度O(n)，栈风险与时间复杂度是两回事。

时间与递归深度分别证明：短侧代价可控不代表栈浅。出现不均衡分治应问能否把代价只记在小的一侧。

个人资料来源：数据结构/模块一_启发式合并与启发式分治.md

字符串、线性代数与几何：模型与推导

字符串、线性代数与几何 / 极角排序

类型与用途：思维型；按半平面再叉积排序非零向量，使圆周相邻关系和连续方向区间可枚举。

复杂度与适用范围：O(n log n)排序时间、O(log n)排序辅助栈。

例题 1 CF598C Nearest vectors

https://codeforces.com/contest/598/problem/C

题意：寻找最小夹角的一对非零向量。

观察与规模：寻找最小夹角的一对非零向量。 最小夹角一定来自极角序的相邻向量；包含首尾圆周相邻。 一个模型求最近方向夹角，另一个求若干向量和长度最大；长度和的选择不是逐个取大向量。

朴素思路：所有夹角点对O(n^2)；枚举向量子集2^n。

关键突破：最小夹角必发生在圆周排序相邻方向。最优向量和S的选中向量投影到S方向非负、未选非正，因此可表示为一个半平面内连续方向段。

状态与建模：CF598C对带原编号的向量按半平面及叉积排序，比较相邻含首尾。ABC139F枚举排序后环形连续段，维护(sumx,sumy)和平方长度。

本题映射：寻找最小夹角的一对非零向量。 最小夹角一定来自极角序的相邻向量；包含首尾圆周相邻。按cmp排序带原编号向量，逐对用atan2(abs(det),dot)取夹角。

转移与正确性：CF非邻接两方向之间若夹了另一方向，则至少一个相邻间距更小。ABC若选中投影负向量，去掉使|S|增大；未选正投影向量加入也增大，故最优集合满足连续半平面结构，边界零投影可纳入枚举。

初始化与顺序：方向排序必须区分上下半平面，叉积0再确定稳定顺序；环形复制或取模枚举。空集合长度0；零向量方向未定义，几何接口应单列或跳过。

优化与限制：CF O(nlog n)，ABC n≤100，O(n^2)枚举适用；结果sqrt后用足够精度输出。叉积用__int128避免坐标乘法溢出。

本题复杂度：O(n log n)时间、O(n)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

using db = long double;
const db EPS = 1e-9L;
int sign(db x) {
    return x < -EPS ? -1 : x > EPS ? 1 : 0;
}
struct P {
    db x = 0, y = 0;
    P() {
    }
    P(db x, db y) : x(x), y(y) {
    }
    P operator+(P p) const {
        return {x + p.x, y + p.y};
    }
    P operator-(P p) const {
        return {x - p.x, y - p.y};
    }
```

```
    P operator*(db k) const {
        return {x * k, y * k};
    }
    P operator/(db k) const {
        return {x / k, y / k};
    }
    bool operator<(P p) const {
        return x != p.x ? x < p.x : y < p.y;
    }
    db dot(P p) const {
        return x * p.x + y * p.y;
    }
    db det(P p) const {
        return x * p.y - y * p.x;
    }
    db abs2() const {
        return dot(*this);
    }
    db abs() const {
        return sqrtl(abs2());
    }
    db distTo(P p) const {
        return (*this - p).abs();
    }
    db alpha() const {
        return atan2l(y, x);
    }
    P rot90() const {
        return {-y, x};
    }
    P rot(db a) const {
        return {x * cosl(a) - y * sinl(a), x * sinl(a) + y * cosl(a)};
    }
    P unit() const {
        assert(abs2() > 0);
        return *this / abs();
    }
    int quad() const {
        return y > 0 || (y == 0 && x >= 0);
    }
    void read() {
        cin >> x >> y;
    }
};
struct Geometry {
    db cross(P a, P b, P c) {
        return (b - a).det(c - a);
    }
    bool intersect(db a, db b, db c, db d) {
        if (a > b)
            swap(a, b);
        if (c > d)
            swap(c, d);
        return sign(b - c) >= 0 && sign(d - a) >= 0;
    }
    bool onSeg(P a, P b, P p) {
        return !sign(cross(a, b, p)) && sign((p - a).dot(p - b)) <= 0;
    }
```

```
    bool onSeg_strict(P a, P b, P p) {
        return !sign(cross(a, b, p)) && sign((p - a).dot(p - b)) < 0;
    }
    bool isSS(P a, P b, P c, P d) {
        return intersect(a.x, b.x, c.x, d.x) && intersect(a.y, b.y, c.y, d.y) &&
               sign(cross(a, b, c)) * sign(cross(a, b, d)) <= 0 &&
               sign(cross(c, d, a)) * sign(cross(c, d, b)) <= 0;
    }
    bool isSS_strict(P a, P b, P c, P d) {
        return sign(cross(a, b, c)) * sign(cross(a, b, d)) < 0 &&
               sign(cross(c, d, a)) * sign(cross(c, d, b)) < 0;
    }
    bool chkLL(P a, P b, P c, P d) {
        return sign((b - a).det(d - c)) != 0;
    }
    P isLL(P a, P b, P c, P d) {
        assert(chkLL(a, b, c, d));
        return a + (b - a) * ((c - a).det(d - c) / (b - a).det(d - c));
    }
    P proj(P a, P b, P p) {
        P v = b - a;
        assert(v.abs2() > 0);
        return a + v * (v.dot(p - a) / v.abs2());
    }
    P reflect(P a, P b, P p) {
        return proj(a, b, p) * 2 - p;
    }
    db nearest(P a, P b, P p) {
        if ((b - a).abs2() == 0)
            return p.distTo(a);
        P v = b - a;
        db t = clamp(v.dot(p - a) / v.abs2(), 0.0L, 1.0L);
        return p.distTo(a + v * t);
    }
    db disSS(P a, P b, P c, P d) {
        if (isSS(a, b, c, d))
            return 0;
        return min(
            {nearest(a, b, c), nearest(a, b, d), nearest(c, d, a), nearest(c, d, b)});
    }
};

static bool cmp(P a, P b) {
    if (a.quad() != b.quad())
        return a.quad() > b.quad();
    db z = a.det(b);
    if (z != 0)
        return z > 0;
    return a.abs2() < b.abs2();
}
void run(vector<P> &a) {
    sort(a.begin(), a.end(), cmp);
}

void init_model() {
}

void solve() {
    int n;
    cin >> n;
```

```
    vector<pair<P, int>> a(n);
    for (int i = 0; i < n; i++)
        a[i].first.read(), a[i].second = i + 1;
    sort(a.begin(), a.end(), [](auto a, auto b) { return ::cmp(a.first, b.first); });
    db best = 10;
    pair<int, int> ans;
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        db ang = atan2l(fabsl(a[i].first.det(a[j].first)), a[i].first.dot(a[j].first));
        if (ang < best)
            best = ang, ans = {a[i].second, a[j].second};
    }
    cout << ans.first << ' ' << ans.second << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先用交换/增删论证找出最优集合必须满足的方向条件，再考虑排序连续段；极角排序本身并没有解决子集最大化。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 ABC139F Engines

https://atcoder.jp/contests/abc139/tasks/abc139_f

题意：选择若干向量，使和的长度最大。

观察与规模：选择若干向量，使和的长度最大。 最优向量集合可取为沿某个方向点积非负的半平面；极角序中是连续区间。枚举所有圆周连续区间必覆盖最优。 一个模型求最近方向夹角，另一个求若干向量和长度最大；长度和的选择不是逐个取大向量。

朴素思路：所有夹角点对O(n^2)；枚举向量子集2^n。

关键突破：最小夹角必发生在圆周排序相邻方向。最优向量和S的选中向量投影到S方向非负、未选非正，因此可表示为一个半平面内连续方向段。

状态与建模：CF598C对带原编号的向量按半平面及叉积排序，比较相邻含首尾。ABC139F枚举排序后环形连续段，维护(sumx,sumy)和平方长度。

本题映射：选择若干向量，使和的长度最大。 最优向量集合可取为沿某个方向点积非负的半平面；极角序中是连续区间。枚举所有圆周连续区间必覆盖最优。 过滤零向量，run排序，枚举起点并累加最多n个向量。

转移与正确性：CF非邻接两方向之间若夹了另一方向，则至少一个相邻间距更小。ABC若选中投影负向量，去掉使|S|增大；未选正投影向量加入也增大，故最优集合满足连续半平面结构，边界零投影可纳入枚举。

初始化与顺序：方向排序必须区分上下半平面，叉积0再确定稳定顺序；环形复制或取模枚举。空集合长度0；零向量方向未定义，几何接口应单列或跳过。

优化与限制：CF O(nlog n)，ABC n≤100，O(n^2)枚举适用；结果sqrt后用足够精度输出。叉积用__int128避免坐标乘法溢出。

本题复杂度：O(nˆ2+n log n)时间、O(n)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

using db = long double;
const db EPS = 1e-9L;
int sign(db x) {
    return x < -EPS ? -1 : x > EPS ? 1 : 0;
}
struct P {
    db x = 0, y = 0;
    P() {
    }
    P(db x, db y) : x(x), y(y) {
    }
    P operator+(P p) const {
        return {x + p.x, y + p.y};
    }
    P operator-(P p) const {
        return {x - p.x, y - p.y};
    }
    P operator*(db k) const {
        return {x * k, y * k};
    }
    P operator/(db k) const {
        return {x / k, y / k};
    }
    bool operator<(P p) const {
        return x != p.x ? x < p.x : y < p.y;
    }
    db dot(P p) const {
        return x * p.x + y * p.y;
    }
    db det(P p) const {
        return x * p.y - y * p.x;
    }
    db abs2() const {
        return dot(*this);
    }
    db abs() const {
        return sqrtl(abs2());
    }
    db distTo(P p) const {
        return (*this - p).abs();
    }
    db alpha() const {
        return atan2l(y, x);
    }
    P rot90() const {
        return {-y, x};
    }
    P rot(db a) const {
        return {x * cosl(a) - y * sinl(a), x * sinl(a) + y * cosl(a)};
    }
```

```
    P unit() const {
        assert(abs2() > 0);
        return *this / abs();
    }
    int quad() const {
        return y > 0 || (y == 0 && x >= 0);
    }
    void read() {
        cin >> x >> y;
    }
};
struct Geometry {
    db cross(P a, P b, P c) {
        return (b - a).det(c - a);
    }
    bool intersect(db a, db b, db c, db d) {
        if (a > b)
            swap(a, b);
        if (c > d)
            swap(c, d);
        return sign(b - c) >= 0 && sign(d - a) >= 0;
    }
    bool onSeg(P a, P b, P p) {
        return !sign(cross(a, b, p)) && sign((p - a).dot(p - b)) <= 0;
    }
    bool onSeg_strict(P a, P b, P p) {
        return !sign(cross(a, b, p)) && sign((p - a).dot(p - b)) < 0;
    }
    bool isSS(P a, P b, P c, P d) {
        return intersect(a.x, b.x, c.x, d.x) && intersect(a.y, b.y, c.y, d.y) &&
               sign(cross(a, b, c)) * sign(cross(a, b, d)) <= 0 &&
               sign(cross(c, d, a)) * sign(cross(c, d, b)) <= 0;
    }
    bool isSS_strict(P a, P b, P c, P d) {
        return sign(cross(a, b, c)) * sign(cross(a, b, d)) < 0 &&
               sign(cross(c, d, a)) * sign(cross(c, d, b)) < 0;
    }
    bool chkLL(P a, P b, P c, P d) {
        return sign((b - a).det(d - c)) != 0;
    }
    P isLL(P a, P b, P c, P d) {
        assert(chkLL(a, b, c, d));
        return a + (b - a) * ((c - a).det(d - c) / (b - a).det(d - c));
    }
    P proj(P a, P b, P p) {
        P v = b - a;
        assert(v.abs2() > 0);
        return a + v * (v.dot(p - a) / v.abs2());
    }
    P reflect(P a, P b, P p) {
        return proj(a, b, p) * 2 - p;
    }
    db nearest(P a, P b, P p) {
        if ((b - a).abs2() == 0)
            return p.distTo(a);
        P v = b - a;
        db t = clamp(v.dot(p - a) / v.abs2(), 0.0L, 1.0L);
        return p.distTo(a + v * t);
    }
```

```
    db disSS(P a, P b, P c, P d) {
        if (isSS(a, b, c, d))
            return 0;
        return min(
            {nearest(a, b, c), nearest(a, b, d), nearest(c, d, a), nearest(c, d, b)});
    }
};

static bool cmp(P a, P b) {
    if (a.quad() != b.quad())
        return a.quad() > b.quad();
    db z = a.det(b);
    if (z != 0)
        return z > 0;
    return a.abs2() < b.abs2();
}
void run(vector<P> &a) {
    sort(a.begin(), a.end(), cmp);
}

void init_model() {
}

void solve() {
    int n;
    cin >> n;
    vector<P> a;
    while (n--) {
        P p;
        p.read();
        if (p.abs2() > 0)
            a.push_back(p);
    }
    ::init_model();
    ::run(a);
    n = a.size();
    db ans = 0;
    for (int i = 0; i < n; i++) {
        P s;
        for (int k = 0; k < n; k++) {
            s = s + a[(i + k) % n];
            ans = max(ans, s.abs2());
        }
    }
    cout << fixed << setprecision(12) << sqrtl(ans) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先用交换/增删论证找出最优集合必须满足的方向条件，再考虑排序连续段；极角排序本身并没有解决子集最大化。

验证：C++17独立程序编译通过；6组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 零向量应预先过滤；对任意浮点数叉积仍有精度风险，整数大坐标用__int128。
- 圆周相邻还需考虑最后和第一个的跨0夹角。

先用交换/增删论证找出最优集合必须满足的方向条件，再考虑排序连续段；极角排序本身并没有解决子集最大化。

个人资料来源：杂/点线.cpp

转化、构造与计数：模型与推导

转化、构造与计数 / 两互质面额不可表示数

类型与用途：思维型；两个互质正面额的非负整数线性组合，最大不可表示数ab-a-b，不可表示正整数个数(a-1)(b-1)/2。

复杂度与适用范围：算式O(1)，断言gcd检查O(log min(a,b))；O(1)空间。

例题 1 P3951 小凯的疑惑

https://www.luogu.com.cn/problem/P3951

题意：两种无限互质金币面额，求不能支付的最高价格。

观察与规模：两种无限互质金币面额，求不能支付的最高价格。 每个模a余数由0..a-1个b恰好遍历；每个余数类最高空缺在其最小可表示数之前一个a，最大值ab-a-b。 两个正面额、数量无限且互质；不是一般多面额背包问题。

朴素思路：向上枚举金额无终止依据。有限容量DP可验证小数据，但无法从经验猜出最大缺口。

关键突破：0,b,2b,...,(a-1)b遍历模a的全部余数，每个余数类的最小可表示数因此确定。

状态与建模：余数r的最小表示是kb，k∈[0,a-1]唯一；其后每隔a均可表示，之前同余的正数都不可表示。

本题映射：两种无限互质金币面额，求不能支付的最高价格。 每个模a余数由0..a-1个b恰好遍历；每个余数类最高空缺在其最小可表示数之前一个a，最大值ab-a-b。 maximum(a,b)，题目保证存在正数不可表示。

转移与正确性：各类最高缺口为kb-a，最大在k=a-1，得到ab-a-b。总缺口Σfloor(kb/a)=(a-1)(b-1)/2，因为非零余数恰为1..a-1。类间余数不同，不重不漏。

初始化与顺序：a或b为1时没有正缺口，最大公式给-1，计数0；P3951保证存在正不可表示金额。HDU到EOF独立计算。

优化与限制：O(log min(a,b))验证GCD后O(1)公式，中间乘法__int128。非互质时有无限正数不能表示，公式失效；三个以上面额也不能套用。

本题复杂度：O(log min(a,b))时间/O(1)空间（含gcd断言）。

完整函数题解

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

int maximum(int a, int b) {
    assert(a > 0 && b > 0 && std::gcd(a, b) == 1);
    __int128 z = (__int128)a * b - a - b;
    assert(z <= LLONG_MAX);
    return z;
}
```

```
int count(int a, int b) {
    assert(a > 0 && b > 0 && std::gcd(a, b) == 1);
    __int128 z = (__int128)(a - 1) * (b - 1) / 2;
    assert(z <= LLONG_MAX);
    return z;
}

void init_model() {
}

void solve() {
    int a, b;
    cin >> a >> b;
    ::init_model();
    cout << ::maximum(a, b) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：小数据规律应落实到余数类最小代表的证明。公式里除2来自完整余数求和，不需要模逆元。

验证：C++17独立程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 HDU1792 A New Change Problem

https://acm.hdu.edu.cn/showproblem.php?pid=1792

题意：两种互质正面额，求最大不可表示金额及不可表示正金额个数。

观察与规模：两种互质正面额，求最大不可表示金额及不可表示正金额个数。 最高空缺公式给最大值；每个余数类空缺求和给(a-1)(b-1)/2，补充计数应用。 两个正面额、数量无限且互质；不是一般多面额背包问题。

朴素思路：向上枚举金额无终止依据。有限容量DP可验证小数据，但无法从经验猜出最大缺口。

关键突破：0,b,2b,...,(a-1)b遍历模a的全部余数，每个余数类的最小可表示数因此确定。

状态与建模：余数r的最小表示是kb，k∈[0,a-1]唯一；其后每隔a均可表示，之前同余的正数都不可表示。

本题映射：两种互质正面额，求最大不可表示金额及不可表示正金额个数。 最高空缺公式给最大值；每个余数类空缺求和给(a-1)(b-1)/2，补充计数应用。 循环到EOF读a,b，输出maximum和count。

转移与正确性：各类最高缺口为kb-a，最大在k=a-1，得到ab-a-b。总缺口Σfloor(kb/a)=(a-1)(b-1)/2，因为非零余数恰为1..a-1。类间余数不同，不重不漏。

初始化与顺序：a或b为1时没有正缺口，最大公式给-1，计数0；P3951保证存在正不可表示金额。HDU到EOF独立计算。

优化与限制：O(log min(a,b))验证GCD后O(1)公式，中间乘法__int128。非互质时有无限正数不能表示，公式失效；三个以上面额也不能套用。

本题复杂度：O(log min(a,b))时间/O(1)空间。

完整函数题解

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

int maximum(int a, int b) {
    assert(a > 0 && b > 0 && std::gcd(a, b) == 1);
    __int128 z = (__int128)a * b - a - b;
    assert(z <= LLONG_MAX);
    return z;
}
int count(int a, int b) {
    assert(a > 0 && b > 0 && std::gcd(a, b) == 1);
    __int128 z = (__int128)(a - 1) * (b - 1) / 2;
    assert(z <= LLONG_MAX);
    return z;
}

void init_model() {
}

void solve() {
    int a, b;
    ::init_model();
    while (cin >> a >> b)
        cout << ::maximum(a, b) << ' ' << ::count(a, b) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：小数据规律应落实到余数类最小代表的证明。公式里除2来自完整余数求和，不需要模逆元。

验证：C++17独立程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 必须gcd(a,b)=1；非互质时有无限多个不可表示数，没有最大值。
- 非负组合和Bezout允许负系数的线性组合不同。
- 乘积使用__int128，返回值必须落入long long。

小数据规律应落实到余数类最小代表的证明。公式里除2来自完整余数求和，不需要模逆元。

个人资料来源：tirick/总结.md

转化、构造与计数 / 前缀关系计数

类型与用途：思维型；把满足条件的区间转为两个前缀的相同键值，按顺序统计此前相同前缀数。

复杂度与适用范围：n次add期望O(n)时间/O(n)空间；unordered_map最坏可O(n^2)。

例题 1 CF1398C Good Subarrays

https://codeforces.com/problemset/problem/1398/C

题意：数字字符串中求元素和等于长度的子区间数。

观察与规模：数字字符串中求元素和等于长度的子区间数。 s[r]-s[l]=r-l转为s[r]-r=s[l]-l，键为前缀和减前缀长度。 区间和/长度来自两个前缀差，求的是区间数量而非具体位置。

朴素思路：枚举左右端点O(n^2)；只存每种键是否出现会漏掉重复前缀形成的多个区间。

关键突破：把条件移项成两个前缀键相等，就能用历史频次一次统计所有合法左端点。

状态与建模：CF1398C键为prefixSum[i]-i；ABC105D键为prefixSum[i] mod M。cnt[key]为当前右端点之前相同键的前缀数量。

本题映射：数字字符串中求元素和等于长度的子区间数。 s[r]-s[l]=r-l转为s[r]-r=s[l]-l，键为前缀和减前缀长度。 init(n+1)，add(0)，逐位计算sum-i并add，输出ans。

转移与正确性：处理右端点r时，答案加cnt[key[r]]，然后才把当前前缀加入。每个区间只在自己的右端点计一次，不重不漏。也可离线用ΣC(cnt,2)。

初始化与顺序：必须先加入空前缀key[0]=0。多组数据清map和ans；先查询后插入，否则把空区间也算进去。

优化与限制：哈希期望O(n)，恶意冲突有退化风险，排序离线O(nlog n)可替代。答案最大n(n+1)/2用64位；负数同余需统一归一化。

本题复杂度：期望O(n)时间/O(n)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

unordered_map<int, int> cnt;
int ans = 0;
void init(int reserve_n = 0) {
    cnt.clear();
    cnt.reserve(reserve_n * 2 + 1);
    ans = 0;
}
int add(int key) {
    int old = cnt[key];
    ans += old;
    cnt[key]++;
    return old;
}

void init_model() {
}

void solve() {
    int T;
    cin >> T;
```

```
    while (T--) {
        int n;
        string s;
        cin >> n >> s;
        ::init_model();
        ::init(n + 1);
        ::add(0);
        int sum = 0;
        for (int i = 1; i <= n; i++) {
            sum += s[i - 1] - '0';
            ::add(sum - i);
        }
        cout << ::ans << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：区间条件先写成前缀差的方程，再移项寻找可比较的键。计数对象是有序位置对l<r，不能只统计值集合。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 ABC105D Candy Distribution

https://atcoder.jp/contests/abc105/tasks/abc105_d

题意：求数组区间和能被m整除的区间数。

观察与规模：求数组区间和能被m整除的区间数。 两前缀和同余，键为前缀和模m。 区间和/长度来自两个前缀差，求的是区间数量而非具体位置。

朴素思路：枚举左右端点O(n^2)；只存每种键是否出现会漏掉重复前缀形成的多个区间。

关键突破：把条件移项成两个前缀键相等，就能用历史频次一次统计所有合法左端点。

状态与建模：CF1398C键为prefixSum[i]-i；ABC105D键为prefixSum[i] mod M。cnt[key]为当前右端点之前相同键的前缀数量。

本题映射：求数组区间和能被m整除的区间数。 两前缀和同余，键为前缀和模m。 add空前缀0，逐个取模加入，输出ans。

转移与正确性：处理右端点r时，答案加cnt[key[r]]，然后才把当前前缀加入。每个区间只在自己的右端点计一次，不重不漏。也可离线用ΣC(cnt,2)。

初始化与顺序：必须先加入空前缀key[0]=0。多组数据清map和ans；先查询后插入，否则把空区间也算进去。

优化与限制：哈希期望O(n)，恶意冲突有退化风险，排序离线O(nlog n)可替代。答案最大n(n+1)/2用64位；负数同余需统一归一化。

本题复杂度：期望O(n)时间/O(min(n,m))空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

unordered_map<int, int> cnt;
int ans = 0;
void init(int reserve_n = 0) {
    cnt.clear();
    cnt.reserve(reserve_n * 2 + 1);
    ans = 0;
}
int add(int key) {
    int old = cnt[key];
    ans += old;
    cnt[key]++;
    return old;
}

void init_model() {
}

void solve() {
    int n, m;
    cin >> n >> m;
    ::init_model();
    ::init(n + 1);
    ::add(0);
    int s = 0;
    while (n--) {
        int x;
        cin >> x;
        s = (s + x) % m;
        ::add(s);
    }
    cout << ::ans << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：区间条件先写成前缀差的方程，再移项寻找可比较的键。计数对象是有序位置对l<r，不能只统计值集合。

验证：C++17独立程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 先加入空前缀，再逐点加入，才能统计从位置1开始的区间。
- 区间计数最高n(n+1)/2，需要64位。
- 使用模前缀时负余数要归一化；对恶意哈希可改map获得O(n log n)最坏界。

区间条件先写成前缀差的方程，再移项寻找可比较的键。计数对象是有序位置对l<r，不能只统计值集合。

个人资料来源：tirick/总结.md

转化、构造与计数 / 同余差与GCD

类型与用途：思维型；所有数同余模m当且仅当m整除全部差；最大可行模或步长来自差值GCD。

复杂度与适用范围：stride/from O(n log A)时间/O(1)额外空间；partitions O(n*tau(n)*log A)时间。

例题 1 CF1920C Partitioning the Array

https://codeforces.com/problemset/problem/1920/C

题意：对n的每个因数k将数组分成长度k块，存在m≥2使各块对应位置模m相同则计数。

观察与规模：对n的每个因数k将数组分成长度k块，存在m≥2使各块对应位置模m相同则计数。 对应位置的相邻块差为a[i]-a[i-k]；它们的GCD不为1正是存在合法模数。 “存在某模数使一组值同余”等价于模数整除所有两两差；移动固定步长也有同样不变量。

朴素思路：枚举所有可能模数没有可靠上界；CF1920C直接比较所有块对会重复。

关键突破：同一位置相邻块的差已经足够，其余块间差可连加；最大公共模数由差的GCD刻画。

状态与建模：CF1920C对每个n的因数k，g=gcd(|a[i]-a[i-k]|)。ABC109C求所有|city-X|的GCD。

本题映射：对n的每个因数k将数组分成长度k块，存在m≥2使各块对应位置模m相同则计数。 对应位置的相邻块差为a[i]-a[i-k]；它们的GCD不为1正是存在合法模数。 partitions(a)，枚举因数并调用stride。

转移与正确性：CF存在m≥2当且仅当g!=1：g≥2可取g，g=0表示所有差0，可取任意m≥2。移动题D必须整除全部差，取GCD既满足又最大。

初始化与顺序：GCD初始0，gcd(0,x)=x；只枚举整除n的k。城市题原输入保证有异于X的城市，正最大步长存在。

优化与限制：CF O(nτ(n)log A)，城市O(nlog A)。abs(LLONG_MIN)溢出，实际数值差需在64位内；g=0不得当无解。

本题复杂度：O(n*tau(n)*log A)时间/O(n)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int stride(const vector<int> &a, int k) {
    int g = 0;
    for (int i = k; i < (int)a.size(); i++)
        g = std::gcd(g, llabs(a[i] - a[i - k]));
    return g;
}
int partitions(const vector<int> &a) {
    int n = a.size(), ans = 0;
    for (int k = 1; k <= n; k++)
        if (n % k == 0 && stride(a, k) != 1)
            ans++;
    return ans;
}

void init_model() {
}

```

```
void solve() {
    int T;
    cin >> T;
    ::init_model();
    while (T--) {
        int n;
        cin >> n;
        vector<int> a(n);
        for (int &x : a)
            cin >> x;
        cout << ::partitions(a) << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：寻找不变量时先减掉公共基准，再问“所有差的公因数”。存在性和求最大值使用同一GCD，但0的含义要单独解释。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 ABC109C Skip

https://atcoder.jp/contests/abc109/tasks/abc109_c

题意：从X出发每次左右移动固定D，访问所有给定城市，求最大的正D。

观察与规模：从X出发每次左右移动固定D，访问所有给定城市，求最大的正D。 每个城市与X的差必须是D整数倍，最大D是这些差的GCD。 “存在某模数使一组值同余”等价于模数整除所有两两差；移动固定步长也有同样不变量。

朴素思路：枚举所有可能模数没有可靠上界；CF1920C直接比较所有块对会重复。

关键突破：同一位置相邻块的差已经足够，其余块间差可连加；最大公共模数由差的GCD刻画。

状态与建模：CF1920C对每个n的因数k，g=gcd(|a[i]-a[i-k]|)。ABC109C求所有|city-X|的GCD。

本题映射：从X出发每次左右移动固定D，访问所有给定城市，求最大的正D。 每个城市与X的差必须是D整数倍，最大D是这些差的GCD。 from(a,X)。

转移与正确性：CF存在m≥2当且仅当g!=1：g≥2可取g，g=0表示所有差0，可取任意m≥2。移动题D必须整除全部差，取GCD既满足又最大。

初始化与顺序：GCD初始0，gcd(0,x)=x；只枚举整除n的k。城市题原输入保证有异于X的城市，正最大步长存在。

优化与限制：CF O(nτ(n)log A)，城市O(nlog A)。abs(LLONG_MIN)溢出，实际数值差需在64位内；g=0不得当无解。

本题复杂度：O(n log A)时间/O(n)空间（保存输入）。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int from(const vector<int> &a, int x) {
    int g = 0;
    for (int y : a)
        g = std::gcd(g, llabs(y - x));
    return g;
}

void init_model() {
}

void solve() {
    int n, x;
    cin >> n >> x;
    vector<int> a(n);
    for (int &y : a)
        cin >> y;
    ::init_model();
    cout << ::from(a, x) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：寻找不变量时先减掉公共基准，再问“所有差的公因数”。存在性和求最大值使用同一GCD，但0的含义要单独解释。

验证：C++17独立程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- GCD=0表示所有差为0，任意正模均可，不是无解。
- CF1920C需要存在m≥2，因此g!=1，0也可行。
- llabs(a-b)先相减可能溢出，本例限定差值在long long内。

寻找不变量时先减掉公共基准，再问“所有差的公因数”。存在性和求最大值使用同一GCD，但0的含义要单独解释。

个人资料来源：tirick/总结.md

转化、构造与计数 / 异或比较与最高不同位

类型与用途：思维型；非负数满足|x-y|≤x xor y≤x+y；异或比较由最高不同位决定，低位值域可截断DP转移。

复杂度与适用范围：subsequence O(n*2^B)时间/O(n)空间；max_xor O(log r)时间/O(1)空间。

例题 1 CF1720D1 Xor-Subsequence (easy version)

https://codeforces.com/problemset/problem/1720/D1

题意：ai≤200，选最长子序列，使相邻所选下标i<j满足ai xor j < aj xor i，下标从0起。

观察与规模：ai≤200，选最长子序列，使相邻所选下标i<j满足ai xor j < aj xor i，下标从0起。 高于第7位只有下标贡献，间距≥256必然失败；仅枚举最近255个前驱。 异或的大小由最高不同位决定；D1中a[i]≤200，而下标可很大。

朴素思路：最长子序列枚举所有前驱O(n^2)，区间最大异或枚举所有点对更不可行。

关键突破：D1中高于第7位只有下标贡献；若j<i且不在同一个256块，高位会让a[j] xor i大于a[i] xor j。合法前驱必距i不到256。

状态与建模：D1 dp[i]是以0基i结尾的最长合法子序列。CF276D令k为l xor r最高置位，区间上方所有位固定，只有k及以下可能产生差异。

本题映射：ai≤200，选最长子序列，使相邻所选下标i<j满足ai xor j < aj xor i，下标从0起。 高于第7位只有下标贡献，间距≥256必然失败；仅枚举最近255个前驱。 读T、a，subsequence(a,8)。

转移与正确性：D1枚举最近255个j，验证原不等式后dp[i]=max(dp[i],dp[j]+1)。CF276D在最高不同位处分成0/1两半，选边界的互补后缀可令下k+1位全1，且不超过公共前缀界。

初始化与顺序：dp全1，i正序；l=r时异或0。生成全1答案用逐位累积而非有符号1<<63。

优化与限制：D1 O(256n)，依赖小ai；D2的值域扩大后此窗口无效。区间公式仅是任意选两数最大异或，不用于固定数组最大异或。

本题复杂度：O(256n)时间/O(n)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int subsequence(const vector<int> &a, int B = 8) {
    int n = a.size(), ans = 0, lim = 1LL << B;
    vector<int> dp(n, 1);
    for (int i = 0; i < n; i++) {
        assert(0 <= a[i] && a[i] < lim);
        for (int j = max(0LL, i - lim + 1); j < i; j++)
            if ((a[j] ^ i) < (a[i] ^ j))
                dp[i] = max(dp[i], dp[j] + 1);
        ans = max(ans, dp[i]);
    }
    return ans;
}

void init_model() {
}

```

```
void solve() {
    int T;
    cin >> T;
    ::init_model();
    while (T--) {
        int n;
        cin >> n;
        vector<int> a(n);
        for (int &x : a)
            cin >> x;
        cout << ::subsequence(a) << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：看到异或比较先定位最高不同位，再利用值域把“不可能的远处候选”排掉；不能把经验窗口当证明。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF276D Little Girl and Maximum XOR

https://codeforces.com/problemset/problem/276/D

题意：从整数区间[l,r]选两数使异或最大。

观察与规模：从整数区间[l,r]选两数使异或最大。 l,r最高不同位以下可以构造全1；最高不同位以上必须一致。 异或的大小由最高不同位决定；D1中a[i]≤200，而下标可很大。

朴素思路：最长子序列枚举所有前驱O(n^2)，区间最大异或枚举所有点对更不可行。

关键突破：D1中高于第7位只有下标贡献；若j<i且不在同一个256块，高位会让a[j] xor i大于a[i] xor j。合法前驱必距i不到256。

状态与建模：D1 dp[i]是以0基i结尾的最长合法子序列。CF276D令k为l xor r最高置位，区间上方所有位固定，只有k及以下可能产生差异。

本题映射：从整数区间[l,r]选两数使异或最大。 l,r最高不同位以下可以构造全1；最高不同位以上必须一致。 max_xor(l,r)。

转移与正确性：D1枚举最近255个j，验证原不等式后dp[i]=max(dp[i],dp[j]+1)。CF276D在最高不同位处分成0/1两半，选边界的互补后缀可令下k+1位全1，且不超过公共前缀界。

初始化与顺序：dp全1，i正序；l=r时异或0。生成全1答案用逐位累积而非有符号1<<63。

优化与限制：D1 O(256n)，依赖小ai；D2的值域扩大后此窗口无效。区间公式仅是任意选两数最大异或，不用于固定数组最大异或。

本题复杂度：O(log r)时间/O(1)空间。

完整函数题解

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

int max_xor(int l, int r) {
    assert(0 <= l && l <= r);
    int x = l ^ r, ans = 0;
    while (x) {
        ans = (ans << 1) | 1;
        x >>= 1;
    }
    return ans;
}

void init_model() {
}

void solve() {
    int l, r;
    cin >> l >> r;
    ::init_model();
    cout << ::max_xor(l, r) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：看到异或比较先定位最高不同位，再利用值域把“不可能的远处候选”排掉；不能把经验窗口当证明。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 下标差≥2^B时，高位大小已经使CF1720条件不可能成立；只能在值域小于2^B时截断。
- D2没有D1的ai≤200限制，不能照用窗口256。
- max_xor由l xor r最高位确定，l=r时答案0。

看到异或比较先定位最高不同位，再利用值域把“不可能的远处候选”排掉；不能把经验窗口当证明。

个人资料来源：tirick/总结.md

转化、构造与计数 / 置换环分解

类型与用途：思维型；排列分解为互不相交的环；任意交换最小次数n-环数。环上的局部选择可独立计数。

复杂度与适用范围：build/min_swaps/sort_swaps O(n)时间与空间；cover_ways O(n)时间与空间。

例题 1 ABC350C Sort

https://atcoder.jp/contests/abc350/tasks/abc350_c

题意：将排列用最多n-1次任意交换排好，并输出交换。

观察与规模：题目只要求≤n-1次操作，贪心同时达到最少次数；交换后的两个pos都要改，否则后续会引用旧位置。 每个值恰出现一次，因此函数图每点入度出度均1，必为若干互不相交的环。

朴素思路：盲目交换可能把两个环合并，使离目标更远；卡牌子集逐个枚举是2^n。

关键突破：一次任意交换能令环数改变±1；排序目标有n个单点环，最优必须每次拆环。

状态与建模：build得到cycle。ABC350C维护a与逆位置pos；ABC436E只需每环长度。ABC247F通过第一个排列的逆位置映射，将每张卡连接为环上的相邻覆盖关系。

本题映射：将排列用最多n-1次任意交换排好，并输出交换。 每次把位置i正确值换入，拆分置换环，总次数正好n-环数。 init，sort_swaps，输出操作列表。

转移与正确性：排序把正确值i换到位置i，不破坏更早位置；每次拆一个环，恰n-环数次。最优第一步任意同环两点都能拆环，共ΣL(L-1)/2。选卡计数等价环顶点覆盖，按第一点是否选分类得到F[L-1]+F[L+1]，自环只能选。

初始化与顺序：排序逐位置扫描并同步修改两个pos；环构建vis清0。F[0]=0,F[1]=1，正序递推。长1环单独用1，避免错误把两种边界方案重复。

优化与限制：全部O(n)/O(n)，取模计数乘积必须用64位。环顶点覆盖是卡片至少覆盖，不等价于每个数恰好覆盖；非排列输入也不能直接环分解。

本题复杂度：O(n)时间/O(n)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n;
vector<int> a, pos;
vector<vector<int>> cycle;
void init(vector<int> p) {
    a = p;
    n = a.size() - 1;
    pos.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        assert(1 <= a[i] && a[i] <= n && !pos[a[i]]);
        pos[a[i]] = i;
    }
}
void build() {
    cycle.clear();
    vector<int> vis(n + 1);
    for (int u = 1; u <= n; u++)
        if (!vis[u]) {
            vector<int> c;
            int v = u;
```

```
            while (!vis[v]) {
                vis[v] = 1;
                c.push_back(v);
                v = a[v];
            }
            cycle.push_back(c);
        }
}
int min_swaps() {
    build();
    return n - cycle.size();
}
vector<pair<int, int>> sort_swaps() {
    vector<pair<int, int>> ans;
    for (int i = 1; i <= n; i++)
        if (a[i] != i) {
            int j = pos[i];
            ans.push_back({i, j});
            swap(a[i], a[j]);
            pos[a[i]] = i;
            pos[a[j]] = j;
        }
    return ans;
}
int cover_ways(int mod = 998244353) {
    build();
    vector<int> f(n + 2);
    f[1] = 1;
    for (int i = 2; i <= n + 1; i++)
        f[i] = (f[i - 1] + f[i - 2]) % mod;
    int ans = 1;
    for (auto &c : cycle) {
        int L = c.size();
        int ways = L == 1 ? 1 : (f[L - 1] + f[L + 1]) % mod;
        ans = ans * ways % mod;
    }
    return ans;
}

void init_model() {
}

void solve() {
    int n;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    ::init_model();
    ::init(a);
    auto ans = ::sort_swaps();
    cout << ans.size() << '\n';
    for (auto [u, v] : ans)
        cout << u << ' ' << v << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
```

```
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先利用双射结构拆环，再分别证明操作次数、可行动作或环上的计数。构造与计数共享结构，但答案公式各有自己的不重不漏依据。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 ABC247F Cards

https://atcoder.jp/contests/abc247/tasks/abc247_f

题意：每张卡两面分别来自两个排列，选卡使1..n每个数至少出现在一张所选卡的任一面上，计数。

观察与规模：卡片两面都可贡献覆盖，不是只能选择一面；映射卡片为置换环后才有顶点覆盖计数。 每个值恰出现一次，因此函数图每点入度出度均1，必为若干互不相交的环。

朴素思路：盲目交换可能把两个环合并，使离目标更远；卡牌子集逐个枚举是2^n。

关键突破：一次任意交换能令环数改变±1；排序目标有n个单点环，最优必须每次拆环。

状态与建模：build得到cycle。ABC350C维护a与逆位置pos；ABC436E只需每环长度。ABC247F通过第一个排列的逆位置映射，将每张卡连接为环上的相邻覆盖关系。

本题映射：每张卡两面分别来自两个排列，选卡使1..n每个数至少出现在一张所选卡的任一面上，计数。 用排列p的逆位置把卡映射成i→pos[q[i]]的置换。每个数是相邻两卡共享的环边，选卡等价于环顶点覆盖。长度L方案F[L-1]+F[L+1]。 构造a，init，cover_ways；自环专门处理。

转移与正确性：排序把正确值i换到位置i，不破坏更早位置；每次拆一个环，恰n-环数次。最优第一步任意同环两点都能拆环，共ΣL(L-1)/2。选卡计数等价环顶点覆盖，按第一点是否选分类得到F[L-1]+F[L+1]，自环只能选。

初始化与顺序：排序逐位置扫描并同步修改两个pos；环构建vis清0。F[0]=0,F[1]=1，正序递推。长1环单独用1，避免错误把两种边界方案重复。

优化与限制：全部O(n)/O(n)，取模计数乘积必须用64位。环顶点覆盖是卡片至少覆盖，不等价于每个数恰好覆盖；非排列输入也不能直接环分解。

本题复杂度：O(n)时间/O(n)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n;
vector<int> a, pos;
vector<vector<int>> cycle;
void init(vector<int> p) {
    a = p;
    n = a.size() - 1;
    pos.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        assert(1 <= a[i] && a[i] <= n && !pos[a[i]]);
        pos[a[i]] = i;
    }
}
```

```
void build() {
    cycle.clear();
    vector<int> vis(n + 1);
    for (int u = 1; u <= n; u++)
        if (!vis[u]) {
            vector<int> c;
            int v = u;
            while (!vis[v]) {
                vis[v] = 1;
                c.push_back(v);
                v = a[v];
            }
            cycle.push_back(c);
        }
}
int min_swaps() {
    build();
    return n - cycle.size();
}
vector<pair<int, int>> sort_swaps() {
    vector<pair<int, int>> ans;
    for (int i = 1; i <= n; i++)
        if (a[i] != i) {
            int j = pos[i];
            ans.push_back({i, j});
            swap(a[i], a[j]);
            pos[a[i]] = i;
            pos[a[j]] = j;
        }
    return ans;
}
int cover_ways(int mod = 998244353) {
    build();
    vector<int> f(n + 2);
    f[1] = 1;
    for (int i = 2; i <= n + 1; i++)
        f[i] = (f[i - 1] + f[i - 2]) % mod;
    int ans = 1;
    for (auto &c : cycle) {
        int L = c.size();
        int ways = L == 1 ? 1 : (f[L - 1] + f[L + 1]) % mod;
        ans = ans * ways % mod;
    }
    return ans;
}

void init_model() {
}

void solve() {
    int n;
    cin >> n;
    vector<int> p(n + 1), pos(n + 1), a(n + 1);
    for (int i = 1; i <= n; i++) {
        cin >> p[i];
        pos[p[i]] = i;
    }
    for (int i = 1, q; i <= n; i++) {
        cin >> q;
        a[i] = pos[q];
```

```
    }
    ::init_model();
    ::init(a);
    cout << ::cover_ways() << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先利用双射结构拆环，再分别证明操作次数、可行动作或环上的计数。构造与计数共享结构，但答案公式各有自己的不重不漏依据。

验证：C++17独立程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 3 ABC436E Minimum Swap

https://atcoder.jp/contests/abc436/tasks/abc436_e

题意：计数能作为最少交换排序方案第一步的交换。

观察与规模：问最少交换排序方案的首步数；只有原来同环两点能把环数加1，跨环交换会使剩余最少次数增加。 每个值恰出现一次，因此函数图每点入度出度均1，必为若干互不相交的环。

朴素思路：盲目交换可能把两个环合并，使离目标更远；卡牌子集逐个枚举是2^n。

关键突破：一次任意交换能令环数改变±1；排序目标有n个单点环，最优必须每次拆环。

状态与建模：build得到cycle。ABC350C维护a与逆位置pos；ABC436E只需每环长度。ABC247F通过第一个排列的逆位置映射，将每张卡连接为环上的相邻覆盖关系。

本题映射：计数能作为最少交换排序方案第一步的交换。 最优步骤必须令环数加1，因此两点必须在同一个原置换环内；任意同环两点都能拆成两个环。 init、build，对每个cycle长度L累加L(L-1)/2。

转移与正确性：排序把正确值i换到位置i，不破坏更早位置；每次拆一个环，恰n-环数次。最优第一步任意同环两点都能拆环，共ΣL(L-1)/2。选卡计数等价环顶点覆盖，按第一点是否选分类得到F[L-1]+F[L+1]，自环只能选。

初始化与顺序：排序逐位置扫描并同步修改两个pos；环构建vis清0。F[0]=0,F[1]=1，正序递推。长1环单独用1，避免错误把两种边界方案重复。

优化与限制：全部O(n)/O(n)，取模计数乘积必须用64位。环顶点覆盖是卡片至少覆盖，不等价于每个数恰好覆盖；非排列输入也不能直接环分解。

本题复杂度：O(n)时间/O(n)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n;
vector<int> a, pos;
vector<vector<int>> cycle;
```

```
void init(vector<int> p) {
    a = p;
    n = a.size() - 1;
    pos.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) {
        assert(1 <= a[i] && a[i] <= n && !pos[a[i]]);
        pos[a[i]] = i;
    }
}
void build() {
    cycle.clear();
    vector<int> vis(n + 1);
    for (int u = 1; u <= n; u++)
        if (!vis[u]) {
            vector<int> c;
            int v = u;
            while (!vis[v]) {
                vis[v] = 1;
                c.push_back(v);
                v = a[v];
            }
            cycle.push_back(c);
        }
}
int min_swaps() {
    build();
    return n - cycle.size();
}
vector<pair<int, int>> sort_swaps() {
    vector<pair<int, int>> ans;
    for (int i = 1; i <= n; i++)
        if (a[i] != i) {
            int j = pos[i];
            ans.push_back({i, j});
            swap(a[i], a[j]);
            pos[a[i]] = i;
            pos[a[j]] = j;
        }
    return ans;
}
int cover_ways(int mod = 998244353) {
    build();
    vector<int> f(n + 2);
    f[1] = 1;
    for (int i = 2; i <= n + 1; i++)
        f[i] = (f[i - 1] + f[i - 2]) % mod;
    int ans = 1;
    for (auto &c : cycle) {
        int L = c.size();
        int ways = L == 1 ? 1 : (f[L - 1] + f[L + 1]) % mod;
        ans = ans * ways % mod;
    }
    return ans;
}

void init_model() {
}

```

```
void solve() {
    int n;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    ::init_model();
    ::init(a);
    ::build();
    int ans = 0;
    for (auto &c : ::cycle) {
        int L = c.size();
        ans += L * (L - 1) / 2;
    }
    cout << ans << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先利用双射结构拆环，再分别证明操作次数、可行动作或环上的计数。构造与计数共享结构，但答案公式各有自己的不重不漏依据。

验证：C++17独立程序编译通过；3组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 任意交换和相邻交换不同；相邻交换应计算逆序对。
- 一个长度L环最少L-1次交换。
- ABC247F长度1的自环只有选此卡一种；长度2允许平行边，方案数3。

先利用双射结构拆环，再分别证明操作次数、可行动作或环上的计数。构造与计数共享结构，但答案公式各有自己的不重不漏依据。

个人资料来源：tirick/总结.md

数论：模型与推导

数论 / LCM单行求和

类型与用途：思维型；求Σ(1<=i<=n)lcm(i,n)=n*(1+Σ(d|n)d*phi(d))/2。保留课堂按约数累加思路。

复杂度与适用范围：批量init O(N log N)、O(N)空间；单点calc试除O(sqrt n)、O(log n)空间。

例题 1 SP5971 LCMSUM - LCM Sum

https://www.luogu.com.cn/problem/SP5971

题意：多组n求lcm(1,n)+...+lcm(n,n)。

观察与规模：批量读询问共用筛，固定n后按gcd分类；与P1891相同公式，仅询问规模不同。 固定n求lcm(1,n)..lcm(n,n)；P1891有30万次询问，n≤10^6，预处理比逐次分解重要。

朴素思路：每问枚举i并求gcd需O(nlog n)，多询问不可行。直接引用公式不解释约数类别容易把phi参数写反。

关键突破：按g=gcd(i,n)分组，令d=n/g、i=gk，则gcd(k,d)=1且lcm(i,n)=nk。

状态与建模：每个d|n对应一组单位k∈[1,d]。d>1时单位成对k与d-k，总和dφ(d)/2；d=1只有k=1，须单独修正。

本题映射：多组n求lcm(1,n)+...+lcm(n,n)。 同一筛可预处理全部约数和，适合n<=1e6的多次查询。 读全部n，init(max n)，每次qur并print128。

转移与正确性：答案=n/2×(1+Σ_(d|n)dφ(d))。括号中的额外1正是d=1的修正，各gcd类别互不相交。筛f[j]+=dφ(d)得到所有约数和，也可用积性函数按素数幂计算。

初始化与顺序：f全0，从d=1枚举到最大询问值，再给所有倍数加贡献。先读全部询问确定上界；n=1答案1。整数先算n×(f[n]+1)再精确除2。

优化与限制：预处理O(Vlog V)，查询O(1)，空间O(V)；__int128保留中间值。SP5971/P1891是同一公式不同询问规模，作为批量策略对照，不冒充两种推导。

本题复杂度：O(N log N+T)时间、O(N+T)空间。

组合依赖：莫比乌斯反演与线性筛 的定义

题解代码（需组合上述依赖）

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 组合依赖：本册 莫比乌斯反演与线性筛 的 Mobius 定义。


vector<int> f;
void init(int n) {
    Mobius t(n);
    f.assign(n + 1, 0);
    for (int d = 1; d <= n; d++)
        for (int j = d; j <= n; j += d)
            f[j] += d * t.phi[d];
}
__int128 qur(int n) {
    assert(1 <= n && n < (int)f.size());
    return (__int128)n * (f[n] + 1) / 2;
}
```

```
__int128 calc(int n) {
    vector<pair<int, int>> fac;
    int x = n;
    for (int p = 2; p <= x / p; p++)
        if (x % p == 0) {
            int e = 0;
            while (x % p == 0)
                x /= p, e++;
            fac.push_back({p, e});
        }
    if (x > 1)
        fac.push_back({x, 1});
    __int128 s = 1;
    for (auto [p, e] : fac) {
        __int128 z = 1, pow = 1;
        for (int k = 1; k <= e; k++) {
            pow *= p;
            z += pow * (pow - pow / p);
        }
        s *= z;
    }
    return (__int128)n * (s + 1) / 2;
}

void print128(__int128 x) {
    if (!x) {
        cout << 0;
        return;
    }
    string s;
    while (x)
        s.push_back('0' + x % 10), x /= 10;
    reverse(s.begin(), s.end());
    cout << s;
}

void init_model() {
}

void solve() {
    int T;
    cin >> T;
    vector<int> a(T);
    for (int &x : a)
        cin >> x;
    ::init_model();
    ::init(*max_element(a.begin(), a.end()));
    for (int x : a) {
        print128(::qur(x));
        cout << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
```

```
    return 0;
}
```

启发式总结：固定参数的GCD/LCM和常按公共因子分类。最终除2应来自配对，d=1这种配对不成立的边界必须单列。

验证：C++17组合程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P1891 疯狂LCM

https://www.luogu.com.cn/problem/P1891

题意：最多300000次查询n≤1000000的单行LCM和。

观察与规模：30万询问使每问试除不合适；一次筛到10^6后O(1)查询是这道题额外的工程限制。 固定n求lcm(1,n)..lcm(n,n)；P1891有30万次询问，n≤10^6，预处理比逐次分解重要。

朴素思路：每问枚举i并求gcd需O(nlog n)，多询问不可行。直接引用公式不解释约数类别容易把phi参数写反。

关键突破：按g=gcd(i,n)分组，令d=n/g、i=gk，则gcd(k,d)=1且lcm(i,n)=nk。

状态与建模：每个d|n对应一组单位k∈[1,d]。d>1时单位成对k与d-k，总和dφ(d)/2；d=1只有k=1，须单独修正。

本题映射：最多300000次查询n≤1000000的单行LCM和。 与SP5971使用同一约数和预处理；本题大量查询不可逐次试除，必须一次筛到最大n。 读全部n，init(max n)，依次qur并精确输出。

转移与正确性：答案=n/2×(1+Σ_(d|n)dφ(d))。括号中的额外1正是d=1的修正，各gcd类别互不相交。筛f[j]+=dφ(d)得到所有约数和，也可用积性函数按素数幂计算。

初始化与顺序：f全0，从d=1枚举到最大询问值，再给所有倍数加贡献。先读全部询问确定上界；n=1答案1。整数先算n×(f[n]+1)再精确除2。

优化与限制：预处理O(Vlog V)，查询O(1)，空间O(V)；__int128保留中间值。SP5971/P1891是同一公式不同询问规模，作为批量策略对照，不冒充两种推导。

本题复杂度：O(N log N+T)时间、O(N+T)空间。

组合依赖：莫比乌斯反演与线性筛 的定义

题解代码（需组合上述依赖）

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 组合依赖：本册 莫比乌斯反演与线性筛 的 Mobius 定义。


vector<int> f;
void init(int n) {
    Mobius t(n);
    f.assign(n + 1, 0);
    for (int d = 1; d <= n; d++)
        for (int j = d; j <= n; j += d)
            f[j] += d * t.phi[d];
}
__int128 qur(int n) {
    assert(1 <= n && n < (int)f.size());
    return (__int128)n * (f[n] + 1) / 2;
}
```

```
__int128 calc(int n) {
    vector<pair<int, int>> fac;
    int x = n;
    for (int p = 2; p <= x / p; p++)
        if (x % p == 0) {
            int e = 0;
            while (x % p == 0)
                x /= p, e++;
            fac.push_back({p, e});
        }
    if (x > 1)
        fac.push_back({x, 1});
    __int128 s = 1;
    for (auto [p, e] : fac) {
        __int128 z = 1, pow = 1;
        for (int k = 1; k <= e; k++) {
            pow *= p;
            z += pow * (pow - pow / p);
        }
        s *= z;
    }
    return (__int128)n * (s + 1) / 2;
}

void print128(__int128 x) {
    if (!x) {
        cout << 0;
        return;
    }
    string s;
    while (x)
        s.push_back('0' + x % 10), x /= 10;
    reverse(s.begin(), s.end());
    cout << s;
}

void init_model() {
}

void solve() {
    int T;
    cin >> T;
    vector<int> a(T);
    for (int &x : a)
        cin >> x;
    ::init_model();
    ::init(*max_element(a.begin(), a.end()));
    for (int x : a) {
        print128(::qur(x));
        cout << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
```

```
    return 0;
}
```

启发式总结：固定参数的GCD/LCM和常按公共因子分类。最终除2应来自配对，d=1这种配对不成立的边界必须单列。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 整数式必须先保留精确分子再除2，不能在偶数模数下直接乘逆元。
- 返回__int128，n<=1e9时可用此范围；打印使用print128。

固定参数的GCD/LCM和常按公共因子分类。最终除2应来自配对，d=1这种配对不成立的边界必须单列。

个人资料来源：数论/数论10_课程复习.md

数论 / LCM矩形与集合求和

类型与用途：思维型；用h(t)=t*Π(p|t)(1-p)整理莫比乌斯反演；矩形中用三角数，集合中用整倍数权重和。

复杂度与适用范围：筛O(V)；矩形查询O(sqrt min(a,b))；集合O(V log V+n)；空间O(V+n)。

例题 1 P1829 Crash的数字表格

https://www.luogu.com.cn/problem/P1829

题意：求n乘m表格内全部lcm(i,j)，模20101009。

观察与规模：矩形包含全部有序对，包括i=j；不能扣对角和除2。两个floor商必须同时在分块区间内恒定。 矩形求所有有序(i,j)，数组求下标无序对；两者都含lcm=ij/gcd，但计数对象不同。

朴素思路：直接枚举全部数对O(nm)或O(N^2)；使用1/gcd的模逆既不普适又掩盖整除结构。

关键突破：把1/g改写成因数卷积：设c(t)=Σ_(k|t)mu(k)k，则Σ_(t|g)c(t)/t=1/g；于是h(t)=t c(t)为可筛积性函数。

状态与建模：矩形的t类权重为h(t)S(floor(n/t))S(floor(m/t))，S(x)=x(x+1)/2。数组s[t]=Σ_(t|a_i)a_i/t，有序对贡献h(t)s[t]^2。

本题映射：求n乘m表格内全部lcm(i,j)，模20101009。 反演后仅需Σh(t)S(n/t)S(m/t)，两个商相同的区间可合并。 init(min(n,m),20101009)，输出qur(n,m)。

转移与正确性：交换有限求和得到上述式；素数幂h(p^e)=p^e(1-p)，据此线性筛。数组先减Σa_i去掉i=j，再乘2的逆元，把两个方向合为一个i<j，不遗漏重复值不同下标。

初始化与顺序：h[1]=1，线性筛积性递推，pre求前缀。矩形按两个商都相同的区间聚合h；数组频次按倍数累加。S(x)先把偶因子除2，再取模。

优化与限制：矩形预处理O(V)，每问整除分块；数组O(Vlog V+N)。数组除2要求奇模数，矩形整数三角和不要求2可逆。乘法使用__int128，值域大仍有内存限制。

本题复杂度：O(min(n,m))预处理，O(sqrt min(n,m))查询，O(min(n,m))空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n, mod;
```

```
vector<int> h, pre, p, pr;

int norm(int x) {
    return (x % mod + mod) % mod;
}
int mul(int a, int b) {
    return (__int128)a * b % mod;
}
int S(int x) {
    int a = x, b = x + 1;
    if (a % 2 == 0)
        a /= 2;
    else
        b /= 2;
    return mul(a % mod, b % mod);
}
void init(int n_, int m) {
    n = n_;
    mod = m;
    assert(mod > 0);
    h.assign(n + 1, 0);
    pre = p = h;
    pr.clear();
    if (n)
        h[1] = 1 % mod;
    for (int i = 2; i <= n; i++) {
        if (!p[i])
            p[i] = i, pr.push_back(i), h[i] = norm(i * (1 - i));
        for (int x : pr) {
            if (x > n / i)
                break;
            p[i * x] = x;
            if (i % x == 0) {
                h[i * x] = mul(h[i], x);
                break;
            }
            h[i * x] = mul(h[i], norm(x * (1 - x)));
        }
    }
    for (int i = 1; i <= n; i++)
        pre[i] = (pre[i - 1] + h[i]) % mod;
}
int qur(int a, int b) {
    if (a > b)
        swap(a, b);
    assert(a <= n);
    int ans = 0;
    for (int l = 1, r; l <= a; l = r + 1) {
        int x = a / l, y = b / l;
        r = min(a / x, b / y);
        ans = (ans + mul(norm(pre[r] - pre[l - 1]), mul(S(x), S(y)))) % mod;
    }
    return ans;
}

void init_model(int n = 0, int mod = 20101009) {
    init(n, mod);
}

```

```
void solve() {
    int n, m;
    cin >> n >> m;
    ::init_model(min(n, m), 20101009);
    cout << ::qur(n, m) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先统一成按公共因子可分离的权重，再区别矩形累加和集合频次。最后“扣对角、除2”取决于题目要有序还是无序。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 AGC038C LCMs

https://atcoder.jp/contests/agc038/tasks/agc038_c

题意：求数组中所有i<j的lcm(ai,aj)之和，模998244353。

观察与规模：求下标i<j，重复数值仍是不同元素；有序平方统计后先扣对角Σa_i，再除2。 矩形求所有有序(i,j)，数组求下标无序对；两者都含lcm=ij/gcd，但计数对象不同。

朴素思路：直接枚举全部数对O(nm)或O(N^2)；使用1/gcd的模逆既不普适又掩盖整除结构。

关键突破：把1/g改写成因数卷积：设c(t)=Σ_(k|t)mu(k)k，则Σ_(t|g)c(t)/t=1/g；于是h(t)=t c(t)为可筛积性函数。

状态与建模：矩形的t类权重为h(t)S(floor(n/t))S(floor(m/t))，S(x)=x(x+1)/2。数组s[t]=Σ_(t|a_i)a_i/t，有序对贡献h(t)s[t]^2。

本题映射：求数组中所有i<j的lcm(ai,aj)之和，模998244353。 倍数元素的单位权重和S(d)替代矩形三角数；同一h反演后用平方统计有序对，再扣对角除2。 init(max a,998244353)，pairs(a)。

转移与正确性：交换有限求和得到上述式；素数幂h(p^e)=p^e(1-p)，据此线性筛。数组先减Σa_i去掉i=j，再乘2的逆元，把两个方向合为一个i<j，不遗漏重复值不同下标。

初始化与顺序：h[1]=1，线性筛积性递推，pre求前缀。矩形按两个商都相同的区间聚合h；数组频次按倍数累加。S(x)先把偶因子除2，再取模。

优化与限制：矩形预处理O(V)，每问整除分块；数组O(Vlog V+N)。数组除2要求奇模数，矩形整数三角和不要求2可逆。乘法使用__int128，值域大仍有内存限制。

本题复杂度：O(V log V+n)时间，O(V+n)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n, mod;
vector<int> h, pre, p, pr;
```

```

int norm(int x) {
    return (x % mod + mod) % mod;
}
int mul(int a, int b) {
    return (__int128)a * b % mod;
}

void init(int n_, int m) {
    n = n_;
    mod = m;
    assert(mod > 0);
    h.assign(n + 1, 0);
    pre = p = h;
    pr.clear();
    if (n)
        h[1] = 1 % mod;
    for (int i = 2; i <= n; i++) {
        if (!p[i])
            p[i] = i, pr.push_back(i), h[i] = norm(i * (1 - i));
        for (int x : pr) {
            if (x > n / i)
                break;
            p[i * x] = x;
            if (i % x == 0) {
                h[i * x] = mul(h[i], x);
                break;
            }
            h[i * x] = mul(h[i], norm(x * (1 - x)));
        }
    }
    for (int i = 1; i <= n; i++)
        pre[i] = (pre[i - 1] + h[i]) % mod;
}

int pairs(const vector<int> &a) {
    vector<int> s(n + 1), freq(n + 1);
    int diag = 0;
    for (int x : a) {
        assert(1 <= x && x <= n);
        freq[x]++;
        diag = (diag + x) % mod;
    }
    for (int d = 1; d <= n; d++)
        for (int x = d; x <= n; x += d)
            s[d] = (s[d] + mul(freq[x], x / d)) % mod;
    int ans = 0;
    for (int d = 1; d <= n; d++)
        ans = (ans + mul(h[d], mul(s[d], s[d]))) % mod;
    assert(mod & 1);
    return mul(norm(ans - diag), (mod + 1) / 2);
}

void init_model(int n = 0, int mod = 20101009) {
    init(n, mod);
}

void solve() {
    int n;
    cin >> n;
```

```
    vector<int> a(n);
    for (int &x : a)
        cin >> x;
    ::init_model(*max_element(a.begin(), a.end()), 998244353);
    cout << ::pairs(a) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先统一成按公共因子可分离的权重，再区别矩形累加和集合频次。最后“扣对角、除2”取决于题目要有序还是无序。

验证：C++17独立程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- h的负数需要归一化；课堂模2^60变体不能直接用有符号溢出替代。
- 三角数先整数除2再取模，支持偶数mod的矩形查询。
- pairs扣去对角再乘2的逆元，所以限定奇数mod。

先统一成按公共因子可分离的权重，再区别矩形累加和集合频次。最后“扣对角、除2”取决于题目要有序还是无序。

个人资料来源：数论/数论10_课程复习.md

数论 / 互质集合与GCD分类计数

类型与用途：思维型；统计非空下标子集的gcd；按倍数统计与反演恢复恰好gcd=d的子集权重。

复杂度与适用范围：O(V log V+n)时间、O(V+n)空间，V=max(a)。

例题 1 CF803F Coprime Subsequences

https://codeforces.com/contest/803/problem/F

题意：求gcd为1的非空子序列数。

观察与规模：空子集没有GCD且必须去掉；重复值按不同下标计，频次进入2^cnt而不是只看distinct值。 子序列由选中下标确定，重复数值仍是不同选择；值域允许枚举倍数。

朴素思路：枚举非空子集2^n；直接按GCD做DP在大值域上可能状态和转移过多。

关键突破：“GCD等于d”难直接数，“所有数被d整除”容易数，再按倍数关系反演。

状态与建模：cnt[d]是d的倍数元素个数。CF803F统计非空下标子集；CF839D的exact[d]是GCD恰为d的所有子集的长度和，不是集合个数。

本题映射：求gcd为1的非空子序列数。 选择下标后顺序唯一，因此子序列数等于下标子集数。每个d的候选有2^cnt[d]-1个。 读数组构造CoprimeSet，输出qur。

转移与正确性：CF803F答案Σmu[d](2^cnt[d]-1)，每个集合贡献Σ_(d|gcd)mu[d]，恰gcd=1时为1。CF839D总长度和为c×2^(c-1)，因为固定每个元素有2^(c-1)个包含它的子集；减所有更大倍数exact后乘d累加d>1。

初始化与顺序：频次先按值累加，cnt按倍数求和；pw[0]=1。exact从大d到小d计算，c=0贡献0，不能访问pw[-1]。模减统一归一化。

优化与限制：O(Vlog V+n)/O(V+n)，线性筛mu依赖已收录Mobius板子；无需逆元，模数条件比阶乘法宽。元素0不能按正整数倍数枚举。

本题复杂度：O(V log V+n)时间、O(V+n)空间。

组合依赖：莫比乌斯反演与线性筛 的定义

题解代码（需组合上述依赖）

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 组合依赖：本册 莫比乌斯反演与线性筛 的 Mobius 定义。


int n, mod;
vector<int> cnt, pw, exact;
Mobius sieve;

void init(const vector<int> &a, int mod_ = 1000000007) {
    mod = mod_;
    n = a.empty() ? 0 : *max_element(a.begin(), a.end());
    sieve.init(n);
    vector<int> freq(n + 1);
    for (int x : a) {
        assert(x > 0);
        freq[x]++;
    }
    cnt.assign(n + 1, 0);
    for (int d = 1; d <= n; d++)
        for (int j = d; j <= n; j += d)
            cnt[d] += freq[j];
    pw.assign(a.size() + 1, 1);
    for (int i = 1; i < (int)pw.size(); i++)
        pw[i] = pw[i - 1] * 2 % mod;
}
int qur() {
    int ans = 0;
    for (int d = 1; d <= n; d++)
        ans = (ans + sieve.mu[d] * (pw[cnt[d]] - 1)) % mod;
    return (ans + mod) % mod;
}

void init_model(const vector<int> &a, int mod_ = 1000000007) {
    init(a, mod_);
}

void solve() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a)
        cin >> x;
    ::init_model(a);
    cout << ::qur() << '\n';
}

```

```
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：计数先写清对象是下标子集，再选择“整除”这一容易统计的上集；带权计数应先推导所有上集的权重和再反演。

验证：C++17组合程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF839D Winter is here

https://codeforces.com/contest/839/problem/D

题意：累加gcd>1子序列的长度乘gcd。

观察与规模：目标含子序列长度权重；gcd=1不计入，exact[d]保存长度和，不是子序列个数。 子序列由选中下标确定，重复数值仍是不同选择；值域允许枚举倍数。

朴素思路：枚举非空子集2^n；直接按GCD做DP在大值域上可能状态和转移过多。

关键突破：“GCD等于d”难直接数，“所有数被d整除”容易数，再按倍数关系反演。

状态与建模：cnt[d]是d的倍数元素个数。CF803F统计非空下标子集；CF839D的exact[d]是GCD恰为d的所有子集的长度和，不是集合个数。

本题映射：累加gcd>1子序列的长度乘gcd。 含c个候选的所有子集长度和为c*2^(c-1)；从大到小减掉更大gcd的贡献。 构造CoprimeSet后调用weighted，精确类别存exact。

转移与正确性：CF803F答案Σmu[d](2^cnt[d]-1)，每个集合贡献Σ_(d|gcd)mu[d]，恰gcd=1时为1。CF839D总长度和为c×2^(c-1)，因为固定每个元素有2^(c-1)个包含它的子集；减所有更大倍数exact后乘d累加d>1。

初始化与顺序：频次先按值累加，cnt按倍数求和；pw[0]=1。exact从大d到小d计算，c=0贡献0，不能访问pw[-1]。模减统一归一化。

优化与限制：O(Vlog V+n)/O(V+n)，线性筛mu依赖已收录Mobius板子；无需逆元，模数条件比阶乘法宽。元素0不能按正整数倍数枚举。

本题复杂度：O(V log V+n)时间、O(V+n)空间。

组合依赖：莫比乌斯反演与线性筛 的定义

题解代码（需组合上述依赖）

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 组合依赖：本册 莫比乌斯反演与线性筛 的 Mobius 定义。


int n, mod;
vector<int> cnt, pw, exact;
Mobius sieve;

```

```
void init(const vector<int> &a, int mod_ = 1000000007) {
    mod = mod_;
    n = a.empty() ? 0 : *max_element(a.begin(), a.end());
    sieve.init(n);
    vector<int> freq(n + 1);
    for (int x : a) {
        assert(x > 0);
        freq[x]++;
    }
    cnt.assign(n + 1, 0);
    for (int d = 1; d <= n; d++)
        for (int j = d; j <= n; j += d)
            cnt[d] += freq[j];
    pw.assign(a.size() + 1, 1);
    for (int i = 1; i < (int)pw.size(); i++)
        pw[i] = pw[i - 1] * 2 % mod;
}

int weighted() {
    exact.assign(n + 1, 0);
    int ans = 0;
    for (int d = n; d >= 1; d--) {
        int c = cnt[d];
        exact[d] = c ? c * pw[c - 1] % mod : 0;
        for (int j = d + d; j <= n; j += d)
            exact[d] = (exact[d] - exact[j] + mod) % mod;
        if (d > 1)
            ans = (ans + d * exact[d]) % mod;
    }
    return ans;
}

void init_model(const vector<int> &a, int mod_ = 1000000007) {
    init(a, mod_);
}

void solve() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int &x : a)
        cin >> x;
    ::init_model(a);
    cout << ::weighted() << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：计数先写清对象是下标子集，再选择“整除”这一容易统计的上集；带权计数应先推导所有上集的权重和再反演。

验证：C++17组合程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 不同下标视为不同选择，重复值不可去重。
- 减法取模需归一化；空集必须扣掉。
- 不是两两互质，而是整个子集gcd为1。

计数先写清对象是下标子集，再选择“整除”这一容易统计的上集；带权计数应先推导所有上集的权重和再反演。

个人资料来源：数论/数论10_ACM板子.md

数论 / 卡特兰数与反射原理

类型与用途：思维型；计数栈出入、合法括号与不越界格路径；精确递推与反射法的组合数差。

复杂度与适用范围：exact O(n)时间/O(1)空间；ballot O(n+m)时间/空间。

例题 1 P1044 栈

https://www.luogu.com.cn/problem/P1044

题意：顺序入栈的n个数有多少种合法出栈序列。

观察与规模：出栈序列与合法操作路径一一对应，因为入栈顺序固定；精确答案n≤18，无需取模。 入栈/出栈或A/B排列都有“任何前缀余额不负”的限制；总数量只是必要条件。

朴素思路：直接组合数C(n+m,m)包含先出后入或B先占优的非法前缀；枚举串指数级。

关键突破：把首次跌到-1之前的上/下步交换，得到与非法路径一一对应的无约束路径。

状态与建模：P1044是n次入n次出的合法操作序列，与出栈排列一一对应。P1641是n个A上步、m个B下步的合法路径。

本题映射：顺序入栈的n个数有多少种合法出栈序列。 入栈和出栈分别是上步和下步，任意前缀不能出栈多于入栈。 调用exact(n)。

转移与正确性：总数C(n+m,m)，非法数C(n+m,m-1)，答案相减；反射的逆映射取首次到+1，证明双射。n=m得Catalan C(2n,n)/(n+1)，也可按第一个匹配对拆成C[i]C[n-1-i]递推。

初始化与顺序：n<m直接0，n=0的Catalan=1。精确递推C_i=C_(i-1)×2(2i-1)/(i+1)，每一步整除；模版用组合差，避免不合法逆元。

优化与限制：精确64位只支持n≤35，计算中间__int128。模组合需素数p且n+m<p；减法加模数归一化。边界限制改变，反射计数也会改变。

本题复杂度：O(n)时间、O(1)空间。

完整函数题解

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

int exact(int n) {
    assert(0 <= n && n <= 35);
    __int128 c = 1;
    for (int i = 1; i <= n; i++)
        c = c * 2 * (2 * i - 1) / (i + 1);
    return c;
}
```

```
void solve() {
    int n;
    cin >> n;
    cout << exact(n) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：遇到前缀不等式先转格路，找第一次违反的位置构造双射；不能只背Catalan数字而忽略总步数是否相等。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P1641 生成字符串

https://www.luogu.com.cn/problem/P1641

题意：由n个A和m个B组成字符串，每个前缀A数不少于B数。

观察与规模：n和m可不相等，不能使用仅n=m的Catalan公式；答案C(n+m,m)-C(n+m,m-1)。 入栈/出栈或A/B排列都有“任何前缀余额不负”的限制；总数量只是必要条件。

朴素思路：直接组合数C(n+m,m)包含先出后入或B先占优的非法前缀；枚举串指数级。

关键突破：把首次跌到-1之前的上/下步交换，得到与非法路径一一对应的无约束路径。

状态与建模：P1044是n次入n次出的合法操作序列，与出栈排列一一对应。P1641是n个A上步、m个B下步的合法路径。

本题映射：由n个A和m个B组成字符串，每个前缀A数不少于B数。 将A/B映射上/下步；总数减去第一次越界后反射的非法路径数。 ballot(n,m,20100403)，不等数量也可使用。

转移与正确性：总数C(n+m,m)，非法数C(n+m,m-1)，答案相减；反射的逆映射取首次到+1，证明双射。n=m得Catalan C(2n,n)/(n+1)，也可按第一个匹配对拆成C[i]C[n-1-i]递推。

初始化与顺序：n<m直接0，n=0的Catalan=1。精确递推C_i=C_(i-1)×2(2i-1)/(i+1)，每一步整除；模版用组合差，避免不合法逆元。

优化与限制：精确64位只支持n≤35，计算中间__int128。模组合需素数p且n+m<p；减法加模数归一化。边界限制改变，反射计数也会改变。

本题复杂度：O(n+m)时间与空间。

组合依赖：素数模组合数 的定义

题解代码（需组合上述依赖）

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 组合依赖：本册 素数模组合数 的 Comb 定义。


```

```
int exact(int n) {
    assert(0 <= n && n <= 35);
    __int128 c = 1;
    for (int i = 1; i <= n; i++)
        c = c * 2 * (2 * i - 1) / (i + 1);
    return c;
}
int ballot(int n, int m, int mod) {
    if (n < m)
        return 0;
    Comb t;
    t.init(n + m, mod);
    return (t.binom(n + m, m) - t.binom(n + m, m - 1) + mod) % mod;
}

void init_model() {
}

void solve() {
    int n, m;
    cin >> n >> m;
    ::init_model();
    cout << ::ballot(n, m, 20100403) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：遇到前缀不等式先转格路，找第一次违反的位置构造双射；不能只背Catalan数字而忽略总步数是否相等。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- Catalan(0)=1。35之后精确结果可能超long long。

- 组合数差公式不要求除n+1，可避免该分母模p为0的问题；本实现仍要求总长度<p。

遇到前缀不等式先转格路，找第一次违反的位置构造双射；不能只背Catalan数字而忽略总步数是否相等。

个人资料来源：卡特兰数复习资料.md

第四部分 混合与综合应用

先写出原问题的选择、费用或转移，再明确哪些部分成为图的边、区间统计或子树接口。代码复用第二部分的结构；含普通DP的解法保留函数形式。

数据结构 / DSU on Tree

类型与用途：混合型；轻子树算完清空、重子树保留，以单一统计容器回答所有子树询问。

复杂度与适用范围：O(n log n)次增删；总时间再乘增删与询问成本；O(n)树结构空间。

复用模板：DSU on Tree（本册第二部分）

例题 1 CF600E Lomsat gelral

https://codeforces.com/problemset/problem/600/E

题意：求子树众数颜色编号和。

观察与规模：求子树众数颜色编号和。 DSU保留重子树，颜色增删同时更新每个频率的颜色和与种类数。 子树询问具有可插入/删除的频率统计，重儿子可复用；CF375D还要问频率至少k。

朴素思路：每个节点清空并扫描子树O(n^2)。普通map小并大有额外log开销。

关键突破：保留重子树计数，轻子树完成答案后清理；最后统一加入轻子树和当前点。

状态与建模：cnt[color]是当前保留集合的颜色频次；sum/mx记录众数信息。ge[k]是当前出现次数≥k的颜色数，频次从f到f+1仅使ge[f+1]加1。

本题映射：求子树众数颜色编号和。 DSU保留重子树，颜色增删同时更新每个频率的颜色和与种类数。change维护cnt、freqSum、freqCount及mx；answer读freqSum[mx]。

转移与正确性：轻孩子keep=false，重孩子keep=true；add轻侧时跳过重子树，只增需要新进入的元素。读出答案后若本节点不保留，则删除整个子树。重链贡献不被反复扫描。

初始化与顺序：先dfs1求siz/heavy和Euler序，再dfs2统计；空计数重置mx/sum，删除时正确维护频次统计。CF600E输出众数编号和，CF375D取ge[k]。

优化与限制：每点经过轻边数量O(log n)，总O(nlog n)，计数O(1)时优于map。删改后的最大频率维护依赖当前实现整体清理，不可随意改为任意在线删除。

本题复杂度：O(n log n)时间、O(n+颜色值域)空间。

复用模板：DSU on Tree（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    vector<int> c(n + 1), ans(n + 1), cnt(n + 1), fc(n + 1), sum(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> c[i];
    DSUOnTree t(n);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        t.add_edge(u, v);
    }
    int mx = 0;
```

```
    auto change = [&](int u, int d) {
        int x = c[u], k = cnt[x];
        if (k)
            fc[k]--, sum[k] -= x;
        k = cnt[x] += d;
        if (k)
            fc[k]++, sum[k] += x;
        mx = max(mx, k);
        while (mx && !fc[mx])
            mx--;
    };
    auto answer = [&](int u) { ans[u] = sum[mx]; };
    t.run(change, answer);
    for (int i = 1; i <= n; i++)
        cout << ans[i] << " \n"[i == n];
}
```

启发式总结：先找可增删的答案统计，再应用重轻复用；遍历顺序是正确性的一部分，不是无条件替换树形DP。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF375D Tree and Queries

https://codeforces.com/contest/375/problem/D

题意：求子树内至少出现k次的颜色种数。

观察与规模：求子树内至少出现k次的颜色种数。 单次颜色频率从x变x+1，只使“至少x+1次”计数增加，删除逆操作；无需树状数组。 子树询问具有可插入/删除的频率统计，重儿子可复用；CF375D还要问频率至少k。

朴素思路：每个节点清空并扫描子树O(n^2)。普通map小并大有额外log开销。

关键突破：保留重子树计数，轻子树完成答案后清理；最后统一加入轻子树和当前点。

状态与建模：cnt[color]是当前保留集合的颜色频次；sum/mx记录众数信息。ge[k]是当前出现次数≥k的颜色数，频次从f到f+1仅使ge[f+1]加1。

本题映射：求子树内至少出现k次的颜色种数。 单次颜色频率从x变x+1，只使“至少x+1次”计数增加，删除逆操作；无需树状数组。 change对atleast新旧边界加减；answer回答挂在该点的全部询问。

转移与正确性：轻孩子keep=false，重孩子keep=true；add轻侧时跳过重子树，只增需要新进入的元素。读出答案后若本节点不保留，则删除整个子树。重链贡献不被反复扫描。

初始化与顺序：先dfs1求siz/heavy和Euler序，再dfs2统计；空计数重置mx/sum，删除时正确维护频次统计。CF600E输出众数编号和，CF375D取ge[k]。

优化与限制：每点经过轻边数量O(log n)，总O(nlog n)，计数O(1)时优于map。删改后的最大频率维护依赖当前实现整体清理，不可随意改为任意在线删除。

本题复杂度：O(n log n+m)时间、O(n+m+颜色值域)空间。

复用模板：DSU on Tree（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    DSUOnTree t(n);
    vector<int> c(n + 1), cnt(100001), atleast(n + 1), ans(m);
```

```
    for (int i = 1; i <= n; i++)
        cin >> c[i];
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        t.add_edge(u, v);
    }
    vector<vector<pair<int, int>>> q(n + 1);
    for (int i = 0; i < m; i++) {
        int u, k;
        cin >> u >> k;
        q[u].push_back({k, i});
    }
    auto change = [&](int u, int d) {
        int &x = cnt[c[u]];
        if (d == 1)
            atleast[++x]++;
        else
            atleast[x--]--;
    };
    auto answer = [&](int u) {
        for (auto [k, i] : q[u])
            ans[i] = k <= n ? atleast[k] : 0;
    };
    t.run(change, answer);
    for (int x : ans)
        cout << x << '\n';
}
```

启发式总结：先找可增删的答案统计，再应用重轻复用；遍历顺序是正确性的一部分，不是无条件替换树形DP。

验证：C++17组合程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 移除必须与加入完全互逆；原“直接清零”仅限清空整个容器的特定统计，不适合通用回调。
- run最终清空统计；多次运行须初始化外部答案和桶。
- 递归深度O(n)，链树可能需要较大栈。

先找可增删的答案统计，再应用重轻复用；遍历顺序是正确性的一部分，不是无条件替换树形DP。

个人资料来源：杂/dsu on tree.cpp；数据结构/模块二_DSU_on_Tree与树上统计.md

数据结构 / 启发式合并

类型与用途：混合型；每次枚举小容器合入大容器；保留颜色桶句柄交换与map统计两个原资料模型。

复杂度与适用范围：位置桶总O(n log n+q)，map树上合并O(n log^2n)；空间O(n+颜色数)。

复用模板：启发式合并（本册第二部分）

例题 1 P3201 梦幻布丁

https://www.luogu.com.cn/problem/P3201

题意：把一种颜色的全部位置改色，并查询颜色段数。

观察与规模：把一种颜色的全部位置改色，并查询颜色段数。 改色等价合并位置集合，段数只在搬运位置两侧变化。 每次整色重命名或合并孩子的统计容器，搬运代价与容器大小相关。

朴素思路：始终把x枚举到y可能反复搬同一大集合；每点重新统计整棵子树O(n^2)。

关键突破：物理上小容器并入大容器，每个被搬元素所在集合至少翻倍，因此最多搬O(log n)次。

状态与建模：P3201逻辑颜色id指向物理桶，a[pos]存物理桶号；CF600E每点容器存颜色频次及最高频率对应编号和。

本题映射：把一种颜色的全部位置改色，并查询颜色段数。 改色等价合并位置集合，段数只在搬运位置两侧变化。 离线读取操作以压缩所有颜色；build、change，查询segments。

转移与正确性：重命名x到y必须保留逻辑语义：先必要时交换id[x],id[y]，再搬物理小桶并重算每个搬运点相邻边的段数。树上先合并孩子map，再记录当前根sum。

初始化与顺序：build先建立位置桶和原段数；map加入当前点后递归合并孩子。释放小桶或清map，所有答案先记录再由父亲破坏合并。

优化与限制：位置桶总搬运O(nlog n)，map每次操作还需log因子，为O(nlog^2 n)。交换桶句柄不等于交换x->y操作方向；否则外部颜色指令失真。

本题复杂度：O((n+q)log(n+q)+n log n)时间，O(n+q)空间。

复用模板：启发式合并（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<int> a(n + 1), all;
    for (int i = 1; i <= n; i++)
        cin >> a[i], all.push_back(a[i]);
    struct op {
        int t, x, y;
    };
    vector<op> q(m);
    for (auto &z : q) {
        cin >> z.t;
        if (z.t == 1) {
            cin >> z.x >> z.y;
            all.push_back(z.x);
            all.push_back(z.y);
        }
    }
    sort(all.begin(), all.end());
    all.erase(unique(all.begin(), all.end()), all.end());
    auto id = [&](int x) {
        return lower_bound(all.begin(), all.end(), x) - all.begin() + 1;
    };
    for (int i = 1; i <= n; i++)
        a[i] = id(a[i]);
    SmallMerge t(n);
    t.build(a, all.size());
    for (auto z : q)
        if (z.t == 1)
            t.change(id(z.x), id(z.y));
        else
            cout << t.segments << '\n';
}
```

启发式总结：证明小并大要追踪具体元素所处集合如何翻倍；带有外部名字时把逻辑名字与存储位置分离。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF600E Lomsat gelral

https://codeforces.com/problemset/problem/600/E

题意：对每棵子树求出现最多的颜色的编号和。

观察与规模：对每棵子树求出现最多的颜色的编号和。 子树统计是孩子容器的并集；小map并入大map并维护频率最大值。 每次整色重命名或合并孩子的统计容器，搬运代价与容器大小相关。

朴素思路：始终把x枚举到y可能反复搬同一大集合；每点重新统计整棵子树O(n^2)。

关键突破：物理上小容器并入大容器，每个被搬元素所在集合至少翻倍，因此最多搬O(log n)次。

状态与建模：P3201逻辑颜色id指向物理桶，a[pos]存物理桶号；CF600E每点容器存颜色频次及最高频率对应编号和。

本题映射：对每棵子树求出现最多的颜色的编号和。 子树统计是孩子容器的并集；小map并入大map并维护频率最大值。 每点insert自身颜色，DFS后merge孩子；合并后记录sum。

转移与正确性：重命名x到y必须保留逻辑语义：先必要时交换id[x],id[y]，再搬物理小桶并重算每个搬运点相邻边的段数。树上先合并孩子map，再记录当前根sum。

初始化与顺序：build先建立位置桶和原段数；map加入当前点后递归合并孩子。释放小桶或清map，所有答案先记录再由父亲破坏合并。

优化与限制：位置桶总搬运O(nlog n)，map每次操作还需log因子，为O(nlog^2 n)。交换桶句柄不等于交换x->y操作方向；否则外部颜色指令失真。

本题复杂度：O(n log^2 n)时间、O(n)活动数据与递归空间。

复用模板：启发式合并（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    vector<int> c(n + 1), ans(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> c[i];
    vector<vector<int>> g(n + 1);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    vector<SmallMerge::info> f(n + 1);
    function<void(int, int)> dfs = [&](int u, int p) {
        SmallMerge::insert(f[u], c[u], 1);
        for (int v : g[u])
            if (v != p) {
                dfs(v, u);
                SmallMerge::merge(f[u], f[v]);
            }
        ans[u] = f[u].sum;
    };
    dfs(1, 0);
    for (int i = 1; i <= n; i++)
        cout << ans[i] << " \n"[i == n];
}
```

启发式总结：证明小并大要追踪具体元素所处集合如何翻倍；带有外部名字时把逻辑名字与存储位置分离。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 交换的是逻辑颜色句柄，不能简单颠倒x->y语义。
- map单次操作还有log因子；复杂度不能仅写O(n log n)。
- 数组边界使用显式判断，不依赖颜色0哨兵。

证明小并大要追踪具体元素所处集合如何翻倍；带有外部名字时把逻辑名字与存储位置分离。

个人资料来源：数据结构/模块一_启发式合并与启发式分治.md

数据结构 / 离线二维数点

类型与用途：混合型；按x扫描，树状数组维护y；四次前缀容斥回答闭矩形计数，允许重复点。

复杂度与适用范围：O((n+q)log(n+q))时间、O(n+q)空间。

复用模板：离线二维数点（本册第二部分）

例题 1 P2163 园丁的烦恼

https://www.luogu.com.cn/problem/P2163

题意：查询二维平面闭矩形内树木棵数。

观察与规模：查询二维平面闭矩形内树木棵数。 静态点与离线矩形查询正好对应二维前缀容斥。 矩形计数和区间不同数都可表示二维点的支配计数，所有询问已知。

朴素思路：每问扫描所有点O(nq)；二维动态结构比题目所需更复杂。

关键突破：矩形用四次前缀包含-排除；按x排序消除一维，BIT维护另一维累计频次。

状态与建模：P2163点为(x,y)，事件为前缀矩形(X,Y)。P1972对位置i取点(i,pre[i])，区间[l,r]中一个值仅其首次出现位置满足pre[i]<l。

本题映射：查询二维平面闭矩形内树木棵数。 静态点与离线矩形查询正好对应二维前缀容斥。 逐点add，逐询问ask，run后输出每个答案。

转移与正确性：矩形答案F(x2,y2)-F(x1-1,y2)-F(x2,y1-1)+F(x1-1,y1-1)。扫描事件X之前加入x≤X点，查询BIT的y≤Y前缀。不同数为F(r,l-1)-F(l-1,l-1)。

初始化与顺序：y离散化保留顺序，加入点用lower_bound，查询边界用upper_bound数出≤Y的坐标。空前缀自然为0，同x必须先纳入点再答包含边界事件。

优化与限制：O((n+q)log(n+q))，空间O(n+q)。不是在线修改模型；坐标减1必须在原题范围内，闭区间条件不可混成<。

本题复杂度：O((n+m)log(n+m))时间与O(n+m)空间。

复用模板：离线二维数点（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    RectangleCounter t;
    while (n--) {
        int x, y;
        cin >> x >> y;
        t.add(x, y);
    }
```

```
    while (m--) {
        int a, b, c, d;
        cin >> a >> b >> c >> d;
        t.ask(a, b, c, d);
    }
    for (int x : t.run())
        cout << x << '\n';
}
```

启发式总结：二维限制先尝试排序其中一维，再用一维前缀结构；重复元素计数的突破是给每个出现指定唯一前驱。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P1972 HH的项链

https://www.luogu.com.cn/problem/P1972

题意：询问区间内不同颜色数。

观察与规模：询问区间内不同颜色数。 颜色只统计区间内第一次出现的位置；位置i贡献当且仅当pre[i]<l且l<=i<=r。映射为点(pre[i],i)。 矩形计数和区间不同数都可表示二维点的支配计数，所有询问已知。

朴素思路：每问扫描所有点O(nq)；二维动态结构比题目所需更复杂。

关键突破：矩形用四次前缀包含-排除；按x排序消除一维，BIT维护另一维累计频次。

状态与建模：P2163点为(x,y)，事件为前缀矩形(X,Y)。P1972对位置i取点(i,pre[i])，区间[l,r]中一个值仅其首次出现位置满足pre[i]<l。

本题映射：询问区间内不同颜色数。 颜色只统计区间内第一次出现的位置；位置i贡献当且仅当pre[i]<l且l<=i<=r。映射为点(pre[i],i)。 读数组时用map保存上次位置并add；ask(0,l,l-1,r)。

转移与正确性：矩形答案F(x2,y2)-F(x1-1,y2)-F(x2,y1-1)+F(x1-1,y1-1)。扫描事件X之前加入x≤X点，查询BIT的y≤Y前缀。不同数为F(r,l-1)-F(l-1,l-1)。

初始化与顺序：y离散化保留顺序，加入点用lower_bound，查询边界用upper_bound数出≤Y的坐标。空前缀自然为0，同x必须先纳入点再答包含边界事件。

优化与限制：O((n+q)log(n+q))，空间O(n+q)。不是在线修改模型；坐标减1必须在原题范围内，闭区间条件不可混成<。

本题复杂度：O((n+m)log(n+m))时间与O(n+m)空间。

复用模板：离线二维数点（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    RectangleCounter t;
    map<int, int> pre;
    for (int i = 1; i <= n; i++) {
        int x;
        cin >> x;
        t.add(pre[x], i);
        pre[x] = i;
    }
    int m;
    cin >> m;
```

```
    while (m--) {
        int l, r;
        cin >> l >> r;
        t.ask(0, l, l - 1, r);
    }
    for (int x : t.run())
        cout << x << '\n';
}
```

启发式总结：二维限制先尝试排序其中一维，再用一维前缀结构；重复元素计数的突破是给每个出现指定唯一前驱。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 严格小于端点的事件在等坐标点加入前处理；无需端点减1，支持LLONG_MIN。
- y仅离散化点坐标，空点集也能运行。
- 原笔记部分标点损坏，整理版补全了可编译事件类型。

二维限制先尝试排序其中一维，再用一维前缀结构；重复元素计数的突破是给每个出现指定唯一前驱。

个人资料来源：数据结构/扫描线/二维数点.md

数据结构 / 线段树优化建图

类型与用途：混合型；点到区间、区间到点建有向边；两棵树方向相反，原点共享叶子。

复杂度与适用范围：建图O(n+q log n)，非负最短路O((n+q log n)log(n+q))，空间O(n+q log n)。

复用模板：线段树优化建图（本册第二部分）

例题 1 CF786B Legacy

https://codeforces.com/problemset/problem/786/B

题意：三类点/区间边，求源点到所有原点的距离。

观察与规模：三类点/区间边，求源点到所有原点的距离。 每个区间拆成O(log n)个树节点，保持原图可达与代价。 单点到区间/区间到单点的边大量出现，朴素真实边数可达nq。

朴素思路：把每条区间边展开成所有端点后，最短路时间和内存随nq增长。

关键突破：区间只需拆成O(log n)个线段树节点，节点之间用0费边表达包含关系。

状态与建模：建立入树和出树：入树孩子到父亲，出树父亲到孩子；叶子与真实点连0边。区间到点从入树节点出边，点到区间向出树节点入边。

本题映射：三类点/区间边，求源点到所有原点的距离。 每个区间拆成O(log n)个树节点，保持原图可达与代价。 按op1/2/3调用add_edge/add_out/add_in，dijkstra只输出原点。

转移与正确性：区间边费用只在实际查询边付一次；其他0边选择区间中的真实端点。两棵树方向相反防止普通节点通过同一树无条件跳到别点。

初始化与顺序：先init/build建立节点编号；读操作后add单边或区间边；最后以真实起点跑Dijkstra，未到达输出-1。所有新增边要求非负权。

优化与限制：O(n+qlog n)节点/边规模，Dijkstra再乘堆log。P6348区间到区间用中间点桥接两方向，避免两区间节点笛卡尔积；不要在两段桥接边上重复付1。

本题复杂度：O((n+q log n)log n)时间、O(n+q log n)空间。

复用模板：线段树优化建图（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, q, s;
    cin >> n >> q >> s;
    SegGraph sg(n);
    while (q--) {
        int op, v, l, r, w;
        cin >> op;
        if (op == 1) {
            cin >> v >> l >> w;
            sg.add_edge(v, l, w);
        } else {
            cin >> v >> l >> r >> w;
            if (op == 2)
                sg.add_out(v, l, r, w);
            else
                sg.add_in(l, r, v, w);
        }
    }
    auto d = sg.dijkstra(s);
    for (int i = 1; i <= n; i++)
        cout << (d[i] == INF ? -1 : d[i]) << (i == n ? '\n' : ' ');
}
```

启发式总结：先把大批量的同操作对象压成层次节点，再检验方向与费用是否一一对应。数据结构在这里压缩图的表达，没有改变最短路模型。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P6348 Journeys

https://www.luogu.com.cn/problem/P6348

题意：区间间建立双向通信线路，求起点到各站的最少线路数。

观察与规模：区间间建立双向通信线路，求起点到各站的最少线路数。 区间到辅助点0，辅助点到另一段1；反向另建辅助点。 单点到区间/区间到单点的边大量出现，朴素真实边数可达nq。

朴素思路：把每条区间边展开成所有端点后，最短路时间和内存随nq增长。

关键突破：区间只需拆成O(log n)个线段树节点，节点之间用0费边表达包含关系。

状态与建模：建立入树和出树：入树孩子到父亲，出树父亲到孩子；叶子与真实点连0边。区间到点从入树节点出边，点到区间向出树节点入边。

本题映射：区间间建立双向通信线路，求起点到各站的最少线路数。 区间到辅助点0，辅助点到另一段1；反向另建辅助点。 每条线路新增两个点，add_in与add_out组合；Dijkstra求代价。

转移与正确性：区间边费用只在实际查询边付一次；其他0边选择区间中的真实端点。两棵树方向相反防止普通节点通过同一树无条件跳到别点。

初始化与顺序：先init/build建立节点编号；读操作后add单边或区间边；最后以真实起点跑Dijkstra，未到达输出-1。所有新增边要求非负权。

优化与限制：O(n+qlog n)节点/边规模，Dijkstra再乘堆log。P6348区间到区间用中间点桥接两方向，避免两区间节点笛卡尔积；不要在两段桥接边上重复付1。

本题复杂度：O((n+m log n)log(n+m))时间、O(n+m log n)空间。

复用模板：线段树优化建图（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m, p;
    cin >> n >> m >> p;
    SegGraph sg(n);
    while (m--) {
        int a, b, c, d;
        cin >> a >> b >> c >> d;
        int x = ++sg.tot;
        sg.add_in(a, b, x, 0);
        sg.add_out(x, c, d, 1);
        int y = ++sg.tot;
        sg.add_in(c, d, y, 0);
        sg.add_out(y, a, b, 1);
    }
    auto dis = sg.dijkstra(p);
    for (int i = 1; i <= n; i++)
        cout << (dis[i] == INF ? -1 : dis[i]) << '\n';
}
```

启发式总结：先把大批量的同操作对象压成层次节点，再检验方向与费用是否一一对应。数据结构在这里压缩图的表达，没有改变最短路模型。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 不能把in/out两棵树方向反写。
- INF需大于可达最短路，d+w不溢出。
- 辅助点需要先++tot，add_edge动态扩容。

先把大批量的同操作对象压成层次节点，再检验方向与费用是否一一对应。数据结构在这里压缩图的表达，没有改变最短路模型。

个人资料来源：数据结构/线段树/线段树优化建图.cpp

图论 / Kruskal重构树

类型与用途：混合型；按权值排序建立连通块合并历史；支持不连通森林与LCA瓶颈查询。

复杂度与适用范围：排序O(m log m)，仅路径压缩的并查集按O((n+m)log n)保守估计，倍增O(n log n)；空间O(n log n+m)。

复用模板：Kruskal重构树（本册第二部分）

例题 1 P2245 星际导航

https://www.luogu.com.cn/problem/P2245

题意：求两点间路径最大边尽可能小的值。

观察与规模：求两点间路径最大边尽可能小的值。 升序合并首次连通的权值就是所需最大边阈值。 询问路径上的最大/最小边界，普通最短距离与瓶颈值不是同一目标。

朴素思路：每次跑瓶颈搜索O(qm)，或者沿生成树逐边找极值，询问多时不可行。

关键突破：按阈值加边的连通分量只会合并；把每次合并记为一个新父节点，形成阈值历史树。

状态与建模：原顶点作叶子，新点值是当前合并边权，两个分量根接为孩子。相同原分量无需新建；不连通得到森林。

本题映射：求两点间路径最大边尽可能小的值。 升序合并首次连通的权值就是所需最大边阈值。build(false)，qur；不连通按题面输出impossible。

转移与正确性：升序Kruskal时两点首次连通阈值即最小可能的路径最大边；降序时为最大可能的路径最小边。首次同属一个分量的新点正是重构树LCA。

初始化与顺序：DSU初始化原点，边排序后merge创建新点，再对每棵根树build倍增。查询先判原图是否连通，同点按题目约定处理。

优化与限制：建树O(m log m+(n+m)log n)保守上界，查询O(log n)；原实现仅压缩路径，不声称凭空获得秩合并复杂度。重构树可深O(n)，递归栈风险明确保留。

本题复杂度：O(m log m+n log n+q log n)时间、O(m+n log n)空间。

复用模板：Kruskal重构树（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    KruskalTree tr;
    tr.init(n);
    while (m--) {
        int u, v, w;
        cin >> u >> v >> w;
        tr.addedge(u, v, w);
    }
    tr.build(false);
    int q;
    cin >> q;
    while (q--) {
        int u, v;
        cin >> u >> v;
        int p = tr.lca(u, v);
        if (!p)
            cout << "impossible\n";
        else
            cout << tr.val[p] << '\n';
    }
}
```

启发式总结：离线阈值连通性可以变为可查询的合并历史；先确定是最小化最大边还是最大化最小边，再决定排序方向。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P1967 货车运输

https://www.luogu.com.cn/problem/P1967

题意：求路径上最小载重尽可能大的容量。

观察与规模：求路径上最小载重尽可能大的容量。 降序合并首次连通阈值是最大最小边。 询问路径上的最大/最小边界，普通最短距离与瓶颈值不是同一目标。

朴素思路：每次跑瓶颈搜索O(qm)，或者沿生成树逐边找极值，询问多时不可行。

关键突破：按阈值加边的连通分量只会合并；把每次合并记为一个新父节点，形成阈值历史树。

状态与建模：原顶点作叶子，新点值是当前合并边权，两个分量根接为孩子。相同原分量无需新建；不连通得到森林。

本题映射：求路径上最小载重尽可能大的容量。 降序合并首次连通阈值是最大最小边。 build(true)，qur，不连通-1。

转移与正确性：升序Kruskal时两点首次连通阈值即最小可能的路径最大边；降序时为最大可能的路径最小边。首次同属一个分量的新点正是重构树LCA。

初始化与顺序：DSU初始化原点，边排序后merge创建新点，再对每棵根树build倍增。查询先判原图是否连通，同点按题目约定处理。

优化与限制：建树O(m log m+(n+m)log n)保守上界，查询O(log n)；原实现仅压缩路径，不声称凭空获得秩合并复杂度。重构树可深O(n)，递归栈风险明确保留。

本题复杂度：O(m log m+n log n+q log n)时间、O(m+n log n)空间。

复用模板：Kruskal重构树（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    KruskalTree tr;
    tr.init(n);
    while (m--) {
        int u, v, w;
        cin >> u >> v >> w;
        tr.addedge(u, v, w);
    }
    tr.build(true);
    int q;
    cin >> q;
    while (q--) {
        int u, v;
        cin >> u >> v;
        cout << tr.qur(u, v) << '\n';
    }
}
```

启发式总结：离线阈值连通性可以变为可查询的合并历史；先确定是最小化最大边还是最大化最小边，再决定排序方向。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 森林必须逐根预处理，不能只dfs(tot)。
- 同点的空路径瓶颈由题目约定；本接口返回0。
- 归并代表形成长链，递归find/dfs有栈风险；每次新建图先init。

离线阈值连通性可以变为可查询的合并历史；先确定是最小化最大边还是最大化最小边，再决定排序方向。

个人资料来源：图论/Kruskal重构树.cpp；图论/Kruskal重构树板子 (1).md

图论 / 基环树最大权独立集

类型与用途：混合型；每个函数图分量一个环；外挂树选/不选DP后，环枚举首点选择并检查首尾冲突。

复杂度与适用范围：O(n)时间与空间。

例题 1 P2607 骑士

https://www.luogu.com.cn/problem/P2607

题意：每位骑士有收益和一个不能同时选的对象，最大总收益。

观察与规模：每位骑士有收益和一个不能同时选的对象，最大总收益。 每点恰好一条指向冲突对象的边，弱分量是基环树。 每点指定一个冲突关系，底层每个分量至多一环；独立集不能选相邻点。

朴素思路：枚举点子集2^n；直接树DP遇到环会反复访问并违反首尾约束。

关键突破：先剥离附着树，得到环，再固定环首点是否选，使环上选择变成链DP。

状态与建模：树f0/f1分别是不选/选根的最大权；环每个点用附着树的两个代价作局部权，链状态记录上一个环点是否选。

本题映射：每位骑士有收益和一个不能同时选的对象，最大总收益。 每点恰好一条指向冲突对象的边，弱分量是基环树。 输入收益val及后继p，run。

转移与正确性：树f0=Σmax(child0,child1)，f1=a+Σchild0。环转移未选取两态最大，选只接前一个未选；固定首态后最后检查首尾不能都选。各连通分量答案相加。

初始化与顺序：树叶f0=0、f1=a；链不可达-INF。边id用于区分反向边，不能仅用父顶点跳边；自环禁止选其唯一点，平行双边环也须识别。

优化与限制：O(n)/O(n)，负权允许全不选，题意若要求非空需另设状态。FunctionalGraph分解模块与独立集转移分开阅读；旧继承版本保留，函数版不用继承。

本题复杂度：O(n)时间/空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n;
vector<int> p, w, val;
vector<int32_t> head, nxt;
vector<char> vis, on;
vector<vector<int>> cycles;

void init(int n_) {
    n = n_;
    p.assign(n + 1, 0);
    w = val = p;
    head.assign(n + 1, -1);
    nxt = head;
    vis.assign(n + 1, 0);
    on = vis;
    cycles.clear();
}
void build() {
    fill(head.begin(), head.end(), -1);
    fill(vis.begin(), vis.end(), 0);
    fill(on.begin(), on.end(), 0);
```

```
    cycles.clear();
    for (int u = 1; u <= n; u++) {
        assert(1 <= p[u] && p[u] <= n);
        nxt[u] = head[p[u]];
        head[p[u]] = u;
    }
    for (int i = 1; i <= n; i++)
        if (!vis[i]) {
            int u = i;
            while (!vis[u])
                vis[u] = 1, u = p[u];
            if (vis[u] == 1) {
                vector<int> c;
                int v = u;
                do {
                    c.push_back(v);
                    on[v] = 1;
                    v = p[v];
                } while (v != u);
                cycles.push_back(c);
            }
            u = i;
            while (vis[u] == 1)
                vis[u] = 2, u = p[u];
        }
}
void orient(const vector<array<int, 3>> &edges) {
    vector<vector<array<int, 3>>> g(n + 1);
    vector<int> deg(n + 1);
    int eid = 0;
    for (auto [u, v, c] : edges) {
        g[u].push_back({v, c, eid});
        g[v].push_back({u, c, eid++});
        deg[u]++;
        deg[v]++;
    }
    queue<int> q;
    for (int u = 1; u <= n; u++)
        if (deg[u] == 1)
            q.push(u);
    vector<char> removed(n + 1);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        removed[u] = 1;
        for (auto [v, c, id] : g[u])
            if (!removed[v]) {
                p[u] = v;
                w[u] = c;
                if (--deg[v] == 1)
                    q.push(v);
                break;
            }
    }
    vector<char> used(n + 1);
    for (int i = 1; i <= n; i++)
```

```
        if (!removed[i] && !used[i]) {
            int u = i, prev = -1;
            do {
                used[u] = 1;
                array<int, 3> z = {0, 0, -1};
                for (auto [v, c, id] : g[u])
                    if (!removed[v] && id != prev) {
                        z = {v, c, id};
                        break;
                    }
                assert(z[0]);
                p[u] = z[0];
                w[u] = z[1];
                prev = z[2];
                u = z[0];
            } while (u != i);
        }
    build();
}

static constexpr int INF = 4000000000000000000LL;
vector<array<int, 2>> dp;

void dfs(int u) {
    dp[u] = {0, val[u]};
    for (int v = head[u]; v != -1; v = nxt[v])
        if (!on[v]) {
            dfs(v);
            dp[u][0] += max(dp[v][0], dp[v][1]);
            dp[u][1] += dp[v][0];
        }
}
int run() {
    build();
    dp.assign(n + 1, {0, 0});
    int ans = 0;
    for (auto c : cycles) {
        for (int u : c)
            dfs(u);
        if (c.size() == 1) {
            ans += dp[c[0]][0];
            continue;
        }
        int best = -INF;
        for (int first = 0; first < 2; first++) {
            int f = first ? -INF : dp[c[0]][0], g = first ? dp[c[0]][1] : -INF;
            for (int i = 1; i < (int)c.size(); i++) {
                int u = c[i];
                int nf = max(f, g) + dp[u][0], ng = f == -INF ? -INF : f + dp[u][1];
                f = nf;
                g = ng;
            }
            best = max(best, first ? f : max(f, g));
        }
        ans += best;
    }
    return ans;
}

```

```
void init_model(int n = 0) {
    init(n);
}

void solve() {
    int n;
    cin >> n;
    ::init_model(n);
    for (int u = 1; u <= n; u++)
        cin >> ::val[u] >> ::p[u];
    cout << ::run() << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：环把局部树接口闭合成首尾约束；只要接口状态很小，枚举首态即可消掉环的耦合。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P1453 城市环路

https://www.luogu.com.cn/problem/P1453

题意：城市构成无向基环图，不能选相邻城市建店，求利润。

观察与规模：城市构成无向基环图，不能选相邻城市建店，求利润。 无向唯一环可定向为函数图，之后独立集模型与骑士相同。 每点指定一个冲突关系，底层每个分量至多一环；独立集不能选相邻点。

朴素思路：枚举点子集2^n；直接树DP遇到环会反复访问并违反首尾约束。

关键突破：先剥离附着树，得到环，再固定环首点是否选，使环上选择变成链DP。

状态与建模：树f0/f1分别是不选/选根的最大权；环每个点用附着树的两个代价作局部权，链状态记录上一个环点是否选。

本题映射：城市构成无向基环图，不能选相邻城市建店，求利润。 无向唯一环可定向为函数图，之后独立集模型与骑士相同。 0基城市转1基，orient(edges)，run后乘系数k，保留1位小数。

转移与正确性：树f0=Σmax(child0,child1)，f1=a+Σchild0。环转移未选取两态最大，选只接前一个未选；固定首态后最后检查首尾不能都选。各连通分量答案相加。

初始化与顺序：树叶f0=0、f1=a；链不可达-INF。边id用于区分反向边，不能仅用父顶点跳边；自环禁止选其唯一点，平行双边环也须识别。

优化与限制：O(n)/O(n)，负权允许全不选，题意若要求非空需另设状态。FunctionalGraph分解模块与独立集转移分开阅读；旧继承版本保留，函数版不用继承。

本题复杂度：O(n)时间/空间。

完整函数题解

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n;
vector<int> p, w, val;
vector<int32_t> head, nxt;
vector<char> vis, on;
vector<vector<int>> cycles;

void init(int n_) {
    n = n_;
    p.assign(n + 1, 0);
    w = val = p;
    head.assign(n + 1, -1);
    nxt = head;
    vis.assign(n + 1, 0);
    on = vis;
    cycles.clear();
}
void build() {
    fill(head.begin(), head.end(), -1);
    fill(vis.begin(), vis.end(), 0);
    fill(on.begin(), on.end(), 0);
    cycles.clear();
    for (int u = 1; u <= n; u++) {
        assert(1 <= p[u] && p[u] <= n);
        nxt[u] = head[p[u]];
        head[p[u]] = u;
    }
    for (int i = 1; i <= n; i++)
        if (!vis[i]) {
            int u = i;
            while (!vis[u])
                vis[u] = 1, u = p[u];
            if (vis[u] == 1) {
                vector<int> c;
                int v = u;
                do {
                    c.push_back(v);
                    on[v] = 1;
                    v = p[v];
                } while (v != u);
                cycles.push_back(c);
            }
            u = i;
            while (vis[u] == 1)
                vis[u] = 2, u = p[u];
        }
}
void orient(const vector<array<int, 3>> &edges) {
    vector<vector<array<int, 3>>> g(n + 1);
    vector<int> deg(n + 1);
    int eid = 0;
    for (auto [u, v, c] : edges) {
        g[u].push_back({v, c, eid});
        g[v].push_back({u, c, eid++});
        deg[u]++;
```

```
        deg[v]++;
    }
    queue<int> q;
    for (int u = 1; u <= n; u++)
        if (deg[u] == 1)
            q.push(u);
    vector<char> removed(n + 1);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        removed[u] = 1;
        for (auto [v, c, id] : g[u])
            if (!removed[v]) {
                p[u] = v;
                w[u] = c;
                if (--deg[v] == 1)
                    q.push(v);
                break;
            }
    }
    vector<char> used(n + 1);
    for (int i = 1; i <= n; i++)
        if (!removed[i] && !used[i]) {
            int u = i, prev = -1;
            do {
                used[u] = 1;
                array<int, 3> z = {0, 0, -1};
                for (auto [v, c, id] : g[u])
                    if (!removed[v] && id != prev) {
                        z = {v, c, id};
                        break;
                    }
                assert(z[0]);
                p[u] = z[0];
                w[u] = z[1];
                prev = z[2];
                u = z[0];
            } while (u != i);
        }
    build();
}

static constexpr int INF = 4000000000000000000LL;
vector<array<int, 2>> dp;

void dfs(int u) {
    dp[u] = {0, val[u]};
    for (int v = head[u]; v != -1; v = nxt[v])
        if (!on[v]) {
            dfs(v);
            dp[u][0] += max(dp[v][0], dp[v][1]);
            dp[u][1] += dp[v][0];
        }
}
int run() {
    build();
    dp.assign(n + 1, {0, 0});
    int ans = 0;
```

```
    for (auto c : cycles) {
        for (int u : c)
            dfs(u);
        if (c.size() == 1) {
            ans += dp[c[0]][0];
            continue;
        }
        int best = -INF;
        for (int first = 0; first < 2; first++) {
            int f = first ? -INF : dp[c[0]][0], g = first ? dp[c[0]][1] : -INF;
            for (int i = 1; i < (int)c.size(); i++) {
                int u = c[i];
                int nf = max(f, g) + dp[u][0], ng = f == -INF ? -INF : f + dp[u][1];
                f = nf;
                g = ng;
            }
            best = max(best, first ? f : max(f, g));
        }
        ans += best;
    }
    return ans;
}

void init_model(int n = 0) {
    init(n);
}

void solve() {
    int n;
    cin >> n;
    ::init_model(n);
    for (int i = 1; i <= n; i++)
        cin >> ::val[i];
    vector<array<int, 3>> e(n);
    for (auto &z : e) {
        cin >> z[0] >> z[1];
        z[0]++;
        z[1]++;
        z[2] = 0;
    }
    long double k;
    cin >> k;
    ::orient(e);
    cout << fixed << setprecision(1) << ::run() * k << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：环把局部树接口闭合成首尾约束；只要接口状态很小，枚举首态即可消掉环的耦合。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 自环点不能选；二点互指构成重边环，也不能同时选。
- 压平原vector<vector<ll>>的两状态表减少百万点内存，递归树DP和环递推保持。
- orient仅接受每个分量恰好一个环；树或多环图不适用。

环把局部树接口闭合成首尾约束；只要接口状态很小，枚举首态即可消掉环的耦合。

个人资料来源：图/图论竞赛复习_严格课件注释版.md

图论 / 基环树最长路径与断环直径

类型与用途：混合型；外挂树求向下最长链与内部直径；环复制后滑窗维护d[j]-前缀距离，最长简单路径可取长弧。

复杂度与适用范围：O(n)时间与空间；断环版本用双栈聚合队列同为O(n)。

例题 1 P4381 Island

https://www.luogu.com.cn/problem/P4381

题意：多个基环分量各取最长简单路径，再把长度相加。

观察与规模：多个基环分量各取最长简单路径，再把长度相加。 路径要么在一个外挂树内，要么经环连接不同树根，分别计算取最大。 Island求每个分量最长简单路之和；CF835F选一环边断开后求最小直径，优化目标不同。

朴素思路：枚举所有简单路径指数级；每次删一边重算树直径需O(n^2)。

关键突破：附着树只向环输出向下最长距离及内部直径；环上跨附着树的路径由两端距离和连续环段组成。

状态与建模：每个环点h为附着树最长下行，inside为附着树内部直径；将环权前缀坐标化，候选h[i]+h[j]+dist(i,j)。

本题映射：多个基环分量各取最长简单路径，再把长度相加。 路径要么在一个外挂树内，要么经环连接不同树根，分别计算取最大。 输入p[u],w[u]，run。

转移与正确性：最长简单路在环上不能走满一圈，双倍环用窗口长度小于环长维护最大h[i]-pos[i]。断环时每个起点窗口表示剩余链，链内最远对可用区间合并信息最大化，再对所有切边取最小。

初始化与顺序：附着树先后序算两条最长分支；非环分量路径退化为树直径。自环不属于可走简单路，平行双边有两种不同边长可选。

优化与限制：原实现按对应接口给出线性/带log的界，附着树递归O(n)栈风险。不能将“原图最长路”作为“断一边后最小直径”；权值需非负才能使用这些距离最优性质。

本题复杂度：O(n)时间与空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n;
vector<int> p, w, val;
vector<int32_t> head, nxt;
vector<char> vis, on;
vector<vector<int>> cycles;

```

```
void init(int n_) {
    n = n_;
    p.assign(n + 1, 0);
    w = val = p;
    head.assign(n + 1, -1);
    nxt = head;
    vis.assign(n + 1, 0);
    on = vis;
    cycles.clear();
}
void build() {
    fill(head.begin(), head.end(), -1);
    fill(vis.begin(), vis.end(), 0);
    fill(on.begin(), on.end(), 0);
    cycles.clear();
    for (int u = 1; u <= n; u++) {
        assert(1 <= p[u] && p[u] <= n);
        nxt[u] = head[p[u]];
        head[p[u]] = u;
    }
    for (int i = 1; i <= n; i++)
        if (!vis[i]) {
            int u = i;
            while (!vis[u])
                vis[u] = 1, u = p[u];
            if (vis[u] == 1) {
                vector<int> c;
                int v = u;
                do {
                    c.push_back(v);
                    on[v] = 1;
                    v = p[v];
                } while (v != u);
                cycles.push_back(c);
            }
            u = i;
            while (vis[u] == 1)
                vis[u] = 2, u = p[u];
        }
}
void orient(const vector<array<int, 3>> &edges) {
    vector<vector<array<int, 3>>> g(n + 1);
    vector<int> deg(n + 1);
    int eid = 0;
    for (auto [u, v, c] : edges) {
        g[u].push_back({v, c, eid});
        g[v].push_back({u, c, eid++});
        deg[u]++;
        deg[v]++;
    }
    queue<int> q;
    for (int u = 1; u <= n; u++)
        if (deg[u] == 1)
            q.push(u);
    vector<char> removed(n + 1);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        removed[u] = 1;
        for (auto [v, c, id] : g[u])
```

```
            if (!removed[v]) {
                p[u] = v;
                w[u] = c;
                if (--deg[v] == 1)
                    q.push(v);
                break;
            }
    }
    vector<char> used(n + 1);
    for (int i = 1; i <= n; i++)
        if (!removed[i] && !used[i]) {
            int u = i, prev = -1;
            do {
                used[u] = 1;
                array<int, 3> z = {0, 0, -1};
                for (auto [v, c, id] : g[u])
                    if (!removed[v] && id != prev) {
                        z = {v, c, id};
                        break;
                    }
                assert(z[0]);
                p[u] = z[0];
                w[u] = z[1];
                prev = z[2];
                u = z[0];
            } while (u != i);
        }
    build();
}

vector<int> d;
int inside;

void dfs(int u) {
    d[u] = 0;
    for (int v = head[u]; v != -1; v = nxt[v])
        if (!on[v]) {
            dfs(v);
            int x = d[v] + w[v];
            inside = max(inside, d[u] + x);
            d[u] = max(d[u], x);
        }
}
int ring_path(const vector<int> &a, const vector<int> &b) {
    int k = a.size();
    if (k <= 1)
        return 0;
    vector<int> s(2 * k);
    for (int i = 1; i < 2 * k; i++)
        s[i] = s[i - 1] + b[(i - 1) % k];
    deque<int> q;
    int ans = 0;
    for (int i = 0; i < 2 * k; i++) {
        while (!q.empty() && i - q.front() >= k)
            q.pop_front();
        if (!q.empty())
            ans = max(ans, a[i % k] + s[i] + a[q.front() % k] - s[q.front()]);
        while (!q.empty() && a[q.back() % k] - s[q.back()] <= a[i % k] - s[i])
            q.pop_back();
        q.push_back(i);
```

```
    }
    return ans;
}
int run() {
    build();
    d.assign(n + 1, 0);
    int ans = 0;
    for (auto c : cycles) {
        inside = 0;
        vector<int> a, b;
        for (int u : c) {
            dfs(u);
            a.push_back(d[u]);
            b.push_back(w[u]);
        }
        ans += max(inside, ring_path(a, b));
    }
    return ans;
}
struct info {
    int minus, plus, best;
};
info merge(info a, info b) {
    return {max(a.minus, b.minus), max(a.plus, b.plus),
            max({a.best, b.best, a.minus + b.plus})};
}
int minimum_cut_diameter() {
    build();
    assert(cycles.size() == 1);
    auto c = cycles[0];
    int k = c.size();
    d.assign(n + 1, 0);
    inside = 0;
    for (int u : c)
        dfs(u);
    if (k <= 1)
        return inside;
    vector<int> s(2 * k);
    for (int i = 1; i < 2 * k; i++)
        s[i] = s[i - 1] + w[c[(i - 1) % k]];
    vector<pair<info, info>> in, out;
    auto push = [&](info z) {
        in.push_back({z, in.empty() ? z : merge(in.back().second, z)});
    };
    auto pop = [&]() {
        if (out.empty())
            while (!in.empty()) {
                info z = in.back().first;
                in.pop_back();
                out.push_back({z, out.empty() ? z : merge(z, out.back().second)});
            }
        out.pop_back();
    };
    auto qur = [&]() {
        if (in.empty())
            return out.back().second.best;
        if (out.empty())
            return in.back().second.best;
        return merge(out.back().second, in.back().second).best;
    };
```

```
    int ans = 4000000000000000000LL;
    for (int i = 0; i < 2 * k - 1; i++) {
        push({d[c[i % k]] - s[i], d[c[i % k]] + s[i], 0});
        if (i >= k)
            pop();
        if (i >= k - 1)
            ans = min(ans, max(inside, qur()));
    }
    return ans;
}

void init_model(int n = 0) {
    init(n);
}

void solve() {
    int n;
    cin >> n;
    ::init_model(n);
    for (int i = 1; i <= n; i++)
        cin >> ::p[i] >> ::w[i];
    cout << ::run() << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先把复杂图拆成小环与独立树，再用树输出的接口简化环问题；极大化与切边后极小化分别推导，不能凭同名直径混用。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF835F Roads in the Kingdom

https://codeforces.com/contest/835/problem/F

题意：删去基环图的一条边保持连通，使剩余图最大点距最小。

观察与规模：删去基环图的一条边保持连通，使剩余图最大点距最小。 只能删环边；每种断法对应复制环的k点连续窗口，窗口内最长路径由有序聚合信息维护。 Island求每个分量最长简单路之和；CF835F选一环边断开后求最小直径，优化目标不同。

朴素思路：枚举所有简单路径指数级；每次删一边重算树直径需O(n^2)。

关键突破：附着树只向环输出向下最长距离及内部直径；环上跨附着树的路径由两端距离和连续环段组成。

状态与建模：每个环点h为附着树最长下行，inside为附着树内部直径；将环权前缀坐标化，候选h[i]+h[j]+dist(i,j)。

本题映射：删去基环图的一条边保持连通，使剩余图最大点距最小。 只能删环边；每种断法对应复制环的k点连续窗口，窗口内最长路径由有序聚合信息维护。 读边orient；调用minimum_cut_diameter，与树内直径取max。

转移与正确性：最长简单路在环上不能走满一圈，双倍环用窗口长度小于环长维护最大h[i]-pos[i]。断环时每个起点窗口表示剩余链，链内最远对可用区间合并信息最大化，再对所有切边取最小。

初始化与顺序：附着树先后序算两条最长分支；非环分量路径退化为树直径。自环不属于可走简单路，平行双边有两种不同边长可选。

优化与限制：原实现按对应接口给出线性/带log的界，附着树递归O(n)栈风险。不能将“原图最长路”作为“断一边后最小直径”；权值需非负才能使用这些距离最优性质。

本题复杂度：O(n)时间与空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

int n;
vector<int> p, w, val;
vector<int32_t> head, nxt;
vector<char> vis, on;
vector<vector<int>> cycles;

void init(int n_) {
    n = n_;
    p.assign(n + 1, 0);
    w = val = p;
    head.assign(n + 1, -1);
    nxt = head;
    vis.assign(n + 1, 0);
    on = vis;
    cycles.clear();
}
void build() {
    fill(head.begin(), head.end(), -1);
    fill(vis.begin(), vis.end(), 0);
    fill(on.begin(), on.end(), 0);
    cycles.clear();
    for (int u = 1; u <= n; u++) {
        assert(1 <= p[u] && p[u] <= n);
        nxt[u] = head[p[u]];
        head[p[u]] = u;
    }
    for (int i = 1; i <= n; i++)
        if (!vis[i]) {
            int u = i;
            while (!vis[u])
                vis[u] = 1, u = p[u];
            if (vis[u] == 1) {
                vector<int> c;
                int v = u;
                do {
                    c.push_back(v);
                    on[v] = 1;
                    v = p[v];
                } while (v != u);
                cycles.push_back(c);
            }
            u = i;
            while (vis[u] == 1)
```

```
                vis[u] = 2, u = p[u];
        }
}
void orient(const vector<array<int, 3>> &edges) {
    vector<vector<array<int, 3>>> g(n + 1);
    vector<int> deg(n + 1);
    int eid = 0;
    for (auto [u, v, c] : edges) {
        g[u].push_back({v, c, eid});
        g[v].push_back({u, c, eid++});
        deg[u]++;
        deg[v]++;
    }
    queue<int> q;
    for (int u = 1; u <= n; u++)
        if (deg[u] == 1)
            q.push(u);
    vector<char> removed(n + 1);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        removed[u] = 1;
        for (auto [v, c, id] : g[u])
            if (!removed[v]) {
                p[u] = v;
                w[u] = c;
                if (--deg[v] == 1)
                    q.push(v);
                break;
            }
    }
    vector<char> used(n + 1);
    for (int i = 1; i <= n; i++)
        if (!removed[i] && !used[i]) {
            int u = i, prev = -1;
            do {
                used[u] = 1;
                array<int, 3> z = {0, 0, -1};
                for (auto [v, c, id] : g[u])
                    if (!removed[v] && id != prev) {
                        z = {v, c, id};
                        break;
                    }
                assert(z[0]);
                p[u] = z[0];
                w[u] = z[1];
                prev = z[2];
                u = z[0];
            } while (u != i);
        }
    build();
}

vector<int> d;
int inside;

void dfs(int u) {
    d[u] = 0;
    for (int v = head[u]; v != -1; v = nxt[v])
```

```
        if (!on[v]) {
            dfs(v);
            int x = d[v] + w[v];
            inside = max(inside, d[u] + x);
            d[u] = max(d[u], x);
        }
}
int ring_path(const vector<int> &a, const vector<int> &b) {
    int k = a.size();
    if (k <= 1)
        return 0;
    vector<int> s(2 * k);
    for (int i = 1; i < 2 * k; i++)
        s[i] = s[i - 1] + b[(i - 1) % k];
    deque<int> q;
    int ans = 0;
    for (int i = 0; i < 2 * k; i++) {
        while (!q.empty() && i - q.front() >= k)
            q.pop_front();
        if (!q.empty())
            ans = max(ans, a[i % k] + s[i] + a[q.front() % k] - s[q.front()]);
        while (!q.empty() && a[q.back() % k] - s[q.back()] <= a[i % k] - s[i])
            q.pop_back();
        q.push_back(i);
    }
    return ans;
}
int run() {
    build();
    d.assign(n + 1, 0);
    int ans = 0;
    for (auto c : cycles) {
        inside = 0;
        vector<int> a, b;
        for (int u : c) {
            dfs(u);
            a.push_back(d[u]);
            b.push_back(w[u]);
        }
        ans += max(inside, ring_path(a, b));
    }
    return ans;
}
struct info {
    int minus, plus, best;
};
info merge(info a, info b) {
    return {max(a.minus, b.minus), max(a.plus, b.plus),
            max({a.best, b.best, a.minus + b.plus})};
}
int minimum_cut_diameter() {
    build();
    assert(cycles.size() == 1);
    auto c = cycles[0];
    int k = c.size();
    d.assign(n + 1, 0);
    inside = 0;
    for (int u : c)
        dfs(u);
    if (k <= 1)
```

```
        return inside;
    vector<int> s(2 * k);
    for (int i = 1; i < 2 * k; i++)
        s[i] = s[i - 1] + w[c[(i - 1) % k]];
    vector<pair<info, info>> in, out;
    auto push = [&](info z) {
        in.push_back({z, in.empty() ? z : merge(in.back().second, z)});
    };
    auto pop = [&]() {
        if (out.empty())
            while (!in.empty()) {
                info z = in.back().first;
                in.pop_back();
                out.push_back({z, out.empty() ? z : merge(z, out.back().second)});
            }
        out.pop_back();
    };
    auto qur = [&]() {
        if (in.empty())
            return out.back().second.best;
        if (out.empty())
            return in.back().second.best;
        return merge(out.back().second, in.back().second).best;
    };
    int ans = 4000000000000000000LL;
    for (int i = 0; i < 2 * k - 1; i++) {
        push({d[c[i % k]] - s[i], d[c[i % k]] + s[i], 0});
        if (i >= k)
            pop();
        if (i >= k - 1)
            ans = min(ans, max(inside, qur()));
    }
    return ans;
}

void init_model(int n = 0) {
    init(n);
}

void solve() {
    int n;
    cin >> n;
    vector<array<int, 3>> e(n);
    for (auto &z : e)
        cin >> z[0] >> z[1] >> z[2];
    ::init_model(n);
    ::orient(e);
    cout << ::minimum_cut_diameter() << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先把复杂图拆成小环与独立树，再用树输出的接口简化环问题；极大化与切边后极小化分别推导，不能凭同名直径混用。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 最长简单路径与最短路意义的直径不同，前者可以选择较长环弧。
- 环滑窗严格排除两端同环点，不能绕完整一圈。
- 断环队列信息有方向，左右merge顺序不能交换。

先把复杂图拆成小环与独立树，再用树输出的接口简化环问题；极大化与切边后极小化分别推导，不能凭同名直径混用。

个人资料来源：图/图论竞赛复习_严格课件注释版.md

图论 / 最小割建模

类型与用途：混合型；复用Dinic，支持最大权闭合子图、二元选择与二分图最大权独立集。

复杂度与适用范围：最大流一般O(V^2 E)，源侧恢复O(V+E)；空间O(V+E)。

复用模板：最小割建模（本册第二部分）

例题 1 P2774 方格取数问题

https://www.luogu.com.cn/problem/P2774

题意：选不相邻的格子，使权值和最大。

观察与规模：选不相邻的格子，使权值和最大。 棋盘二分图最大权独立集=总权-最小点覆盖；冲突边给INF。 取舍收益与两对象不可同时选择的冲突，能映射到s/t两侧；不是所有独立集都可用二分图最小割。

朴素思路：枚举所有选择2^n；逐个选最大收益会与多个中等收益冲突。

关键突破：总收益固定时，最大合法收益=总收益-最小舍弃代价。用无限容量边禁止非法的跨割关系。

状态与建模：P2774黑白格二分图，s到黑、白到t容量为点权，黑到相邻白为INF。P2762项目在源侧表示选，依赖设备也须在源侧：项目到设备INF。

本题映射：选不相邻的格子，使权值和最大。 棋盘二分图最大权独立集=总权-最小点覆盖；冲突边给INF。 S到黑格点权，白格到T点权，黑到相邻白INF，输出total-dinic。

转移与正确性：有限割对应合法选择，割掉的点/项目/设备边准确计损失或成本；任意合法方案也有同费用割，双向对应证明最优。P4126对最大流后残量图缩SCC判断边能否/是否必在最小割。

初始化与顺序：init(n,s,t)，全部add后flow，答案总收益-flow。需要恢复方案时从s沿正残量走出源侧；P4126要额外使用SCC板子，不能把原容量图缩点。

优化与限制：INF取严格大于任意有限可行割上界，求和避免溢出；最大流复杂度沿本册Dinic说明。冲突图非二分时P2774模型失效。

本题复杂度：O(V^2 E)时间、O(V+E)空间。

复用模板：最小割建模（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> a(n, vector<int>(m));
    int total = 0;
```

```
    for (auto &row : a)
        for (int &x : row) {
            cin >> x;
            total += x;
        }
    int s = n * m + 1, t = s + 1;
    FlowGraph<int> g;
    g.init(s, t, t);
    int dx[4] = {1, -1, 0, 0}, dy[4] = {0, 0, 1, -1};
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            int u = i * m + j + 1;
            if ((i + j) % 2 == 0) {
                g.addedge(s, u, a[i][j]);
                for (int k = 0; k < 4; k++) {
                    int x = i + dx[k], y = j + dy[k];
                    if (0 <= x && x < n && 0 <= y && y < m)
                        g.addedge(u, x * m + y + 1, total + 1);
                }
            } else
                g.addedge(u, t, a[i][j]);
        }
    cout << total - g.dinic() << '\n';
}
```

启发式总结：先明确每个点在两侧的含义，再对每类边写割费用；不依赖一句“建图跑流”。恢复方案与多种最小割判断还需残量结构。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P2762 太空飞行计划问题

https://www.luogu.com.cn/problem/P2762

题意：实验带收益并依赖设备，设备有费用，求最优实验设备集合。

观察与规模：实验带收益并依赖设备，设备有费用，求最优实验设备集合。 选择实验必须选择依赖设备，是最大权闭合子图。 取舍收益与两对象不可同时选择的冲突，能映射到s/t两侧；不是所有独立集都可用二分图最小割。

朴素思路：枚举所有选择2^n；逐个选最大收益会与多个中等收益冲突。

关键突破：总收益固定时，最大合法收益=总收益-最小舍弃代价。用无限容量边禁止非法的跨割关系。

状态与建模：P2774黑白格二分图，s到黑、白到t容量为点权，黑到相邻白为INF。P2762项目在源侧表示选，依赖设备也须在源侧：项目到设备INF。

本题映射：实验带收益并依赖设备，设备有费用，求最优实验设备集合。 选择实验必须选择依赖设备，是最大权闭合子图。 收益S->实验，费用设备->T，实验->设备INF；cut_side恢复选点。

转移与正确性：有限割对应合法选择，割掉的点/项目/设备边准确计损失或成本；任意合法方案也有同费用割，双向对应证明最优。P4126对最大流后残量图缩SCC判断边能否/是否必在最小割。

初始化与顺序：init(n,s,t)，全部add后flow，答案总收益-flow。需要恢复方案时从s沿正残量走出源侧；P4126要额外使用SCC板子，不能把原容量图缩点。

优化与限制：INF取严格大于任意有限可行割上界，求和避免溢出；最大流复杂度沿本册Dinic说明。冲突图非二分时P2774模型失效。

本题复杂度：O(V^2 E)时间、O(V+E)空间。

复用模板：最小割建模（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int m, n;
    cin >> m >> n;
    string line;
    getline(cin, line);
    vector<int> profit(m + 1);
    vector<vector<int>> need(m + 1);
    int total = 0;
    for (int i = 1; i <= m; i++) {
        getline(cin, line);
        istringstream in(line);
        in >> profit[i];
        total += profit[i];
        int x;
        while (in >> x)
            need[i].push_back(x);
    }
    vector<int> cost(n + 1);
    for (int j = 1; j <= n; j++)
        cin >> cost[j];
    int s = m + n + 1, t = s + 1;
    FlowGraph<int> g;
    g.init(s, t, t);
    for (int i = 1; i <= m; i++) {
        g.addedge(s, i, profit[i]);
        for (int j : need[i])
            g.addedge(i, m + j, total + 1);
    }
    for (int j = 1; j <= n; j++)
        g.addedge(m + j, t, cost[j]);
    int ans = total - g.dinic();
    auto vis = g.cut_side();
    for (int i = 1; i <= m; i++)
        if (vis[i])
            cout << i << ' ';
    cout << '\n';
    for (int j = 1; j <= n; j++)
        if (vis[m + j])
            cout << j << ' ';
    cout << '\n' << ans << '\n';
}
```

启发式总结：先明确每个点在两侧的含义，再对每类边写割费用；不依赖一句“建图跑流”。恢复方案与多种最小割判断还需残量结构。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 3 P4126 最小割边判定

https://www.luogu.com.cn/problem/P4126

题意：判断每条边是否可出现在某个或所有最小割中。

观察与规模：判断每条边是否可出现在某个或所有最小割中。 最大流后残量SCC：饱和且两端不同SCC可属于某割；起点在源SCC且终点在汇SCC属于所有割。 取舍收益与两对象不可同时选择的冲突，能映射到s/t两侧；不是所有独立集都可用二分图最小割。

朴素思路：枚举所有选择2^n；逐个选最大收益会与多个中等收益冲突。

关键突破：总收益固定时，最大合法收益=总收益-最小舍弃代价。用无限容量边禁止非法的跨割关系。

状态与建模：P2774黑白格二分图，s到黑、白到t容量为点权，黑到相邻白为INF。P2762项目在源侧表示选，依赖设备也须在源侧：项目到设备INF。

本题映射：判断每条边是否可出现在某个或所有最小割中。 最大流后残量SCC：饱和且两端不同SCC可属于某割；起点在源SCC且终点在汇SCC属于所有割。 dinic后将正残量边加入SCC；按保存原边号检查f=0和col条件。

转移与正确性：有限割对应合法选择，割掉的点/项目/设备边准确计损失或成本；任意合法方案也有同费用割，双向对应证明最优。P4126对最大流后残量图缩SCC判断边能否/是否必在最小割。

初始化与顺序：init(n,s,t)，全部add后flow，答案总收益-flow。需要恢复方案时从s沿正残量走出源侧；P4126要额外使用SCC板子，不能把原容量图缩点。

优化与限制：INF取严格大于任意有限可行割上界，求和避免溢出；最大流复杂度沿本册Dinic说明。冲突图非二分时P2774模型失效。

本题复杂度：最大流一般O(V^2E)，SCC O(V+E)，O(V+E)空间。

复用模板：最小割建模（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m, s, t;
    cin >> n >> m >> s >> t;
    FlowGraph<int> g;
    g.init(s, t, n);
    vector<array<int, 3>> a(m);
    for (auto &z : a) {
        int c;
        cin >> z[0] >> z[1] >> c;
        z[2] = g.etot;
        g.addedge(z[0], z[1], c);
    }
    g.dinic();
    SCC sc;
    sc.init(n);
    for (int u = 1; u <= n; u++)
        for (int i = g.head[u]; i != -1; i = g.e[i].nxt)
            if (g.e[i].f > 0)
                sc.addedge(u, g.e[i].v);
    sc.run();
    for (auto [u, v, id] : a) {
        bool possible = g.e[id].f == 0 && sc.bel[u] != sc.bel[v];
        bool necessary = possible && sc.bel[u] == sc.bel[s] && sc.bel[v] == sc.bel[t];
        cout << possible << ' ' << necessary << '\n';
    }
}
```

启发式总结：先明确每个点在两侧的含义，再对每类边写割费用；不依赖一句“建图跑流”。恢复方案与多种最小割判断还需残量结构。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 选A的代价接u->T，选B的代价接S->u，先写清源侧语义。
- 饱和边不一定属于同一个最小割，按源侧跨边恢复。

- 最小割边分类须在最大流完成后，以正残量图求SCC。

- 最小割边分类须在最大流完成后，以正残量图求SCC。

先明确每个点在两侧的含义，再对每类边写割费用；不依赖一句“建图跑流”。恢复方案与多种最小割判断还需残量结构。

个人资料来源：图/图论竞赛复习_严格课件注释版.md

动态规划 / DFS序依赖背包

类型与用途：混合型；选节点则继续DFS下一点，不选则跳过其整个子树；依赖环先缩点为共同选择的软件包。

复杂度与适用范围：O(nM)时间/空间；SCC另加O(n+m)。

例题 1 P2014 选课 DFS序版

https://www.luogu.com.cn/problem/P2014

题意：按先修条件恰好选m门课。

观察与规模：恰选m门课且每门至多一个先修；虚根0不计数量，必须允许森林但不能丢掉恰好数量约束。 每点只有一个直接前置依赖；P2515允许依赖形成环，P2014要求恰好选m门。

朴素思路：普通树背包要枚举两个数量分配；环上按点逐个选择会破坏“全部选或全部不选”。

关键突破：先把依赖环缩成SCC；DFS先序中子树连续，不选一个点就能直接跳过整棵子树。

状态与建模：id[i]为DFS序第i个点，r[u]为其子树结束后的第一位置；dp[i][j]为从位置i以后合法选择、容量为j的最大收益。exact=true时j是恰好用量，否则是不超过容量。

本题映射：按先修条件恰好选m门课。 先修树的合法点集在DFS序中只能“选当前”或“跳整子树”。 真实课w=1、虚根w=0，run(0,true)。

转移与正确性：不选u：dp[r[u]][j]；选u：dp[i+1][j-w[u]]+a[u]。选择父亲后才能走进下一位置的后代，跳子树保证没有选父亲时后代也不选。缩点累加重量和价值，依赖边连接不同SCC。

初始化与顺序：逆DFS序计算。exact末尾仅dp[tot+1][0]=0，其余-INF；不超过容量时末尾全0。虚根0重量价值均0，用它接多个无依赖分量。SCC需先init、加边、run。

优化与限制：缩点O(n+m)，DFS序DP O(nM)/O(nM)。依赖任意多前驱时缩点后可能是一般DAG，不能当森林跳子树；本题单前驱是关键。

本题复杂度：O(nm)时间/空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

static constexpr int INF = 4000000000000000000LL;
int n, M, tot;
vector<vector<int>> son;
vector<int> id, r, w, a;
vector<vector<int>> dp;

void init(int n_, int m) {
    n = n_;
    M = m;
    tot = 0;
    son.assign(n + 1, {});
```

```
    id.assign(n + 2, 0);
    r = w = a = id;
    dp.clear();
}
void add_edge(int u, int v) {
    son[u].push_back(v);
}
void dfs(int u) {
    id[++tot] = u;
    for (int v : son[u])
        dfs(v);
    r[u] = tot + 1;
}
int run(int root, bool exact = false) {
    tot = 0;
    dfs(root);
    dp.assign(tot + 2, vector<int>(M + 1, exact ? -INF : 0));
    dp[tot + 1][0] = 0;
    for (int i = tot; i >= 1; i--) {
        int u = id[i];
        for (int j = 0; j <= M; j++) {
            dp[i][j] = dp[r[u]][j];
            if (j >= w[u] && dp[i + 1][j - w[u]] > -INF)
                dp[i][j] = max(dp[i][j], dp[i + 1][j - w[u]] + a[u]);
        }
    }
    return dp[1][M];
}

void init_model(int n = 0, int M = 0) {
    init(n, M);
}

void solve() {
    int n, m;
    cin >> n >> m;
    ::init_model(n, m);
    for (int i = 1; i <= n; i++) {
        int p;
        cin >> p >> ::a[i];
        ::w[i] = 1;
        ::add_edge(p, i);
    }
    cout << ::run(0, true) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：看到“不选后全部后代失效”可尝试DFS序跳跃。优化来自状态的两种去向，不是把树背包循环机械删一层。

验证：C++17独立程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 P2515 软件安装

https://www.luogu.com.cn/problem/P2515

题意：有最多一个依赖的软件选择，可能存在依赖环，限定容量求最大价值。

观察与规模：软件可能相互依赖，SCC内全部选或全部不选；每点最多一个直接依赖，缩点后才可按森林做DFS序跳跃。 每点只有一个直接前置依赖；P2515允许依赖形成环，P2014要求恰好选m门。

朴素思路：普通树背包要枚举两个数量分配；环上按点逐个选择会破坏“全部选或全部不选”。

关键突破：先把依赖环缩成SCC；DFS先序中子树连续，不选一个点就能直接跳过整棵子树。

状态与建模：id[i]为DFS序第i个点，r[u]为其子树结束后的第一位置；dp[i][j]为从位置i以后合法选择、容量为j的最大收益。exact=true时j是恰好用量，否则是不超过容量。

本题映射：有最多一个依赖的软件选择，可能存在依赖环，限定容量求最大价值。 环中软件必须全选或全不选；SCC缩点后依赖变森林，再DFS序背包。 SCC.scc取得col，合计包w/a；无前提包接虚根；run(0)。

转移与正确性：不选u：dp[r[u]][j]；选u：dp[i+1][j-w[u]]+a[u]。选择父亲后才能走进下一位置的后代，跳子树保证没有选父亲时后代也不选。缩点累加重量和价值，依赖边连接不同SCC。

初始化与顺序：逆DFS序计算。exact末尾仅dp[tot+1][0]=0，其余-INF；不超过容量时末尾全0。虚根0重量价值均0，用它接多个无依赖分量。SCC需先init、加边、run。

优化与限制：缩点O(n+m)，DFS序DP O(nM)/O(nM)。依赖任意多前驱时缩点后可能是一般DAG，不能当森林跳子树；本题单前驱是关键。

本题复杂度：O(nM+n)时间、O(nM)空间。

组合依赖：Tarjan强连通分量 的定义

题解代码（需组合上述依赖）

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

// 组合依赖：本册 Tarjan强连通分量 的 SCC 定义。


static constexpr int INF = 4000000000000000000LL;
int n, M, tot;
vector<vector<int>> son;
vector<int> id, r, w, a;
vector<vector<int>> dp;

void init(int n_, int m) {
    n = n_;
    M = m;
    tot = 0;
    son.assign(n + 1, {});
    id.assign(n + 2, 0);
    r = w = a = id;
    dp.clear();
}
void add_edge(int u, int v) {
    son[u].push_back(v);
}
void dfs(int u) {
    id[++tot] = u;
    for (int v : son[u])
        dfs(v);
```

```
    r[u] = tot + 1;
}
int run(int root, bool exact = false) {
    tot = 0;
    dfs(root);
    dp.assign(tot + 2, vector<int>(M + 1, exact ? -INF : 0));
    dp[tot + 1][0] = 0;
    for (int i = tot; i >= 1; i--) {
        int u = id[i];
        for (int j = 0; j <= M; j++) {
            dp[i][j] = dp[r[u]][j];
            if (j >= w[u] && dp[i + 1][j - w[u]] > -INF)
                dp[i][j] = max(dp[i][j], dp[i + 1][j - w[u]] + a[u]);
        }
    }
    return dp[1][M];
}

void init_model(int n = 0, int M = 0) {
    init(n, M);
}

void solve() {
    int n, M;
    cin >> n >> M;
    vector<int> w(n + 1), a(n + 1), p(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> w[i];
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    SCC s;
    s.init(n);
    for (int i = 1; i <= n; i++) {
        cin >> p[i];
        if (p[i])
            s.addedge(p[i], i);
    }
    s.run();
    ::init_model(s.cnt, M);
    vector<int> in(s.cnt + 1);
    for (int i = 1; i <= n; i++) {
        ::w[s.bel[i]] += w[i];
        ::a[s.bel[i]] += a[i];
        if (p[i] && s.bel[p[i]] != s.bel[i])
            ::add_edge(s.bel[p[i]], s.bel[i]), in[s.bel[i]] = 1;
    }
    for (int i = 1; i <= s.cnt; i++)
        if (!in[i])
            ::add_edge(0, i);
    cout << ::run(0) << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
```

```
90 }
```

启发式总结：看到“不选后全部后代失效”可尝试DFS序跳跃。优化来自状态的两种去向，不是把树背包循环机械删一层。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 图必须是依赖森林，一般多前提DAG不能直接套。
- 虚根0的w=a=0，SCC缩点后每包最多一个外部先修。
- 原课堂固定规模数组省略了tot+1行；整理版显式分配结束状态。

看到“不选后全部后代失效”可尝试DFS序跳跃。优化来自状态的两种去向，不是把树背包循环机械删一层。

个人资料来源：dp/树形动态规划进阶_比赛板子.md

动态规划 / 树上不相交给定路径

类型与用途：混合型；选择给定带权路径，每个顶点至多使用一次。在LCA处决策，沿路径累加sdp-dp；保留课堂逐点版并提供DFS序树状数组优化。

复杂度与适用范围：build O(n log n)；逐点版O(nm)，fast版O((n+m)log n)；空间O(n log n+m)。

例题 1 HDU5293 Tree chain problem

https://acm.hdu.edu.cn/showproblem.php?pid=5293

题意：树上给出带权链，选择顶点互不相交的链使权值和最大。

观察与规模：树上给出带权链，选择顶点互不相交的链使权值和最大。 与课堂路径1的点不相交带权模型一致；delta之和扣掉所选路径占用顶点造成的子树最优损失。 给定路径可带权，选出的路径不能共享顶点；路径是已给候选，不是任意自创。

朴素思路：枚举路径子集2^m；将每条路径沿树逐点更新可能达O(nm)。

关键突破：按LCA把路径挂在最高节点；在u处要么不用经过u的路径，要么仅选一条，因为所有这类路径共享u。

状态与建模：dp[u]是u子树最大选路权重，sdp[u]=Σdp[v]是不用u时各孩子独立最优。选路径x-y会占用它经过的每个祖先节点，必须释放相应子树最优占用。

本题映射：树上给出带权链，选择顶点互不相交的链使权值和最大。 与课堂路径1的点不相交带权模型一致；delta之和扣掉所选路径占用顶点造成的子树最优损失。 读T；每组n,m、n-1条边、m条x,y,w；build后add_path，run使用优化版。

转移与正确性：候选=sdp[u]+w+Σ_(两端到u且不含u)(sdp[z]-dp[z])。这保留路径旁的侧枝方案并去掉与路径冲突的最优项；每条路径仅在其LCA处理一次。

初始化与顺序：先build得到父亲、Euler序和倍增表，再add_path挂LCA，后序dfs完成子树dp。计算u时BIT只保存已经完成的后代，u的delta在本节点候选全部计算后插入。

优化与限制：朴素沿路径累加改为Euler区间加、端点单查：delta[z]影响整棵z子树，两端各查询一次。O((n+m)log n)，不能提前插入u造成u计算两次。HDU4912把w设1，HDU5293读实际权值。

本题复杂度：O((n+m)log n)时间、O(n log n+m)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

struct info {
    int x, y, w;
};
int n, lg, timer;
vector<vector<int>> e;
vector<vector<info>> path;
vector<int> fa, dep, l, r, dp, sdp, bit;
vector<vector<int32_t>> up;

void init(int n_) {
    n = n_;
    lg = 1;
    while ((1LL << lg) <= n)
        lg++;
    timer = 0;
    e.assign(n + 1, {});
    path.assign(n + 1, {});
    fa.assign(n + 1, 0);
    dep = fa;
    l = fa;
    r = fa;
    dp = fa;
    sdp = fa;
    bit.assign(n + 2, 0);
    up.assign(lg, vector<int32_t>(n + 1));
}
void add_edge(int u, int v) {
    e[u].push_back(v);
    e[v].push_back(u);
}
void build(int u, int p) {
    fa[u] = p;
    dep[u] = dep[p] + 1;
    l[u] = ++timer;
    up[0][u] = p;
    for (int k = 1; k < lg; k++)
        up[k][u] = up[k - 1][up[k - 1][u]];
    for (int v : e[u])
        if (v != p)
            build(v, u);
    r[u] = timer;
}
void build(int root = 1) {
    timer = 0;
    build(root, 0);
}
int lca(int x, int y) {
    if (dep[x] < dep[y])
        swap(x, y);
    int d = dep[x] - dep[y];
    for (int k = 0; k < lg; k++)
        if (d >> k & 1)
            x = up[k][x];
    if (x == y)
```

```
        return x;
    for (int k = lg - 1; k >= 0; k--)
        if (up[k][x] != up[k][y]) {
            x = up[k][x];
            y = up[k][y];
        }
    return fa[x];
}
void add_path(int x, int y, int w) {
    path[lca(x, y)].push_back({x, y, w});
}
void change(int x, int v) {
    for (; x <= n + 1; x += x & -x)
        bit[x] += v;
}
int qur(int x) {
    int ans = 0;
    for (; x; x -= x & -x)
        ans += bit[x];
    return ans;
}
void dfs(int u, bool fast) {
    sdp[u] = 0;
    for (int v : e[u])
        if (v != fa[u]) {
            dfs(v, fast);
            sdp[u] += dp[v];
        }
    dp[u] = sdp[u];
    for (auto p : path[u]) {
        int tmp = sdp[u] + p.w;
        if (fast)
            tmp += qur(l[p.x]) + qur(l[p.y]);
        else {
            int x = p.x, y = p.y;
            while (x != u) {
                tmp += sdp[x] - dp[x];
                x = fa[x];
            }
            while (y != u) {
                tmp += sdp[y] - dp[y];
                y = fa[y];
            }
        }
        dp[u] = max(dp[u], tmp);
    }
    if (fast) {
        int delta = sdp[u] - dp[u];
        change(l[u], delta);
        change(r[u] + 1, -delta);
    }
}
int run(int root = 1, bool fast = true) {
    fill(bit.begin(), bit.end(), 0);
    dfs(root, fast);
    return dp[root];
}

```

```
void init_model(int n = 0) {
    init(n);
}

void solve() {
    int T;
    cin >> T;
    while (T--) {
        int n, m;
        cin >> n >> m;
        ::init_model(n);
        for (int i = 1; i < n; i++) {
            int u, v;
            cin >> u >> v;
            ::add_edge(u, v);
        }
        ::build();
        while (m--) {
            int x, y, w;
            cin >> x >> y >> w;
            ::add_path(x, y, w);
        }
        cout << ::run() << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：优化前先写出需要的路径和，再把“祖先贡献”转换成“子树覆盖端点”。点不相交与边不相交模型不同，不能仅改标题。

验证：C++17独立程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 HDU4912 Paths on the tree

https://acm.hdu.edu.cn/showproblem.php?pid=4912

题意：从给定树上路径中选尽量多条，使所选路径顶点互不相交。

观察与规模：从给定树上路径中选尽量多条，使所选路径顶点互不相交。 把每条路径权重设为1就是同一带权选择问题；原题可用贪心，本例展示通用DP的等权特例。 给定路径可带权，选出的路径不能共享顶点；路径是已给候选，不是任意自创。

朴素思路：枚举路径子集2^m；将每条路径沿树逐点更新可能达O(nm)。

关键突破：按LCA把路径挂在最高节点；在u处要么不用经过u的路径，要么仅选一条，因为所有这类路径共享u。

状态与建模：dp[u]是u子树最大选路权重，sdp[u]=Σdp[v]是不用u时各孩子独立最优。选路径x-y会占用它经过的每个祖先节点，必须释放相应子树最优占用。

本题映射：从给定树上路径中选尽量多条，使所选路径顶点互不相交。 把每条路径权重设为1就是同一带权选择问题；原题可用贪心，本例展示通用DP的等权特例。 循环到EOF读n,m及树边、路径端点；build，add_path(x,y,1)，run。

转移与正确性：候选=sdp[u]+w+Σ_(两端到u且不含u)(sdp[z]-dp[z])。这保留路径旁的侧枝方案并去掉与路径冲突的最优项；每条路径仅在其LCA处理一次。

初始化与顺序：先build得到父亲、Euler序和倍增表，再add_path挂LCA，后序dfs完成子树dp。计算u时BIT只保存已经完成的后代，u的delta在本节点候选全部计算后插入。

优化与限制：朴素沿路径累加改为Euler区间加、端点单查：delta[z]影响整棵z子树，两端各查询一次。O((n+m)log n)，不能提前插入u造成u计算两次。HDU4912把w设1，HDU5293读实际权值。

本题复杂度：O((n+m)log n)时间、O(n log n+m)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

struct info {
    int x, y, w;
};
int n, lg, timer;
vector<vector<int>> e;
vector<vector<info>> path;
vector<int> fa, dep, l, r, dp, sdp, bit;
vector<vector<int32_t>> up;

void init(int n_) {
    n = n_;
    lg = 1;
    while ((1LL << lg) <= n)
        lg++;
    timer = 0;
    e.assign(n + 1, {});
    path.assign(n + 1, {});
    fa.assign(n + 1, 0);
    dep = fa;
    l = fa;
    r = fa;
    dp = fa;
    sdp = fa;
    bit.assign(n + 2, 0);
    up.assign(lg, vector<int32_t>(n + 1));
}
void add_edge(int u, int v) {
    e[u].push_back(v);
    e[v].push_back(u);
}
void build(int u, int p) {
    fa[u] = p;
    dep[u] = dep[p] + 1;
    l[u] = ++timer;
    up[0][u] = p;
    for (int k = 1; k < lg; k++)
        up[k][u] = up[k - 1][up[k - 1][u]];
    for (int v : e[u])
        if (v != p)
            build(v, u);
    r[u] = timer;
}
```

```
void build(int root = 1) {
    timer = 0;
    build(root, 0);
}
int lca(int x, int y) {
    if (dep[x] < dep[y])
        swap(x, y);
    int d = dep[x] - dep[y];
    for (int k = 0; k < lg; k++)
        if (d >> k & 1)
            x = up[k][x];
    if (x == y)
        return x;
    for (int k = lg - 1; k >= 0; k--)
        if (up[k][x] != up[k][y]) {
            x = up[k][x];
            y = up[k][y];
        }
    return fa[x];
}
void add_path(int x, int y, int w) {
    path[lca(x, y)].push_back({x, y, w});
}
void change(int x, int v) {
    for (; x <= n + 1; x += x & -x)
        bit[x] += v;
}
int qur(int x) {
    int ans = 0;
    for (; x; x -= x & -x)
        ans += bit[x];
    return ans;
}
void dfs(int u, bool fast) {
    sdp[u] = 0;
    for (int v : e[u])
        if (v != fa[u]) {
            dfs(v, fast);
            sdp[u] += dp[v];
        }
    dp[u] = sdp[u];
    for (auto p : path[u]) {
        int tmp = sdp[u] + p.w;
        if (fast)
            tmp += qur(l[p.x]) + qur(l[p.y]);
        else {
            int x = p.x, y = p.y;
            while (x != u) {
                tmp += sdp[x] - dp[x];
                x = fa[x];
            }
            while (y != u) {
                tmp += sdp[y] - dp[y];
                y = fa[y];
            }
        }
        dp[u] = max(dp[u], tmp);
    }
```

```
    if (fast) {
        int delta = sdp[u] - dp[u];
        change(l[u], delta);
        change(r[u] + 1, -delta);
    }
}
int run(int root = 1, bool fast = true) {
    fill(bit.begin(), bit.end(), 0);
    dfs(root, fast);
    return dp[root];
}

void init_model(int n = 0) {
    init(n);
}

void solve() {
    int n, m;
    while (cin >> n >> m) {
        ::init_model(n);
        for (int i = 1; i < n; i++) {
            int u, v;
            cin >> u >> v;
            ::add_edge(u, v);
        }
        ::build();
        while (m--) {
            int x, y;
            cin >> x >> y;
            ::add_path(x, y, 1);
        }
        cout << ::run() << '\n';
    }
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：优化前先写出需要的路径和，再把“祖先贡献”转换成“子树覆盖端点”。点不相交与边不相交模型不同，不能仅改标题。

验证：C++17独立程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 点不相交，不能换成仅边不相交。
- 树状数组对子树加delta；查询端点时，尚未处理的当前LCA及更高祖先不会被计入。
- 可不选任何路径，所以负权自然被忽略。
- 长链递归有栈溢出风险。HDU原站题面与样例已核实，原题n,m≤100000，需使用fast版。

优化前先写出需要的路径和，再把“祖先贡献”转换成“子树覆盖端点”。点不相交与边不相交模型不同，不能仅改标题。

个人资料来源：dp/树形动态规划进阶_比赛板子.md；代码源272（原题不可访问，按已存课堂模型整理）

动态规划 / 祖先路径最小费用覆盖

类型与用途：混合型；从给定非负费用祖先 - 后代路径中选若干条覆盖全部点，可重叠。保留最高到达深度DP，补充适合大数据的可并堆费用扣减版。

复杂度与适用范围：深度DP O(n^2+m)时间/最坏O(n^2)空间；可并堆O((n+m)log(m+1))时间/O(n+m)空间。

例题 1 CF1675D Vertical Paths

https://codeforces.com/problemset/problem/1675/D

题意：将有根树分成最少条向下的互不相交路径，输出路径划分。

观察与规模：n及所有测试n之和≤2×10^5；竖直路径只沿祖先到后代，要求每个顶点恰好出现一次，并输出路径。

朴素思路：套一般有权路径覆盖会多维护费用和候选，本题所有竖直链都可用，且只求最少分组，可以先找结构下界。

关键突破：一条竖直链最多包含一个原树叶子，所以路径条数至少为叶子数。

状态与建模：fa[u]为父亲、has_child[u]标记是否有真孩子、used[u]标记已分组。只从叶子开始向上收集尚未覆盖的连续前缀。

转移与正确性：到已访问点即停，再反转得到从祖先到叶子的路径。各节点只加入一次；任一内部点都有叶子后代，必被某次上爬覆盖。恰每个叶子产一条，达到下界，故构造最优。

初始化与顺序：根以fa[root]=root标识；建has_child时不把自父关系当真孩子。每组清used，单点树产一条含根路径。

优化与限制：上爬过程中每个节点被标记一次，总O(n)时间、O(n)空间，无需可并堆。输出允许多种正确方案，以覆盖、不重和父子连续性验证。

特殊修改：本题是无权竖直链分组，采用O(n)直接构造；一般有权覆盖模型用于下一例CF671D。

本题复杂度：O(n)时间、O(n)空间；每个顶点只被上爬加入一次。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

void solve() {
    int T;
    cin >> T;
    while (T--) {
        int n;
        cin >> n;
        vector<int> fa(n + 1), has_child(n + 1), used(n + 1);
        for (int u = 1; u <= n; u++) {
            cin >> fa[u];
            if (fa[u] != u)
                has_child[fa[u]] = 1;
        }
        vector<vector<int>> paths;
        for (int u = 1; u <= n; u++)
            if (!has_child[u]) {
                vector<int> p;
                int v = u;
```

```
                while (!used[v]) {
                    p.push_back(v);
                    used[v] = 1;
                    if (fa[v] == v)
                        break;
                    v = fa[v];
                }
                reverse(p.begin(), p.end());
                paths.push_back(p);
            }
        cout << paths.size() << '\n';
        for (auto &p : paths) {
            cout << p.size() << '\n';
            for (int x : p)
                cout << x << ' ';
            cout << '\n';
        }
        cout << '\n';
    }
}
signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：先找不可合并的对象提供下界，再构造恰好达到下界的分组。若允许非竖直路径，一条路径能含两个叶子，下界与构造都需重做。

验证：C++17独立程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF671D Roads in Yusland

https://codeforces.com/problemset/problem/671/D

题意：给定工人各可修一条祖先路径，付费雇佣工人覆盖所有树边，求最小总费用。

观察与规模：给定工人各可修一条祖先路径，付费雇佣工人覆盖所有树边，求最小总费用。 把边(parent[x],x)映射为点x。u到v的边集合变成u到v下一层孩子的点路径；忽略u=v的空路径，根用免费单点覆盖。 路径从后代到祖先，费用非负；课堂覆盖点，CF671D覆盖边，CF1675D只需无权竖直路径分组。

朴素思路：枚举路径子集指数级；深度状态合并若枚举两状态O(depth^2)，链上也可能有大量状态。

关键突破：覆盖当前点后，只需记选中路径能向上延伸的最浅深度；合并两个方案的延伸深度取min。

状态与建模：dp[u][d]是覆盖u子树、且所选路径最高能到深度d的最小费用。哨兵depth[u]+1只表示合并尚未覆盖u；返回父亲前必须禁用。堆记录路径top深度和尚未付清的残余费用。

本题映射：给定工人各可修一条祖先路径，付费雇佣工人覆盖所有树边，求最小总费用。 把边(parent[x],x)映射为点x。u到v的边集合变成u到v下一层孩子的点路径；忽略u=v的空路径，根用免费单点覆盖。 DFS建立fa、倍增表；将v下方孩子作为top，add_path；run_heap。原题n,m≤300000，不能调用O(n^2)深度DP。

转移与正确性：合并(a,b)按min深度分类，a[k]+min_(j≥k)b[j]或b[k]+min_(j≥k)a[j]；后缀min把双枚举降一维。堆在每点支付最小残余价，再对所有覆盖该点的候选扣同样的价。把支付量看作该点的对偶价格，每条路径上的累计价格不超过原费用，故价格总和是任何覆盖方案的下界。支付后记录一条残余0的紧路径，按根到叶回溯，只为尚未覆盖点选其当时记录的路径。已变0且能到某祖先的路径会使该祖先

支付0，因此选出的路径不重复覆盖正价格点，原费用总和恰等于下界，证明最优。祖先链结构与非负费用是此证明的关键。

初始化与顺序：本点路径初始化dp[u][top]=cost，未处理孩子前哨兵为0，最终哨兵INF。堆合并孩子后加入本点路径，弹出top>depth[u]，空堆即无解。每次run_heap重建堆并清answer/ok。

优化与限制：深度DP最坏O(n^2)，仅作推导/小数据参考；左偏堆合并及lazy残余扣减约O((n+m)log(n+m))。CF671D把每条边映射为较深端点，路径u到v的top改成v下一层，根免费覆盖。

本题复杂度：O(n log n+(n+m)log(m+1))时间；O(n log n+m)空间。

完整函数题解

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

struct node {
    int ls = 0, rs = 0, dist = 1, key = 0, lazy = 0, top = 0;
};
static constexpr int INF = 4000000000000000000LL;
int n, answer;
bool ok;
vector<vector<int>> son;
vector<vector<pair<int, int>>> path;
vector<int> dep, rt;
vector<vector<int>> dp;
vector<node> tr;

void init(int n_) {
    n = n_;
    son.assign(n + 1, {});
    path.assign(n + 1, {});
    dep.assign(n + 1, 0);
    rt = dep;
    dp.assign(n + 1, {});
    tr.assign(1, node{});
    tr[0].dist = 0;
}
void add_edge(int u, int v) {
    son[u].push_back(v);
}
void build(int u = 1) {
    for (int v : son[u]) {
        dep[v] = dep[u] + 1;
        build(v);
    }
}
void add_path(int bottom, int top, int cost) {
    assert(cost >= 0);
    path[bottom].push_back({dep[top], cost});
}
void dfs(int u) {
    int L = dep[u] + 1;
    dp[u].assign(L + 1, INF);
    dp[u][L] = 0;
    for (auto [d, w] : path[u])
        dp[u][d] = min(dp[u][d], w);
```

```
    for (int v : son[u]) {
        dfs(v);
        vector<int> a = dp[u], b = dp[v], sa(L + 2, INF), sb(L + 2, INF);
        for (int k = L; k >= 0; k--) {
            sa[k] = min(sa[k + 1], a[k]);
            sb[k] = min(sb[k + 1], b[k]);
        }
        for (int k = 0; k <= L; k++)
            dp[u][k] = min(INF, min(a[k] + sb[k], b[k] + sa[k]));
    }
    dp[u][L] = INF; // 返回父亲前，u必须已经被覆盖。
}
int run(int root = 1) {
    dfs(root);
    int z = *min_element(dp[root].begin(), dp[root].end());
    return z >= INF ? -1 : z;
}
void apply(int p, int v) {
    if (p) {
        tr[p].key += v;
        tr[p].lazy += v;
    }
}
void push(int p) {
    if (tr[p].lazy) {
        apply(tr[p].ls, tr[p].lazy);
        apply(tr[p].rs, tr[p].lazy);
        tr[p].lazy = 0;
    }
}
int merge(int x, int y) {
    if (!x || !y)
        return x + y;
    if (tr[x].key > tr[y].key)
        swap(x, y);
    push(x);
    tr[x].rs = merge(tr[x].rs, y);
    if (tr[tr[x].ls].dist < tr[tr[x].rs].dist)
        swap(tr[x].ls, tr[x].rs);
    tr[x].dist = tr[tr[x].rs].dist + 1;
    return x;
}
int pop(int x) {
    push(x);
    return merge(tr[x].ls, tr[x].rs);
}
void dfs_heap(int u) {
    rt[u] = 0;
    for (int v : son[u]) {
        dfs_heap(v);
        rt[u] = merge(rt[u], rt[v]);
    }
    for (auto [d, w] : path[u]) {
        node a;
        a.key = w;
        a.top = d;
        tr.push_back(a);
        rt[u] = merge(rt[u], tr.size() - 1);
    }
    while (rt[u] && tr[rt[u]].top > dep[u])
```

```
        rt[u] = pop(rt[u]);
    if (!rt[u]) {
        ok = false;
        return;
    }
    int val = tr[rt[u]].key;
    answer += val;
    apply(rt[u], -val);
}
int run_heap(int root = 1) {
    tr.assign(1, node{});
    tr[0].dist = 0;
    answer = 0;
    ok = true;
    dfs_heap(root);
    return ok ? answer : -1;
}

void init_model(int n = 0) {
    init(n);
}

void solve() {
    int n, m;
    cin >> n >> m;
    vector<vector<int>> e(n + 1);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        e[u].push_back(v);
        e[v].push_back(u);
    }
    ::init_model(n);
    int L = 1;
    while ((1LL << L) <= n)
        L++;
    vector<vector<int32_t>> up(L, vector<int32_t>(n + 1));
    function<void(int, int)> dfs = [&](int u, int p) {
        up[0][u] = p;
        for (int k = 1; k < L; k++)
            up[k][u] = up[k - 1][up[k - 1][u]];
        for (int v : e[u])
            if (v != p) {
                ::add_edge(u, v);
                ::dep[v] = ::dep[u] + 1;
                dfs(v, u);
            }
    };
    dfs(1, 0);
    ::add_path(1, 1, 0);
    while (m--) {
        int u, v, c;
        cin >> u >> v >> c;
        if (u == v)
            continue;
        int x = u, d = ::dep[u] - ::dep[v] - 1;
        for (int k = 0; k < L; k++)
            if (d >> k & 1)
                x = up[k][x];
        ::add_path(u, x, c);
```

```
    }
    cout << ::run_heap() << '\n';
}

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--)
        solve();
    return 0;
}
```

启发式总结：状态压缩必须解释“向上还保留什么能力”。覆盖边转换点要排除上端点；任意树上路径或负费用破坏此模型，不能把堆维护误读为每点独立选最便宜路径。

验证：C++17独立程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 原稿merge只有f[k]+后缀min(child)一侧；必须补child[k]+后缀min(f)。
- 合并前哨兵表示u未覆盖，合并后必须禁用该哨兵才能返回父亲。
- 堆中的费用是剩余费用：各子树先缴费，再把全部候选统一减去已缴最小值；每个候选只被删除一次。
- 边覆盖改成点覆盖时，路径最高点应为原top下方的孩子；根加费用0的单点路径。
- CF1675D是所有根到叶路径费用1的特例；允许重叠的最优值等于叶子数，可切去重复前缀获得原题恰好覆盖一次的输出。
- 树的长链递归有栈风险。

状态压缩必须解释“向上还保留什么能力”。覆盖边转换点要排除上端点；任意树上路径或负费用破坏此模型，不能把堆维护误读为每点独立选最便宜路径。

个人资料来源：dp/树形动态规划进阶_比赛板子.md；代码源273（原题不可访问，保留点覆盖模型）

字符串、线性代数与几何 / 度数根号分治

类型与用途：混合型；小度点广播直接修改邻居，大度点广播记录时间戳；查询从相邻大点恢复最后一次颜色。

复杂度与适用范围：预处理O(n+m)，每次O(B+m/B)，B约sqrt(m)；空间O(n+m)。

复用模板：度数根号分治（本册第二部分）

例题 1 ABC219G Propagation

https://atcoder.jp/contests/abc219/tasks/abc219_g

题意：查询点将自身数字复制给所有邻居，最终输出所有点数字。

观察与规模：查询点将自身数字复制给所有邻居，最终输出所有点数字。 高次数邻居更新用重度懒广播规避星形图退化。 图中向全部邻居传播颜色的操作，扫描高出度顶点很贵。

朴素思路：每次传播遍历邻居最坏O(qm)；给所有边维护全套历史也浪费。

关键突破：高出度点个数≤2m/B，可以延迟存广播；低出度点直接更新邻居。

状态与建模：val[u]/time[u]是已直接收到的最新信息；tag[h]/stamp[h]是重邻居最后一次广播。heavy_neighbors[u]仅列出u的重邻居。

本题映射：查询点将自身数字复制给所有邻居，最终输出所有点数字。 高次数邻居更新用重度懒广播规避星形图退化。 build初始col[i]=i；每次x先qur(x)，再change(x,c,false)。

转移与正确性：读u前比较自身时间和全部重邻居广播时间，取最新者。轻点传播直接写邻居，重点传播仅改tag；时间戳的最大值确保多次覆盖等价原操作顺序。

初始化与顺序：先按度标重并建重邻居表；初始时间0，每操作递增时间。传播前必须refresh(u)，否则可能广播已过期颜色。

优化与限制：O(m+q(B+m/B))及邻接存储，阈值约sqrt(m)。Typical90-083自己和邻居都改为给定颜色；ABC219G以当前u颜色传邻居，操作语义不同，读取/更新顺序照原题。

本题复杂度：O(n+m+q sqrt m)时间、O(n+m)空间。

复用模板：度数根号分治（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m, q;
    cin >> n >> m >> q;
    DegreeDivide t(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        t.add_edge(u, v);
    }
    vector<int> a(n + 1);
    iota(a.begin(), a.end(), 0);
    t.build(a);
    while (q--) {
        int x;
        cin >> x;
        int c = t.qur(x);
        t.change(x, c);
    }
    for (int i = 1; i <= n; i++)
        cout << t.qur(i) << " \n"[i == n];
}
```

启发式总结：延迟处理不仅需要lazy值，还要能比较多个来源的新旧；时间戳让覆盖操作满足最后写入胜出。

验证：C++17组合程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 Typical90-083 Colorful Graph

https://atcoder.jp/contests/typical90/tasks/typical90_ce

题意：输出指定点当前颜色后，将它及全部邻居改成新颜色。

观察与规模：输出指定点当前颜色后，将它及全部邻居改成新颜色。 邻居广播与前题同模型，区别是本点也赋值。 图中向全部邻居传播颜色的操作，扫描高出度顶点很贵。

朴素思路：每次传播遍历邻居最坏O(qm)；给所有边维护全套历史也浪费。

关键突破：高出度点个数≤2m/B，可以延迟存广播；低出度点直接更新邻居。

状态与建模：val[u]/time[u]是已直接收到的最新信息；tag[h]/stamp[h]是重邻居最后一次广播。heavy_neighbors[u]仅列出u的重邻居。

本题映射：输出指定点当前颜色后，将它及全部邻居改成新颜色。 邻居广播与前题同模型，区别是本点也赋值。 初始全1；qur(x)输出后change(x,y,true)。

转移与正确性：读u前比较自身时间和全部重邻居广播时间，取最新者。轻点传播直接写邻居，重点传播仅改tag；时间戳的最大值确保多次覆盖等价原操作顺序。

初始化与顺序：先按度标重并建重邻居表；初始时间0，每操作递增时间。传播前必须refresh(u)，否则可能广播已过期颜色。

优化与限制：O(m+q(B+m/B))及邻接存储，阈值约sqrt(m)。Typical90-083自己和邻居都改为给定颜色；ABC219G以当前u颜色传邻居，操作语义不同，读取/更新顺序照原题。

本题复杂度：O(n+m+q sqrt m)时间、O(n+m)空间。

复用模板：度数根号分治（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, m;
    cin >> n >> m;
    DegreeDivide t(n);
    while (m--) {
        int u, v;
        cin >> u >> v;
        t.add_edge(u, v);
    }
    t.build(vector<int>(n + 1, 1));
    int q;
    cin >> q;
    while (q--) {
        int x, y;
        cin >> x >> y;
        cout << t.qur(x) << '\n';
        t.change(x, y, true);
    }
}
```

启发式总结：延迟处理不仅需要lazy值，还要能比较多个来源的新旧；时间戳让覆盖操作满足最后写入胜出。

验证：C++17组合程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- qur必须先于广播读取当前颜色，懒标记比较时间戳。
- 建图后build一次，动态增边不适用。
- 大点只有O(m/B)个，因此查询扫描的大点邻居有上界。

延迟处理不仅需要lazy值，还要能比较多个来源的新旧；时间戳让覆盖操作满足最后写入胜出。

个人资料来源：杂/根号分治.md

字符串、线性代数与几何 / 步长根号分治

类型与用途：混合型；小步长预维护/预处理，大步长暴力跳跃；收录动态模下标和与静态跳跃两个模型。

复杂度与适用范围：动态修改O(B)、查询O(1)或O(n/B)，空间O(n+B^2)；跳跃预处理O(nB)、空间O(nB)，B约sqrt(n)。

复用模板：步长根号分治（本册第二部分）

例题 1 CF1207F Remainder Problem

https://codeforces.com/problemset/problem/1207/F

题意：单点增加，查询所有下标模x等于y的位置和。

观察与规模：单点增加，查询所有下标模x等于y的位置和。 小模数一次修改影响B个桶，大模数查询最多n/B个位置。 查询涉及步长b：小步长覆盖点多但种类少，大步长种类多却路径短。

朴素思路：每次沿等差位置遍历最多O(n)，或为所有b预处理O(n^2)。

关键突破：用阈值B把“值小可预处理”和“值大可直接枚举”合起来，平衡n/B与B。

状态与建模：CF1207F对小b维护各余数类之和，更新x时改所有b≤B的sum[b][x%b]。CF797E dp[k][p]是从p反复跳p+a[p]+k到越界的步数。

本题映射：单点增加，查询所有下标模x等于y的位置和。 小模数一次修改影响B个桶，大模数查询最多n/B个位置。 init(500000)，操作1用change，操作2用qur。

转移与正确性：CF1207F大b查询直接遍历余数类；小b直接读表。CF797E小k倒序dp=1+dp[next]，因a[p],k正故next>p；大k每步前进至少k，枚举次数≤n/k。

初始化与顺序：更新前所有表0；离线数组跳跃先读固定a，再预计算小k；越界dp=0。余数类是否含位置0由真实下标条件处理。

优化与限制：B约sqrt(n)，更新/询问O(B+n/B)或预处理O(nB)，空间O(B^2)或O(nB)。CF797E允许改a则预处理失效。

本题复杂度：O(q sqrt N)时间、O(N)空间。

复用模板：步长根号分治（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int q;
    cin >> q;
    SqrtDivide t(500000);
    while (q--) {
        int op, x, y;
        cin >> op >> x >> y;
        if (op == 1)
            t.change(x, y);
        else
            cout << t.qur(x, y) << '\n';
    }
}
```

启发式总结：分治阈值由两种代价公式平衡，不能只背sqrt(n)；明确哪部分是参数种类少、哪部分是操作次数少。

验证：C++17组合程序编译通过；未登记原题样例运行；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 CF797E Array Queries

https://codeforces.com/contest/797/problem/E

题意：多次询问正数数组从p开始跳p+a[p]+k的次数。

观察与规模：多次询问正数数组从p开始跳p+a[p]+k的次数。 k较小时重复查询可复用倒序DP；大k每次跳至少k格。 查询涉及步长b：小步长覆盖点多但种类少，大步长种类多却路径短。

朴素思路：每次沿等差位置遍历最多O(n)，或为所有b预处理O(n^2)。

关键突破：用阈值B把“值小可预处理”和“值大可直接枚举”合起来，平衡n/B与B。

状态与建模：CF1207F对小b维护各余数类之和，更新x时改所有b≤B的sum[b][x%b]。CF797E dp[k][p]是从p反复跳p+a[p]+k到越界的步数。

本题映射：多次询问正数数组从p开始跳p+a[p]+k的次数。 k较小时重复查询可复用倒序DP；大k每次跳至少k格。 读数组后build_jump，再逐次jump_count。

转移与正确性：CF1207F大b查询直接遍历余数类；小b直接读表。CF797E小k倒序dp=1+dp[next]，因a[p],k正故next>p；大k每步前进至少k，枚举次数≤n/k。

初始化与顺序：更新前所有表0；离线数组跳跃先读固定a，再预计算小k；越界dp=0。余数类是否含位置0由真实下标条件处理。

优化与限制：B约sqrt(n)，更新/询问O(B+n/B)或预处理O(nB)，空间O(B^2)或O(nB)。CF797E允许改a则预处理失效。

本题复杂度：O(n sqrt n+q sqrt n)时间、O(n sqrt n)空间。

复用模板：步长根号分治（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    SqrtDivide t(n);
    t.build_jump(a);
    int q;
    cin >> q;
    while (q--) {
        int p, k;
        cin >> p >> k;
        cout << t.jump_count(p, k) << '\n';
    }
}
```

启发式总结：分治阈值由两种代价公式平衡，不能只背sqrt(n)；明确哪部分是参数种类少、哪部分是操作次数少。

验证：C++17组合程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 跳跃版必须a[i]>0，才能保证预处理依赖右侧与大步长次数界。
- 两个模型独立使用同一对象，不要build_jump后再用动态tag。
- jump计数存int32_t避免long long宏把nB内存翻倍。

分治阈值由两种代价公式平衡，不能只背sqrt(n)；明确哪部分是参数种类少、哪部分是操作次数少。

个人资料来源：杂/根号分治.md

转化、构造与计数 / 曼哈顿距离坐标旋转

类型与用途：混合型；用u=x+y,v=x-y把平面曼哈顿距离变为旋转坐标的切比雪夫距离；最大距离只需两维极值。

复杂度与适用范围：add/qur/diameter O(1)；建n点O(n)时间/O(1)额外空间。

复用模板：曼哈顿距离坐标旋转（本册第二部分）

例题 1 ABC178E Dist Max

https://atcoder.jp/contests/abc178/tasks/abc178_e

题意：求平面n点两两曼哈顿距离的最大值。

观察与规模：求平面n点两两曼哈顿距离的最大值。 max(|Δu|,|Δv|)的最大值由每一维最大减最小得到。 平面L1距离有绝对值，求最远点或维护区间极值；不是最近邻问题。

朴素思路：每问遍历全部点O(nq)，动态区间询问还需扫描区间。

关键突破：令u=x+y、v=x-y，|Δx|+|Δy|=max(|Δu|,|Δv|)，消掉绝对值组合。

状态与建模：集合信息仅需u/v各自最小最大四个数；这些信息可逐点加入，也可由两个集合逐维min/max合并。

本题映射：求平面n点两两曼哈顿距离的最大值。 max(|Δu|,|Δv|)的最大值由每一维最大减最小得到。 add所有点，diameter。

转移与正确性：全局直径为max(maxu-minu,maxv-minv)；给定点最远距离比较其到四个极值的距离。动态区间用线段树存同一四元信息，取查询区间的合并结果再计算。

初始化与顺序：空集合极值LLONG_MAX/LLONG_MIN，至少加入一点才能查询；动态每叶先init再add，父节点merge。ABC437F查询不得直接使用全局集合。

优化与限制：静态预处理O(n)、每问O(1)，动态建树/初始化按现实现O(nlog n)、每操作O(log n)。x±y及差必须容纳64位。高维有2^d符号组合，不能只留四极值。

本题复杂度：O(n)时间/O(1)额外空间。

复用模板：曼哈顿距离坐标旋转（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n;
    cin >> n;
    Manhattan t;
    while (n--) {
        int x, y;
        cin >> x >> y;
        t.add(x, y);
    }
    cout << t.diameter() << '\n';
}
```

启发式总结：把绝对值距离改为有限个线性投影，再观察投影统计能否合并；最近距离不由全局极值决定。

验证：C++17组合程序编译通过；1组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 2 Typical90036 Max Manhattan Distance

https://atcoder.jp/contests/typical90/tasks/typical90_aj

题意：多次指定一个已有点，询问到所有点的最大曼哈顿距离。

观察与规模：多次指定一个已有点，询问到所有点的最大曼哈顿距离。 两维极值由全部点共享，每问只比较到四个端点。 平面L1距离有绝对值，求最远点或维护区间极值；不是最近邻问题。

朴素思路：每问遍历全部点O(nq)，动态区间询问还需扫描区间。

关键突破：令u=x+y、v=x-y，|Δx|+|Δy|=max(|Δu|,|Δv|)，消掉绝对值组合。

状态与建模：集合信息仅需u/v各自最小最大四个数；这些信息可逐点加入，也可由两个集合逐维min/max合并。

本题映射：多次指定一个已有点，询问到所有点的最大曼哈顿距离。 两维极值由全部点共享，每问只比较到四个端点。 保留点数组；add预处理，qur(a[q])。

转移与正确性：全局直径为max(maxu-minu,maxv-minv)；给定点最远距离比较其到四个极值的距离。动态区间用线段树存同一四元信息，取查询区间的合并结果再计算。

初始化与顺序：空集合极值LLONG_MAX/LLONG_MIN，至少加入一点才能查询；动态每叶先init再add，父节点merge。ABC437F查询不得直接使用全局集合。

优化与限制：静态预处理O(n)、每问O(1)，动态建树/初始化按现实现O(nlog n)、每操作O(log n)。x±y及差必须容纳64位。高维有2^d符号组合，不能只留四极值。

本题复杂度：O(n+q)时间/O(n)空间（保存输入点）。

复用模板：曼哈顿距离坐标旋转（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, q;
    cin >> n >> q;
    vector<pair<int, int>> a(n + 1);
    Manhattan t;
    for (int i = 1; i <= n; i++) {
        cin >> a[i].first >> a[i].second;
        t.add(a[i].first, a[i].second);
    }
    while (q--) {
        int k;
        cin >> k;
        cout << t.qur(a[k].first, a[k].second) << '\n';
    }
}
```

启发式总结：把绝对值距离改为有限个线性投影，再观察投影统计能否合并；最近距离不由全局极值决定。

验证：C++17组合程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

例题 3 ABC437F Manhattan Christmas Tree 2

https://atcoder.jp/contests/abc437/tasks/abc437_f

题意：支持修改一个点坐标，查询下标区间内到指定坐标最远的曼哈顿距离。

观察与规模：支持修改一个点坐标，查询下标区间内到指定坐标最远的曼哈顿距离。 将Manhattan四个极值作为线段树合并信息，区间极值仍满足旋转距离公式。 平面L1距离有绝对值，求最远点或维护区间极值；不是最近邻问题。

朴素思路：每问遍历全部点O(nq)，动态区间询问还需扫描区间。

关键突破：令u=x+y、v=x-y，|Δx|+|Δy|=max(|Δu|,|Δv|)，消掉绝对值组合。

状态与建模：集合信息仅需u/v各自最小最大四个数；这些信息可逐点加入，也可由两个集合逐维min/max合并。

本题映射：支持修改一个点坐标，查询下标区间内到指定坐标最远的曼哈顿距离。 将Manhattan四个极值作为线段树合并信息，区间极值仍满足旋转距离公式。 完整板子附ManhattanSeg；每点change初值；修改change，查询qur(L,R).qur(x,y)。

转移与正确性：全局直径为max(maxu-minu,maxv-minv)；给定点最远距离比较其到四个极值的距离。动态区间用线段树存同一四元信息，取查询区间的合并结果再计算。

初始化与顺序：空集合极值LLONG_MAX/LLONG_MIN，至少加入一点才能查询；动态每叶先init再add，父节点merge。ABC437F查询不得直接使用全局集合。

优化与限制：静态预处理O(n)、每问O(1)，动态建树/初始化按现实现O(nlog n)、每操作O(log n)。x±y及差必须容纳64位。高维有2^d符号组合，不能只留四极值。

本题复杂度：O((n+q)log n)时间/O(n)空间。

复用模板：曼哈顿距离坐标旋转（本册第二部分）

solve 与必要辅助代码（配合已收录板子）

```
void solve() {
    int n, q;
    cin >> n >> q;
    ManhattanSeg t(n);
    for (int i = 1; i <= n; i++) {
        int x, y;
        cin >> x >> y;
        t.change(i, x, y);
    }
    while (q--) {
        int op;
        cin >> op;
        if (op == 1) {
            int i, x, y;
            cin >> i >> x >> y;
            t.change(i, x, y);
        } else {
            int l, r, x, y;
            cin >> l >> r >> x >> y;
            cout << t.qur(l, r).qur(x, y) << '\n';
        }
    }
}
```

启发式总结：把绝对值距离改为有限个线性投影，再观察投影统计能否合并；最近距离不由全局极值决定。

验证：C++17组合程序编译通过；2组登记运行通过；未在线提交；来源：已读取原站题面并核实题号、模型与输入格式

共同思路、边界与反例

- 坐标和、差、最终距离均须在long long内。
- 旋转后的整数坐标满足奇偶约束，若反求点或统计格点必须检查；本最大值问题不需要。
- 仅维护極值无法回答最小距离。

把绝对值距离改为有限个线性投影，再观察投影统计能否合并；最近距离不由全局极值决定。

个人资料来源：tirick/总结.md

第五部分 附录与索引

常用代码片段与复杂度

```
#include <bits/stdc++.h>
using namespace std;
#define int long long

signed main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int t = 1;
    // cin >> t;
    while (t--) solve();
    return 0;
}
```

线性/树遍历O(n)：通常适合百万级，但递归深度和邻接内存要另查。

排序与平衡结构O(nlog n)：适合十万级常见题；常数、long long表和节点总数仍要预算。

背包O(nW)、区间DP O(n^3)：只在实际容量/区间数足够小时使用，不能把“DP”误当复杂度保证。

枚举集合O(2^n)、枚举全部状态转移O(3^n)：只适合小集合；上界应按操作数及内存具体估算。

__int128乘法后再取模可避免64位中间乘积溢出；仅#define int long long不能解决所有乘法。

常见边界与本版验证范围

区间下标、空集/非空要求、0基/1基、闭区间端点和哨兵需与题面一致。

多测对全局vector、memo、堆、BIT以及时间戳显式init；函数版替代对象默认构造时必须补清空。

深树DFS、Tarjan、重构树和附着树递归都可能栈RE；本册保持用户递归写法，并注明需要足够栈空间。

节点池、二维表和#define int long long使每项8字节，可能MLE；统计表/编号可在确有范围保证时用int32_t。

本版新增ABC361E原站两个样例，普通背包和顶点覆盖另给直接题解；全部177份程序编译及登记输入运行，变更函数与原版还进行独立/回归对照，详情以手册验证记录和验证摘要为准。

没有在线评测AC记录，也没有对所有模型跑最大规模压力测试。课程272/273无法取得原题，以核实的相似题替代，未伪造原题输入格式。

知识点与旧版映射

模板型 / 01Trie

混合型 / DSU on Tree

模板型 / ST表

模板型 / 动态开点权值线段树

模板型 / 势能线段树

模板型 / 区间加线段树

思维型 / 启发式分治

混合型 / 启发式合并

模板型 / 对顶堆与中位数

模板型 / 并查集

模板型 / 普通线段树

模板型 / 权值线段树合并与分裂
模板型 / 树状数组
模板型 / 珂朵莉树
混合型 / 离线二维数点
混合型 / 线段树优化建图
模板型 / 2-SAT
模板型 / DFS增广路匹配
模板型 / Dinic最大流
模板型 / Hopcroft-Karp匹配
模板型 / Kosaraju强连通分量
混合型 / Kruskal重构树
模板型 / Prim最小生成树
模板型 / Tarjan强连通分量
模板型 / 二分图染色
模板型 / 倍增LCA
模板型 / 割点
模板型 / 割边
混合型 / 基环树最大权独立集
混合型 / 基环树最长路径与断环直径
模板型 / 差分约束
模板型 / 拓扑排序
模板型 / 无向欧拉路径
混合型 / 最小割建模
模板型 / 最小费用最大流
模板型 / 有上下界可行流与最大流
模板型 / 有向欧拉路径
模板型 / 点双连通分量
模板型 / 边双连通分量
模板型 / BSGS离散对数
思维型 / LCM单行求和
思维型 / LCM矩形与集合求和
模板型 / Lucas定理
模板型 / 中国剩余定理
思维型 / 互质集合与GCD分类计数
思维型 / 卡特兰数与反射原理
模板型 / 原根
模板型 / 扩展Lucas
模板型 / 扩展欧几里得
模板型 / 扩展欧拉与指数塔
模板型 / 整除分块
模板型 / 素数模组合数
模板型 / 莫比乌斯反演与线性筛
混合型 / DFS序依赖背包

思维型 / 二状态树形背包

思维型 / 二进制位贡献与Popcount计数

思维型 / 区间同色配对与涂色

思维型 / 区间强制对象分治

思维型 / 区间石子合并

思维型 / 子树合并背包

思维型 / 括号计数与无歧义区间DP

思维型 / 换根连通块计数

模板型 / 数位DP与有限状态

思维型 / 数位构造与不同数位限制

思维型 / 数位背包

混合型 / 树上不相交给定路径

思维型 / 树上最大连通块权和

思维型 / 树上行走与返回状态设计

合并覆盖旧专题：dp/整理版/Crystalfly三状态树DP；dp/整理版/Kingdom军队遍历树DP。原模板与例题未删除。

思维型 / 树上距离分层计数

混合型 / 祖先路径最小费用覆盖

思维型 / 自相似序列奇偶分治

思维型 / 进位支付与有符号数位

模板型 / KMP字符串匹配

混合型 / 度数根号分治

思维型 / 极角排序

混合型 / 步长根号分治

模板型 / 点直线与线段几何

模板型 / 线性基

思维型 / 两互质面额不可表示数

思维型 / 前缀关系计数

思维型 / 同余差与GCD

思维型 / 异或比较与最高不同位

混合型 / 曼哈顿距离坐标旋转

思维型 / 置换环分解

dp/区间动态规划进阶_比赛板子.md → 区间同色配对与涂色；区间强制对象分治；区间石子合并；括号计数与无歧义区间DP

dp/数位动态规划进阶_比赛板子.md → 二进制位贡献与Popcount计数；数位DP与有限状态；数位构造与不同数位限制；数位背包；自相似序列奇偶分治；进位支付与有符号数位

dp/树形动态规划进阶_比赛板子.md → DFS序依赖背包；二状态树形背包；子树合并背包；换根连通块计数；树上不相交给定路径；树上最大连通块权和；树上行走与返回状态设计；树上距离分层计数；祖先路径最小费用覆盖

tirick/总结.md → 树状数组；两互质面额不可表示数；前缀关系计数；同余差与GCD；异或比较与最高不同位；曼哈顿距离坐标旋转；置换环分解

代码源272（原题不可访问，按已存课堂模型整理） → 树上不相交给定路径

代码源273（原题不可访问，保留点覆盖模型） → 祖先路径最小费用覆盖

卡特兰数复习资料.md → 卡特兰数与反射原理

图/图论竞赛复习_严格课件注释版.md → 2-SAT；Dinic最大流；Kosaraju强连通分量；Tarjan强连通分量；割点；割边；基环树最大权独立集；基环树最长路径与断环直径；差分约束；最小割建模；最小费用最大流；有上下界可行流与最大流；边双连通分量

图/图论竞赛复习_按课件整理.md → 最小费用最大流；有上下界可行流与最大流；点双连通分量
图论/HK二分图最大匹配.cpp → Hopcroft-Karp匹配
图论/Kruskal重构树.cpp → Kruskal重构树
图论/Kruskal重构树板子 (1).md → Kruskal重构树
图论/SCC.md → Kosaraju强连通分量；Tarjan强连通分量
图论/prim.cpp → Prim最小生成树
图论/二分图匹配.cpp → DFS增广路匹配
图论/二分图染色.cpp → 二分图染色
图论/倍增求lca.cpp → 倍增LCA
图论/差分约束.cpp → 差分约束
图论/拓扑.cpp → 拓扑排序
图论/欧拉路径.md → 无向欧拉路径；有向欧拉路径
数据结构/01tire（支持删）.cpp → 01Trie
数据结构/ST表.cpp → ST表
数据结构/对顶堆（multiset维护中位数）.cpp → 对顶堆与中位数
数据结构/并查集/并查集.md → 并查集
数据结构/并查集/板子.cpp → 并查集
数据结构/扫描线/二维数点.md → 离线二维数点
数据结构/树状数组/封装.cpp → 树状数组
数据结构/模块一_启发式合并与启发式分治.md → 启发式分治；启发式合并
数据结构/模块二_DSU_on_Tree与树上统计.md → DSU on Tree
数据结构/珂朵莉树.cpp → 珂朵莉树
数据结构/线段树/势能线段树.cpp → 势能线段树
数据结构/线段树/区间修改区间查询.cpp → 区间加线段树
数据结构/线段树/区间查询单点修改.cpp → 普通线段树
数据结构/线段树/权值线段树（动态开点基础版）.cpp → 动态开点权值线段树
数据结构/线段树/权值线段树（合并与分裂）.cpp → 权值线段树合并与分裂
数据结构/线段树/线段树优化建图.cpp → 线段树优化建图
数论/CRT（中国剩余定理）.cpp → 中国剩余定理
数论/exgcd.md → 扩展欧几里得
数论/数论10_ACM板子.md → 互质集合与GCD分类计数
数论/数论10_课程复习.md → LCM单行求和；LCM矩形与集合求和
数论/数论6_ACM板子.md → Lucas定理；扩展Lucas
数论/数论6_课程复习.md → Lucas定理；扩展Lucas
数论/数论7_ACM板子.md → 扩展欧拉与指数塔
数论/数论7_课程复习.md → 扩展欧拉与指数塔
数论/数论8_ACM板子.md → BSGS离散对数；原根
数论/数论8_课程复习.md → BSGS离散对数；原根
数论/数论9_ACM板子.md → 莫比乌斯反演与线性筛
数论/数论分块.cpp → 整除分块
数论/数论分块.md → 整除分块
数论/求mo函数.cpp → 莫比乌斯反演与线性筛
数论/组合数.cpp → 素数模组合数

杂/dsu on tree.cpp → DSU on Tree

杂/kmp.cpp → KMP字符串匹配

杂/根号分治.md → 度数根号分治；步长根号分治

杂/点线.cpp → 极角排序；点直线与线段几何

杂/线性基.md → 线性基

题号、位置与原题网址

ABC034C / 経路 / 素数模组合数

https://atcoder.jp/contests/abc034/tasks/abc034_c

ABC065D / Built / Prim最小生成树

https://atcoder.jp/contests/abc065/tasks/arc076_b

ABC105D / Candy Distribution / 前缀关系计数

https://atcoder.jp/contests/abc105/tasks/abc105_d

ABC109C / Skip / 同余差与GCD

https://atcoder.jp/contests/abc109/tasks/abc109_c

ABC120D / Decayed Bridges / 并查集

https://atcoder.jp/contests/abc120/tasks/abc120_d

ABC127F / Absolute Minima / 对顶堆与中位数

https://atcoder.jp/contests/abc127/tasks/abc127_f

ABC139F / Engines / 极角排序

https://atcoder.jp/contests/abc139/tasks/abc139_f

ABC145D / Knight / 素数模组合数

https://atcoder.jp/contests/abc145/tasks/abc145_d

ABC155E / Payment / 进位支付与有符号数位

https://atcoder.jp/contests/abc155/tasks/abc155_e

ABC178E / Dist Max / 曼哈顿距离坐标旋转

https://atcoder.jp/contests/abc178/tasks/abc178_e

ABC186E / Throne / 扩展欧几里得

https://atcoder.jp/contests/abc186/tasks/abc186_e

ABC219G / Propagation / 度数根号分治

https://atcoder.jp/contests/abc219/tasks/abc219_g

ABC223D / Restricted Permutation / 拓扑排序

https://atcoder.jp/contests/abc223/tasks/abc223_d

ABC231E / Minimal payments / 进位支付与有符号数位

https://atcoder.jp/contests/abc231/tasks/abc231_e

ABC241D / Sequence Query / 动态开点权值线段树

https://atcoder.jp/contests/abc241/tasks/abc241_d

ABC242D / ABC Transform / 自相似序列奇偶分治

https://atcoder.jp/contests/abc242/tasks/abc242_d

ABC247F / Cards / 置换环分解

https://atcoder.jp/contests/abc247/tasks/abc247_f

ABC253C / Max-Min Query / 动态开点权值线段树

https://atcoder.jp/contests/abc253/tasks/abc253_c

ABC312D / Count Bracket Sequences / 括号计数与无歧义区间DP

https://atcoder.jp/contests/abc312/tasks/abc312_d

ABC327D / Good Tuple Problem / 二分图染色

https://atcoder.jp/contests/abc327/tasks/abc327_d

ABC350C / Sort / 置换环分解

https://atcoder.jp/contests/abc350/tasks/abc350_c

ABC356D / Masked Popcount / 二进制位贡献与Popcount计数

https://atcoder.jp/contests/abc356/tasks/abc356_d

ABC361E / Tree and Hamilton Path 2 / 树上行走与返回状态设计

https://atcoder.jp/contests/abc361/tasks/abc361_e

ABC436E / Minimum Swap / 置换环分解

https://atcoder.jp/contests/abc436/tasks/abc436_e

ABC437F / Manhattan Christmas Tree 2 / 曼哈顿距离坐标旋转

https://atcoder.jp/contests/abc437/tasks/abc437_f

AGC038C / LCMs / LCM矩形与集合求和

https://atcoder.jp/contests/agc038/tasks/agc038_c

CF1132F / Clear the String / 区间同色配对与涂色

https://codeforces.com/contest/1132/problem/F

CF1156E / Special Segments / 启发式分治

https://codeforces.com/contest/1156/problem/E

CF118E / Bertown Roads / 割边

https://codeforces.com/problemset/problem/118/E

CF1207F / Remainder Problem / 步长根号分治

https://codeforces.com/problemset/problem/1207/F

CF1324F / Maximum White Subtree / 树上最大连通块权和

https://codeforces.com/contest/1324/problem/F

CF1328E / Tree Queries / 倍增LCA

https://codeforces.com/problemset/problem/1328/E

CF1398C / Good Subarrays / 前缀关系计数

https://codeforces.com/problemset/problem/1398/C

CF1551F / Equidistant Vertices / 树上距离分层计数

https://codeforces.com/contest/1551/problem/F

CF1560F1 / Nearest Beautiful Number easy / 数位构造与不同数位限制

https://codeforces.com/contest/1560/problem/F1

CF1560F2 / Nearest Beautiful Number hard / 数位构造与不同数位限制

https://codeforces.com/contest/1560/problem/F2

CF161D / Distance in Tree / 树上距离分层计数

https://codeforces.com/contest/161/problem/D

CF1675D / Vertical Paths / 祖先路径最小费用覆盖

https://codeforces.com/problemset/problem/1675/D

CF1720D1 / Xor-Subsequence (easy version) / 异或比较与最高不同位

https://codeforces.com/problemset/problem/1720/D1

CF1920C / Partitioning the Array / 同余差与GCD

https://codeforces.com/problemset/problem/1920/C

CF276D / Little Girl and Maximum XOR / 异或比较与最高不同位

https://codeforces.com/problemset/problem/276/D

CF284A / Cows and Primitive Roots / 原根

https://codeforces.com/problemset/problem/284/A

CF375D / Tree and Queries / DSU on Tree

https://codeforces.com/contest/375/problem/D

CF438D / The Child and Sequence / 势能线段树

https://codeforces.com/problemset/problem/438/D

CF471D / MUH and Cube Walls / KMP字符串匹配

https://codeforces.com/contest/471/problem/D

CF508D / Tanya and Password / 有向欧拉路径

https://codeforces.com/contest/508/problem/D

CF510E / Fox And Dinner / 有上下界可行流与最大流

https://codeforces.com/contest/510/problem/E

CF543D / Road Improvement / 换根连通块计数

https://codeforces.com/contest/543/problem/D

CF598C / Nearest vectors / 极角排序

https://codeforces.com/contest/598/problem/C

CF600E / Lomsat gelral / DSU on Tree

https://codeforces.com/problemset/problem/600/E

CF600E / Lomsat gelral / 启发式合并

https://codeforces.com/problemset/problem/600/E

CF671D / Roads in Yusland / 祖先路径最小费用覆盖

https://codeforces.com/problemset/problem/671/D

CF706D / Vasiliy’s Multiset / 01Trie

https://codeforces.com/problemset/problem/706/D

CF786B / Legacy / 线段树优化建图

https://codeforces.com/problemset/problem/786/B

CF797E / Array Queries / 步长根号分治

https://codeforces.com/contest/797/problem/E

CF803F / Coprime Subsequences / 互质集合与GCD分类计数

https://codeforces.com/contest/803/problem/F

CF835F / Roads in the Kingdom / 基环树最长路径与断环直径

https://codeforces.com/contest/835/problem/F

CF839D / Winter is here / 互质集合与GCD分类计数

https://codeforces.com/contest/839/problem/D

CF896C / Willem Chtholly and Seniorious / 珂朵莉树

https://codeforces.com/contest/896/problem/C

CF906D / Power Tower / 扩展欧拉与指数塔

https://codeforces.com/problemset/problem/906/D

CGL_1_A / Projection / 点直线与线段几何

https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_1_A

CGL_1_B / Reflection / 点直线与线段几何

https://judge.u-aizu.ac.jp/onlinejudge/description.jsp?id=CGL_1_B

DP_N / Slimes / 区间石子合并

https://atcoder.jp/contests/dp/tasks/dp_n

DP_S / Digit Sum / 数位DP与有限状态

https://atcoder.jp/contests/dp/tasks/dp_s

DP_V / Subtree / 换根连通块计数

https://atcoder.jp/contests/dp/tasks/dp_v

Gym102769K / Kingdom’s Power / 树上行走与返回状态设计

https://codeforces.com/gym/102769/problem/K

Gym102832D / Meaningless Sequence / 二进制位贡献与Popcount计数

https://codeforces.com/gym/102832/problem/D

Gym102992M / Monster Hunter / 二状态树形背包

https://codeforces.com/gym/102992/problem/M

Gym103470H / Crystalfly / 树上行走与返回状态设计

https://codeforces.com/gym/103470/problem/H

HDU1792 / A New Change Problem / 两互质面额不可表示数

https://acm.hdu.edu.cn/showproblem.php?pid=1792

HDU4912 / Paths on the tree / 树上不相交给定路径

https://acm.hdu.edu.cn/showproblem.php?pid=4912

HDU5293 / Tree chain problem / 树上不相交给定路径

https://acm.hdu.edu.cn/showproblem.php?pid=5293

LeetCode480 / 滑动窗口中位数 / 对顶堆与中位数

https://leetcode.cn/problems/sliding-window-median/description/

P1044 / 栈 / 卡特兰数与反射原理

https://www.luogu.com.cn/problem/P1044

P1048 / 采药 / 数位背包

https://www.luogu.com.cn/problem/P1048

P1082 / 同余方程 / 扩展欧几里得

https://www.luogu.com.cn/problem/P1082

P1122 / 最大子树和 / 树上最大连通块权和

https://www.luogu.com.cn/problem/P1122

P1137 / 旅行计划 / 拓扑排序

https://www.luogu.com.cn/problem/P1137

P1168 / 中位数 / 对顶堆与中位数

https://www.luogu.com.cn/problem/P1168

P1198 / 最大数 / 普通线段树

https://www.luogu.com.cn/problem/P1198

P1273 / 有线电视网 / 子树合并背包

https://www.luogu.com.cn/problem/P1273

P1313 / 计算系数 / Lucas定理

https://www.luogu.com.cn/problem/P1313

P1330 / 封锁阳光大学 / 二分图染色

https://www.luogu.com.cn/problem/P1330

P1341 / 无序字母对 / 无向欧拉路径

https://www.luogu.com.cn/problem/P1341

P1403 / 约数研究 / 整除分块

https://www.luogu.com.cn/problem/P1403

P1453 / 城市环路 / 基环树最大权独立集

https://www.luogu.com.cn/problem/P1453

P1495 / 曹冲养猪 / 中国剩余定理

https://www.luogu.com.cn/problem/P1495

P1641 / 生成字符串 / 卡特兰数与反射原理

https://www.luogu.com.cn/problem/P1641

P1656 / 炸铁路 / 割边

https://www.luogu.com.cn/problem/P1656

P1816 / 忠诚 / 普通线段树

https://www.luogu.com.cn/problem/P1816

P1829 / Crash的数字表格 / LCM矩形与集合求和

https://www.luogu.com.cn/problem/P1829

P1880 / 石子合并 / 区间石子合并

https://www.luogu.com.cn/problem/P1880

P1891 / 疯狂LCM / LCM单行求和

https://www.luogu.com.cn/problem/P1891

P1908 / 逆序对 / 树状数组

https://www.luogu.com.cn/problem/P1908

P1966 / 火柴排队 / 树状数组

https://www.luogu.com.cn/problem/P1966

P1967 / 货车运输 / Kruskal重构树

https://www.luogu.com.cn/problem/P1967

P1972 / HH的项链 / 离线二维数点

https://www.luogu.com.cn/problem/P1972

P2014 / 选课 DFS序版 / DFS序依赖背包

https://www.luogu.com.cn/problem/P2014

P2014 / 选课 / 子树合并背包

https://www.luogu.com.cn/problem/P2014

P2016 / 战略游戏 / 二状态树形背包

https://www.luogu.com.cn/problem/P2016

P2163 / 园丁的烦恼 / 离线二维数点

https://www.luogu.com.cn/problem/P2163

P2183 / 礼物 / 扩展Lucas

https://www.luogu.com.cn/problem/P2183

P2245 / 星际导航 / Kruskal重构树

https://www.luogu.com.cn/problem/P2245

P2341 / 受欢迎的牛 / Kosaraju强连通分量

https://www.luogu.com.cn/problem/P2341

P2341 / 受欢迎的牛 / Tarjan强连通分量

https://www.luogu.com.cn/problem/P2341

P2424 / 约数和 / 整除分块

https://www.luogu.com.cn/problem/P2424

P2485 / 计算器 / BSGS离散对数

https://www.luogu.com.cn/problem/P2485

P2515 / 软件安装 / DFS序依赖背包

https://www.luogu.com.cn/problem/P2515

P2522 / ZAP-Queries / 莫比乌斯反演与线性筛

https://www.luogu.com.cn/problem/P2522

P2607 / 骑士 / 基环树最大权独立集

https://www.luogu.com.cn/problem/P2607

P2657 / windy数 / 数位DP与有限状态

https://www.luogu.com.cn/problem/P2657

P2731 / Riding the Fences / 无向欧拉路径

https://www.luogu.com.cn/problem/P2731

P2762 / 太空飞行计划问题 / 最小割建模

https://www.luogu.com.cn/problem/P2762

P2763 / 试题库问题 / Dinic最大流

https://www.luogu.com.cn/problem/P2763

P2774 / 方格取数问题 / 最小割建模

https://www.luogu.com.cn/problem/P2774

P2860 / Redundant Paths / 边双连通分量

https://www.luogu.com.cn/problem/P2860

P2880 / Balanced Lineup / ST表

https://www.luogu.com.cn/problem/P2880

P3188 / 梦幻岛宝珠 / 数位背包

https://www.luogu.com.cn/problem/P3188

P3201 / 梦幻布丁 / 启发式合并

https://www.luogu.com.cn/problem/P3201

P3224 / 永无乡 / 权值线段树合并与分裂

https://www.luogu.com.cn/problem/P3224

P3225 / 矿场搭建 / 点双连通分量

https://www.luogu.com.cn/problem/P3225

P3275 / 糖果 / 差分约束

https://www.luogu.com.cn/problem/P3275

P3366 / 最小生成树 / Prim最小生成树

https://www.luogu.com.cn/problem/P3366

P3369 / 普通平衡树 / 动态开点权值线段树

https://www.luogu.com.cn/problem/P3369

P3372 / 线段树1 / 区间加线段树

https://www.luogu.com.cn/problem/P3372

P3374 / 树状数组1 / 区间加线段树

https://www.luogu.com.cn/problem/P3374

P3374 / 树状数组1 / 树状数组

https://www.luogu.com.cn/problem/P3374

P3375 / KMP / KMP字符串匹配

https://www.luogu.com.cn/problem/P3375

P3376 / 网络最大流 / Dinic最大流

https://www.luogu.com.cn/problem/P3376

P3379 / 最近公共祖先 / 倍增LCA

https://www.luogu.com.cn/problem/P3379

P3381 / 最小费用最大流 / 最小费用最大流

https://www.luogu.com.cn/problem/P3381

P3386 / 二分图最大匹配 / DFS增广路匹配

https://www.luogu.com.cn/problem/P3386

P3386 / 二分图最大匹配 / Hopcroft-Karp匹配

https://www.luogu.com.cn/problem/P3386

P3387 / 缩点 / Kosaraju强连通分量

https://www.luogu.com.cn/problem/P3387

P3387 / 缩点 / Tarjan强连通分量

https://www.luogu.com.cn/problem/P3387

P3388 / 割点 / 割点

https://www.luogu.com.cn/problem/P3388

P3455 / Problem b / 莫比乌斯反演与线性筛

https://www.luogu.com.cn/problem/P3455

P3469 / BLO-Blockade / 割点

https://www.luogu.com.cn/problem/P3469

P3740 / 贴海报 / 珂朵莉树

https://www.luogu.com.cn/problem/P3740

P3807 / Lucas定理 / Lucas定理

https://www.luogu.com.cn/problem/P3807

P3812 / 线性基 / 线性基

https://www.luogu.com.cn/problem/P3812

P3846 / 可爱的质数 / BSGS离散对数

https://www.luogu.com.cn/problem/P3846

P3865 / ST表 / ST表

https://www.luogu.com.cn/problem/P3865

P3951 / 小凯的疑惑 / 两互质面额不可表示数

https://www.luogu.com.cn/problem/P3951

P4015 / 运输问题 / 最小费用最大流

https://www.luogu.com.cn/problem/P4015

P4126 / 最小割边判定 / 最小割建模

https://www.luogu.com.cn/problem/P4126

P4139 / 上帝与集合的正确用法 / 扩展欧拉与指数塔

https://www.luogu.com.cn/problem/P4139

P4145 / 花神游历各国 / 势能线段树

https://www.luogu.com.cn/problem/P4145

P4151 / 最大XOR和路径 / 线性基

https://www.luogu.com.cn/problem/P4151

P4170 / 涂色 / 区间同色配对与涂色

https://www.luogu.com.cn/problem/P4170

P4171 / 满汉全席 / 2-SAT

https://www.luogu.com.cn/problem/P4171

P4317 / 花神的数论题 / 二进制位贡献与Popcount计数

https://www.luogu.com.cn/problem/P4317

P4381 / Island / 基环树最长路径与断环直径

https://www.luogu.com.cn/problem/P4381

P4551 / 最长异或路径 / 01Trie

https://www.luogu.com.cn/problem/P4551

P4570 / 元素 / 线性基

https://www.luogu.com.cn/problem/P4570

P4720 / 扩展Lucas / 扩展Lucas

https://www.luogu.com.cn/problem/P4720

P4766 / Outer space invaders / 区间强制对象分治

https://www.luogu.com.cn/problem/P4766

P4777 / 扩展中国剩余定理 / 中国剩余定理

https://www.luogu.com.cn/problem/P4777

P4782 / 2-SAT / 2-SAT

https://www.luogu.com.cn/problem/P4782

P5192 / Shoot the Bullet / 有上下界可行流与最大流

https://www.luogu.com.cn/problem/P5192

P5494 / 线段树分裂 / 权值线段树合并与分裂

https://www.luogu.com.cn/problem/P5494

P5960 / 差分约束 / 差分约束

https://www.luogu.com.cn/problem/P5960

P6091 / 原根 / 原根

https://www.luogu.com.cn/problem/P6091

P6348 / Journeys / 线段树优化建图

https://www.luogu.com.cn/problem/P6348

P7771 / 欧拉路径 / 有向欧拉路径

https://www.luogu.com.cn/problem/P7771

P7914 / 括号序列 / 括号计数与无歧义区间DP

https://www.luogu.com.cn/problem/P7914

P8435 / 点双连通分量 / 点双连通分量

https://www.luogu.com.cn/problem/P8435

P8436 / 边双连通分量 / 边双连通分量

https://www.luogu.com.cn/problem/P8436

QOJ12409 / Bit Sequence / 自相似序列奇偶分治

https://qoj.ac/problem/12409

SP5971 / LCMSUM - LCM Sum / LCM单行求和

https://www.luogu.com.cn/problem/SP5971

Typical90-083 / Colorful Graph / 度数根号分治

https://atcoder.jp/contests/typical90/tasks/typical90_ce

Typical90036 / Max Manhattan Distance / 曼哈顿距离坐标旋转

https://atcoder.jp/contests/typical90/tasks/typical90_aj

UVA10003 / Cutting Sticks / 区间强制对象分治

https://onlinejudge.org/external/100/10003.pdf

UVA1608 / Non-boring sequences / 启发式分治

https://onlinejudge.org/external/16/1608.pdf

practice2_a / Disjoint Set Union / 并查集

https://atcoder.jp/contests/practice2/tasks/practice2_a

practice2_d / Maxflow / DFS增广路匹配

https://atcoder.jp/contests/practice2/tasks/practice2_d

practice2_d / Maxflow / Hopcroft-Karp匹配

https://atcoder.jp/contests/practice2/tasks/practice2_d